
TotalDepth Guide

Release 0.4.0

Paul Ross

Oct 28, 2024

CONTENTS

1	Introduction to TotalDepth	3
1.1	Petrophysical File Formats Supported by TotalDepth	3
2	TotalDepth Example Outputs	5
2.1	Wireline Plots	5
2.1.1	Plot Examples	5
2.1.2	Plotting from LIS	7
2.1.3	Plotting From LAS	8
2.2	LIS Log HTML Summaries	9
2.2.1	How This HTML was Made	9
2.3	RP66V1 Log HTML Summaries	9
3	Installation	11
3.1	Stable release	11
3.2	From sources	11
3.3	Developing with TotalDepth	12
3.3.1	Test Coverage	12
3.3.2	Documentation	12
3.3.3	System Testing	12
3.3.4	Unit Testing	13
4	HOWTOs: What TotalDepth can do for you	15
4.1	Working with Archives of Mixed Data	15
4.1.1	I have an archive of files and I want to know what is in there.	15
4.1.2	I have an archive and I want copy specific file types	15
4.1.3	I have an archive and I want report or remove duplicate files	15
4.2	Working with Western Atlas BIT Data	15
4.2.1	I have an archive of data and I'd like a summary	15
4.2.2	I'd like to convert BIT files to LAS format files	16
4.2.3	Getting the Frame Data as a <code>numpy</code> Array	16
4.2.4	I have some troublesome BIT files	16
4.3	Working with LAS Data	16
4.3.1	I have an archive of data and I'd like a summary in HTML	16
4.3.2	I have an archive of LAS data and I want to plot it	16
4.3.3	Getting the Frame Data as a <code>numpy</code> Array	16
4.3.4	I have some troublesome LAS files	16
4.4	Working with LIS Data	17
4.4.1	I have an archive of data and I'd like a summary in HTML	17
4.4.2	I have an archive of LIS data and I want to plot it	17
4.4.3	I'd like to convert LIS files to LAS format files	17

4.4.4	Getting the Frame Data as a <code>numpy</code> Array	17
4.4.5	I have some troublesome LIS files	17
4.5	Working with RP66V1 Data	17
4.5.1	I have an archive of RP66V1 data and I'd like a summary in HTML	17
4.5.2	I'd like to create some well plots from RP66V1 data	18
4.5.3	I'd like to convert RP66V1 files to LAS format files	18
4.5.4	Getting the Frame Data as a <code>numpy</code> Array	18
4.5.5	I have some troublesome RP66V1 files	18
4.6	Converting Files to LAS	18
4.6.1	Converting LIS Files to LAS	18
4.6.2	Converting RP66V1 Files to LAS	18
5	Command Line Tools	19
5.1	TotalDepth Command Line Tools	19
5.1.1	Analysing a Directory of Files with <code>tdarchive</code>	19
5.1.2	Copying a Directory of Files with <code>tdcopybinfiles</code>	23
5.1.3	Removing Duplicate Files with <code>tdremovedupefiles</code>	25
5.1.4	Removing TIF Markers From Files with <code>tddetif</code>	26
5.1.5	Plotting Well Logs with <code>tdplotlogs</code>	27
5.2	BIT Command Line Tools	33
5.2.1	Summarise BIT Files with <code>tdbitread</code>	34
5.2.2	Converting BIT Files to LAS Files with <code>tdbittolas</code>	39
5.3	LAS Command Line Tools	44
5.3.1	Summarise LAS Files in HTML with <code>tdlastohtml</code>	44
5.3.2	Summarise LAS Files with <code>tdlasreadlasfiles</code>	47
5.4	LIS Command Line Tools	51
5.4.1	Summarise LIS Files in HTML with <code>tdlistohtml</code>	51
5.4.2	Plots LIS log data as SVG pages.	55
5.4.3	Scanning Physical Records in LIS Files with <code>tdlisscanphysrec</code>	56
5.4.4	Scanning Logical Records in LIS Files with <code>tdlisscanlogirecord</code>	59
5.4.5	Scanning Logical Data in LIS Files with <code>tdlisscanlogidata</code>	60
5.4.6	Extracting Data from LIS with <code>tdlisdumpframeset</code>	62
5.4.7	Analysing Table Data in LIS Files with <code>tdlistablehistogram</code>	64
5.4.8	Indexing LIS Files with <code>tdlisindex</code>	66
5.4.9	<code>tdXlisrandomframesetread</code>	68
5.4.10	Converting LIS Files to LAS Files with <code>tdlistolas</code>	68
5.5	RP66V1 Command Line Tools	72
5.5.1	Creating HTML Pages from RP66V1 Files with <code>tdrp66v1scanhtml</code>	73
5.5.2	Converting RP66V1 Files to LAS Files with <code>tdrp66v1tolas</code>	74
5.5.3	Indexing RP66V1 Files with <code>tdrp66v1indexpickle</code>	78
5.5.4	Indexing RP66V1 Files with <code>tdrp66v1indexxml</code>	79
5.5.5	Scanning RP66V1 Files with <code>tdrp66v1scan</code>	82
6	Importing TotalDepth into your Python Project	103
6.1	Processing Western Atlas BIT Files	103
6.1.1	Reading a BIT File Log Data	103
6.2	Processing LAS Files	104
6.2.1	Reading a LAS File Log Data	104
6.3	Processing LIS Files	105
6.3.1	Important Concepts	105
6.3.2	Important LIS Concepts	106
6.3.3	Reading a LIS File Table Data	106
6.3.4	Reading a LIS File Log Data	108
6.4	Processing RP66V1 Files	110

6.4.1	RP66V1 File Structure	110
6.4.2	Basic Pattern for Reading RP66V1 Files with TotalDepth	112
6.4.3	Inspecting the Logical File	112
6.4.4	More about RP66V1.EFLR Tables	117
6.4.5	Reading the Frame Data and Accessing the <code>numpy</code> Arrays	123
7	Technical Notes	129
7.1	Performance of TotalDepth	129
7.2	Measuring Performance	129
7.2.1	User's Perception of Performance	129
7.2.2	Performance Improvements	130
7.3	LAS Performance	130
7.3.1	Reading LAS Files	130
7.4	Indexing LIS Files	132
7.4.1	Introduction	132
7.4.2	Indexing Design	132
7.4.3	Indexing Performance	135
7.5	LIS Performance	140
7.5.1	File Read Cost	140
7.5.2	Frame Read Cost	142
7.5.3	Performance Improvements	147
7.6	Indexing RP66V1 Files	147
7.6.1	Low-Level Index	148
7.6.2	Mid-Level Index	152
7.7	RP66V1 Performance	156
7.7.1	Scanning RP66V1 to produce an HTML Summary	156
7.7.2	Converting RP66V1 to LAS	158
7.8	Plotting Wireline Data	161
7.8.1	Wireline Files With Integral Plot Data	161
7.8.2	Wireline Files With External Plot Data	163
7.8.3	Internal Plot Algorithms	164
7.9	Data Quality	167
7.10	TotalDepth and SaaS	168
7.10.1	Processing Server Side Files	168
7.10.2	Processing Files that Remain on the Client's Device	168
7.10.3	Rest of Session	170
7.10.4	Data Persistence	171
7.10.5	SaaS Pros and Cons	171
7.11	Process Monitoring with <code>TotalDepth.common.process</code>	171
7.11.1	Command Line Tools	172
7.11.2	Plotting the Data	172
7.11.3	Adding Events as Graph Labels	174
7.12	Performance Graphs via Gnuplot with <code>TotalDepth.util.gnuplot</code>	176
7.13	Total Depth and Units	177
7.13.1	LIS Units	177
7.13.2	General Units	177
7.14	Schlumberger Mnemonics	178
8	TODO's	183
8.1	TotalDepth Improvements (General)	183
8.2	LAS Improvements	184
8.3	LIS Improvements	184
8.4	RP66V1 Improvements	185
8.5	Plotting Improvements	186

8.6	File Formats	186
8.7	Frame Array Improvements	186
9	Contributing	187
9.1	Types of Contributions	187
9.1.1	Report Bugs	187
9.1.2	Fix Bugs	187
9.1.3	Implement Features	187
9.1.4	Write Documentation	187
9.1.5	Submit Feedback	188
9.2	Get Started!	188
9.2.1	Pull Request Guidelines	189
9.2.2	Tips	189
9.3	Release Checklist	189
9.3.1	Increment version	189
9.3.2	Build and Test	190
9.3.3	Build a Development Environment	190
9.3.4	Commit and Tag	191
9.3.5	PyPi	191
9.3.6	ReadTheDocs	191
9.3.7	Prepare Next Release Candidate	191
10	Testing TotalDepth	193
10.1	Unit Tests	193
10.1.1	Basic Testing	193
10.1.2	Testing With Test Coverage	193
10.2	Testing the Plot Package	194
10.2.1	Running the test	194
10.2.2	Results	195
10.3	Testing TotalDepth With Recorded Data	198
10.3.1	Small Test Set	198
10.3.2	Medium Test Set	198
10.3.3	Large Test Set	198
10.3.4	Synthetic Test Set	199
11	History	201
11.1	0.4.0 (2021-09-18)	201
11.2	0.3.1 (2020-06-15)	202
11.3	0.3.0 (2020-01-01)	202
11.4	0.2.1 (2018-04-21)	202
11.5	0.2.0 (2017-09-25)	202
11.6	0.1.0 (2012-03-03)	202
12	Indices and tables	203
A	Glossary	205
B	Licence	215
B.1	OpenSource Licence	215
B.2	Other Licences	222
	Index	223

TotalDepth is an Open Source, cross platform, software collection that processes petrophysical data from the oil field such as wireline logs.

If you are new here then have a look at some [TotalDepth Example Outputs](#). For more detail see [Introduction to TotalDepth](#).

The [TotalDepth project](#) is currently at **Beta**, development version 0.4.0, release version 0.4.0. For the licence see here [Licence](#).

INTRODUCTION TO TOTALDEPTH

TotalDepth is an Open Source, cross platform, software collection that can process petrophysical data from the oil field such as wireline logs, seismic data and so on.

Conventional, proprietary, software for petrophysical data tends to be expensive to licence, restrictive, slow to develop for and tied to expensive hardware. TotalDepth changes all of that.

TotalDepth is open and cross-platform, and produces results straight to the browser. TotalDepth supports such technologies such as HTML5, AJAX, Software as a Service (SaaS) and Cloud Computing.

TotalDepth is currently at **Alpha**, development version 0.4.0, release version 0.4.0. For the licence see [Licence](#).

TotalDepth is written in [Python](#) with performance critical code written in C or C++.

1.1 Petrophysical File Formats Supported by TotalDepth

Format	Support	Since	Notes.
LIS	Full	0.1.0	
LAS version 1.2	Full	0.1.0	
LAS version 2.0	Full	0.1.0	
LAS version 3.0	None		Little evidence that this is used by the industry.
WellLogML	None		No evidence that this is used by the industry.
RP66v1 “DLIS”	Full	0.3.0	Full.
RP66v2 “DLIS”	None		No evidence that this is used by the industry.
DAT	Full	0.4.0	An informal standard commonly used for mud logs.
Western Atlas BIT	Most	0.4.0	An informal standard not publicly documented.

TOTALDEPTH EXAMPLE OUTPUTS

This shows some examples of the kind of thing that TotalDepth can do.

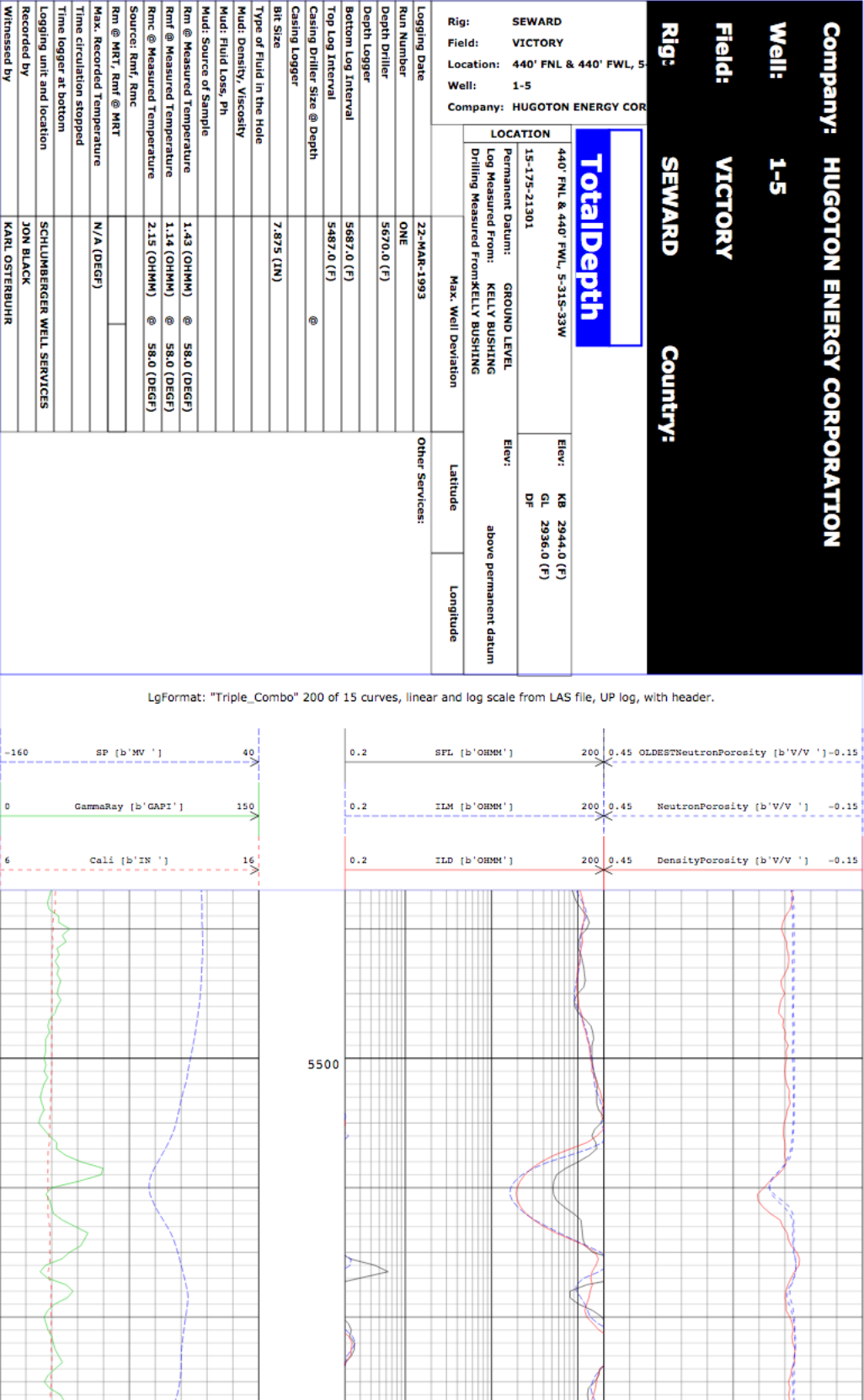
2.1 Wireline Plots

TotalDepth produced these time honoured plots from LIS and LAS wireline logs in SVG format that can be viewed in most browsers¹.

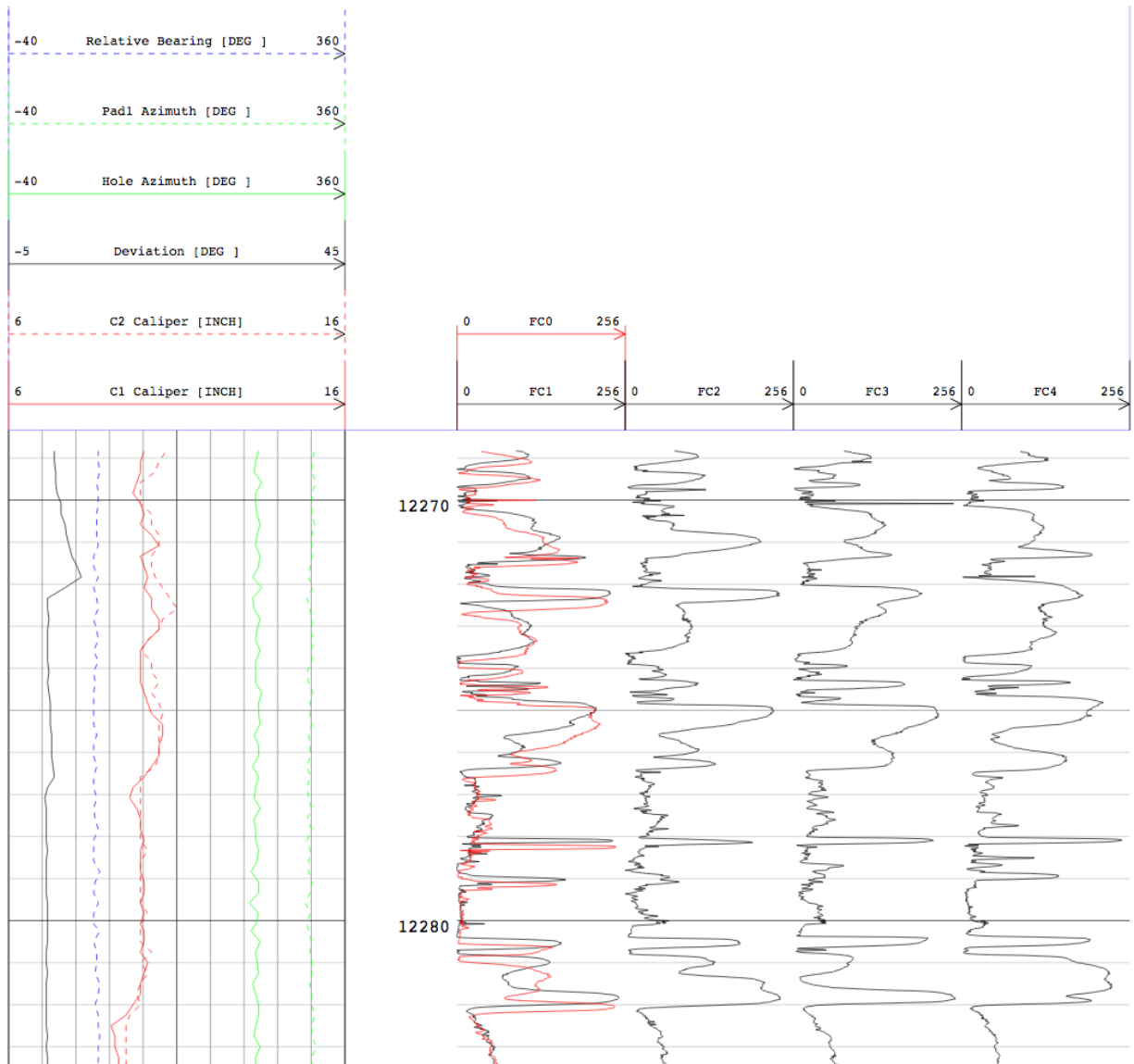
2.1.1 Plot Examples

Here is an example of a LAS file plotted with the Triple Combo plot format including the API header:

¹ There is good SVG support among current browsers such as [Opera](#), [Chrome](#) and [Safari](#). You can find a comparison of browser support for SVG at [Wikipedia](#).



An example of a High Resolution Dipmeter plotted at 1:25 scale:



2.1.2 Plotting from LIS

Some examples of plots generated from LIS79 files:

- A [collection](#) of 9 LIS files where TotalDepth used their internal plot specifications to generate 22 separate plots.
- A [High Resolution Dipmeter](#) plot on a scale of 1:25 with an API header. Fast channel FC0 (red) overlaid on FC1.

Note: These may not be available in the PDF version of the documentation.

2.1.3 Plotting From LAS

This shows off some examples of plots generated from the Canadian Well Logging Society's LAS formatted files².

Single LAS Plot examples

This shows plots of a single LAS file that has 200 feet of 15 curves. TotalDepth can plot this with, linear and log scales and with an API header:

- Plotted with the [Resistivity 3Track Logrithmic](#) format.
- The same data plotted with the [Triple Combo](#) format.

The original LAS file is [here](#).

Note: These may not be available in the PDF version of the documentation.

A Collection of LAS Plots

Here is a [directory of six LAS files](#) that was used to make 31 individual plots complete with an index that summarises them. For each LAS file the plotting program automatically choose from 29 plot formats the formats that produce useful plots³.

Note: These may not be available in the PDF version of the documentation.

Making LAS Plots

The `PlotLogs.py` command line tool was used with the command:

```
$ python3 PlotLogs.py -A -j4 -r -X 4 Data/ Plot/
```

This searched for LAS files in directory `Data/` with the plots being written in directory `Plot/`.

The following options have been set:

- API headers on the top of each plot: `-A`
- Multiprocessing on with 4 simultaneous jobs: `-j4`
- Recursive search of input directory: `-r`
- Uses any available plot specifications from `LgFormat XML` files which result in 4 curves or more being plotted: `-X 4`

This took around six seconds to compute. More detail on the `PlotLogs.py` is here: [Plotting Well Logs with *tdplotlogs*](#)

² Thanks to the [University of Kansas](#) [kgs.ku.edu] for the original data. For these examples that data has been edited or truncated or both.

³ A *useful* plot format is one that can handle at least n curves where n is a number that is specified by the user. If the user specifies 4 then there will be at least 4 curves on each plot.

2.2 LIS Log HTML Summaries

The program `LisToHtml.py` takes LIS file(s) and generates an [LIS HTML summary](#) for each one.

2.2.1 How This HTML was Made

The `LisToHtml.py` command line tool was used with the command:

```
$ python3 LisToHtml.py -k -j4 -r Data/ HTML/
```

This searched for LAS files in directory `Data/` with the files being written to directory `HTML/`.

The following options have been set:

- Keep going as far as possible: `-k`
- Multiprocessing on with 4 simultaneous jobs: `-j4`
- Recursive search of input directory: `-r`

More detail on the `LisToHtml.py` is here: [Summarise LIS Files in HTML with `tdlisthtml`](#)

2.3 RP66V1 Log HTML Summaries

The program `ScanHtml.py` takes RP66V1 file(s) and generates an [RP66V1 HTML summary](#) for each one.

Note: These may not be available in the PDF version of the documentation.

INSTALLATION

3.1 Stable release

To install TotalDepth, run this command in your terminal:

```
$ pip install TotalDepth
```

This is the preferred method to install TotalDepth, as it will always install the most recent stable release.

If you don't have `pip` installed, this [Python installation guide](#) can guide you through the process.

3.2 From sources

If you are using a virtual environment in your `<PYTHONVENV>`, say `~/pyvenvs`:

```
$ python3 -m venv <PYTHONVENV>/TotalDepth
$ . <PYTHONVENV>/TotalDepth/bin/activate
(TotalDepth) $
```

Or if you have a Conda environment:

```
$ conda create --name TotalDepth python=3.6 pip
$ source activate TotalDepth
```

Install the dependencies, `numpy` and `Cython`:

If you are using a virtual environment:

```
(TotalDepth) $ pip install numpy
(TotalDepth) $ pip install Cython
```

Or if you have a Conda environment:

```
(TotalDepth) $ conda install numpy
(TotalDepth) $ conda install Cython
```

The sources for TotalDepth can be downloaded from the [Github repo](#).

You can either clone the public repository:

```
(TotalDepth) $ git clone git://github.com/paulross/TotalDepth.git
```

Or download the [tarball](#):

```
(TotalDepth) $ curl -OL https://github.com/paulross/TotalDepth/tarball/master
```

Once you have a copy of the source, you can install it with:

```
(TotalDepth) $ cd TotalDepth
(TotalDepth) $ python setup.py install
```

Install the test dependencies and run TotalDepth's tests:

```
(TotalDepth) $ pip install pytest
(TotalDepth) $ pip install pytest-runner
(TotalDepth) $ python setup.py test
```

3.3 Developing with TotalDepth

If you are developing with TotalDepth you need the test coverage and documentation tools.

3.3.1 Test Coverage

Install `pytest-cov`:

```
(TotalDepth) $ pip install pytest-cov
```

The most meaningful invocation that eliminates the top level tools is:

```
(TotalDepth) $ pytest --cov=TotalDepth.LAS.core --cov=TotalDepth.LIS.core --
↳ cov=TotalDepth.RP66.core --cov=TotalDepth.util --cov-report html tests/
```

3.3.2 Documentation

To build the documentation you need to:

```
(TotalDepth) $ pip install Sphinx
(TotalDepth) $ cd docs
(TotalDepth) $ make html
```

3.3.3 System Testing

See *Testing the Plot Package* for comprehensive testing of your installation to see if LIS/LAS files can be written, read and plotted. This pretty much executes all TotalDepth code.

3.3.4 Unit Testing

See *Unit Tests* for more information about testing and unit tests.

HOWTOS: WHAT TOTALDEPTH CAN DO FOR YOU

This describes how TotalDepth can help you with various tasks when dealing with petrophysical data.

4.1 Working with Archives of Mixed Data

4.1.1 I have an archive of files and I want to know what is in there.

Sometimes you are given an archive of data and would like to know what file formats, the file sizes and so on. There is a command line tool `tdarchive` [References `TotalDepth.util.archive`] that can give you a summary of the files there, their binary file types and their sizes.

Read more here *Analysing a Directory of Files with `tdarchive`*.

4.1.2 I have an archive and I want copy specific file types

There is a command line tool `tdcopybinfiles` [References `TotalDepth.util.CopyBinFiles`] that can copy specific file types from one directory to another. It can also recursively deflate archives such as ZIP files.

Read more here *Copying a Directory of Files with `tdcopybinfiles`*.

4.1.3 I have an archive and I want report or remove duplicate files

There is a command line tool `tdremovedupefiles` [References `TotalDepth.util.RemoveDupeFiles`] that can find duplicate files based on their checksum and, optionally, remove the duplicates.

Read more here *Removing Duplicate Files with `tdremovedupefiles`*.

4.2 Working with Western Atlas BIT Data

4.2.1 I have an archive of data and I'd like a summary

There is a command line tool `tdbitread` (a wrapper around `TotalDepth.BIT.ReadBIT`) which can generate a summary of a body of BIT files. There is a tutorial here: *Summarise BIT Files with `tdbitread`*

4.2.2 I'd like to convert BIT files to LAS format files

There is a tutorial here : [Converting BIT Files to LAS Files with tdbittolas](#).

4.2.3 Getting the Frame Data as a numpy Array

TotalDepth's BIT parser represents the channel data as Numpy arrays. There is a tutorial here on writing code that allows you to access the Numpy channel data directly: [Reading a BIT File Log Data](#)

4.2.4 I have some troublesome BIT files

This is a highly specialised area. Feel free to contact the author for advice.

4.3 Working with LAS Data

4.3.1 I have an archive of data and I'd like a summary in HTML

There is a command line tool *tdlastohtml* (a wrapper around `TotalDepth.LAS.LASToHtml`) which can generate an HTML summary of a body of LAS files. There is a tutorial here: [Summarise LAS Files in HTML with tdlastohtml](#)

4.3.2 I have an archive of LAS data and I want to plot it

There is a generalised well log plotting command line tool *tdplotlogs* that supports LAS described here: [Plotting Well Logs with tdplotlogs](#) (references `TotalDepth.PlotLogs`).

LAS files do not contain an internal plot specification. For files needing an external plot specification there is some background information on plotting in a technical note here [Wireline Files With External Plot Data](#).

4.3.3 Getting the Frame Data as a numpy Array

TotalDepth's LAS represents the channel data as Numpy arrays. There is a tutorial here on writing code that allows you to access the Numpy channel data directly: [Reading a LAS File Log Data](#)

4.3.4 I have some troublesome LAS files

There are several problem areas for LAS files:

- The LAS specification is fairly weak and provides a lot of uncertainty. So LAS files from some producers are not readable by some other LAS consumers.
- LAS is a 'human readable' format, unfortunately that means it is a human writable format as well. This often means that LAS files can be mangled by well meaning, but mistaken intervention.
- Some LAS file archives have serious errors such as swapping value and description fields. These are not easily fixable by a rule based system.

The advantage, of course, with LAS files is that they can be hacked around with a simple text editor at will. This will often fix small local problems.

Feel free to contact the author for advice.

4.4 Working with LIS Data

4.4.1 I have an archive of data and I'd like a summary in HTML

There is a command line tool *tdlistohtml* (a wrapper around `TotalDepth.LIS.LisToHtml`) which can generate an HTML summary of a body of LIS files. There is a tutorial here: *Summarise LIS Files in HTML with tdlistohtml*

4.4.2 I have an archive of LIS data and I want to plot it

There is a generalised well log plotting command line tool *tdplotlogs* that supports LIS and is described here: *Plotting Well Logs with tdplotlogs* (references `TotalDepth.PlotLogs`).

LIS files may or may not contain an internal plot specification, if so *tdplotlogs* can take advantage of that, if not then external plot specifications can be used. For files needing an external plot specification there is some background information on plotting in a technical note here *Wireline Files With External Plot Data*.

4.4.3 I'd like to convert LIS files to LAS format files

There is a tutorial here : *Converting LIS Files to LAS Files with tdlistolas*.

4.4.4 Getting the Frame Data as a numpy Array

TotalDepth's LIS represents the channel data as Numpy arrays. There is a tutorial here on writing code that allows you to access the Numpy channel data directly: *Reading a LIS File Log Data*

4.4.5 I have some troublesome LIS files

This is a highly specialised area. Feel free to contact the author for advice.

4.5 Working with RP66V1 Data

TotalDepth version 0.3.0 (you are viewing 0.4.0) onwards provides a RP66V1 standard compliant implementation. RP66V1 file handling is work-in-progress.

4.5.1 I have an archive of RP66V1 data and I'd like a summary in HTML

TotalDepth's *tdrp66v1scanhtml* command line tool can do this, it is a wrapper around `TotalDepth.RP66V1.ScanHTML` There is a tutorial here: *Creating HTML Pages from RP66V1 Files with tdrp66v1scanhtml*.

Here is an example of the HTML summary of a [single RP66V1 file](#) .

4.5.2 I'd like to create some well plots from RP66V1 data

Unlike LIS and like LAS, RP66V1 files do not specify a plot format. Some producers include some producer specific information in private EFLRs. TotalDepth version 0.5.0 will provide a simpler, universal, way of specifying plot formats in SVG from LAS, LIS and RP66V1 data.

4.5.3 I'd like to convert RP66V1 files to LAS format files

There is a tutorial here : [Converting RP66V1 Files to LAS Files with tdrp66v1tolas](#).

4.5.4 Getting the Frame Data as a numpy Array

There is a tutorial here on writing code that allows you to access the numpy channel data directly: [Reading the Frame Data and Accessing the numpy Arrays](#). There is some example code in `example_data/RP66V1/demo_read.py`

4.5.5 I have some troublesome RP66V1 files

One problem noticed in RP66V1 data from the wild is that it is often polluted with gratuitous TIF markers which makes the file unreadable by a RP66V1 standard compliant implementation. TotalDepth's `tddetif` command line tool can remove these TIF markers and make the files readable. There is a tutorial here: [Removing TIF Markers From Files with tddetif](#)

Other problems require special skills, feel free to contact the author for advice.

4.6 Converting Files to LAS

4.6.1 Converting LIS Files to LAS

There is a command line tool `tdlistolas` that can convert LIS files to LAS 2.0 files. There is a tutorial here: [Converting LIS Files to LAS Files with tdlistolas](#). This command line tool is a wrapper round the ToLAS module, the reference documentation is here: `TotalDepth.LIS.ToLAS`.

4.6.2 Converting RP66V1 Files to LAS

There is a command line tool `tdrp66v1tolas` that can convert RP66V1 files to LAS 2.0 files. There is a tutorial here: [Converting RP66V1 Files to LAS Files with tdrp66v1tolas](#). This command line tool is a wrapper round the ToLAS module, the reference documentation is here: `TotalDepth.RP66V1.ToLAS`.

There is a technical note about the performance of this conversion here [Converting RP66V1 to LAS](#).

COMMAND LINE TOOLS

TotalDepth provides a number of tools run from the command line that can analyse and visualise petrophysical data. All the command line tools start with `td`.

5.1 TotalDepth Command Line Tools

This describes the command line tools that are available for processing any TotalDepth file.

The tools are located in TotalDepth/

Table 1: LIS Command Line Tools

Tool Name	Description
<code>tdarchive</code>	Scans a directory of files and provides an analysis by file type. Link
<code>tdcopybinfiles</code>	Selects files of a particular type and copies them to a directory. This can also expand compressed archives. Link
<code>tdremovedupefiles</code>	Removes duplicate files based on their checksum. Link
<code>tddetif</code>	Removes TIF markers from a file. Link
<code>tdplotlogs</code>	Plots logs from LIS and LAS data. Link

5.1.1 Analysing a Directory of Files with `tdarchive`

Scans a directory of files and provides an analysis by file type.

Usage

Usage:

```
usage: tdarchive [-h] [--file-type FILE_TYPE] [-b BYTES] [-r]
                [--expand-and-delete] [--histogram] [-n] [-o] [-l LOG_LEVEL]
                [-v]
                path [path_out]
```

Arguments

1. The path to the input directory
2. [Optional] Path to the output directory to write the files to. The results are undefined if path_out conflicts with path_in.

Options

Option	Description
-h, --help	Show this help message and exit.
--file-type	Types of files to analyse, this option is additive so can be used multiple times. Supported (and default) file types are: ASCII, LAS1.2, LAS2.0, LAS3.0, LIS, LISt, LISt, PDF, PS, RP66V1, RP66V1t, RP66V1tr, RP66V2, SEGY, TIFF, XML, ZIP
-b, --bytes=	The number of initial bytes of the file to show [default: 0].
-r, --recursive	Process input recursively. [default: False]
--expand-and-delete	Expand and delete archive files, implies --recurse. [default: False]
--histogram	Plot a histogram of file sizes. [default: False]
-n, --nervous	Nervous mode, does not do anything but report.
-o, --over-write	Overwrite existing files, otherwise warns.
-l, --log-level=	Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 30]
-v, --verbose	Increase verbosity, additive [default: 0]

Examples

Here is an example of scanning the example_data directory (the output is filleted for clarity):

```
$ tdarchive example_data/ -r --histogram
CMD: tdarchive example_data/ -r --histogram
Analysing archive.
Common prefix:
    108,707 .SVG      XML      example_data/LIS/Plot/DILLSON-1_WELL_LOGS_FILE-013.
↳ LIS_0001_HDT.svg
    667 .HTML        XML      example_data/LIS/Plot/index.html
    163,169 .SVG      XML      example_data/LIS/Plot/DILLSON-1_WELL_LOGS_FILE-037.
↳ LIS_0001_1.svg
    690 .CSS         ASCII    example_data/LIS/Plot/index.css
...
    184,084 .LIS      LIS      example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-037.LIS
    96,376 .LIS      LIS      example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-013.LIS
    98,508 .LIS      LIS      example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.LIS
    10,306 .PY       ASCII    example_data/RP66V1/demo_read.py
    462,795 .HTML     XML      example_data/RP66V1/HTML/206_05a-_3_DWL_DWL_WIRE_
↳ 258276498.DLIS.HTML.html
```

(continues on next page)

(continued from previous page)

```

      3,098 .HTML      XML      example_data/RP66V1/HTML/index.html
...
      1,018,327 .PKL      example_data/RP66V1/pickle/206_05a-_3_DWL_DWL_WIRE_
↳258276498.pkl
      276,045 .PKL      example_data/RP66V1/pickle/206_05a-_3_DWL_DWL_WIRE_
↳258276498.DLIS.pkl
      520 .PKL      example_data/RP66V1/pickle/MINIMAL_FILE.dlis.pkl
      53,137 .PKL      example_data/RP66V1/pickle/BASIC_FILE.dlis.pkl
      949 .PKL      example_data/RP66V1/pickle/BASIC_FILE_WITH_TWO_
↳VISIBLE_RECORDS_NO_IFLRS.dlis.pkl
      7,731 .LAS      LAS2.0 example_data/RP66V1/LAS/BASIC_FILE_WITH_TWO_VISIBLE_
↳RECORDS_NO_IFLRS_0_.las
      1,621,374 .LAS      LAS2.0 example_data/RP66V1/LAS/206_05a-_3_DWL_DWL_WIRE_
↳258276498_0_800T.las
...
      44,916 .DLIS      RP66V1 example_data/RP66V1/data/BASIC_FILE.dlis
      540,372 .DLIS      RP66V1 example_data/RP66V1/data/206_05a-_3_DWL_DWL_WIRE_
↳258276498.DLIS
      716 .DLIS      RP66V1 example_data/RP66V1/data/MINIMAL_FILE.dlis
      8,826 .DLIS      RP66V1 example_data/RP66V1/data/BASIC_FILE_WITH_TWO_VISIBLE_
↳RECORDS_NO_IFLRS.dlis
Total number of files 75, total bytes 12,981,766
File extensions:
.CSS :      5
.DLIS :      4
.HTML :     13
.LAS :      6
.LIS :      3
.PKL :      5
.PY :      1
.PYC :      1
.SVG :     36
.XML :      1
Binary file types:
Binary type: ""
Extensions: .PKL, .PYC
Count: 6 [8.000%]
Bytes: 1,354,629 [10.435%] from 520 to 1,018,327
>=2**9 [ 2] |_
↳+++++
>=2**10 [ 0] |
>=2**11 [ 0] |
>=2**12 [ 1] | +++++
>=2**13 [ 0] |
>=2**14 [ 0] |
>=2**15 [ 1] | +++++
>=2**16 [ 0] |
>=2**17 [ 0] |
>=2**18 [ 1] | +++++
>=2**19 [ 1] | +++++
Binary type: "ASCII"

```

(continues on next page)

(continued from previous page)

```

Extensions: .CSS, .PY
  Count: 6 [8.000%]
  Bytes: 15,232 [0.117%] from 690 to 10,306
>=2**9 [ 4] | _
↳ ++++++
>=2**10 [ 0] |
>=2**11 [ 1] | ++++++
>=2**12 [ 0] |
>=2**13 [ 1] | ++++++

Binary type: "LAS2.0"
Extensions: .LAS
  Count: 6 [8.000%]
  Bytes: 1,785,653 [13.755%] from 1,279 to 1,621,374
>=2**10 [ 1] | _
↳ ++++++
>=2**11 [ 1] | _
↳ ++++++
>=2**12 [ 1] | _
↳ ++++++
>=2**13 [ 0] |
>=2**14 [ 0] |
>=2**15 [ 1] | _
↳ ++++++
>=2**16 [ 1] | _
↳ ++++++
>=2**17 [ 0] |
>=2**18 [ 0] |
>=2**19 [ 0] |
>=2**20 [ 1] | _
↳ ++++++

Binary type: "LIS"
Extensions: .LIS
  Count: 3 [4.000%]
  Bytes: 378,968 [2.919%] from 96,376 to 184,084
>=2**16 [ 2] | _
↳ ++++++
>=2**17 [ 1] | ++++++

Binary type: "RP66V1"
Extensions: .DLIS
  Count: 4 [5.333%]
  Bytes: 594,830 [4.582%] from 716 to 540,372
>=2**9 [ 1] | _
↳ ++++++
>=2**10 [ 0] |
>=2**11 [ 0] |
>=2**12 [ 0] |
>=2**13 [ 1] | _
↳ ++++++
>=2**14 [ 0] |

```

(continues on next page)

(continued from previous page)

```

>=2**15 [ 1] |
↳ ++++++
>=2**16 [ 0] |
>=2**17 [ 0] |
>=2**18 [ 0] |
>=2**19 [ 1] |
↳ ++++++

Binary type: "XML"
Extensions: .HTML, .SVG, .XML
Count: 50 [66.667%]
Bytes: 8,852,454 [68.191%] from 667 to 961,863
>=2**9 [ 1] | +++++
>=2**10 [ 1] | +++++
>=2**11 [ 2] | ++++++
>=2**12 [ 2] | ++++++
>=2**13 [ 0] |
>=2**14 [ 0] |
>=2**15 [ 3] | ++++++
>=2**16 [ 16] |
↳ ++++++
>=2**17 [ 18] |
↳ ++++++
>=2**18 [ 5] | ++++++
>=2**19 [ 2] | ++++++

Execution time: 0.067 (s)
Files: 75 rate 1,113.2 (files/s)
Bytes: 1,350 rate 20,038.5 (bytes/s)

```

5.1.2 Copying a Directory of Files with `tdcopybinfiles`

Scans a directory of files and can copy particular file type to another directory.

Usage

Usage:

```

usage: TotalDepth.RP66V1.util.CopyBinFiles.main
       [-h] [--file-types FILE_TYPES] [-m] [-n] [-l LOG_LEVEL]
       path_in path_out

```

Arguments

1. The path to the input directory
2. Path to the output directory to write the files to. The results are undefined if path_out conflicts with path_in.

Options

Option	Description
-h, --help	Show this help message and exit.
--file-types	Types of files to copy, this option is additive so can be used multiple times. Supported (and default) file types are: ASCII, LAS1.2, LAS2.0, LAS3.0, LIS, LISt, LIStR, PDF, PS, RP66V1, RP66V1t, RP66V1tr, RP66V2, SEGY, TIFF, XML, ZIP Use '?' or '??' to see what file types are available
-m, --move	Move rather than copy, Irrelevant for files in ZIP archives which are always copied. [default: False]
-n, --nervous	Nervous mode, does not do anything but report.
-l, --log-level	Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 30]

Examples

To see what file types are supported use '?' and two dummy paths:

```
$ tdcopybinfiles --file-types=? ' ' ' '
Cmd: tdcopybinfiles --file-types=?
Binary file types supported: ASCII, LAS1.2, LAS2.0, LAS3.0, LIS, LISt, LIStR, PDF, PS,
↳RP66V1, RP66V1t, RP66V1tr, RP66V2, SEGY, TIFF, XML, ZIP
Execution time =    0.000 (S) 0 kb/s
```

To get the file type description as well use '??' and two dummy paths:

```
$ tdcopybinfiles --file-types=?? ' ' ' '
Cmd: tdcopybinfiles --file-types=??
Binary file types supported:
ASCII      - American Standard Code for Information Interchange
LAS1.2     - Canadian Well Logging Society Log ASCII Standard version 1.2
LAS2.0     - Canadian Well Logging Society Log ASCII Standard version 2.0
LAS3.0     - Canadian Well Logging Society Log ASCII Standard version 3.0
LIS        - Schlumberger LIS-79 well logging format
LISt       - Schlumberger LIS-79 well logging format with TIF markers
LIStR      - Schlumberger LIS-79 well logging format with reversed TIF markers
PDF        - Portable Document Format
PS         - Postscript
RP66V1     - American Petroleum Institute Recommended Practice 66 version 1
RP66V1t    - American Petroleum Institute Recommended Practice 66 version 1 with TIF
↳markers
RP66V1tr   - American Petroleum Institute Recommended Practice 66 version 1 with reversed
↳TIF markers
```

(continues on next page)

(continued from previous page)

```

RP66V2  - American Petroleum Institute Recommended Practice 66 version 2
SEGY    - Society of Exploration Geophysicists seismic format Y
TIFF    - Tagged Image File Format
XML     - eXtensible Markup Language
ZIP     - ZIP Compressed Archive
Execution time =    0.000 (S) 0 kb/s

```

5.1.3 Removing Duplicate Files with `tdremovedupefiles`

Scans a directory of files and identifies duplicate files by their checksum. It is **strongly recommended** to use `-n` (`--nervous`) first and look at the results before running this without `-n` which is potentially destructive.

Usage

Usage:

```

usage: TotalDepth.RP66V1.util.RemoveDupeFiles.main [-h] [--version] [-k] [-v]
                                                    [-r] [-l LOG_LEVEL] [-n]
                                                    path_in

```

Arguments

1. The path to the directory.

Options

Option	Description
<code>-h, --help</code>	Show this help message and exit.
<code>-r, --recursive</code>	Process input recursively. [default: False]
<code>-n, --nervous</code>	Nervous mode, does not do anything but report [default: False].
<code>-l, --log-level=</code>	Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 20]
<code>-v, --verbose</code>	Increase verbosity, additive [default: 0]

5.1.4 Removing TIF Markers From Files with `tddetif`

Scans a directory of files and removes TIF markers. TIF markers are 12 bytes of data inserted in various places. They are not part of any standard and, except for one case, provide no value. This tool rewrites the input file without TIF markers if they are found.

Usage

Usage:

```
usage: tddetif [-h] [-r] [-n]
              [-l LOG_LEVEL] [-v] [-o]
              path_in [path_out]
```

Arguments

1. The path to the input directory
2. Path to the output directory to write the files to. The results are undefined if `path_out` conflicts with `path_in`.

Options

Option	Description
<code>-h, --help</code>	Show this help message and exit.
<code>-r, --recurse</code>	Process input recursively. [default: False]
<code>-n, --nervous</code>	Nervous mode, does not do anything but report [default: False].
<code>-l, --log-level=</code>	Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 30]
<code>-v, --verbose</code>	Increase verbosity, additive [default: 0]
<code>-o, --over-write</code>	Overwrite existing files if found, otherwise warns of existing target file.

Examples

todo:

```
Put examples here.
```

These command line tools plot wireline data.

5.1.5 Plotting Well Logs with `tdplotlogs`

Produces SVG plots from LIS and LAS files.

Usage

Usage:

```
usage: tdplotlogs [-h] [--version] [-j JOBS] [-k] [-l LOGLEVEL] [-g] [-r]
                [-A] [-x LGFORMAT] [-X LGFORMAT_MIN] [-s SCALE]
                in out
```

Arguments

These are required arguments unless `-h` or `--version` options are specified (in which case no processing is done):

1. The path to the input LAS or LIS file or directory thereof.
2. The path to the output SVG file or directory, any directories will be created as necessary.

Options

Option	Description
<code>--version</code>	Show program's version number and exit
<code>-h, --help</code>	Show this help message and exit.
<code>-j JOBS,</code> <code>--jobs=JOBS</code>	Max processes when multiprocessing. Zero uses number of native CPUs [8]. -1 disables multiprocessing. [default: -1]
<code>-k,</code> <code>--keep-going</code>	Keep going as far as sensible. [default: False]
<code>-l</code> <code>LOGLEVEL,</code> <code>--loglevel=LOGLEVEL</code>	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]
<code>-g, --glob</code>	File pattern match. [default none]
<code>-r,</code> <code>--recursive</code>	Process input recursively. [default: False]
<code>-A, --API</code>	Put an API header on each plot. [default: False]
<code>-x</code> <code>LGFORMAT,</code> <code>--xml</code> <code>LGFORMAT</code>	Use XML LgFormat UniqueId to use for plotting (additive). Use <code>-x?</code> to see what LgFormats (UniqueID+Description) are available. Use <code>-x??</code> to see what curves each format can plot. See also <code>-X</code> . This is additive so can used multiple times to get multiple plots from the same data.
<code>-X</code> <code>LGFORMAT_MIN,</code> <code>--XML</code> <code>LGFORMAT_MIN</code>	Use all available LgFormat XML plots that use LGFORMAT_MIN or more outputs. If <code>-x</code> option present limited by those LgFormats [default: 0]
<code>-s SCALE,</code> <code>--scale</code> <code>SCALE</code>	Scale of X axis to use (an integer). This overrides the scale(s) specified in the LgFormat file or FILM table. [default: 0].

Examples

LgFormat XML

Using `-x?` to see what formats are available:

```
$ python3 tdplotlogs -x? spam eggs
```

The output is something like:

```
Cmd: tdplotlogs -x? spam eggs
XML LgFormats available: [29]
UniqueId                Description
-----
ADN_Image_Format         : ADN Image Log
Azimuthal_Density_3Track.xml : Azimuthal Density 3Track
Azimuthal_Resistivity_3Track.xml : Azimuthal Resistivity 3Track
Blank_3Track_Depth       : Blank 3Track
Blank_3Track_Time.xml    : Blank 3Track Time
FMI_IMAGE_ALIGNED        : FMI Image Aligned
FMI_IMAGE_PROCESSED      : FMI Image Processed
Formation_Test           : Formation Test Time
HDT                      : High Definition Dipmeter
Micro_Resistivity_3Track.xml : Micro Resistivity 3 Track Format
Natural_GR_Spectrometry_3Track.xml : Natural GR Spectrometry 3Track
OBMI_IMAGE_EQUAL         : OBMI Image Equalized
Porosity_GR_3Track       : Standard Porosity Curves
Pulsed_Neutron_3Track.xml : Pulsed Neutron 3Track
Pulsed_Neutron_Time.xml  : Pulsed Neutron Time
RAB_Image_Format_Deep    : Resistivity At the Bit Image
RAB_Image_Format_Medium  : Resistivity At the Bit Image
RAB_Image_Format_Shallow : Resistivity At the Bit Image
RAB_Std_Format           : Resistivity At the Bit
Resistivity_3Track_Correlation.xml : Resistivity Linear Correlation Format
Resistivity_3Track_Logrithmic.xml : Logrithmic Resistivity 3Track
Resistivity_Investigation_Image.xml : AIT Radial Investigation Image
Sonic_3Track.xml         : Sonic DT Porosity 3 Track
Sonic_PWF4              : SONIC Packed Waveform 4
Sonic_SPR1_VDL           : SONIC Receiver Array Lower Dipole VDL
Sonic_SPR2_VDL           : SONIC Receiver Array Upper Dipole VDL
Sonic_SPR3_VDL           : SONIC Receiver Array Stonely VDL
Sonic_SPR4_VDL           : SONIC Receiver Array P and S VDL
Triple_Combo            : Resistivity Density Neutron GR 3Track Format
```

The first column is the UniqueID to be used in identifying plots for the `-x` option.

Using `-x??` to see what formats and what curves would be plotted by each plot specification:

```
$ python3 tdplotlogs -x?? a b
```

The output is something like:

```
Cmd: tdplotlogs -x?? a b
XML LgFormats available: [29]
```

(continues on next page)

(continued from previous page)

UniqueId	Description
ADN_Image_Format	: ADN Image Log DRHB, GR , GR_RAB, ROBB, ROP5, TNPH
Azimuthal_Density_3Track.xml	: Azimuthal Density 3Track BS , DCAL, DRHB, DRHL, DRHO, DRHR, DRHU, DTAB, HDIA, PEB , PEF , PEL PER , PEU , RHOB, ROBB, ROBL, ROBR, ROBU, ROP5, RPM , SCN2, SOAB, SOAL SOAR, SOAU, SONB, SOXB, VDIA
Azimuthal_Resistivity_3Track.xml	: Azimuthal Resistivity 3Track AAI , BS , C1 , C2 , CALI, GR , GRDN_RAB, GRLT_RAB, GRRT_RAB, GRUP_RAB, PCAL, ↪RDBD RDBL, RDBR, RDBU, RLA0, RLA1, RLA2, RLA3, RLA4, RLA5, RMBD, RMBL, RMBR RMBU, ROP5, RPM , RSBD, RSBL, RSBR, RSBU, SP , TENS
Blank_3Track_Depth	: Blank 3Track
Blank_3Track_Time.xml	: Blank 3Track Time
FMI_IMAGE_ALIGNED	: FMI Image Aligned C1 , C2 , GR , HAZIM, P1AZ, SP , TENS
FMI_IMAGE_PROCESSED	: FMI Image Processed C1 , C2 , GR , HAZIM, P1AZ, SP , TENS
Formation_Test	: Formation Test Time B1TR, BFR1, BQP1, BQP1, BQP1, BQP1, BSG1, POHP
HDT	: High Definition Dipmeter C1 , C2 , DEVI, FC0 , FC1 , FC2 , FC3 , FC4 , GR , HAZI, P1AZ, RB
Micro_Resistivity_3Track.xml	: Micro Resistivity 3 Track Format BMIN, BMNO, BS , CALI, GR , HCAL, HMIN, HMNO, MINV, MLL , MNOR, MSFL PROX, RX0 , SP , TENS
Natural_GR_Spectrometry_3Track.xml	: Natural GR Spectrometry 3Track CGR , PCAL, POTA, ROP5, SGR , SIGM, TENS, THOR, URAN
OBMI_IMAGE_EQUAL	: OBMI Image Equalized C1 , C1_OBMT, C2 , C2_OBMT, GR , HAZIM, OBRA3, OBRB3, OBRC3, OBRD3, P1AZ, P1NO_ ↪OBMT TENS
Porosity_GR_3Track	: Standard Porosity Curves APDC, APLC, APSC, BS , C1 , C2 , CALI, CALI_CDN, CMFF, CMRP, DPHB, DPHI DPHZ, DPOR_CDN, DRHO, ENPH, GR , HCAL, NPHI, NPOR, PCAL, RHOB, RHOZ, ROP5 SNP , SP , SPHI, TENS, TNPB, TNPH, TNPH_CDN, TPHI
Pulsed_Neutron_3Track.xml	: Pulsed Neutron 3Track FBAC, GR , INFD, SIGM, TAU , TCAF, TENS, TPHI, TSCF, TSCN
Pulsed_Neutron_Time.xml	: Pulsed Neutron Time FBAC_SL, GR_SL, INFD_SL, SIGM_SL, TAU_SL, TCAF_SL, TENS_SL, TPHI_SL, TSCF_SL, TSCN_SL
RAB_Image_Format_Deep	: Resistivity At the Bit Image GR_RAB, RES_BD, RES_BM, RES_BS, RES_RING, ROP5
RAB_Image_Format_Medium	: Resistivity At the Bit Image GR_RAB, RES_BD, RES_BM, RES_BS, RES_RING, ROP5
RAB_Image_Format_Shallow	: Resistivity At the Bit Image GR_RAB, RES_BD, RES_BM, RES_BS, RES_RING, ROP5
RAB_Std_Format	: Resistivity At the Bit AAI , BDAV, BDM3, BMAV, BMM2, BSAV, BSM1, BTAB, CALI, DEVI, GR_RAB, HAZI OBIT, RBIT, RING, ROP5, RPM , RTAB
Resistivity_3Track_Correlation.xml	: Resistivity Linear Correlation Format AHT20, AHT60, AHT90, ATR , BS , CALI, CATR, CILD, CLLD, GR , HCAL, ILD ILM , LLD , LLS , MSFL, PCAL, PSR , RLA0, ROP5, RT , RX0 , SFL , SP

(continues on next page)

(continued from previous page)

```

TENS
Resistivity_3Track_Logrithmic.xml : Logrithmic Resistivity 3Track
A22H, A34H, AHF10, AHF20, AHF30, AHF60, AHF90, AH010, AH020, AH030, AH060, AH090
AHT10, AHT20, AHT30, AHT60, AHT90, ATR , BS , CALI, GR , HCAL, ILD , ILM
LLD , LLM , MSFL, P16H_RT, P28H_RT, P34H_RT, PCAL, PSR , RLA0, RLA1, RLA2, RLA3
RLA4, RLA5, ROP5, RX0 , SFL , SP , TENS
Resistivity_Investigation_Image.xml : AIT Radial Investigation Image
AHT10, AHT20, AHT30, AHT60, AHT90, BS , GR , HCAL, SP
Sonic_3Track.xml : Sonic DT Porosity 3 Track
BS , CALI, DT , DT0S, DT1R, DT2 , DT2R, DT4S, DTBC, DTCO, DTCU, DTL
DTLF, DTLN, DTR2, DTR5, DTRA, DTRS, DTSH, DTSM, DTST, DTTA, GR , HCAL
PCAL, ROP5, SP , SPHI, TENS
Sonic_PWF4 : SONIC Packed Waveform 4
CALI, DT1 , DT2 , DTCO, DTSM, DTST, GR , HCAL, TENS
Sonic_SPR1_VDL : SONIC Receiver Array Lower Dipole VDL
CALI, DT1 , DT1 , DT2 , DTCO, DTSM, DTST, GR , HCAL, TENS
Sonic_SPR2_VDL : SONIC Receiver Array Upper Dipole VDL
CALI, DT1 , DT2 , DT2 , DTCO, DTSM, DTST, GR , HCAL, TENS
Sonic_SPR3_VDL : SONIC Receiver Array Stonely VDL
CALI, DT1 , DT2 , DT3R, DTCO, DTSM, DTST, GR , HCAL, TENS
Sonic_SPR4_VDL : SONIC Receiver Array P and S VDL
CALI, DT1 , DT2 , DTCO, DTRP, DTRS, DTSM, DTST, GR , HCAL, TENS
Triple_Combo : Resistivity Density Neutron GR 3Track Format
AHT10, AHT20, AHT30, AHT60, AHT90, APDC, APLC, APSC, ATR , BS , C1 , C2
CALI, CMFF, CMRP, DPHB, DPHI, DPHZ, DPOR_CDN, DSOZ, ENPH, GR , HCAL, HMIN
HMNO, ILD , ILM , LLD , LLM , MSFL, NPHI, NPOR, PCAL, PEFZ, PSR , RLA0
RLA1, RLA2, RLA3, RLA4, RLA5, ROP5, RSOZ, RX0 , RXOZ, SFL , SNP , SP
SPHI, TENS, TNPB, TNPH, TNPH_CDN, TPHI

```

Plotting Logs

Here is an example of plotting LIS and LAS files in directory in/ with the plots in directory out/. The following options have been invoked:

- API headers on the top of each plot: -A
- Multiprocessing on with 4 simultaneous jobs: -j4
- Recursive search of input directory: -r
- Uses any available plot specifications from LgFormat XML files which result in 4 curves or more being plotted: -X 4

The command line is:

```
$ python3 tdplotlogs -A -j4 -r -X 4 in/ out/
```

First tdplotlogs echos the command:

```
Cmd: tdplotlogs -A -j4 -r -X 4 in/ out/
```

When complete tdplotlogs writes out a summary, first the number of files read (output is wrapped here with ‘\’ for clarity):

```
plotLogInfo PlotLogInfo <__main__.PlotLogInfo object at 0x101e0da90> \
  Files=23 \
  Bytes=10648531 \
  LogPasses=23 \
  Plots=8 \
  Curve points=229991
```

Then as summary of each plot in detail (output is wrapped here with '\ ' for clarity):

```
('in/1003578128.las', \
  0, \
  'Natural_GR_Spectrometry_3Track.xml', \
  IndexTableValue( \
    scale=100, \
    evFirst='800.5 (FEET)', \
    evLast='3019.5 (FEET)', \
    evInterval='2219.0 (FEET)', \
    curves='CGR_2, POTA, SGR_1, TENS_16, THOR, URAN', \
    numPoints=26213, \
    outPath='out//1003578128.las_0000_Natural_GR_Spectrometry_3Track.xml.svg' \
  )
)
('in/1003578128.las', \
  0, \
  'Porosity_GR_3Track', \
  IndexTableValue( \
    scale=100, \
    evFirst='800.5 (FEET)', \
    evLast='3019.5 (FEET)', \
    evInterval='2219.0 (FEET)', \
    curves='Cali, DRHO, DensityPorosity, GammaRay, NeutronPorosity, \
→OLDESTNeutronPorosity, OLDNeutronPorosity, RHOB, SP, SonicPorosity, Tension', \
    numPoints=46170, \
    outPath='out//1003578128.las_0000_Porosity_GR_3Track.svg' \
  )
)
... 8<----- Snip ----->8
('in/1006346987.las', \
  0, 'Sonic_3Track.xml', \
  IndexTableValue( \
    scale=100, \
    evFirst='4597.5 (FEET)', \
    evLast='5799.5 (FEET)', \
    evInterval='1202.0 (FEET)', \
    curves='Caliper, DT, DTL_DDBHC, GammaRay, SonicPorosity, TENSION', \
    numPoints=14430, \
    outPath='out//1006346987.las_0000_Sonic_3Track.xml.svg' \
  )
)
```

The fields in each tuple are:

- Input file name.
- LogPass number in the file. For example “Repeat Section” might be 0 and “Main Log” 1.
- LgFormat used for the plot (several plots may be generated from one LogPass).
- An IndexTableValue object (used to generate the index.html file) that has the following fields:
 - Plot scale as an integer.
 - First reading and units as an Engineering Value.
 - Last reading and units as an Engineering Value.
 - Log interval and units as an Engineering Value.
 - List of curve names plotted.
 - Total number of data points plotted.
 - The output file.

Finally the total number of curve feet plotted and the time it took:

```
Interval*curves: EngVal: 121020.000 (FEET)
CPU time =    0.043 (S)
Exec. time =   25.119 (S)
Bye, bye!
```

In this case (under Unix) the “CPU Time” is the cumulative amount of CPU time used. As we are using multiprocessing it is the CPU time of the parent process which is very small since it just invokes child processes. The Exec. time is the wall clock time between starting and finishing tdplotlogs.

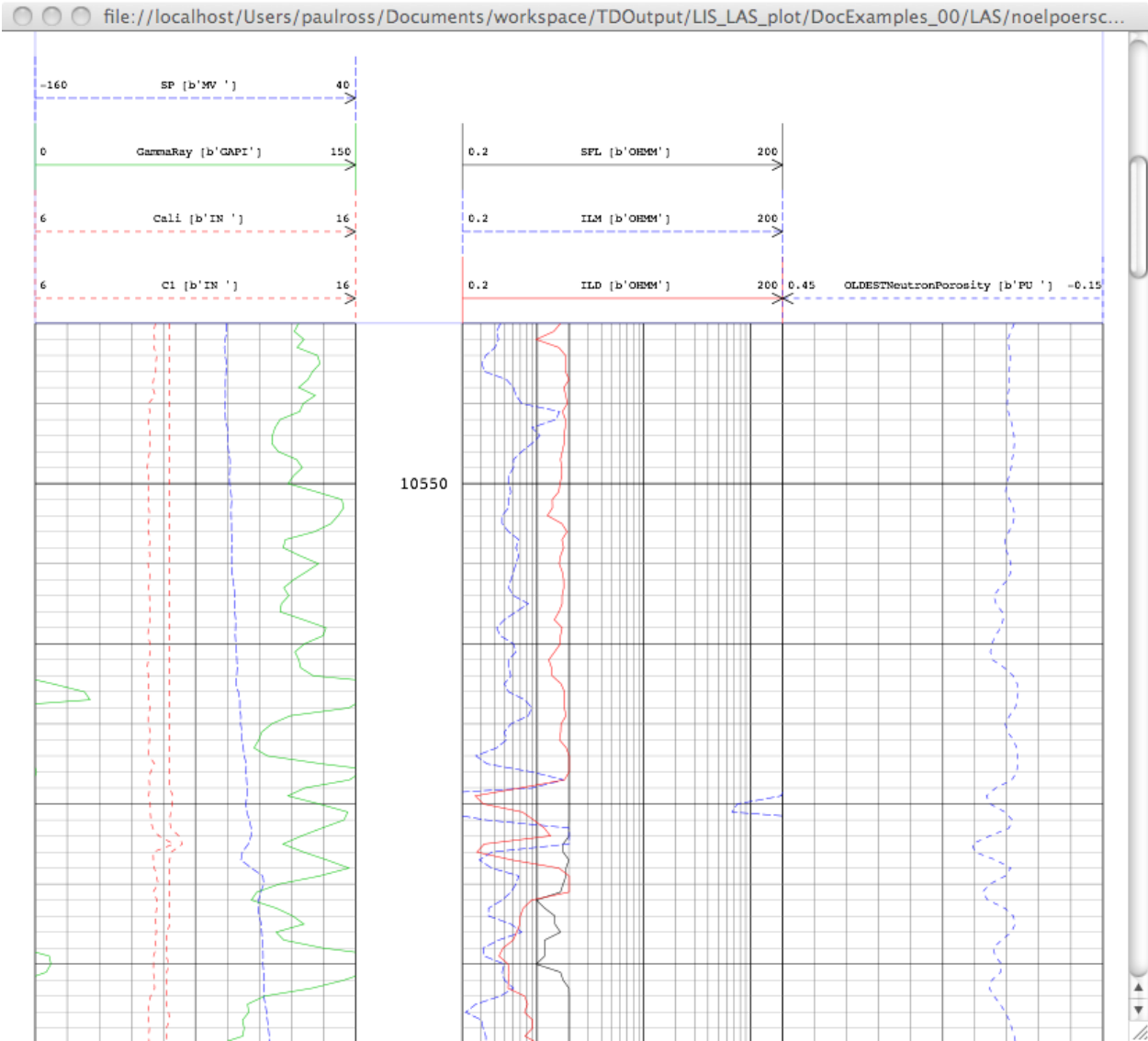
In the output directory will be an index.html file that has a table with the fields that duplicate those on the command line output. It looks like this:

LIS plots in SVG

PlotLogPasses: ../..TDTestData/LAS/uz_small/

Input	Pass	File	Scale	From	To	Interval	Curves	Points	Plot
1003578128.las	0	Natural_GR_Spectrometry_3Track.xml	100	800.5 (FEET)	3019.5 (FEET)	2219.0 (FEET)	CGR_2,POTA,SGR_1, TENS_16,THOR,URAN	28213	1003578128.las 0000 Natural_GR_Spectrometry_3Track.xml.svg
		Porosity_GR_3Track	100	800.5 (FEET)	3019.5 (FEET)	2219.0 (FEET)	Cal, DRHO, DensityPorosity, GammaRay, NeutronPorosity, OLDESTNeutronPorosity, OLDNeutronPorosity, RHOB, SP, SonicPorosity, Tension	46170	1003578128.las 0000 Porosity_GR_3Track.svg
		Resistivity_3Track_Correlation.xml	100	800.5 (FEET)	3019.5 (FEET)	2219.0 (FEET)	CALJ_8,CILD,GR_9,SP_10, TENS_8	22012	1003578128.las 0000 Resistivity_3Track_Correlation.xml.svg
		Resistivity_3Track_Logarithmic.xml	100	800.5 (FEET)	3019.5 (FEET)	2219.0 (FEET)	CALJ_8,GR_9,ILD,ILM,SFL, SP_10,TENS_8	30878	1003578128.las 0000 Resistivity_3Track_Logarithmic.xml.svg
		Sonic_3Track.xml	100	800.5 (FEET)	3019.5 (FEET)	2219.0 (FEET)	Caliper,DTLF,DDBHIC, DTLN,DDBHIC,GammaRay,SP, SonicPorosity,TENSION	24935	1003578128.las 0000 Sonic_3Track.xml.svg
		Triple_Combo	100	800.5 (FEET)	3019.5 (FEET)	2219.0 (FEET)	Cal, DensityPorosity, GammaRay,ILD,ILM, NeutronPorosity, OLDESTNeutronPorosity, OLDNeutronPorosity,SFL,SP, SonicPorosity,Tension	50755	1003578128.las 0000 Triple_Combo.svg
1003489986.las	0	Porosity_GR_3Track	100	4592.0 (FEET)	5808.0 (FEET)	1216.0 (FEET)	Cal, DRHO, DensityPorosity, GammaRay, OLDESTNeutronPorosity, RHOB	14598	1003489986.las 0000 Porosity_GR_3Track.svg
1003489987.las	0	Sonic_3Track.xml	100	4597.5 (FEET)	5799.5 (FEET)	1202.0 (FEET)	Caliper,DT,DTL,DDBHIC, GammaRay,SonicPorosity, TENSION	14430	1003489987.las 0000 Sonic_3Track.xml.svg

The links in the last column are to the SVG plots. Here is a screen shot of one:



Sample Plots

Here is an actual plot from a [LAS file](#) and there are many more examples here: [Wireline Plots](#).

5.2 BIT Command Line Tools

This describes the command line tools that are available for processing BIT files.

Table 2: BIT Command Line Tools

Tool Name	Description
tdbitread	Generates a summary of an archive of BIT file(s). Link
tdbittolas	Converts BIT file(s) to LAS file(s). Link

5.2.1 Summarise BIT Files with tdbitread

Generates a summary from input BIT file or directory.

Arguments

1. The path to the input BIT file or directory.

Options

Option	Description
-h, --help	Show this help message and exit.
--version	Show program's version number and exit
-k, --keep-going	Keep going as far as sensible. [default: False]
-v, --verbose	Verbose output, this outputs a representation of table data and DFSRs.
-r, --recurse	Process input recursively. [default: False]
-l LOGLEVEL, --loglevel=LOGLEVEL	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]
--summary	Summarise the Frame Data. [default False]

Examples

Command to process a directory of BIT:

```
$ tdbitread -r data/DresserAtlasBIT/special/
```

Output:

```
Cmd: src/TotalDepth/BIT/ReadBIT.py data/DresserAtlasBIT/special/ --summary -r
2021-02-04 13:19:42,290 - ReadBIT.py          - 631 - 8000 - (MainThread) - INFO      -
↳Processing: data/DresserAtlasBIT/special/29_10-_3/DWL_FILE/29_10-_3_dwl_DWL_WIRE_
↳1646632.bit
2021-02-04 13:19:42,762 - ReadBIT.py          - 600 - 8000 - (MainThread) - WARNING   -
↳The block length 276 does not have equal data for the channels 10. at tell=922504.
↳Ignoring rest of file.
2021-02-04 13:19:42,946 - ReadBIT.py          - 631 - 8000 - (MainThread) - INFO      -
↳Processing: data/DresserAtlasBIT/special/29_10-_3/DWL_FILE/29_10-_3_dwl_DWL_WIRE_
↳1646636.bit
2021-02-04 13:19:43,175 - ReadBIT.py          - 600 - 8000 - (MainThread) - WARNING   -
↳The block length 276 does not have equal data for the channels 10. at tell=441328.
↳Ignoring rest of file.
2021-02-04 13:19:43,279 - ReadBIT.py          - 631 - 8000 - (MainThread) - INFO      -
↳Processing: data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_
↳1644659.bit
2021-02-04 13:19:43,364 - ReadBIT.py          - 631 - 8000 - (MainThread) - INFO      -
↳Processing: data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_
↳1644660.bit
2021-02-04 13:19:43,582 - ReadBIT.py          - 690 - 8000 - (MainThread) - INFO      -
↳Count of success=4 errors=0
      Size      Time (s) Rate (ms/Mb) File
```

(continues on next page)

(continued from previous page)

```

1004032      0.654      682.705 data/DresserAtlasBIT/special/29_10-_3/DWL_FILE/29_
↪10-_3_dwl_DWL_WIRE_1644632.bit
513536      0.330      673.674 data/DresserAtlasBIT/special/29_10-_3/DWL_FILE/29_
↪10-_3_dwl_DWL_WIRE_1644636.bit
119276      0.084      736.193 data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/
↪29_10-_3Z_dwl_DWL_WIRE_1644659.bit
339916      0.217      667.974 data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/
↪29_10-_3Z_dwl_DWL_WIRE_1644660.bit
Total size 1976760 bytes, total time 1.284 (s)
Rate 681.053 (ms/MB) 1.468 Mb/s
Use -v, --verbose to see more information about each BIT file.
Execution time = 1.293 (S)
Bye, bye!

```

Adding the -v flag for more verbosity gives the following, here for one file:

```

$ tdbitread data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_
↪1644659.bit -v
Cmd: src/TotalDepth/BIT/ReadBIT.py data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_
↪3Z_dwl_DWL_WIRE_1644659.bit -v
2021-02-04 13:21:41,402 - ReadBIT.py - 631 - 8016 - (MainThread) - INFO -
↪Processing: data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_
↪1644659.bit
File size: 119276 0x1dec: data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_
↪DWL_WIRE_1644659.bit
===== 29_10-_3Z_dwl_DWL_WIRE_1644659.bit =====
----- Frame Array [0] -----
BITFrameArray: ident=""
Unknown head: b'\x00\x02\x00\x00'
Description: b'SHELL EXPRO U.K. 24 OCT 84 MANSFIELD/DODDS
↪ '
Unknown A: b'\x00\n\x00\x18\x00'
Unknown B: b'T 2 9 / 1 0 - 3
↪ '
Unknown C: b'\x00\x12\x00\x0b\x00\x06 '
Channels [10]: ['COND', 'SN ', 'SP ', 'GR ', 'CAL ', 'TEN ', 'SPD ', 'ACQ ', 'AC ',
↪ 'RT ']
BIT Log Pass: LogPassRange(depth_from=14950.000891089492, depth_to=14590.000869631818,
↪ spacing=0.2500000149011621, unknown_a=0.0, unknown_b=16.000000953674373)
Unknown tail: b'MN239J 1'
Frame count: 1472
Frame array: FrameArray: ID: 0 b'SHELL EXPRO U.K. 24 OCT 84
↪ MANSFIELD/DODDS
<FrameChannel: 'X ' "Computed X-axis" units: 'b'' count: 1 dimensions: (1,)
↪ frames: 1472>
<FrameChannel: 'COND' "COND" units: 'b'' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'SN ' "SN " units: 'b'' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'SP ' "SP " units: 'b'' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'GR ' "GR " units: 'b'' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'CAL ' "CAL " units: 'b'' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'TEN ' "TEN " units: 'b'' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'SPD ' "SPD " units: 'b'' count: 1 dimensions: (1,) frames: 1472>

```

(continues on next page)

(continued from previous page)

```

<FrameChannel: 'ACQ ' "ACQ " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'AC ' "AC " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'RT ' "RT " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
----- DONE: Frame Array [0] -----
----- Frame Array [1] -----
BITFrameArray: ident="1"
  Unknown head: b'\x00\x02\x00\x00'
  Description: b'SHELL EXPRO U.K.      24 OCT 84      MANSFIELD/DODDS
  ↳ '
    Unknown A: b'\x00\n\x00\x18\x00'
    Unknown B: b'T 2 9 / 1 0 - 3
    ↳ '
    Unknown C: b'\x00\x11\x00/\x00\r '
  Channels [10]: ['COND', 'SN ', 'SP ', 'GR ', 'CAL ', 'TEN ', 'SPD ', 'ACQ ', 'AC ',
  ↳ 'RT ']
  BIT Log Pass: LogPassRange(depth_from=14948.000890970283, depth_to=0.0, spacing=0.
  ↳ 25000000149011621, unknown_a=0.0, unknown_b=16.0000000953674373)
  Unknown tail: b'MN239J 4'
  Frame count: 1440
  Frame array:      FrameArray: ID: 1 b'SHELL EXPRO U.K.      24 OCT 84
  ↳ MANSFIELD/DODDS
    <FrameChannel: 'X ' "Computed X-axis" units: 'b''' count: 1 dimensions: (1,)
  ↳ frames: 1440>
    <FrameChannel: 'COND' "COND" units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'SN ' "SN " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'SP ' "SP " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'GR ' "GR " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'CAL ' "CAL " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'TEN ' "TEN " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'SPD ' "SPD " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'ACQ ' "ACQ " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'AC ' "AC " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'RT ' "RT " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
----- DONE: Frame Array [1] -----
===== DONE 29_10-_3Z_dwl_DWL_WIRE_1644659.bit =====
Result:      119276      0.085      743.316 data/DresserAtlasBIT/special/29_10-_3Z/
  ↳ DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_1644659.bit
Execution time =      0.085 (S)
Bye, bye!

```

Adding the --summary flag for the frame data gives the following:

```

$ tdbitread data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_
  ↳ 1644659.bit -v --summary
Cmd: src/TotalDepth/BIT/ReadBIT.py data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_
  ↳ 3Z_dwl_DWL_WIRE_1644659.bit -v --summary
2021-02-04 13:24:12,756 - ReadBIT.py      - 631 - 8140 - (MainThread) - INFO      -
  ↳ Processing: data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_
  ↳ 1644659.bit
File size: 119276 0x1d1ec: data/DresserAtlasBIT/special/29_10-_3Z/DWL_FILE/29_10-_3Z_dwl_
  ↳ DWL_WIRE_1644659.bit
===== 29_10-_3Z_dwl_DWL_WIRE_1644659.bit =====

```

(continues on next page)

(continued from previous page)

```
----- Frame Array [0] -----
BITFrameArray: ident="0"
  Unknown head: b'\x00\x02\x00\x00'
  Description: b'SHELL EXPRO U.K.      24 OCT 84      MANSFIELD/DODDS
  '
  Unknown A: b'\x00\n\x00\x18\x00'
  Unknown B: b'T  2 9 / 1 0 - 3
  '
  Unknown C: b'\x00\x12\x00\x0b\x00\x06 '
  Channels [10]: ['COND', 'SN ', 'SP ', 'GR ', 'CAL ', 'TEN ', 'SPD ', 'ACQ ', 'AC ',
  'RT ' ]
  BIT Log Pass: LogPassRange(depth_from=14950.000891089492, depth_to=14590.000869631818,
  spacing=0.2500000149011621, unknown_a=0.0, unknown_b=16.000000953674373)
  Unknown tail: b'MN239J 1'
  Frame count: 1472
  Frame array:      FrameArray: ID: 0 b'SHELL EXPRO U.K.      24 OCT 84
  MANSFIELD/DODDS
    <FrameChannel: 'X ' "Computed X-axis" units: 'b''' count: 1 dimensions: (1,)
    frames: 1472>
    <FrameChannel: 'COND' "COND" units: 'b''' count: 1 dimensions: (1,) frames: 1472>
    <FrameChannel: 'SN ' "SN " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
    <FrameChannel: 'SP ' "SP " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
    <FrameChannel: 'GR ' "GR " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
    <FrameChannel: 'CAL ' "CAL " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
    <FrameChannel: 'TEN ' "TEN " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
    <FrameChannel: 'SPD ' "SPD " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
    <FrameChannel: 'ACQ ' "ACQ " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
    <FrameChannel: 'AC ' "AC " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
    <FrameChannel: 'RT ' "RT " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
ID   Length      Shape   Count      Min      Max      Mean      Std.Dev.
  Median Equal   Inc.   Dec.      Activity      Drift      First ->      Last
X     1472      (1472,)   1472      14582.3    14950    14766.1    106.232
  14766.1      0      0    1471    2.4427e-05    -0.25    14950 ->    14582.3
COND  1472      (1472,)   1472      0.0001    2249.65    1016.02    520.471
  1032.29    36    741    694    0.0695624    -0.690478    1015.69 ->    0.0001
SN     1472      (1472,)   1472      -3.18704    24.465    16.718    3.78744
  17.4128    35    762    674      nan    -0.0109108    16.0499 ->    0.0001
SP     1472      (1472,)   1472      -249.709    0.0001    -244.45    35.8542
  -249.709    1470    1      0      nan    0.169755    -249.709 ->    0.0001
GR     1472      (1472,)   1472      0.0001    128.979    64.2715    17.9234
  65.8888    168    637    666    0.0418426    -0.0556673    81.8867 ->    0.0001
CAL    1472      (1472,)   1472      -2.44161    0.0001    -2.36168    0.3465
  -2.41034    216    633    622      nan    0.00163825    -2.40976 ->    0.0001
TEN    1472      (1472,)   1472      0.0001    4385.71    3619.19    543.754
  3680.34    44    780    647    0.0198009    -2.18931    3220.48 ->    0.0001
SPD    1472      (1472,)   1472      0.0001    43.074    30.9595    4.88865
  31.8997    52    682    737    0.0201738    -0.0197124    28.9971 ->    0.0001
ACQ    1472      (1472,)   1472      0      4    0.058426    0.390943
  0    1433    26    12      nan    6.7981e-08    0 ->    0.0001
AC     1472      (1472,)   1472      0.0001    91.9285    73.2971    15.5518
  76.4215    116    630    725    0.0242162    -0.0254478    37.4338 ->    0.0001
RT     1472      (1472,)   1472      0.0001    22.6922    1.81547    3.05078
  0.952113    36    693    742    0.0621832    -0.000669238    0.984549 ->    0.0001
```

(continues on next page)

(continued from previous page)

```

----- DONE: Frame Array [0] -----
----- Frame Array [1] -----
BITFrameArray: ident="1"
  Unknown head: b'\x00\x02\x00\x00'
  Description: b'SHELL EXPRO U.K.      24 OCT 84      MANSFIELD/DODDS
↳ '
  Unknown A: b'\x00\n\x00\x18\x00'
  Unknown B: b'T 29 / 10 - 3
↳ '
  Unknown C: b'\x00\x11\x00/\x00\r '
  Channels [10]: ['COND', 'SN ', 'SP ', 'GR ', 'CAL ', 'TEN ', 'SPD ', 'ACQ ', 'AC ',
↳ 'RT ']
  BIT Log Pass: LogPassRange(depth_from=14948.000890970283, depth_to=0.0, spacing=0.
↳ 25000000149011621, unknown_a=0.0, unknown_b=16.000000953674373)
  Unknown tail: b'MN239J 4'
  Frame count: 1440
  Frame array:      FrameArray: ID: 1 b'SHELL EXPRO U.K.      24 OCT 84
↳ MANSFIELD/DODDS
    <FrameChannel: 'X ' "Computed X-axis" units: 'b''' count: 1 dimensions: (1,)
↳ frames: 1440>
    <FrameChannel: 'COND' "COND" units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'SN ' "SN " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'SP ' "SP " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'GR ' "GR " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'CAL ' "CAL " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'TEN ' "TEN " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'SPD ' "SPD " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'ACQ ' "ACQ " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'AC ' "AC " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
    <FrameChannel: 'RT ' "RT " units: 'b''' count: 1 dimensions: (1,) frames: 1440>
ID   Length      Shape   Count      Min      Max      Mean      Std.Dev.
↳ Median   Equal   Inc.   Dec.   Activity   Drift   First ->   Last
X     1440      (1440,)   1440    14588.3   14948   14768.1   103.923
↳ 14768.1    0      0    1439  2.44237e-05   -0.25   14948 ->   14588.3
COND  1440      (1440,)   1440      0.0001   2261.8   1016.27   509.346
↳ 1053.65   20     757   662   0.0685063   -0.708206   1019.11 ->   0.0001
SN     1440      (1440,)   1440      0.0001   91.6549   18.5481   4.23407
↳ 19.0641   20     756   663   0.0256921   -0.0636934   91.6549 ->   0.0001
SP     1440      (1440,)   1440   -249.709   0.0001   -239.304   29.3708
↳ -242.363   54     708   677      nan    0.164374   -236.534 ->   0.0001
GR     1440      (1440,)   1440      0.0001   131.253   65.1255   17.9535
↳ 66.5528   155    631   653   0.0393008   -0.055561   79.9523 ->   0.0001
CAL    1440      (1440,)   1440   -2.43995   0.0001   -2.37981   0.289624
↳ -2.41493   80     673   686      nan    0.00167468   -2.40976 ->   0.0001
TEN    1440      (1440,)   1440      0.0001   6564.66   3783.73   721.861
↳ 3678.22   21     786   632   0.0212608   -1.99216   2866.71 ->   0.0001
SPD    1440      (1440,)   1440      0      39.5108   29.5586   4.05069
↳ 29.9188   26     683   730      inf   6.94927e-08   0 ->   0.0001
ACQ    1440      (1440,)   1440      0      2   0.00972368   0.12322
↳ 0      1426     8     5      nan   6.94927e-08   0 ->   0.0001
AC     1440      (1440,)   1440      0.0001   91.9285   76.2796   10.8951
↳ 76.8658   91     633   715   0.023214   -0.0523857   75.3831 ->   0.0001

```

(continues on next page)

(continued from previous page)

```

RT      1440      (1440,)    1440      0.0001      21.3526      1.79013      2.73227
↪ 0.92928      20      661      758      0.060755 -0.000681828      0.98125 ->      0.0001
----- DONE: Frame Array [1] -----
===== DONE 29_10-_3Z_dwl_DWL_WIRE_1644659.bit =====
Result:      119276      0.097      848.759 data/DresserAtlasBIT/special/29_10-_3Z/
↪DWL_FILE/29_10-_3Z_dwl_DWL_WIRE_1644659.bit
Execution time =      0.097 (S)
Bye, bye!

```

5.2.2 Converting BIT Files to LAS Files with `tdbittolas`

This takes a BIT file or directory of them and writes out a set of LAS files. A single LAS file is written for each Log Pass so a single BIT file produces one or more LAS files.

The frames in the log pass can be sub-sampled by using `--frame-slice` which speeds things up when processing large files. The `--channels` option can be used to limit channels.

BIT does not allow multiple values per channel.

As BIT files contain very little other than the frame data the generated LAS files are very simple and are missing what many processors would regard as essential data such as well name. These LAS files may have to be edited with data from other sources than the original BIT file to be useful.

LAS File Naming Convention

One BIT file produces one or more LAS files. LAS file names are of the form:

```
{BIT_File}_{logical_file_number:04d}.las
```

Processing a Single BIT File

Given the path out the LAS files will be named `{path_out}_{logical_file_number}.las`

For example `tdbittolas foo.bit bar/baz` might create:

```
bar/baz.bit_0000.las
bar/baz.bit_0001.las
```

and so on.

Processing a Directory of BIT Files

Given the path out the LAS files will be named:

```
{path_out}/{BIT_File}_{logical_file_number}.las
```

For example `tdbittolas foo/ bar/baz` might create:

```
bar/baz.bit_0000.las
bar/baz.bit_0001.las
```

and so on.

The output directory structure will mirror the input directory structure.

Arguments

The first argument is the path to a BIT file or directory. The second argument is the path to write the output to.

Options

- h, --help** show this help message and exit
- version** show program's version number and exit
- k, --keep-going** Keep going as far as sensible. Default: False.
- v, --verbose** Increase verbosity, additive [default: 0]
- r, --recurse** Process the input recursively. Default: False.
- l LOG_LEVEL, --log-level LOG_LEVEL** Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 20]
- j JOBS, --jobs JOBS** Max processes when multiprocessing. Zero uses number of native CPUs [8]. Negative value disables multiprocessing code. Default: -1.
- frame-slice FRAME_SLICE** Do not process all frames but sample or slice the frames. SAMPLE: Sample is of the form "N" so a maximum of N frames, roughly regularly spaced, will be processed. N must be +ve, non-zero integer. Example: "64" - process a maximum of 64 frames. SLICE: Slice the frames is of the form start,stop,step as a comma separated list. Values can be absent or "None". Examples: ".,," - every frame, ".,2" - every other frame, ".,10," - frames 0 to 9, "4,10,2" - frames 4, 6, 8, "40,-1,4" - every fourth frame from 40 to the end. Results will be truncated by frame array length. Use "?" to see what frames are available [default: ".,," i.e. all frames]
- log-process LOG_PROCESS** Writes process data such as memory usage as a log INFO line every LOG_PROCESS seconds. If 0.0 no process data is logged. [default: 0.0]
- gnuplot GNUPLOT** Directory to write the gnuplot data.
- array-reduction ARRAY_REDUCTION** Method to reduce multidimensional channel data to a single value. One of {first,max,mean,median,min} [default: first]
- channels CHANNELS** Comma separated list of channels to write out (X axis is always included). Use "?" to see what channels exist without writing anything. [default: ""]
- field-width FIELD_WIDTH** Field width for array data [default: 16].
- float-format FLOAT_FORMAT** Floating point format for array data [default: ".3f"].

Examples

Finding out what Channels and Frames Exist:

Use `--channels=?` and/or `--frame-slice=?` to see what channels and frames exist in the original BIT file.

```
$ tdbittolas --channels=? --frame-slice=? example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_
↳1644659.bit example_data/BIT/LIS
===== File example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_1644659.bit =====

Frame Array: 0
Channels: "X  ", "COND", "SN  ", "SP  ", "GR  ", "CAL ", "TEN ", "SPD ", "ACQ ", "AC  ", "RT  "
X axis: <FrameChannel: 'X  ' "Computed X-axis" units: 'b'' count: 1 dimensions: (1,)
↳frames: 1472>
Frames: 1472 from 14950.000891089492 to 14582.250869169884 interval -0.
↳2500000149011612 [b'']

Frame Array: 1
Channels: "X  ", "COND", "SN  ", "SP  ", "GR  ", "CAL ", "TEN ", "SPD ", "ACQ ", "AC  ", "RT  "
X axis: <FrameChannel: 'X  ' "Computed X-axis" units: 'b'' count: 1 dimensions: (1,)
↳frames: 1440>
Frames: 1440 from 14948.000890970283 to 14588.250869527512 interval -0.
↳2500000149011612 [b'']

===== END File example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_1644659.bit =====
```

Processing a Single File

```
$ tdbittolas example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_1644659.bit example_data/BIT/
↳LAS/29_10-_3Z_dwl_DWL_WIRE_1644659.bit
Cmd: /Users/paulross/pyenvs/TotalDepth3.8_v0.3/bin/tdbittolas example_data/BIT/data/29_
↳10-_3Z_dwl_DWL_WIRE_1644659.bit example_data/BIT/LAS/29_10-_3Z_dwl_DWL_WIRE_1644659.bit
gnuplot version: "b'gnuplot 5.4 patchlevel 1'"
2021-02-05 12:58:18,749 - WriteLAS.py - 191 - 28222 - (MainThread) - INFO -
↳process_to_las(): Namespace(array_reduction='first', channels='', field_width=16,
↳float_format='.3f', frame_slice=',', gnuplot=None, jobs=-1, keepGoing=False, log_
↳level=20, log_process=0.0, path_in='example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_
↳1644659.bit', path_out='example_data/BIT/LAS/29_10-_3Z_dwl_DWL_WIRE_1644659.bit',
↳recurse=False, verbose=0)
2021-02-05 12:58:18,749 - WriteLAS.py - 167 - 28222 - (MainThread) - INFO -
↳index_dir_or_file(): "example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_1644659.bit" to
↳"example_data/BIT/LAS/29_10-_3Z_dwl_DWL_WIRE_1644659.bit" recurse: False
2021-02-05 12:58:18,750 - ToLAS.py - 117 - 28222 - (MainThread) - INFO -
↳Found file type BIT on path example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_1644659.bit
2021-02-05 12:58:18,750 - ToLAS.py - 119 - 28222 - (MainThread) - INFO -
↳Reading BIT file example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_1644659.bit
2021-02-05 12:58:18,846 - ToLAS.py - 125 - 28222 - (MainThread) - INFO -
↳Writing frame array 0 to example_data/BIT/LAS/29_10-_3Z_dwl_DWL_WIRE_1644659.bit_0000.
↳las
2021-02-05 12:58:18,848 - WriteLAS.py - 521 - 28222 - (MainThread) - INFO -
↳Writing array section with 1,472 frames, 11 channels and 11 values per frame, total:
↳16,192 input values. (continues on next page)
```

(continued from previous page)

```

2021-02-05 12:58:18,994 - ToLAS.py          - 125 - 28222 - (MainThread) - INFO      -
↳ Writing frame array 1 to example_data/BIT/LAS/29_10-_3Z_dwl_DWL_WIRE_1644659.bit_0001.
↳ las
2021-02-05 12:58:18,995 - WriteLAS.py      - 521 - 28222 - (MainThread) - INFO      -
↳ Writing array section with 1,440 frames, 11 channels and 11 values per frame, total:
↳ 15,840 input values.
  Input      Type  Output LAS Count  Time  Ratio  ms/Mb Exception
  ↳          ↳          ↳          ↳          ↳          ↳          ↳
  ↳          ↳          ↳          ↳          ↳          ↳          ↳
  ↳          ↳          ↳          ↳          ↳          ↳          ↳
-----
↳ -----
119,276 BIT      549,613          2 0.370 460.8% 3249.8      False "example_data/BIT/data/
↳ 29_10-_3Z_dwl_DWL_WIRE_1644659.bit"
Writing results returned: 0 files failed.
Execution time =    0.370 (S)
Out of 1 processed 1 files of total size 119,276 input bytes
Wrote 549,613 output bytes, ratio: 460.791% at 3256.2 ms/Mb
Execution time: 0.370 (s)
Bye, bye!

```

The LAS files look like this:

```

~Version Information Section
VERS.          2.0                      : CWLS Log ASCII Standard - VERSION
↳ 2.0
WRAP.          NO                      : One Line per depth step
PROD.          TotalDepth              : LAS Producer
PROG.          TotalDepth.BIT.ToLAS 0.1.1 : LAS Program name and version
CREA.          2021-02-05 12:58:18.847493 UTC : LAS Creation date [YYYY-mm-dd HH
↳ MM SS.us UTC]
SOURCE.        29_10-_3Z_dwl_DWL_WIRE_1644659.bit : Source File Name
LOGICAL-FILE.  0                      : Logical File number in the Source
↳ file
SOURCE_FORMAT. WESTERN ATLAS BIT FORMAT      : File format of Source file.
#
# Binary block A: b'SHELL EXPRO U.K.      24 OCT 84      MANSFIELD/DODDS
↳
# Binary block B: b'T  2 9 / 1 0 - 3
↳
# BIT Log Pass (claimed): LogPassRange(depth_from=14950.000891089492, depth_to=14590.
↳ 000869631818, spacing=0.2500000149011621, unknown_a=0.0, unknown_b=16.0000000953674373)
#
~Well Information Section
#MNEM.UNIT  DATA                      DESCRIPTION
#-----
STRT.       14950.000891089492          : START
STOP.       14582.250869169884          : STOP
STRP.       -0.2500000149011612         : STEP
~Curve Information Section
#MNEM.UNIT  Curve Description
#-----
X .         : Computed X-axis Dimensions (1,)
COND.       : COND Dimensions (1,)

```

(continues on next page)

(continued from previous page)

```

SN      : SN      Dimensions (1,)
SP      : SP      Dimensions (1,)
GR      : GR      Dimensions (1,)
CAL     : CAL     Dimensions (1,)
TEN     : TEN     Dimensions (1,)
SPD     : SPD     Dimensions (1,)
ACQ     : ACQ     Dimensions (1,)
AC      : AC      Dimensions (1,)
RT      : RT      Dimensions (1,)
# Array processing information:
# Frame Array: ID: 0 description: b'SHELL EXPRO U.K.      24 OCT 84      MANSFIELD/DODDS_
↳
# All [11] original channels reproduced here.
# Where a channel has multiple values the reduction method is by "first" value.
# Maximum number of original frames: 1472
# Requested frame slicing: <Slice on length=1472 start=0 stop=1472 step=1>, total number_
↳ of frames presented here: 1472
~A      X      COND      SN      SP      GR      _
↳      CAL      TEN      SPD      ACQ      AC      _
↳      RT
14950.001      1015.693      16.050      -249.709      81.887      _
↳      -2.410      3220.477      28.997      0.000      37.434      _
↳      0.985
14949.751      1015.693      16.050      -249.709      81.887      _
↳      -2.410      3220.477      28.997      0.000      37.434      _
↳      0.985
14949.501      1015.693      16.050      -249.709      81.887      _
↳      -2.410      3220.477      28.997      0.000      37.434      _
↳      0.985
14949.251      1015.693      16.050      -249.709      81.887      _
↳      -2.410      3220.477      28.997      0.000      37.434      _
↳      0.985

```

Processing a Directory

Use the `-r` option to process recursively. The output directory will mirror the input directory.

```

$ tdbittolas -r example_data/BIT/data example_data/BIT/LAS
Cmd: /Users/paulross/pyenvs/TotalDepth_3.8_v0.3/bin/tdbittolas -r example_data/BIT/data_
↳ example_data/BIT/LAS
gnuplot version: "b'gnuplot 5.4 patchlevel 1'"
2021-02-05 13:00:32,879 - WriteLAS.py      - 191 - 28324 - (MainThread) - INFO      _
↳ process_to_las(): Namespace(array_reduction='first', channels='', field_width=16,
↳ float_format='.3f', frame_slice=',', gnuplot=None, jobs=-1, keepGoing=False, log_
↳ level=20, log_process=0.0, path_in='example_data/BIT/data', path_out='example_data/BIT/
↳ LAS', recurse=True, verbose=0)
2021-02-05 13:00:32,879 - WriteLAS.py      - 167 - 28324 - (MainThread) - INFO      _
↳ index_dir_or_file(): "example_data/BIT/data" to "example_data/BIT/LAS" recurse: True
2021-02-05 13:00:32,880 - ToLAS.py          - 117 - 28324 - (MainThread) - INFO      _
↳ Found file type BIT on path example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_1644659.bit
2021-02-05 13:00:32,880 - ToLAS.py          - 119 - 28324 - (MainThread) - INFO      _
↳ Reading BIT file example_data/BIT/data/29_10-_3Z_dwl_DWL_WIRE_1644659.bi (continues on next page)

```

(continued from previous page)

```

2021-02-05 13:00:32,962 - ToLAS.py - 125 - 28324 - (MainThread) - INFO -
↳ Writing frame array 0 to example_data/BIT/LAS/29_10-_3Z_dwl_DWL_WIRE_1644659.bit_0000.
↳ las
2021-02-05 13:00:32,964 - WriteLAS.py - 521 - 28324 - (MainThread) - INFO -
↳ Writing array section with 1,472 frames, 11 channels and 11 values per frame, total:
↳ 16,192 input values.
2021-02-05 13:00:33,076 - ToLAS.py - 125 - 28324 - (MainThread) - INFO -
↳ Writing frame array 1 to example_data/BIT/LAS/29_10-_3Z_dwl_DWL_WIRE_1644659.bit_0001.
↳ las
2021-02-05 13:00:33,076 - WriteLAS.py - 521 - 28324 - (MainThread) - INFO -
↳ Writing array section with 1,440 frames, 11 channels and 11 values per frame, total:
↳ 15,840 input values.
Input      Type  Output LAS Count  Time  Ratio  ms/Mb  Exception
↳
↳ Path
-----
↳ -----
119,276 BIT      549,613          2 0.300 460.8% 2634.7      False "example_data/BIT/data/
↳ 29_10-_3Z_dwl_DWL_WIRE_1644659.bit"
Writing results returned: 0 files failed.
Execution time = 0.301 (S)
Out of 1 processed 1 files of total size 119,276 input bytes
Wrote 549,613 output bytes, ratio: 460.791% at 2644.1 ms/Mb
Execution time: 0.301 (s)
Bye, bye!

```

5.3 LAS Command Line Tools

This describes the command line tools that are available for processing LAS files.

Table 3: LAS Command Line Tools

Tool Name	Description
tdlstohtml	Generates a HTML page(s) about LAS file(s). Link
tdlasreadlasfiles	Summarises LAS file(s). Link

5.3.1 Summarise LAS Files in HTML with tdlastohtml

Generates HTML from input LAS file or directory to an output destination.

Arguments

1. The path to the input LAS file or directory.
2. The path to the output file or directory, any directories will be created as necessary.

Options

- h, --help** show this help message and exit
- version** show program's version number and exit
- k, --keep-going** Keep going as far as sensible. Default: False.
- v, --verbose** Increase verbosity, additive [default: 0]
- r, --recurse** Process the input recursively. Default: False.
- l LOG_LEVEL, --log-level LOG_LEVEL** Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 20]
- j JOBS, --jobs JOBS** Max processes when multiprocessing. Zero uses number of native CPUs [8]. Negative value disables multiprocessing code. Default: -1.
- frame-slice FRAME_SLICE** Do not process all frames but sample or slice the frames. SAMPLE: Sample is of the form "N" so a maximum of N frames, roughly regularly spaced, will be processed. N must be +ve, non-zero integer. Example: "64" - process a maximum of 64 frames. SLICE: Slice the frames is of the form start,stop,step as a comma separated list. Values can be absent or "None". Examples: ".,," - every frame, ".,2" - every other frame, ".,10," - frames 0 to 9, "4,10,2" - frames 4, 6, 8, "40,-1,4" - every fourth frame from 40 to the end. Results will be truncated by frame array length. Use '?' to see what frames are available [default: ".,," i.e. all frames]
- log-process LOG_PROCESS** Writes process data such as memory usage as a log INFO line every LOG_PROCESS seconds. If 0.0 no process data is logged. [default: 0.0]
- gnuplot GNUPLOT** Directory to write the gnuplot data.
- g, --glob** File match pattern. Default: None.

Examples

Command to process a directory of LAS:

```
$ tdlastohtml example_data/LAS/data/ example_data/LAS/HTML/
```

Output:

```
$ tdlastohtml example_data/LAS/data/ example_data/LAS/HTML/
Cmd: /Users/paulross/pyenvs/TotalDepth_3.8_v0.3/bin/tdlastohtml example_data/LAS/data/
↳ example_data/LAS/HTML/
gnuplot version: "b'gnuplot 5.4 patchlevel 1'"
2021-02-06 11:13:20,527 - LASToHTML.py - 440 - 41351 - (MainThread) - INFO -
↳ scan_dir_or_file(): "example_data/LAS/data" to "example_data/LAS/HTML" recurse: False
2021-02-06 11:13:20,529 - LASToHTML.py - 351 - 41351 - (MainThread) - INFO -
↳ Scanning file type "ASCII" from "example_data/LAS/data/.DS_Store" to "example_data/LAS/
↳ HTML/.DS_Store.html"
2021-02-06 11:13:20,530 - LASToHTML.py - 351 - 41351 - (MainThread) - INFO -
↳ Scanning file type "LAS2.0" from "example_data/LAS/data/1000079714.las" to "example_
↳ data/LAS/HTML/1000079714.las.html"
```

(continues on next page)

(continued from previous page)

```

2021-02-06 11:13:20,530 - LASToHTML.py      - 353 - 41351 - (MainThread) - INFO      -
↳scan_a_single_file(): "example_data/LAS/data/1000079714.las" to "example_data/LAS/HTML/
↳1000079714.las.html"
2021-02-06 11:13:20,614 - LASToHTML.py      - 351 - 41351 - (MainThread) - INFO      -
↳Scanning file type "LAS2.0" from "example_data/LAS/data/206_05a-_3_DWL_DWL_WIRE_
↳258276498_0_2000T.las" to "example_data/LAS/HTML/206_05a-_3_DWL_DWL_WIRE_258276498_0_
↳2000T.las.html"
2021-02-06 11:13:20,614 - LASToHTML.py      - 353 - 41351 - (MainThread) - INFO      -
↳scan_a_single_file(): "example_data/LAS/data/206_05a-_3_DWL_DWL_WIRE_258276498_0_2000T.
↳las" to "example_data/LAS/HTML/206_05a-_3_DWL_DWL_WIRE_258276498_0_2000T.las.html"
2021-02-06 11:13:20,679 - LASToHTML.py      - 351 - 41351 - (MainThread) - INFO      -
↳Scanning file type "LAS2.0" from "example_data/LAS/data/BASIC_FILE_0_50.las" to
↳"example_data/LAS/HTML/BASIC_FILE_0_50.las.html"
2021-02-06 11:13:20,680 - LASToHTML.py      - 353 - 41351 - (MainThread) - INFO      -
↳scan_a_single_file(): "example_data/LAS/data/BASIC_FILE_0_50.las" to "example_data/LAS/
↳HTML/BASIC_FILE_0_50.las.html"
2021-02-06 11:13:20,724 - LASToHTML.py      - 382 - 41351 - (MainThread) - INFO      -
↳write_indexes(): result map size 4
2021-02-06 11:13:20,725 - ToHTML.py         - 207 - 41351 - (MainThread) - INFO      -
↳Opening index file at /Users/paulross/PycharmProjects/TotalDepth/example_data/LAS/HTML/
↳index.html
2021-02-06 11:13:20,730 - ToHTML.py         - 240 - 41351 - (MainThread) - INFO      -
↳Completed index file at /Users/paulross/PycharmProjects/TotalDepth/example_data/LAS/
↳HTML/index.html
2021-02-06 11:13:20,730 - LASToHTML.py      - 399 - 41351 - (MainThread) - INFO      -
↳Wrote indexes: ['/Users/paulross/PycharmProjects/TotalDepth/example_data/LAS/HTML/
↳index.html']
Common path: example_data/LAS/data
      Size In   Size Out      Time  Ratio %    ms/Mb Fail? Path
-----
      6,148         0    0.000   0.000%     0.0 False ".DS_Store"
     80,697     12,892    0.083  15.976%   1078.0 False "1000079714.las"
     87,448     53,444    0.065  61.115%    779.8 False "206_05a-_3_DWL_DWL_WIRE_
↳258276498_0_2000T.las"
     62,494     26,021    0.044  41.638%    736.1 False "BASIC_FILE_0_50.las"
Processed 4 files and 236,787 bytes in 0.203 s, 900.0 ms/Mb
Bye, bye!

```

For each file the output lists:

- Input file.
- Output HTML file.
- File size.
- Execution time.

In the output directory there will be an index.html file which has the columns:

Path File Type Sections Channels Frames STRT STOP STEP Size Time

- The name of the LAS file.
- LAS file type.
- Number of sections.

- Recorded channels.
- Number of data frames.
- Start of log pass.
- End of log pass.
- Frame step.
- The size of the LAS file.
- Execution time.

In the linked HTML file is a summary of the content of the LAS file.

5.3.2 Summarise LAS Files with `tdlasreadlasfiles`

Reads an input LAS file or directory and summarises it by showing mnemonics, curves and well site data.

Arguments

1. The path to the input LAS file or directory.

Options

-h, --help	show this help message and exit
--version	show program's version number and exit
-k, --keep-going	Keep going as far as sensible. Default: False.
-v, --verbose	Increase verbosity, additive [default: 0]
-r, --recurse	Process the input recursively. Default: False.
-l LOG_LEVEL, --log-level LOG_LEVEL	Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 20]
--log-process LOG_PROCESS	Writes process data such as memory usage as a log INFO line every LOG_PROCESS seconds. If 0.0 no process data is logged. [default: 0.0]
--gnuplot GNUPLOT	Directory to write the gnuplot data.
-m, --mnemonic	Output Mnemonic map. Default: False.
-c, --curve	Output Curve map. Default: False.
-u, --unit	Output Units map. Default: False.
-w, --wsd	Output Well Site Data map. Default: False.
-p, --param	Output Parameter section mnemonics and their most popular description and a map of themnemonic frequency. Default: False.
-s, --size-time	Output parser's size vs time performance. Default: False.
-a, --all	Output all, equivalent to -mcuwps. Default: False.

Examples

Listing Menmonics

Use the `-m` option to summarise the menmonics and their descriptions:

```
$ tdlasreadlasfiles example_data/LAS/data/ -m
Cmd: /Users/paulross/pyenvs/TotalDepth_3.8_v0.3/bin/tdlasreadlasfiles example_data/LAS/
↳data/ -m
gnuplot version: "b'gnuplot 5.4 patchlevel 1'"
----- All mnemonics and their (most popular) description -----
{
"ALTDPCCHAN"           : "Name Of Alternate Depth Channel"
↳           ", # Out of 1
"AMD"                  : "Azimuth Of Maximum Deviation"
↳           ", # Out of 1
"AOFF"                 : "Alphanumeric To Film Flag"
↳           ", # Out of 1
"APD"                  : "Depth Above Pd"
↳           ", # Out of 2
"API"                  : "
↳           ", # Out of 2
"APIN"                 : "Api S/N"
↳           ", # Out of 2
"BG"                   : "Gas Formation Volume Factor, Bg"
↳           ", # Out of 1
"BHS"                  : "Borehole Status"
↳           ", # Out of 1
"BHT"                  : "Bottom Hole Temperature (Used In Calculations)"
↳           ", # Out of 1
"BLI"                  : "Bottom Log Interval"
↳           ", # Out of 1
"BO"                   : "Oil Formation Volume Factor, Bo"
↳           ", # Out of 1
"BPP"                  : "Bubble Point Pressure"
↳           ", # Out of 1
...

```

Listing Curves

Use the `-c` option to summarise the curves and their descriptions:

```
$ tdlasreadlasfiles example_data/LAS/data/ -c
Cmd: /Users/paulross/pyenvs/TotalDepth_3.8_v0.3/bin/tdlasreadlasfiles example_data/LAS/
↳data/ -c
gnuplot version: "b'gnuplot 5.4 patchlevel 1'"
----- All Curve mnemonics and their (most popular) description -----
{
"DEPT"                 : "Depth Curve"
↳           ", # Out of 2
"DEPT_SL"              : "Station Logging Depth Dimensions (1,)"
↳           ", # Out of 1

```

(continues on next page)

(continued from previous page)

```

"DHTN"                : "Dhtn/Ch Tension Dimensions (1,)"
↪      ", # Out of 1
"ETIM"                : "Etim/Elapsed Time Dimensions (1,)"
↪      ", # Out of 1
"GR"                  : "Gamma Ray"
↪      ", # Out of 2
"TDEP"                : "Second River Depth Dimensions (1,)"
↪      ", # Out of 1
"TENS"                : "Tens/Tension Dimensions (1,)"
↪      ", # Out of 1
"TENS_SL"             : "Cable Tension Dimensions (1,)"
↪      ", # Out of 1
"TIME"                : "Second River Time Dimensions (1,)"
↪      ", # Out of 1
}
----- DONE: All Curve mnemonics and their (most popular) description -----

```

Listing Units

Use the `-u` option to summarise the channels and their units:

```

$ tdlasreadlasfiles example_data/LAS/data/ -u
Cmd: /Users/paulross/pyenvs/TotalDepth_3.8_v0.3/bin/tdlasreadlasfiles example_data/LAS/
↪data/ -u
gnuplot version: "b'gnuplot 5.4 patchlevel 1'"
----- Channels and their Units -----
{
"DEPT"      : "Counter({'F': 1, 'm': 1})",
"DEPT_SL"   : "Counter({'0.1': 1})",
"DHTN"      : "Counter({'lbs': 1})",
"ETIM"      : "Counter({'min': 1})",
"GR"        : "Counter({'GAPI': 1, 'api': 1})",
"TDEP"      : "Counter({'0.1': 1})",
"TENS"      : "Counter({'lbs': 1})",
"TENS_SL"   : "Counter({'lbf': 1})",
"TIME"      : "Counter({'ms': 1})",
}
----- DONE: Channels and their Units -----

```

Listing Well Site Data

Use the `-w` option to summarise the well site data and its frequency:

```

$ tdlasreadlasfiles example_data/LAS/data/ -w
Cmd: /Users/paulross/pyenvs/TotalDepth_3.8_v0.3/bin/tdlasreadlasfiles example_data/LAS/
↪data/ -w
gnuplot version: "b'gnuplot 5.4 patchlevel 1'"
----- Count of well site mnemonics and the % of files that have them -----
{

```

(continues on next page)

(continued from previous page)

"API"				:	""	
↪	,	#	3	100.00%		
"CNTY"				:	""	
↪	,	#	2	66.67%		
"COMP"				:	""	
↪	,	#	3	100.00%		
"CORN"				:	"Reference Section Corner For Footage"	
↪	,	#	1	33.33%		
"COUN"				:	"County"	
↪	,	#	1	33.33%		
"CTRY"				:	""	
↪	,	#	2	66.67%		
"DATE"				:	""	
↪	,	#	3	100.00%		
"FLD"				:	""	
↪	,	#	3	100.00%		
"FTE"				:	"Feet East From Reference Section Corner"	
↪	,	#	1	33.33%		
"FTN"				:	"Feet North From Reference Section Corner"	
↪	,	#	1	33.33%		
"LAT"				:	"Latitude North (Kgs,Leo3.6)"	
↪	,	#	1	33.33%		
"LEAS"				:	"Lease Name"	
↪	,	#	1	33.33%		
"LOC"				:	""	
↪	,	#	3	100.00%		
"LON"				:	"Longitude West (Kgs, Leo3.6)"	
↪	,	#	1	33.33%		
"NULL"				:	""	
↪	,	#	3	100.00%		
"PM"				:	"Principal Meridian"	
↪	,	#	1	33.33%		
"PROV"				:	""	
↪	,	#	2	66.67%		
"RANG"				:	"Range"	
↪	,	#	1	33.33%		
"SECT"				:	"Section"	
↪	,	#	1	33.33%		
"SPOT"				:	"Spot Location"	
↪	,	#	1	33.33%		
"SRVC"				:	""	
↪	,	#	2	66.67%		
"STAT"				:	"State Name"	
↪	,	#	3	100.00%		
"STEP"				:	"Step (Average)"	
↪	,	#	3	100.00%		
"STOP"				:	"Stop Depth"	
↪	,	#	3	100.00%		
"STRT"				:	"Start X"	
↪	,	#	3	100.00%		
"TOWN"				:	"Township"	
↪	,	#	1	33.33%		

(continues on next page)

(continued from previous page)

```

"UWI"                                : ""
↪      , #          2    66.67%
"WELL"                                : ""
↪      , #          3    100.00%
}
--- DONE: Count of well site mnemonics and the % of files that have them ---

```

5.4 LIS Command Line Tools

This describes the command line tools that are available for processing LIS files.

Table 4: LIS Command Line Tools

Tool Name	Description
tdlistohtml	Generates a HTML page(s) about LIS file(s). Link
tdlisplotlogpasses	Plots LIS log data as SVG pages. Link
tdlisdumpframeset	Writes out the frame data as a CSV file. Link
tdlisindex	Indexes a LIS file. Link
tdlistablehistogram	Analyses the contents of table Logical Records. Link
tdlisscanphysrec	Scans all the Physical Records. Link
tdlisscanlogidata	Scans the logical data. Link
tdlisscanlogirecord	Scans all Logical records. Link

5.4.1 Summarise LIS Files in HTML with tdlistohtml

Generates HTML from input LIS file or directory to an output destination.

Arguments

1. The path to the input LIS file or directory.
2. The path to the output file or directory, any directories will be created as necessary.

Options

Option	Description
--version	Show program's version number and exit
-h, --help	Show this help message and exit.
-g, --glob	File pattern match. [default none]
-j JOBS, --jobs=JOBS	Max processes when multiprocessing. Zero uses number of native CPUs [8]. -1 disables multiprocessing. [default: -1]
-k, --keep-going	Keep going as far as sensible. [default: False]
-l LOGLEVEL, --loglevel=LOGLEVEL	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]
-v, --verbose	Verbose output, this outputs a representation of table data and DFSRs.
-r, --recursive	Process input recursively. [default: False]

Examples

Command to process a directory of LIS:

```
$ ``tdlisttohtml`` Simple/ LIS_plot/Simple_00/
```

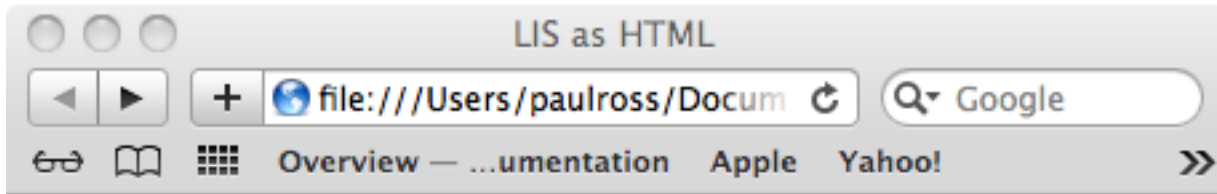
Output:

```
Cmd: ``tdlisttohtml`` Simple/ LIS_plot/Simple_00/
plotLogInfo:
FileInfo: "Simple/LIS.lis" -> "LIS_plot/Simple_00/LIS.lis.html" 17 (kb) LR count=4 t=0.
↳070
FileInfo: "Simple/RW.lis" -> "LIS_plot/Simple_00/RW.lis.html" 843 (kb) LR count=12 t=3.
↳206
FileInfo: "Simple/RW_No_TIF.lis" -> "LIS_plot/Simple_00/RW_No_TIF.lis.html" 833 (kb) LR_
↳count=12 t=3.200
CPU time = 6.568 (S)
Exec. time = 6.568 (S)
Bye, bye!
```

For each file the output lists:

- Input file.
- Output HTML file.
- File size.
- Count of Logical Records.
- Execution time.

In the output directory there will be an index.html file, for example:



LIS File	Size (MB)	Record Entries	CPU Time (s)	Rate (MB/s)
2851.S1	0.026	14	0.117	0.219
2851.S2	0.176	11	1.110	0.159
2851.S3	0.161	11	1.012	0.159
2851.S4	0.052	11	0.310	0.169
2851.S5	0.058	11	0.346	0.169
2851.S6	0.028	11	0.149	0.186
2851.S7	0.236	11	1.504	0.157
2951.S1	0.182	18	0.659	0.277
2951.S2	0.097	15	0.367	0.264
2953.S10	0.041	9	0.230	0.179
2953.S12	0.016	10	0.068	0.236
2953.S13	0.084	10	0.582	0.145
2953.S14	0.031	19	0.136	0.227
2953.S15	0.015	11	0.065	0.232
2953.S16	0.071	11	0.484	0.148
2953.S17	0.031	17	0.133	0.233
2953.S18	0.015	10	0.065	0.234
2953.S19	0.074	10	0.437	0.169
2955.S1	0.028	10	0.117	0.238
2955.S2	1.487	12	9.463	0.157
Totals	2.910	242	17.352	0.168

The columns are:

- The name of the LIS file.
- The size of the LIS file.

- Count of Logical Records.
- Execution time.
- Processing rate.

In the linked HTML file is a summary of the content of the LIS file.

The Log Pass merits several entries, the first summarises the frame shape and the shape of each channel, for example:

LIS analysis of ../../../../TDTestData/LIS/pLogicTestData_med/LIS/13615.S3

Overview — Documentation Apple Yahoo! Google Maps YouTube Wikipedia News (225) Popular

Log Pass at 0x3f6

Frame Information

	Value
Frame length	118 bytes
Number of Channels	8
Indirect X size	4 bytes

Channels

RHDT, P1AZ (DEG), DEVI (DEG), HAZI (DEG), C1 (INCH), C2 (INCH), FEP (VOLT), RB (DEG)

Name	Service ID	Service Order	Units	API	File	Size	Rep Code	Sub-channels	Values per frame	Shape [sc, sa, bu]
RHDT	HDT			00-000-00-0	3	90	234	15	90	(0, 16, 1), (1, 16, 1), (2, 16, 1), (3, 16, 1), (4, 16, 1), (5, 1, 1), (6, 1, 1), (7, 1, 1), (8, 1, 1), (9, 1, 1), (10, 1, 1), (11, 1, 1), (12, 1, 1), (13, 1, 1), (14, 1, 1)
P1AZ	HDT		DEG	00-000-00-0	3	4	68	1	1	(0, 1, 1)
DEVI	HDT		DEG	00-000-00-0	3	4	68	1	1	(0, 1, 1)
HAZI	HDT		DEG	00-000-00-0	3	4	68	1	1	(0, 1, 1)
C1	HDT		INCH	00-000-00-0	3	4	68	1	1	(0, 1, 1)
C2	HDT		INCH	00-000-00-0	3	4	68	1	1	(0, 1, 1)
FEP	HDT		VOLT	00-000-00-0	3	4	68	1	1	(0, 1, 1)
RB	HDT		DEG	00-000-00-0	3	4	68	1	1	(0, 1, 1)

Then there is a couple of tables, the first summarises the X axis and the second summarises each channel (min, max mean etc.), for example:

LIS analysis of ../../../../TDTestData/LIS/pLogicTestData_med/LIS/13615.S3

Overview — Documentation Apple Yahoo! Google Maps YouTube Wikipedia News (225) Popular ▾

X Axis Information

	Value
From	17119.000 (FEET)
To	16632.883 (FEET)
Interval	-486.117 (FEET)
Total number of frames	1824
Overall frame spacing	-0.267 (FEET)
Original recording units	b'.1IN'

Frame Data

Sc Name	Units	Count	Min	Mean	Max	StdDev	--	==	++
FC0		29184	1	100.457	255	79.2557	9633	9633	9917
FC1		29184	0	88.7988	255	82.9932	10219	8393	10571
FC2		29184	0	93.4661	255	83.5306	9801	9250	10132
FC3		29184	1	114.45	255	76.1916	9543	9686	9954
FC4		29184	1	92.0765	255	83.5217	9908	9148	10127
STAT		1824	0	0	0	nan	0	1823	0
REF		1824	0	0	0	nan	0	1823	0
REFC		1824	0	0	0	nan	0	1823	0
EMEX		1824	0	0	0	nan	0	1823	0
PADP		1824	0	0	0	nan	0	1823	0
TEMP		1824	0	0	0	nan	0	1823	0
FEP1		1824	0	0	0	nan	0	1823	0
FEP2		1824	0	0	0	nan	0	1823	0
RAC1		1824	0	0	0	nan	0	1823	0
RAC2		1824	0	0	0	nan	0	1823	0
PIAZ	DEG	1824	1.69282	173.73	359.983	104.172	78	41	1704
DEVI	DEG	1824	-0.2	4.11568	5.8	1.09273	41	1713	69
HAZI	DEG	1824	127.8	215.584	347.4	30.3939	256	1288	279
C1	INCH	1824	4	6.40827	8.51	0.531151	173	1486	164
C2	INCH	1824	4	6.49762	8.51	0.51583	180	1473	170
FEP	VOLT	1824	1	172.815	194	45.16	198	1451	174
RB	DEG	1824	0	187.687	360	103.195	63	69	1691

[To top](#)

5.4.2 Plots LIS log data as SVG pages.

TODO:

Generates HTML from input LIS file or directory to an output destination.

Arguments

1. The path to the input LIS file or directory.
2. The path to the output file or directory, any directories will be created as necessary.

Options

Option	Description
<code>--version</code>	Show program's version number and exit
<code>-h, --help</code>	Show this help message and exit.
<code>-g, --glob</code>	File pattern match. [default none]
<code>-j JOBS, --jobs=JOBS</code>	Max processes when multiprocessing. Zero uses number of native CPUs [8]. -1 disables multiprocessing. [default: -1]
<code>-k, --keep-going</code>	Keep going as far as sensible. [default: False]
<code>-l LOGLEVEL, --loglevel=LOGLEVEL</code>	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]
<code>-v, --verbose</code>	Verbose output, this outputs a representation of table data and DFSRs.
<code>-r, --recursive</code>	Process input recursively. [default: False]

Examples

Command to process a directory of LIS:

```
$ ``tdlisthtml`` Simple LIS_plot/Simple_00
```

Output:

```
x
```

5.4.3 Scanning Physical Records in LIS Files with `tdlisscanphysrec`

Scans a LIS79 file and reports the Physical Record structure.

Arguments

One argument that will be treated as a path to a LIS file.

Options

Option	Description
<code>--version</code>	Show program's version number and exit
<code>-h, --help</code>	Show this help message and exit.
<code>-k, --keep-going</code>	Keep going as far as sensible. [default: False]
<code>-l LOGLEVEL, --loglevel=LOGLEVEL</code>	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]

Examples

Example of scanning a non-TIF encoded file:

```
$ ``tdlisscanphysrec`` LIS.lis
Cmd: ScanPhysRec.py LIS.lis
PR:      tell()  Length    Attr  LD_len  RecNum  FilNum  ChkSum    LR Attr [Total LD]
----- start -----
PR: 0x      0      62 0x  0      58 ----- ----- ----- 0x80 0x00 [      58]
PR: 0x     3e    1024 0x  1    1020 ----- ----- ----- 0x22 0x00
PR: 0x    43e    1024 0x  3    1020 ----- ----- ----- + -- --
PR: 0x    83e    1024 0x  3    1020 ----- ----- ----- + -- --
PR: 0x    c3e    1024 0x  3    1020 ----- ----- ----- + -- --
PR: 0x   103e    1024 0x  3    1020 ----- ----- ----- + -- --
PR: 0x   143e    1024 0x  3    1020 ----- ----- ----- + -- --
PR: 0x   183e    1024 0x  3    1020 ----- ----- ----- + -- --
PR: 0x   1c3e    1024 0x  3    1020 ----- ----- ----- + -- --
PR: 0x   203e     34 0x  2     30 ----- ----- ----- + -- -- [ 8190]
PR: 0x   2060    304 0x  0    300 ----- ----- ----- 0x40 0x00 [   300]
PR: 0x   2190   1014 0x  0   1010 ----- ----- ----- 0x00 0x00 [  1010]
PR: 0x   2586   1014 0x  0   1010 ----- ----- ----- 0x00 0x00 [  1010]
PR: 0x   297c   1014 0x  0   1010 ----- ----- ----- 0x00 0x00 [  1010]
PR: 0x   2d72   1014 0x  0   1010 ----- ----- ----- 0x00 0x00 [  1010]
PR: 0x   3168   1014 0x  0   1010 ----- ----- ----- 0x00 0x00 [  1010]
PR: 0x   355e   1014 0x  0   1010 ----- ----- ----- 0x00 0x00 [  1010]
PR: 0x   3954   1014 0x  0   1010 ----- ----- ----- 0x00 0x00 [  1010]
PR: 0x   3d4a   1014 0x  0   1010 ----- ----- ----- 0x00 0x00 [  1010]
PR: 0x   4140    942 0x  0    938 ----- ----- ----- 0x00 0x00 [   938]
PR: 0x   44ee     62 0x  0     58 ----- ----- ----- 0x81 0x00 [   58]
PR: EOF
----- EOF -----
PR Count: 21
Histogram of Physical Record lengths:
Bytes
  34 [1] | ++++++++
  62 [2] | ++++++
 304 [1] | ++++++++
  942 [1] | ++++++++
1014 [8] | _
↪ ++++++
1024 [8] | _
↪ ++++++
CPU time =    0.001 (S)
Bye, bye!
```

First tdlisscanphysrec echo's the command line then it scans the Physical Records an writes out a table that has the following columns:

Heading	Description
tell()	The file position of the start of the Physical Record as a hex integer.
Length	The length of the Physical Record as a decimal integer.
Attr	The Physical Record Header attributes as a hex integer.
LD_len	The length of the logical data payload contained in this Physical Record.
RecNum	A record number from the Physical Record trailer if present, otherwise: -----
FilNum	A file number from the Physical Record trailer if present, otherwise: -----
ChkSum	A checksum from the Physical Record trailer if present, otherwise: -----
LR	Logical Record type from the Logical Record Header as a hex integer.
Attr	Logical Record attributes from the Logical Record Header as a hex integer. This is (almost?) always 0x00
[Total LD]	The total length of the logical data in the Logical Record if a terminator Physical Record, otherwise blank.

This is followed by an ASCII histogram of the lengths of all Physical Records with the following columns:

1. The size in bytes.
2. The frequency count.
3. A series of + that is proportionate to the frequency count.

If TIF markers are detected then the output adds TIF columns thus:

TIF	?	:	Type	Back	Next	PR:	tell()	Length	Attr	LD_len
→ RecNum	FilNum	ChkSum	LR Attr	[Total LD]						
----- start -----										
→										
TIF	True	>: 0x	0 0x	0 0x	4a	PR: 0x	0	62	0x 0	58
→	-----	-----	-----	0x80 0x00	[58]					
TIF	True	>: 0x	0 0x	0 0x	456	PR: 0x	4a	1024	0x 1	1020
→	-----	-----	-----	0x22 0x00						
TIF	True	>: 0x	0 0x	4a 0x	862	PR: 0x	456	1024	0x 3	1020
→	-----	-----	-----	+ --	--					
TIF	True	>: 0x	0 0x	456 0x	c6e	PR: 0x	862	1024	0x 3	1020
→	-----	-----	-----	+ --	--					
TIF	True	>: 0x	0 0x	862 0x	107a	PR: 0x	c6e	1024	0x 3	1020
→	-----	-----	-----	+ --	--					
TIF	True	>: 0x	0 0x	c6e 0x	1486	PR: 0x	107a	1024	0x 3	1020
→	-----	-----	-----	+ --	--					
TIF	True	>: 0x	0 0x	107a 0x	1892	PR: 0x	1486	1024	0x 3	1020
→	-----	-----	-----	+ --	--					
TIF	True	>: 0x	0 0x	1486 0x	1c9e	PR: 0x	1892	1024	0x 3	1020
→	-----	-----	-----	+ --	--					
TIF	True	>: 0x	0 0x	1892 0x	20aa	PR: 0x	1c9e	1024	0x 3	1020
→	-----	-----	-----	+ --	--					
TIF	True	>: 0x	0 0x	1c9e 0x	20ec	PR: 0x	20aa	54	0x 2	50
→	-----	-----	-----	+ --	-- [8210]					

The additional columns are:

Heading	Description
?	?
Type	TIF marker type, 0 for in-file record, 1 for EOF.
Back	The file position of the precious TIF marker as a hex integer.
Next	The file position of the next TIF marker as a hex integer.

5.4.4 Scanning Logical Records in LIS Files with `tdlisscanlogirecord`

Scans a LIS79 file and reports the Logical Record structure.

Arguments

One argument that will be treated as a path to a LIS file.

Options

Option	Description
<code>--version</code>	Show program's version number and exit
<code>-h, --help</code>	Show this help message and exit.
<code>-k, --keep-going</code>	Keep going as far as sensible. [default: False]
<code>-l LOGLEVEL,</code> <code>--loglevel=LOGLEVEL</code>	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]
<code>-v, --verbose</code>	Verbose output, this outputs a representation of table data and DFSRs.

Examples

Example of scanning a LIS file:

```
$ tdlisscanlogirecord RW.lis
Cmd: ScanLogiRec.py RW.lis
0x00000000 <TotalDepth.LIS.core.LogiRec.LrFileHeadRead object at 0x1007981d0>: "File
↳header"
2012-02-08 17:43:45,078 WARNING LrTableRead(): Discarding duplicate row b'BS7 ' in
↳table b'CONS'
2012-02-08 17:43:45,087 WARNING LrTableRead.__init__(): Tell: 0x4a LD index: 0x32
↳Error: FileRead.unpack(): Bytes: b'\x00' not enough for struct that needs: 12 bytes.
0x0000004a <TotalDepth.LIS.core.LogiRec.LrTableRead object at 0x100798210>: "Well site
↳data"
0x000020ec <TotalDepth.LIS.core.LogiRec.LrDFSRRead object at 0x1007981d0>: "Data format
↳specification record"
0x0006141c <TotalDepth.LIS.core.LogiRec.LrFileTailRead object at 0x10058e7d0>: "File
↳trailer"
0x00061466 <TotalDepth.LIS.core.LogiRec.LrFileHeadRead object at 0x10058e850>: "File
↳header"
2012-02-08 17:43:45,103 WARNING LrTableRead(): Discarding duplicate row b'BS7 ' in
↳table b'CONS'
0x000614b0 <TotalDepth.LIS.core.LogiRec.LrTableRead object at 0x10058e7d0>: "Well site
↳data"
```

(continues on next page)

(continued from previous page)

```

0x0006353e <TotalDepth.LIS.core.LogiRec.LrDFSRRead object at 0x10058e850>: "Data format.
↳ specification record"
0x00065a44 <TotalDepth.LIS.core.LogiRec.LrFileTailRead object at 0x10058e850>: "File.
↳ trailer"
0x00065a8e <TotalDepth.LIS.core.LogiRec.LrFileHeadRead object at 0x10058e7d0>: "File.
↳ header"
2012-02-08 17:43:45,116 WARNING LrTableRead(): Discarding duplicate row b'BS7 ' in
↳ table b'CONS'
2012-02-08 17:43:45,124 WARNING LrTableRead.__init__(): Tell: 0x65ad8 LD index: 0x32.
↳ Error: FileRead.unpack(): Bytes: b'\x00' not enough for struct that needs: 12 bytes.
0x00065ad8 <TotalDepth.LIS.core.LogiRec.LrTableRead object at 0x10058e850>: "Well site.
↳ data"
0x00067b7a <TotalDepth.LIS.core.LogiRec.LrDFSRRead object at 0x10058e7d0>: "Data format.
↳ specification record"
0x000d2c44 <TotalDepth.LIS.core.LogiRec.LrFileTailRead object at 0x10058e7d0>: "File.
↳ trailer"
CPU time = 0.064 (S)
Bye, bye!

```

5.4.5 Scanning Logical Data in LIS Files with `tdlisscanlogidata`

Scans a LIS79 file and reports the Logical Record structure.

Arguments

One argument that will be treated as a path to a LIS file.

Options

Option	Description
<code>--version</code>	Show program's version number and exit
<code>-h, --help</code>	Show this help message and exit.
<code>-k, --keep-going</code>	Keep going as far as sensible. [default: False]
<code>-d DUMP, --dump=DUMP</code>	Dump complete data at these integer positions (ws separated, hex/dec). [default:]
<code>-l LOGLEVEL, --loglevel=LOGLEVEL</code>	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]
<code>-v, --verbose</code>	Verbose output, this outputs a representation of table data and DFSRs.

Examples

Example of scanning a LIS file:

```
$ ``tdlisscanlogidata`` LIS.lis
Cmd: ScanLogiData.py LIS.lis
Offset      Length  Type  Logical Data
0x00000000      58   128  b'\x80\x00RUN1R .S01\x00\x00DAT2TF          '...
0x0000003E     8190    34  b'" \x00IA\x04\x00TYPE  CONS\x00A\x04\x00MNEM  HI'...
0x00002060      300    64  b'@\x00\x01\x020\x00\x00\x02\x020\x00\x00\x03\x04I\x00\x00\
↳x00\x18\x04\x020\x00\x01\x08\x04D?N\x07_\t'...
0x00002190     1010     0  b"\x00\x00F@'\xde\xbe76\xfb@\xd6\x1a\xc0@\xd0\xdc\xcd\xce0\
↳xa6P\xba\x83\x18\x00F@&\xa6\xbe-"...
0x00002586     1010     0  b'\x00\x00E\xff\xe9S\xbe:\x1f\x82@\xfe%\xc9@\xf7\xd5\xb7EA\
↳x90\xda\xba\x83\x18\x00E\xff\xe6\xe3\xbe\x8a'...
0x0000297C     1010     0  b'\x00\x00E\xff\x82\xea\xbe-\xe1\xa8@\xd83\xb6@\xf0\x0f\
↳x0fET\x149D\xc8\x08\xc5E\xff\x80y\xbe-'...
0x00002D72     1010     0  b'\x00\x00E\xff\x1c\x80\xbd\xba\x7f\x19@\xc4\xbf\xe8@y\x0b\
↳xb3E\xc0\x08\x03D\xd5\xednE\xff\x1a\x10\xbd\xba4'...
0x00003168     1010     0  b'\x00\x00E\xfe\x8b\x16\xbe\x12\xde\x0f@\xcbl\xe7@zF\xc2Ew\
↳xba/D\xd0\xca\xd9E\xfe\x8b\x16\xbe\x12'...
0x0000355E     1010     0  b'\x00\x00E\xfe0\xac\xbe40\x85@\xcc4\x8d@of\xd9E\xc1F\xd8D\
↳xd4\xaa+E\xfeM<\x8b6'...
0x00003954     1010     0  b"\x00\x00E\xfd\xe9C\xbd\x82\x19\x0f\xba\x83\x18\x00AK'%D\
↳xed\x9d\xbdD\xd0\x17RE\xfd\xe6\xd3\xbd\xba"...
0x00003D4A     1010     0  b'\x00\x00E\xfd\x82\xd9\xba\x83\x18\x00\xba\x83\x18\x00\xba\
↳x83\x18\x00\xba\x83\x18\x00D\xd0\xad\x0f3E\xfd\x80i\xba\x83'...
0x00004140      938     0  b'\x00\x00E\xfd\x1c0\xba\x83\x18\x00\xba\x83\x18\x00\xba\x83\
↳x18\x00\xba\x83\x18\x00D\xd8\x8c\x8b5E\xfd\x19\xff\xba\x83'...
0x000044EE      58     0  b'\x81\x00RUN1R .S01\x00\x00DAT2TF          '...
Histogram of Logical Data lengths:
Bytes
  58 [1] | ++++++++
 300 [1] | ++++++++
 938 [1] | ++++++++
1010 [8] | _
↳ ++++++++
8190 [1] | ++++++++
Histogram of Logical Record types:
  0 [9] | _
↳ ++++++++
 34 [1] | ++++++++
 64 [1] | ++++++++
128 [1] | ++++++++
CPU time = 0.001 (S)
Bye, bye!
```

First tdlisscanlogidata echo's the command line then it scans the file an writes out a table that has the following columns:

Head- ing	Description
Offset	The file position of the start of the Physical Record as a hex integer.
Length	The length of the Logical Record as a decimal integer.
Type	The Logical Record type as a decimal integer.
Logical Data	The logical data payload. Only the first 32 bytes are shown. . . . is shown if the payload is longer than 32 bytes. If the verbose or dump options are given then all bytes are shown.

This is followed by an ASCII histogram of the lengths of all logical data with the following columns:

1. The size in bytes.
2. The frequency count.
3. A series of + that is proportionate to the frequency count.

This is followed by an ASCII histogram of the lengths of all Logical Record types with the following columns:

1. The size in bytes.
2. The frequency count.
3. A series of + that is proportionate to the frequency count.

Using the -d option expands the output when the file position value matches. So given the above then adding -d 0x44EE changes this:

```
...
0x000044EE      58      0  b'\x81\x00RUN1R .S01\x00\x00DAT2TF      '...
...
```

To this:

```
...
0x000044EE      58      0  b'\x81\x00RUN1R .S01\x00\x00DAT2TF      \x00 1024\
  ↳\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
...
```

5.4.6 Extracting Data from LIS with `tdlisdumpframeset`

Reads a LIS file and writes out tab separated values of each frame.

Arguments

1. The path to the LIS file.

Options

Option	Description
--version	Show program's version number and exit
-h, --help	Show this help message and exit.
-k, --keep-going	Keep going as far as sensible. [default: False]
-l LOGLEVEL, --loglevel=LOGLEVEL	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]
-s, --summary	Display summary only [default: False].

Examples

```
$ ``tdlisdumpframeset`` LIS.lis
Cmd: DumpFrameSet.py LIS.lis
2012-02-09 08:41:38,372 INFO      Index.indexFile(): LIS.lis
<TotalDepth.LIS.core.LogPass.LogPass object at 0x101a0c510>
b'DEPT' [b'M ' ] b'SP ' [b'MV ' ] b'SN ' [b'OHMM' ] b'ILD ' [b'OHMM' ] b'CILD'
↳ [b'MMHO' ] b'DT ' [b'US/M' ]
2052.98 -4.54908      1.34538 1.26348 386.599 -999.25
2052.83 -5.1372      1.36062 1.29521 500.511 -999.25
2052.68 -6.66747      1.38543 1.45786 595.623 -999.25
2052.53 -6.69616      1.43226 1.61085 592.447 -999.25
2052.37 -4.93782      1.51647 1.6622  590.846 -999.25
2052.22 -4.38823      1.66883 1.70584 586.092 -999.25
2052.07 -4.70347      1.8102  1.70607 577.873 -999.25
...
1996.44 -999.25      -999.25 -999.25 -999.25 -999.25
1996.29 -999.25      -999.25 -999.25 -999.25 -999.25
1996.14 -999.25      -999.25 -999.25 -999.25 -999.25
1995.99 -999.25      -999.25 -999.25 -999.25 -999.25

Sc Name      Count      Min      Mean      Max Std Dev.      --      ==      ++
↳ Bias      Drift Activity
DEPT [M ]      375      2e+03 2.02e+03 2.05e+03      16.5      374      0      0
↳ 1      -0.152 0.000144
SP [MV ]      262      -13.7      -5.67      -0.769      2.66      124      0      137 -
↳ 0.0498 0.0144 0.678
SN [OHMM]      252      0.866      1.36      1.98      0.277      123      0      128 -
↳ 0.0199 -0.000719 0.0425
ILD [OHMM]      253      0.361      1.31      2.35      0.412      95      0      157 -
↳ 0.246 0.00429 0.134
CILD [MMHO]      253      387      787 1.75e+03      236      130      0      122
↳ 0.0317 0.205 0.101
DT [US/M]      292      133      320      460      42.5      139      0      152 -
↳ 0.0447 -0.451 0.106
CPU time = 0.047 (S)
Bye, bye!
```

The summary table at the end has the following columns:

Heading	Description
Sc Name	The sub-channel name and units of measure.
Count	The number of non-null values.
Min	Minimum value.
Mean	Arithmetic mean of values.
Max	Maximum value.
Std Dev.	Standard deviation of values.
--	Number of values that are a decrease over the previous value.
==	Number of values that are equal to the previous value.
++	Number of values that are an increase over the previous value.
Bias	(-- - ++) / total
Drift	(last value - first value) / number of values
Activity	The RMS exponent change.

5.4.7 Analysing Table Data in LIS Files with `tdlistablehistogram`

Provides a count of elements in LIS tables.

Arguments

1. A path to a LIS file or directory of LIS files.

Options

Option	Description
--version	Show program's version number and exit
-h, --help	Show this help message and exit.
-k, --keep-going	Keep going as far as sensible. [default: False]
-r, --recursive	Process input recursively. [default: False]
-s, --structure	Display table structure (row/col range). [default: False]
--type=LRTYPE	Logical record table type e.g. 34. [default: 34]
--name=NAME	Logical record table name e.g. PRES. [default:]
--row=ROW	Logical record table row e.g. "GR ". [default:]
--col=COL	Logical record table column e.g. "LEDG". [default:]
-l LOGLEVEL, --loglevel=LOGLEVEL	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]

Examples

Count of all entries regardless of the table/row/column that they appear in:

```
$ ``tdlistablehistogram`` -l 40 Simple/
Cmd: TableHistogram.py -l 40 Simple/
===== Count of all table entries =====
{"(34, b'      ')": 1414,
 "(34, b'0.445')": 5,
 "(34, b'0.621')": 5,
 "(34, b'013529700231')": 7,
```

(continues on next page)

(continued from previous page)

```

"(34, b'1')": 5,
"(34, b'1.22')": 5,
"(34, b'1.70')": 2,
"(34, b'116')": 2,
"(34, b'12.25')": 2,
"(34, b'15')": 5,
"(34, b'15-4-76')": 5,
"(34, b'17')": 5,
"(34, b'17.5')": 5,
"(34, b'19')": 5,
"(34, b'1976')": 7,
"(34, b'2')": 2,
"(34, b'2055.0')": 2,
"(34, b'2071.2')": 4,
"(34, b'25')": 2,
"(34, b'25-6-76')": 2,
"(34, b'257.0')": 7,
...
"(34, b'WN  ')": 7,
"(34, b'YEAR')": 7,
'(34,)': 443}
===== Count of all table entries END =====
CPU time =    0.205 (S)
Bye, bye!

```

The result is a dictionary that has the key as a pair (lr_type, cell_value) and the value as a count of the number of occurrences.

If the -s option is used then an additional summary is provided:

```

===== Row entries =====
{(34, b'CONS', b'APIN'): 7,
 (34, b'CONS', b'BLI  '): 7,
 (34, b'CONS', b'BS1  '): 7,
 (34, b'CONS', b'BS2  '): 7,
 (34, b'CONS', b'BS3  '): 7,
 ...
 (34, b'CONS', b'WN   '): 7,
 (34, b'CONS', b'YEAR'): 7}
===== Row entries END =====
===== Column entries =====
{(34, b'CONS', b'ALLO'): 707,
 (34, b'CONS', b'MNEM'): 707,
 (34, b'CONS', b'PUNI'): 707,
 (34, b'CONS', b'TUNI'): 707,
 (34, b'CONS', b'VALU'): 707}
===== Column entries END =====

```

These are dictionaries that have the key as a tripple (lr_type, table_name, row_name) and (lr_type, table_name, column_name) respectively and the value as a count of the number of occurrences.

Filtering by Logical Record type, table name, row name and column name (note quoting of spaces):

```
$ ``tdlistablehistogram`` -l 40 --type=34 --name=CONS --row="WN " --col=VALU Simple/
Cmd: TableHistogram.py -l 40 --type=34 --name=CONS --row=WN --col=VALU Simple/
===== Count of all table entries =====
{'(34, b'CONS', b'WN ', b'VALU', b'B897 - 14')': 1,
 '(34, b'CONS', b'WN ', b'VALU', b'DIEKSAND 111A')': 3,
 '(34, b'CONS', b'WN ', b'VALU', b'VOELKERSEN AZ4')': 3}
===== Count of all table entries END =====
CPU time = 0.174 (S)
Bye, bye!
```

The result is a dictionary that has the key as a quadruple (lr_type, table_name, row_name, column_name, cell_value) and the value as a count of the number of occurrences.

5.4.8 Indexing LIS Files with `tdlisindex`

This indexes a LIS file and prints out the result. It can also provide some performance measurements of the indexing operation. See [Indexing LIS Files](#) for more information about the design and performance of LIS indexing.

Arguments

1. The path to a LIS file or a directory of LIS files.

Options

Option	Description
<code>--version</code>	Show program's version number and exit
<code>-h, --help</code>	Show this help message and exit.
<code>-k, --keep-going</code>	Keep going as far as sensible. [default: False]
<code>-l LOGLEVEL, --loglevel=LOGLEVEL</code>	Log Level (debug=10, info=20, warning=30, error=40, critical=50) [default: 20]
<code>-j JOBS, --jobs=JOBS</code>	Max processes when multiprocessing. Zero uses number of native CPUs [8]. -1 disables multiprocessing. [default: 1]
<code>-t TIMES, --times=TIMES</code>	Number of times to repeat the read [default: 1]
<code>-s, --statistics</code>	Dump timing statistics. [default: False]
<code>-v, --verbose</code>	Verbose output, this outputs a representation of table data and DFSRs.
<code>-r, --recursive</code>	Process input recursively. [default: False]

Examples

Simple `tif_scan_path` of a single file:

```
$ ``tdlisindex`` Simple/LIS.lis
Cmd: Index.py Simple/LIS.lis
2012-02-09 09:36:28,039 INFO      Index.indexFile(): Simple/LIS.lis
File size: 17708 (0.017 MB) Reference Time: 0.002459 (s) for Simple/LIS.lis
↳ pickleLen=4351 jsonLen=-1
Summary:
```

(continues on next page)

(continued from previous page)

```
Results:      1
Errors:       0
Total:        1
CPU time =    0.004 (S)
Bye, bye!
```

Simple tif_scan_path of a single file with verbose output:

```
$ ``tdlisindex`` -v Simple/LIS.lis
Cmd: Index.py -v Simple/LIS.lis
2012-02-09 09:39:29,493 INFO      Index.indexFile(): Simple/LIS.lis
<TotalDepth.LIS.core.FileIndexer.FileIndex object at 0x10197fdd0> "Simple/LIS.lis" [4]:
  tell: 0x00000000 type=128 <TotalDepth.LIS.core.FileIndexer.IndexFileHead object at 0x10197fe10>
  tell: 0x0000003e type= 34 name=b'CONS' <TotalDepth.LIS.core.FileIndexer.IndexTable object at 0x10197fe90>
  <TotalDepth.LIS.core.LogPass.LogPass object at 0x101b071d0>
  tell: 0x000044ee type=129 <TotalDepth.LIS.core.FileIndexer.IndexFileTail object at 0x101b07790>

===== All records =====
tell: 0x00000000 type=128 <TotalDepth.LIS.core.FileIndexer.IndexFileHead object at 0x10197fe10>
tell: 0x0000003e type= 34 name=b'CONS' <TotalDepth.LIS.core.FileIndexer.IndexTable object at 0x10197fe90>
<TotalDepth.LIS.core.LogPass.LogPass object at 0x101b071d0>
tell: 0x000044ee type=129 <TotalDepth.LIS.core.FileIndexer.IndexFileTail object at 0x101b07790>

===== All records DONE =====
===== Log Passes =====
LogPass <TotalDepth.LIS.core.LogPass.LogPass object at 0x101b071d0>:
  DFSR: <TotalDepth.LIS.core.LogiRec.LrDFSRRead object at 0x10197ff90>: "Data format specification record"
  Frame plan: <TotalDepth.LIS.core.Type01Plan.FrameSetPlan object at 0x101b07210>: indr=0 frame length=24 channels=6
    Channels: [b'DEPT', b'SP ', b'SN ', b'ILD ', b'CILD', b'DT ']
    RLE: <TotalDepth.LIS.core.Rle.RLEType01 object at 0x101b07250>: func=None: [RLEItemType01: datum=8592 stride=1014 repeat=7 frames=42, RLEItemType01: datum=16704 stride=None repeat=0 frames=39]
    X axis: first=2052.983 last=1995.986 frames=375 overall spacing=-0.1524 in optical units=b'M ' (actual units=b'M ')
  Frame set: None

===== Log Passes DONE =====
===== Plot Records =====
===== Plot Records DONE =====

  Min: 0.003 (s)
  Max: 0.003 (s)
  Mean: 0.003 (s)
File size: 17708 (0.017 MB) Reference Time: 0.002529 (s) for Simple/LIS.lis
pickleLen=4351 jsonLen=-1
Summary:
Results:      1
```

(continues on next page)

(continued from previous page)

```

Errors:      0
Total:      1
CPU time =   0.004 (S)
Bye, bye!

```

Scan of a directory (recursively) indexing each file 11 times and writing out statistics:

```

$ ``tdlindex`` -t11 -s -l 40 Simple/
Cmd: Index.py -t11 -s -l 40 ../../../../TDTestData/LIS/Simple
File size: 17708 (0.017 MB) Reference Time: 0.001670 (s) for Simple/LIS.lis
↪pickleLen=4351 jsonLen=-1
File size: 863374 (0.823 MB) Reference Time: 0.043411 (s) for Simple/RW.lis
↪pickleLen=18231 jsonLen=-1
File size: 853030 (0.814 MB) Reference Time: 0.039238 (s) for Simple/RW_No_TIF.lis
↪pickleLen=18238 jsonLen=-1
Summary:
Size (kb)   Time (s)
17.293      0.001670
843.139     0.043411
833.037     0.039238

Files: 3
Errors: 0
CPU time =   0.938 (S)
Bye, bye!

```

5.4.9 tdxlrandomframesetread

For developers only. This may not be present in some distributions. This is designed to measure the performance of loading and iterating across a frame-set.

5.4.10 Converting LIS Files to LAS Files with tdlistolas

This takes a LIS file or directory of them and writes out a set of LAS files. A single LAS file is written for each Log Pass so a single LIS file produces one or more LAS files.

The frames in the log pass can be sub-sampled by using `--frame-slice` which speeds things up when processing large files. The `--channels` option can be used to limit channels.

Where a channel has multiple values, and LAS can only record a single value, then the `--array-reduction` flag can be used to specify how the single value is computed. The allowable values are `{first,max,mean,median,min}` and the default is `mean`.

LAS File Naming Convention

One LIS file produces one or more LAS files. LAS file names are of the form:

```
{LIS_File_no_extension}_{logical_file_number}.las
```

Processing a Single LIS File

Given the path out the LAS files will be named {path_out}_{logical_file_number}.las

For example `tdlistolas foo.lis bar/baz` might create:

```
bar/baz_0.las
bar/baz_1.las
```

and so on.

Processing a Directory of LIS Files

Given the path out the LAS files will be named:

```
{path_out}/{LIS_File}_{logical_file_number}.las
```

For example `tdlistolas foo/ bar/baz` might create:

```
bar/baz/bit_0.las
bar/baz/bit_1.las
```

and so on.

The output directory structure will mirror the input directory structure.

Arguments

The first argument is the path to a LIS file or directory. The second argument is the path to write the output to.

Options

- h, --help** show this help message and exit
- version** show program's version number and exit
- k, --keep-going** Keep going as far as sensible. Default: False.
- v, --verbose** Increase verbosity, additive [default: 0]
- r, --recurse** Process the input recursively. Default: False.
- l LOG_LEVEL, --log-level LOG_LEVEL** Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 20]
- j JOBS, --jobs JOBS** Max processes when multiprocessing. Zero uses number of native CPUs [8]. Negative value disables multiprocessing code. Default: -1.

- frame-slice FRAME_SLICE** Do not process all frames but sample or slice the frames. SAMPLE: Sample is of the form “N” so a maximum of N frames, roughly regularly spaced, will be processed. N must be +ve, non-zero integer. Example: “64” - process a maximum of 64 frames. SLICE: Slice the frames is of the form start,stop,step as a comma separated list. Values can be absent or “None”. Examples: “,” - every frame, “,2” - every other frame, “,10,” - frames 0 to 9, “4,10,2” - frames 4, 6, 8, “40,-1,4” - every fourth frame from 40 to the end. Results will be truncated by frame array length. Use “?” to see what frames are available [default: “,” i.e. all frames]
- log-process LOG_PROCESS** Writes process data such as memory usage as a log INFO line every LOG_PROCESS seconds. If 0.0 no process data is logged. [default: 0.0]
- gnuplot GNUPLOT** Directory to write the gnuplot data.
- array-reduction ARRAY_REDUCTION** Method to reduce multidimensional channel data to a single value. One of { first,max,mean,median,min } [default: first]
- channels CHANNELS** Comma separated list of channels to write out (X axis is always included). Use “?” to see what channels exist without writing anything. [default: “”]
- field-width FIELD_WIDTH** Field width for array data [default: 16].
- float-format FLOAT_FORMAT** Floating point format for array data [default: “.3f”].

Examples

Finding out what Channels and Frames Exist:

Use `--channels=?` and/or `--frame-slice=?` to see what channels and frames exist in the LIS file.

```
$ tdlistolas --channels=? --frame-slice=? example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-
↳049.LIS tmp/scrap/
===== File example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.LIS =====
Log pass 1:
Available channels: ['FC0 ', 'FC1 ', 'FC2 ', 'FC3 ', 'FC4 ', 'STAT', 'REF ', 'REFC',
↳ 'EMEX', 'PADP', 'TEMP', 'FEP1', 'FEP2', 'RAC1', 'RAC2', 'P1AZ', 'DEVI', 'HAZI', 'C1 ',
↳ 'C2 ', 'FEP ', 'RB ']
X axis: first=5280.792 last=5079.725 frames=755 overall spacing=-0.2667 in optical_
↳units=b'FEET' (actual units=b'.1IN')
===== END File example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.LIS =====
```

Processing a Single File

```
$ tdlistolas example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.LIS tmp/scrap/
Cmd: /Users/engun/venvs/TotalDepth36_00/bin/tdlistolas example_data/LIS/data/DILLSON-1_
↳WELL_LOGS_FILE-049.LIS tmp/scrap/
gnuplot version: "b'gnuplot 5.2 patchlevel 6'"
2020-09-01 13:31:24,159 - WriteLAS.py - 117 - 39415 - (MainThread) - INFO -
↳process_to_las(): Namespace(array_reduction='first', channels='', field_width=16,
↳float_format='.3f', frame_slice=',', gnuplot=None, jobs=-1, keepGoing=False, log_
↳level=20, log_process=0.0, path_in='example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.
↳LIS', path_out='tmp/scrap/', recurse=False, verbose=0)
2020-09-01 13:31:24,159 - WriteLAS.py - 93 - 39415 - (MainThread) - INFO -
↳index_dir_or_file(): "example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.LIS" to "tmp/
↳scrap/" recurse: False
2020-09-01 13:31:24,159 - ToLAS.py - 328 - 39415 - (MainThread) - INFO -
↳single_lis_file_to_las(): path_in: example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.
↳LIS path_out: tmp/scrap/
2020-09-01 13:31:24,161 - File.py - 254 - 39415 - (MainThread) - INFO -
↳Finding best PR settings for: <_io.BufferedReader name='example_data/LIS/data/DILLSON-
↳1_WELL_LOGS_FILE-049.LIS'>
2020-09-01 13:31:24,171 - File.py - 260 - 39415 - (MainThread) - INFO -
↳Best pad options, first of 5: <PhysicalRecordSettings(pad_modulo=0, pad_non_null:
↳False> giving 100 Physical Records.
2020-09-01 13:31:24,177 - ToLAS.py - 337 - 39415 - (MainThread) - INFO -
↳Reading LIS in example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.LIS
2020-09-01 13:31:24,177 - ToLAS.py - 338 - 39415 - (MainThread) - INFO -
↳Index.indexFile(): example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.LIS
2020-09-01 13:31:24,189 - ToLAS.py - 358 - 39415 - (MainThread) - INFO -
↳LIS Logical Files: [<TotalDepth.LIS.ToLAS.LisLogicalFile object at 0x111a2e390>]
2020-09-01 13:31:24,190 - ToLAS.py - 291 - 39415 - (MainThread) - INFO -
↳write_las_file(): path_in: example_data/LIS/data/DILLSON-1_WELL_LOGS_FILE-049.LIS path_
↳out: tmp/scrap/
2020-09-01 13:31:24,190 - ToLAS.py - 294 - 39415 - (MainThread) - INFO -
↳Writing to LAS tmp/scrap/_0.las
Input Output LAS Count Time Ratio ms/Mb Exception
↳
↳ Path
-----
↳
98,508 285,487 1 0.201 289.8% 2139.3 False "example_data/LIS/data/DILLSON-1_
↳WELL_LOGS_FILE-049.LIS"
Total files: 1
Failed files: 0
Execution time = 0.220 (S)
Out of 1 processed 1 files of total size 98,508 input bytes
Wrote 285,487 output bytes, ratio: 289.811% at 2344.7 ms/Mb
Execution time: 0.220 (s)
Bye, bye!
```

The LAS files look like this:

```
$ head -n20 tmp/scrap/_0.las
~Version Information Section
VERS. 2.0 : CWLS Log ASCII Standard - VERSION 2.0
```

(continues on next page)

(continued from previous page)

```

WRAP.          NO                      : One Line per depth step
PROD.          TotalDepth              : LAS Producer
PROG.          TotalDepth.LIS.ToLAS 0.1.1 : LAS Program name and version
CREA.          2020-09-01 12:31:24.190938 UTC : LAS Creation date [YYYY-mm-dd.
↳HH:MM:SS.us UTC]
SOURCE.        DILLSON-1_WELL_LOGS_FILE-049.LIS : LIS File Name
LOGICAL-FILE.  0                      : Logical File number in the LIS file
~Well Information Section
#MNEM.UNIT  Value      Description
#-----
STRT.FEET    5280.792    : START
STOP.FEET    5079.725    : STOP
STEP.FEET    -0.267     : STEP
NULL.        -999.250    : NULL VALUE
COUN.        N.A.       : County
CTRY.        AUSTRALIA   : COUNTRY
LATI.        21 23 06.314 : Latitude
LONG.        115 10 56.336 : Longitude
STAT.        WEST AUSTRALIA : STATE

```

Processing a Directory

Use the `-r` option to process recursively. The output directory will mirror the input directory.

```

$ tdrp66v1scanhtml -r example_data/LIS/data tmp/LAS
Input Output LAS Count Time Ratio ms/Mb Exception
↳-----
↳-----
↳-----
96,376 341,102      1 0.151 353.9% 1645.0      False "example_data/LIS/data/DILLSON-1_
↳WELL_LOGS_FILE-013.LIS"
184,084 741,708     1 0.300 402.9% 1709.6      False "example_data/LIS/data/DILLSON-1_
↳WELL_LOGS_FILE-037.LIS"
98,508 285,487      1 0.154 289.8% 1639.0      False "example_data/LIS/data/DILLSON-1_
↳WELL_LOGS_FILE-049.LIS"
Total files: 3
Failed files: 0

```

5.5 RP66V1 Command Line Tools

This describes the command line tools that are available for processing RP66V1 files. They are:

Tool Name	Description
<code>tdrp66v1scanhtml</code>	Scans RP66V1 file(s) and writes out a summary in HTML.
<code>tdrp66v1tolas</code>	Converts RP66V1 file(s) to a set of LAS files.
<code>tdrp66v1indexpickle</code>	Indexes RP66V1 file(s) and writes the indexes for future use as Python pickle files.
<code>tdrp66v1indexxml</code>	Indexes RP66V1 file(s) and writes the indexes as XML files.
<code>tdrp66v1scan</code>	Scans RP66V1 file at various levels of structure.

5.5.1 Creating HTML Pages from RP66V1 Files with `tdrp66v1scanhtml`

This takes a RP66V1 file or directory of them and writes out an HTML summary of each Logical File. The summary includes each non-encrypted EFLR and Log Pass. The frames in the log pass can be sub-sampled by using `--frame-slice` which speeds things up when processing large files.

Arguments

The first argument is the path to a RP66V1 file or directory. The second argument is the path to write the output to.

Options

- h, --help** show this help message and exit
- version** show program's version number and exit
- k, --keep-going** Keep going as far as sensible. Default: False.
- v, --verbose** Increase verbosity, additive [default: 0]
- r, --recurse** Process the input recursively. Default: False.
- l LOG_LEVEL, --log-level LOG_LEVEL** Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 20]
- j JOBS, --jobs JOBS** Max processes when multiprocessing. Zero uses number of native CPUs [8]. Negative value disables multiprocessing code. Default: -1.
- e, --encrypted** Output encrypted Logical Records as well. [default: False]
- frame-slice FRAME_SLICE** Do not process all frames but sample or slice the frames. SAMPLE: Sample is of the form "N" so a maximum of N frames, roughly regularly spaced, will be processed. N must be +ve, non-zero integer. Example: "64" - process a maximum of 64 frames. SLICE: Slice the frames is of the form start,stop,step as a comma separated list. Values can be absent or "None". Examples: ".,," - every frame, ".,2" - every other frame, ".,10," - frames 0 to 9, "4,10,2" - frames 4, 6, 8, "40,-1,4" - every fourth frame from 40 to the end. Results will be truncated by frame array length. [default: ".,," i.e. all frames]
- log-process LOG_PROCESS** Writes process data such as memory usage as a log INFO line every LOG_PROCESS seconds. If 0.0 no process data is logged. [default: 0.0]
- gnuplot GNUPLOT** Directory to write the gnuplot data.

Here is an example of the [HTML summary of a single RP66V1 file](#) .

5.5.2 Converting RP66V1 Files to LAS Files with `tdrp66v1tolas`

This takes a RP66V1 file or directory of them and writes out a set of LAS files. A single LAS file is written for each Log Pass in each Logical Record.

The frames in the log pass can be sub-sampled by using `--frame-slice` which speeds things up when processing large files. The `--channels` option can be used to limit channels.

Where a channel has multiple values, and LAS can only record a single value, then the `--array-reduction` flag can be used to specify how the single value is computed. The allowable values are `{first,max,mean,median,min}` and the default is `mean`.

LAS File Naming Convention

One RP66V1 file produces one or more LAS files. LAS file names are of the form:

`{RP66V1_File_no_extension}_{logical_file_number}_{frame_array_name}`

Processing a Single RP66V1 File

Given the path out the LAS files will be named `{path_out}_{logical_file_number}_{frame_array_name}.las`

For example `tdrp66v1tolas foo.dlis bar/baz` might create:

`bar/baz_0_2000T.las
bar/baz_0_800T.las
bar/baz_1_2000T.las
bar/baz_1_800T.las`

and so on.

Processing a Directory of RP66V1 Files

Given the path out the LAS files will be named:

`{path_out}/{RP66V1_File}_{logical_file_number}_{frame_array_name}.las`

For example `tdrp66v1tolas foo/ bar/baz` might create:

`bar/baz/bit_0_2000T.las
bar/baz/bit_0_800T.las
bar/baz/bit_1_2000T.las
bar/baz/bit_1_800T.las`

and so on.

The output directory structure will mirror the input directory structure.

Arguments

The first argument is the path to a RP66V1 file or directory. The second argument is the path to write the output to.

Options

- h, --help** show this help message and exit
- version** show program's version number and exit
- k, --keep-going** Keep going as far as sensible. Default: False.
- v, --verbose** Increase verbosity, additive [default: 0]
- r, --recurse** Process the input recursively. Default: False.
- l LOG_LEVEL, --log-level LOG_LEVEL** Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 20]
- j JOBS, --jobs JOBS** Max processes when multiprocessing. Zero uses number of native CPUs [8]. Negative value disables multiprocessing code. Default: -1.
- frame-slice FRAME_SLICE** Do not process all frames but sample or slice the frames. SAMPLE: Sample is of the form "N" so a maximum of N frames, roughly regularly spaced, will be processed. N must be +ve, non-zero integer. Example: "64" - process a maximum of 64 frames. SLICE: Slice the frames is of the form start,stop,step as a comma separated list. Values can be absent or "None". Examples: ".,," - every frame, ".,2" - every other frame, ".,10," - frames 0 to 9, "4,10,2" - frames 4, 6, 8, "40,-1,4" - every fourth frame from 40 to the end. Results will be truncated by frame array length. Use "?" to see what frames are available [default: ".,," i.e. all frames]
- log-process LOG_PROCESS** Writes process data such as memory usage as a log INFO line every LOG_PROCESS seconds. If 0.0 no process data is logged. [default: 0.0]
- gnuplot GNUPLOT** Directory to write the gnuplot data.
- array-reduction ARRAY_REDUCTION** Method to reduce multidimensional channel data to a single value. One of {first,max,mean,median,min} [default: first]
- channels CHANNELS** Comma separated list of channels to write out (X axis is always included). Use "?" to see what channels exist without writing anything. [default: ""]
- field-width FIELD_WIDTH** Field width for array data [default: 16].
- float-format FLOAT_FORMAT** Floating point format for array data [default: ".3f"].

Examples

Finding out what Channels and Frames Exist:

Use `--channels=?` and/or `--frame-slice=?` to see what channels and frames exist in the RP66V1 file.

```
$ tdrp66v1tolas --channels=? --frame-slice=? example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_
↳ 258276498.DLIS example_data/LAS/206_05a-_3_DWL_DWL_WIRE_258276498
Logical file [0000]: <TotalDepth.RP66V1.core.LogicalFile.LogicalFile object at 0x109fd50f0>
↳ 0x109fd50f0>
  Frame Array: b'2000T'
  Channels: b'TIME,TDEP,TENS_SL,DEPT_SL'
  X axis: FrameChannel: OBNAME: 0: 2 C: 4 I: b'TIME'                      Rc: 2 Co: 1 Un: b'ms
↳ '          Di: [1] b'1 second River Time'
  Frames: 921 from 16677259.0 to 17597260.0 interval 1000.0010869565217 [b'ms']

  Frame Array: b'800T'
  Channels: b'TIME,TDEP,ETIM,LMVL,UMVL,CFLA,OCD,RCMD,RCPP,CMRT,RCNU,DCFL,DFS,DZER,RHMD,
↳ HMRT,RHV,RLSW,MNU,S1CY,S2CY,RSCU,RSTS,UCFL,CARC,CMDV,CMPP,CNU,HMDV,HV,LSWI,SCUR,SSTA,
↳ RCMP,RHPP,RRPP,CMPP,HPPR,RPPV,SMSC,CMCU,HMCU,CMLP'
  X axis: FrameChannel: OBNAME: 0: 2 C: 5 I: b'TIME'                      Rc: 2 Co: 1 Un: b'ms
↳ '          Di: [1] b'400 milli-second time channel'
  Frames: 2301 from 16677259.0 to 17597260.0 interval 400.0004347826087 [b'ms']
```

Processing a Single File

```
$ tdrp66v1tolas example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS example_data/
↳ LAS/206_05a-_3_DWL_DWL_WIRE_258276498
  Input   Output LAS Count  Time  Ratio  ms/Mb Exception
↳ -----
  Path
  -----
540,372 1,812,131          2 1.816 335.3% 3524.1      False "example_data/RP66V1/206_05a-_
↳ 3_DWL_DWL_WIRE_258276498.DLIS"
Execution time = 1.819 (S)
Out of 1 processed 1 files of total size 540,372 input bytes
Wrote 1,812,131 output bytes, ratio: 335.349% at 3529.3 ms/Mb
$ ll example_data/LAS/
total 4600
-rw-r--r-- 1 xxxxxxxx staff 94317 28 Oct 10:19 206_05a-_3_DWL_DWL_WIRE_258276498_0_
↳ 2000T.las
-rw-r--r-- 1 xxxxxxxx staff 1717814 28 Oct 10:20 206_05a-_3_DWL_DWL_WIRE_258276498_0_
↳ 800T.las
```

The LAS files look like this:

```
$ head -n20 example_data/LAS/206_05a-_3_DWL_DWL_WIRE_258276498_0_2000T.las
~Version Information Section
VERS.          2.0                                : CWLS Log ASCII Standard -
↳ VERSION 2.0
WRAP.          NO                                : One Line per depth step
```

(continues on next page)

(continued from previous page)

```

PROD.          TotalDepth          : LAS Producer
PROG.          TotalDepth.RP66V1.ToLAS 0.1.1 : LAS Program name and version
CREA.          2019-10-28 10:30      : LAS Creation date [YYYY-mm-dd
↳HH:MM]
DLIS_CREA.     2011-08-20 22:48      : DLIS Creation date and time
↳[YYYY-mm-dd HH:MM]
SOURCE.        206_05a-_3_DWL_DWL_WIRE_258276498.DLIS : DLIS File Name
FILE-ID.       MSCT_197LTP          : File Identification Number
LOGICAL-FILE.  0                    : Logical File number in the DLIS
↳file
FRAME-ARRAY.   2000T                : Identity of the Frame Array in
↳the Logical File
~Well Information Section
#MNEM.UNIT  DATA                    DESCRIPTION
#-----
STRT.ms     16677259.0                : Start X
STOP.ms     17597260.0                : Stop X, frames: 921
STEP.ms     1000.0010869565217        : Step (average)
NULL.       :
COMP.       Faroe Petroleum           :
WELL.       206/05a-3                :

$ head -n20 example_data/LAS/206_05a-_3_DWL_DWL_WIRE_258276498_0_800T.las
~Version Information Section
VERS.        2.0                      : CWLS Log ASCII Standard -
↳VERSION 2.0
WRAP.        NO                       : One Line per depth step
PROD.        TotalDepth               : LAS Producer
PROG.        TotalDepth.RP66V1.ToLAS 0.1.1 : LAS Program name and version
CREA.        2019-10-28 10:30         : LAS Creation date [YYYY-mm-dd
↳HH:MM]
DLIS_CREA.   2011-08-20 22:48         : DLIS Creation date and time
↳[YYYY-mm-dd HH:MM]
SOURCE.      206_05a-_3_DWL_DWL_WIRE_258276498.DLIS : DLIS File Name
FILE-ID.     MSCT_197LTP              : File Identification Number
LOGICAL-FILE. 0                      : Logical File number in the DLIS
↳file
FRAME-ARRAY. 800T                    : Identity of the Frame Array in
↳the Logical File
~Well Information Section
#MNEM.UNIT  DATA                    DESCRIPTION
#-----
STRT.ms     16677259.0                : Start X
STOP.ms     17597260.0                : Stop X, frames: 2,301
STEP.ms     400.0004347826087         : Step (average)
NULL.       :
COMP.       Faroe Petroleum           :
WELL.       206/05a-3                :

```

Processing a Directory

Use the `-r` option to process recursively. The output directory will mirror the input directory.

```
$ tdrp66v1tolas -r example_data/ tmp/LAS
Input      Output LAS Count  Time  Ratio  ms/Mb Exception
↳          Path
-----
↳ -----
540,372 1,812,131      2 1.874 335.3% 3636.8      False "example_data/RP66V1/206_05a-_
↳ 3_DWL_DWL_WIRE_258276498.DLIS"
Execution time =    1.884 (S)
Out of 6 processed 1 files of total size 540,372 input bytes
Wrote 1,812,131 output bytes, ratio: 335.349% at 3655.1 ms/Mb
$ find tmp/LAS -name '*.las'
tmp/LAS/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498_0_800T.las
tmp/LAS/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498_0_2000T.las
```

5.5.3 Indexing RP66V1 Files with `tdrp66v1indexpickle`

`tdrp66v1indexpickle` reads a RP66V1 file and dumps the index to a pickle file.

Arguments

The first argument is the path to a RP66V1 file or directory. The second argument is the path to write the output to.

Options

- h, --help** show this help message and exit
- r, --recurse** Process recursively. [default: False]
- read-back** Read and time the output. [default: False]
- l LOG_LEVEL, --log-level LOG_LEVEL** Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 30]
- log-process LOG_PROCESS** Writes process data such as memory usage as a log INFO line every LOG_PROCESS seconds. If 0.0 no process data is logged. [default: 0.0]
- v, --verbose** Increase verbosity, additive [default: 0]
- gnuplot GNUPLOT** Directory to write the gnuplot data.

Examples

Processing a Single File

```
$ tdrp66v1indexpickle --read-back example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.
↳DLIS example_data/pickle/206_05a-_3_DWL_DWL_WIRE_258276498
Common path prefix: example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
Size (b) | Index (b) | Ratio (%) | Index (s) | Index (ms/Mb) | Read (s) | Read (ms/Mb) |
↳Except | Path
----- | ----- | ----- | ----- | ----- | ----- | ----- |
↳----- | ----
540,372 | 1,018,327 | 188.449% | 0.330 | 639.9 | 0.041 | 78.96 |
↳ False |
Execution time = 0.379 (S)
Out of 1 processed 1 files of total size 540,372 input bytes
Wrote 1,018,327 output bytes, ratio: 188.449% at 651.0 ms/Mb
$ ll example_data/pickle/
total 1992
-rw-r--r-- 1 xxxxxxxx staff 1018327 28 Oct 12:11 206_05a-_3_DWL_DWL_WIRE_258276498.pkl
```

Processing a Directory

Use the `-r` option to process recursively. The output directory will mirror the input directory.

5.5.4 Indexing RP66V1 Files with `tdrp66v1indexxml`

`tdrp66v1indexxml` reads a RP66V1 file and dumps the index to an XML file.

Arguments

The first argument is the path to a RP66V1 file or directory. The second argument is the path to write the output to.

Options

optional arguments:

- h, --help** show this help message and exit
- r, --recurse** Process files recursively. [default: False]
- p, --private** Also write out private EFLRs. [default: False]
- l LOG_LEVEL, --log-level LOG_LEVEL** Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 20]
- log-process LOG_PROCESS** Writes process data such as memory usage as a log INFO line every LOG_PROCESS seconds. If 0.0 no process data is logged. [default: 0.0]
- v, --verbose** Increase verbosity, additive [default: 0]
- gnuplot GNUPLOT** Directory to write the gnuplot data.

Examples

Processing a Single File

```
$ tdrp66v1indexxml example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS example_
↳data/XML/206_05a-_3_DWL_DWL_WIRE_258276498
2019-10-28 11:58:55,498 - 74153 - MainThread - INFO      - IndexXML.py      - index_dir_
↳or_file(): "example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS" to "example_
↳data/XML/206_05a-_3_DWL_DWL_WIRE_258276498" recurse: False
2019-10-28 11:58:55,499 - 74153 - MainThread - INFO      - IndexXML.py      - Making_
↳directory: example_data/XML
2019-10-28 11:58:55,499 - 74153 - MainThread - INFO      - IndexXML.py      - Indexing_
↳example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS to example_data/XML/206_05a-
↳_3_DWL_DWL_WIRE_258276498
2019-10-28 11:58:55,939 - 74153 - MainThread - INFO      - IndexXML.py      - Length of_
↳XML: 428622
-----
      Size In      Size Out      Time  Ratio %    ms/Mb Fail? Path
-----
      540,372      428,622    0.440  79.320%    854.6 False "example_data/RP66V1/
↳206_05a-_3_DWL_DWL_WIRE_258276498.DLIS"
Execution time =    0.443 (S)
Out of 1 processed 1 files of total size 540,372 input bytes
Wrote 428,622 output bytes, ratio: 79.320% at 860.4 ms/Mb
```

The XML looks something like this:

```
<?xml version='1.0' encoding="utf-8"?>
<RP66V1FileIndex creator="TotalDepth.RP66V1.core.Index" path="example_data/RP66V1/206_
↳05a-_3_DWL_DWL_WIRE_258276498.DLIS" schema_version="0.1.0" size="540372" utc_file_
↳mtime="2019-06-22 09:10:59.512253" utc_now="2019-10-28 11:58:55.799047">
  <StorageUnitLabel dlis_version="V1.00" maximum_record_length="8192" sequence_number="1
↳" storage_set_identifier="Default Storage Set
↳" storage_unit_structure="RECORD"/>
  <LogicalFiles count="1">
    <LogicalFile has_log_pass="True" index="0">
      <EFLR lr_type="0" lrsh_position="0x54" object_count="1" set_name="" set_type="FILE-
↳HEADER" vr_position="0x50">
        <Object C="0" I="5" O="2">
          <Attribute count="1" label="SEQUENCE-NUMBER" rc="20" rc_ascii="ASCII" units="">
            <Value type="bytes" value="      197"/>
          </Attribute>
          <Attribute count="1" label="ID" rc="20" rc_ascii="ASCII" units="">
            <Value type="bytes" value="MSCT_197LTP
↳" />
          </Attribute>
        </Object>
      </EFLR>
      <!-- More EFLRs ... -->
      <LogPass count="2">
        <FrameArray C="0" I="2000T" O="2" description="" x_axis="TIME" x_units="ms">
          <Channels count="4">
            <Channel C="4" I="TIME" O="2" count="1" dimensions="1" long_name="1 second_
↳River Time" rep_code="2" units="ms"/>
```

(continues on next page)

(continued from previous page)

```

    <Channel C="4" I="TDEP" O="2" count="1" dimensions="1" long_name="1 second_
↳River Depth" rep_code="2" units="0.1 in"/>
    <Channel C="0" I="TENS_SL" O="2" count="1" dimensions="1" long_name="Cable_
↳Tension" rep_code="2" units="lbf"/>
    <Channel C="0" I="DEPT_SL" O="2" count="1" dimensions="1" long_name="Station_
↳logging depth" rep_code="2" units="0.1 in"/>
  </Channels>
  <IFLR count="921">
    <FrameNumbers count="921" rle_len="1">
      <RLE datum="1" repeat="920" stride="1"/>
    </FrameNumbers>
    <LRSH count="921" rle_len="400">
      <RLE datum="0x13254" repeat="1" stride="0x190"/>
      <!-- ... -->
      <RLE datum="0x83ba4" repeat="1" stride="0x198"/>
    </LRSH>
    <Xaxis count="921" rle_len="2">
      <RLE datum="16677259.0" repeat="99" stride="1000.0"/>
      <RLE datum="16777260.0" repeat="820" stride="1000.0"/>
    </Xaxis>
  </IFLR>
</FrameArray>
<FrameArray C="0" I="800T" O="2" description="" x_axis="TIME" x_units="ms">
  <Channels count="43">
    <Channel C="5" I="TIME" O="2" count="1" dimensions="1" long_name="400 milli-
↳second time channel" rep_code="2" units="ms"/>
    <Channel C="5" I="TDEP" O="2" count="1" dimensions="1" long_name="MSCT depth_
↳channel" rep_code="2" units="0.1 in"/>
    <Channel C="1" I="ETIM" O="2" count="1" dimensions="1" long_name="Elapsed_
↳Logging Time" rep_code="2" units="s"/>
    <!-- ... -->
    <Channel C="0" I="HMCU" O="2" count="1" dimensions="1" long_name="Hydraulic_
↳Motor Current" rep_code="2" units="mA"/>
    <Channel C="0" I="CMLP" O="2" count="1" dimensions="1" long_name="Coring_
↳Motor Linear Position" rep_code="2" units="in"/>
  </Channels>
  <IFLR count="2301">
    <FrameNumbers count="2301" rle_len="1">
      <RLE datum="1" repeat="2300" stride="1"/>
    </FrameNumbers>
    <LRSH count="2301" rle_len="937">
      <RLE datum="0x13274" repeat="1" stride="0xb8"/>
      <!-- ... -->
      <RLE datum="0x83d5c" repeat="1" stride="0xbc"/>
    </LRSH>
    <Xaxis count="2301" rle_len="2">
      <RLE datum="16677259.0" repeat="249" stride="400.0"/>
      <RLE datum="16777260.0" repeat="2050" stride="400.0"/>
    </Xaxis>
  </IFLR>
</FrameArray>
</LogPass>

```

(continues on next page)

(continued from previous page)

```
</LogicalFile>
</LogicalFiles>
<VisibleRecords count="66" rle_len="15">
  <RLE datum="0x50" repeat="3" stride="0x2000"/>
  <!-- ... -->
  <RLE datum="0x81f70" repeat="0" stride="0x0"/>
</VisibleRecords>
</RP66V1FileIndex>
```

Processing a Directory

Use the `-r` option to process recursively. The output directory will mirror the input directory.

5.5.5 Scanning RP66V1 Files with `tdrp66v1scan`

`tdrp66v1scan` scans a RP66V1 file and dumps data about the file to stdout. This is useful for examining the details of RP66V1 files and can dump data at various levels of encapsulation, from the lowest level upwards:

- `--VR` - Visible Records only.
- `--LRSH` - Logical Record segments.
- `--LD` - Logical data i.e. all Logical Record segments concatenated for each Logical Record.
- `--EFLR` - Explicitly Formatted Logical Records.
- `--IFLR` - Implicitly Formatted Logical Records.
- `--LR` - All data, including the numerical analysis of frame data.

If these options are combined then the input is scanned, and reported, multiple times.

Arguments

The first argument is the path to a RP66V1 file. An optional second argument is the path to write the output to. If absent then output is written to stdout.

Options

-h, --help	show this help message and exit
-V, --VR	Dump the Visible Records. [default: False]
-L, --LRSH	Summarise the Visible Records and the Logical Record Segment Headers, use <code>-v</code> to dump records. [default: False]
-D, --LD	Summarise logical data, use <code>-v</code> to dump records. See also <code>--dump-bytes</code> , <code>--dump-raw-bytes</code> . [default: False]
-E, --EFLR	Dump EFLR Set. [default: False]
--eflr-set-type EFLR_SET_TYPE	List of EFLR Set Types to output, additive, if absent then dump all. [default: []]
-I, --IFLR	Dump IFLRs. [default: False]

--iflr-set-type IFLR_SET_TYPE List of IFLR Set Types to output, additive, if absent then dump all. [default: []]

-R, --LR Dump all data, including frame data from Logical Records. [default: False]

-d DUMP_BYTES, --dump-bytes DUMP_BYTES Dump X leading raw bytes for certain options, if -1 all bytes are dumped. [default: 0]

--dump-raw-bytes Dump the raw bytes for certain options in raw format, otherwise Hex format is used. [default: False]

-r, --recurse Process files recursively. [default: False]

-e, --encrypted Output encrypted Logical Records as well. [default: False]

-k, --keep-going Keep going as far as sensible. [default: False]

--frame-slice FRAME_SLICE NOTE: Requires -R, -LR. Do not process all frames but sample or slice the frames. SAMPLE: Sample is of the form "N" so a maximum of N frames, roughly regularly spaced, will be processed. N must be +ve, non-zero integer. Example: "64" - process a maximum of 64 frames. SLICE: Slice the frames is of the form start,stop,step as a comma separated list. Values can be absent or "None". Examples: ".,," - every frame, ".,2" - every other frame, ",10," - frames 0 to 9, "4,10,2" - frames 4, 6, 8, "40,-1,4" - every fourth frame from 40 to the end. Results will be truncated by frame array length. [default: ".,," i.e. all frames]

--eflr-as-table When with -LR and not -html then dump EFLRs as tables, otherwise every EFLR object. [default: False]

-l LOG_LEVEL, --log-level LOG_LEVEL Log Level as an integer or symbol. (0<->NOTSET, 10<->DEBUG, 20<->INFO, 30<->WARNING, 40<->ERROR, 50<->CRITICAL) [default: 30]

-v, --verbose Increase verbosity, additive [default: 0]

--gnuplot GNUPLOT Directory to write the gnuplot data.

-T, --test-data Dump the file as annotated bytes, useful for creating test data. [default: False]

Examples

Scanning Visible Records with --VR

Example of scanning a RP66V1 file:

```
$ tdrp66v1scan --VR example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
***** RP66V1 Visible and LRSH Records*****
↳ *****
===== Summary of Visible Records=====
↳ =====
Visible records: 66
----- RLE Visible Record Position -----
↳ -----
Datum:          80 0x000000050 Repeat:      3 Stride:  8,192 0x2000
```

(continues on next page)

(continued from previous page)

```

Datum:      32,844 0x0000804c Repeat:    5 Stride:  8,192 0x2000
Datum:      81,988 0x00014044 Repeat:    4 Stride:  8,192 0x2000
Datum:     122,940 0x0001e03c Repeat:   10 Stride:  8,192 0x2000
Datum:     213,016 0x00034018 Repeat:    3 Stride:  8,192 0x2000
Datum:     245,764 0x0003c004 Repeat:    3 Stride:  8,192 0x2000
Datum:     278,516 0x00043ff4 Repeat:    3 Stride:  8,192 0x2000
Datum:     311,268 0x0004bfe4 Repeat:    3 Stride:  8,192 0x2000
Datum:     344,020 0x00053fd4 Repeat:    2 Stride:  8,192 0x2000
Datum:     368,576 0x00059fc0 Repeat:    4 Stride:  8,192 0x2000
Datum:     409,524 0x00063fb4 Repeat:    2 Stride:  8,192 0x2000
Datum:     434,080 0x00069fa0 Repeat:    3 Stride:  8,192 0x2000
Datum:     466,832 0x00071f90 Repeat:    3 Stride:  8,192 0x2000
Datum:     499,584 0x00079f80 Repeat:    3 Stride:  8,192 0x2000
Datum:     532,336 0x00081f70 Repeat:    0 Stride:    0 0x0000
----- END RLE Visible Record Position -----
<-----
===== END Summary of Visible Records
<=====
***** END RP66V1 Visible and LRSH Records
<*****

```

And with the -v option:

```

$ tdrp66v1scan --VR -v example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
***** RP66V1 Visible and LRSH Records
<*****
<VisibleRecord: position=0x00000050 length=0x2000 version=0xff01> Stride: 0x00000050
<80
<VisibleRecord: position=0x00002050 length=0x2000 version=0xff01> Stride: 0x00002000 8,
<192
<VisibleRecord: position=0x00004050 length=0x2000 version=0xff01> Stride: 0x00002000 8,
<192
<VisibleRecord: position=0x00006050 length=0x1ffc version=0xff01> Stride: 0x00002000 8,
<192
<VisibleRecord: position=0x0000804c length=0x2000 version=0xff01> Stride: 0x00001ffc 8,
<188
<VisibleRecord: position=0x0000a04c length=0x2000 version=0xff01> Stride: 0x00002000 8,
<192
<VisibleRecord: position=0x0000c04c length=0x2000 version=0xff01> Stride: 0x00002000 8,
<192
<VisibleRecord: position=0x0000e04c length=0x2000 version=0xff01> Stride: 0x00002000 8,
<192
<VisibleRecord: position=0x0001004c length=0x2000 version=0xff01> Stride: 0x00002000 8,
<192
<VisibleRecord: position=0x0001204c length=0x1ff8 version=0xff01> Stride: 0x00002000 8,
<192
<VisibleRecord: position=0x00014044 length=0x2000 version=0xff01> Stride: 0x00001ff8 8,
<184
<VisibleRecord: position=0x00016044 length=0x2000 version=0xff01> Stride: 0x00002000 8,
<192
<VisibleRecord: position=0x00018044 length=0x2000 version=0xff01> Stride: 0x00002000 8,
<192

```

(continues on next page)

(continued from previous page)

```

<VisibleRecord: position=0x0001a044 length=0x2000 version=0xff01> Stride: 0x00002000 8,
↪192
<VisibleRecord: position=0x0001c044 length=0x1ff8 version=0xff01> Stride: 0x00002000 8,
↪192
<VisibleRecord: position=0x0001e03c length=0x2000 version=0xff01> Stride: 0x00001ff8 8,
↪184
<VisibleRecord: position=0x0002003c length=0x2000 version=0xff01> Stride: 0x00002000 8,
↪192
...
<VisibleRecord: position=0x0007df80 length=0x2000 version=0xff01> Stride: 0x00002000 8,
↪192
<VisibleRecord: position=0x0007ff80 length=0x1ff0 version=0xff01> Stride: 0x00002000 8,
↪192
<VisibleRecord: position=0x00081f70 length=0x1f64 version=0xff01> Stride: 0x00001ff0 8,
↪176
===== Summary of Visible Records↪
↪=====

```

Scanning Logical Record Segments with --LRSH

Example of scanning a RP66V1 file for Logical Record Segments, this gives just a summary:

```

$ tdrp66v1scan --LRSH example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
...
===== Summary of LRSH↪
↪=====
LRSH: total=3,303 is_first=3252
LRSH: record types and counts (first segments only) [9]:
  0 :    3,223
  1 :         1
  3 :         1
  4 :         1
  5 :        10
128 :         2
129 :         2
132 :        10
133 :         2
LRSH: record lengths and counts (all segments) [62] range: 16...8188
===== END Summary of LRSH↪
↪=====

```

And with the -v option gives the Visible Records and Logical Record Segments:

```

***** RP66V1 Visible and LRSH Records↪
↪*****
<VisibleRecord: position=0x00000050 length=0x2000 version=0xff01> Stride: 0x00000050 ↪
↪80
  <LogicalRecordSegmentHeader: position=0x00000054 length=0x007c attributes=0x80 LR↪
↪type= 0> Stride: 0x00000054      84
  <LogicalRecordSegmentHeader: position=0x000000d0 length=0x0504 attributes=0x81 LR↪
↪type= 1> Stride: 0x0000007c     124

```

(continues on next page)

(continued from previous page)

```

<LogicalRecordSegmentHeader: position=0x000005d4 length=0x05e0 attributes=0x81 LR_
↪type= 5> Stride: 0x00000504 1,284
<LogicalRecordSegmentHeader: position=0x00000bb4 length=0x03e4 attributes=0x99 LR_
↪type=132> Stride: 0x000005e0 1,504
<LogicalRecordSegmentHeader: position=0x00000f98 length=0x0254 attributes=0x99 LR_
↪type=132> Stride: 0x000003e4 996
<LogicalRecordSegmentHeader: position=0x000011ec length=0x0588 attributes=0x81 LR_
↪type= 5> Stride: 0x00000254 596
<LogicalRecordSegmentHeader: position=0x00001774 length=0x023c attributes=0x98 LR_
↪type=132> Stride: 0x00000588 1,416
<LogicalRecordSegmentHeader: position=0x000019b0 length=0x0084 attributes=0x98 LR_
↪type=132> Stride: 0x0000023c 572
<LogicalRecordSegmentHeader: position=0x00001a34 length=0x061c attributes=0xa0 LR_
↪type=132> Stride: 0x00000084 132
<VisibleRecord: position=0x00002050 length=0x2000 version=0xff01> Stride: 0x00002000 8,
↪192
--<LogicalRecordSegmentHeader: position=0x00002054 length=0x0304 attributes=0xc1 LR_
↪type=132> Stride: 0x00000620 1,568
<LogicalRecordSegmentHeader: position=0x00002358 length=0x0e3c attributes=0x81 LR_
↪type= 5> Stride: 0x00000304 772
<LogicalRecordSegmentHeader: position=0x00003194 length=0x0ebc attributes=0xb9 LR_
↪type=132> Stride: 0x00000e3c 3,644
<VisibleRecord: position=0x00004050 length=0x2000 version=0xff01> Stride: 0x00002000 8,
↪192
--<LogicalRecordSegmentHeader: position=0x00004054 length=0x0110 attributes=0xd9 LR_
↪type=132> Stride: 0x00000ec0 3,776
<LogicalRecordSegmentHeader: position=0x00004164 length=0x1eec attributes=0xa0 LR_
↪type= 5> Stride: 0x00000110 272
<VisibleRecord: position=0x00006050 length=0x1ffc version=0xff01> Stride: 0x00002000 8,
↪192
--<LogicalRecordSegmentHeader: position=0x00006054 length=0x1864 attributes=0xc1 LR_
↪type= 5> Stride: 0x00001ef0 7,920
<LogicalRecordSegmentHeader: position=0x000078b8 length=0x0794 attributes=0xb9 LR_
↪type=132> Stride: 0x00001864 6,244
<VisibleRecord: position=0x0000804c length=0x2000 version=0xff01> Stride: 0x00001ffc 8,
↪188
--<LogicalRecordSegmentHeader: position=0x00008050 length=0x1080 attributes=0xd9 LR_
↪type=132> Stride: 0x00000798 1,944
<LogicalRecordSegmentHeader: position=0x000090d0 length=0x01e0 attributes=0x81 LR_
↪type= 5> Stride: 0x00001080 4,224
<LogicalRecordSegmentHeader: position=0x000092b0 length=0x023c attributes=0x99 LR_
↪type=132> Stride: 0x000001e0 480
<LogicalRecordSegmentHeader: position=0x000094ec length=0x0314 attributes=0x81 LR_
↪type= 5> Stride: 0x0000023c 572
<LogicalRecordSegmentHeader: position=0x00009800 length=0x0154 attributes=0x99 LR_
↪type=128> Stride: 0x00000314 788
<LogicalRecordSegmentHeader: position=0x00009954 length=0x0238 attributes=0x81 LR_
↪type= 5> Stride: 0x00000154 340
<LogicalRecordSegmentHeader: position=0x00009b8c length=0x0270 attributes=0x81 LR_
↪type= 5> Stride: 0x00000238 568
<LogicalRecordSegmentHeader: position=0x00009dfc length=0x0250 attributes=0xa0 LR_
↪type= 5> Stride: 0x00000270 624

```

(continues on next page)

(continued from previous page)

...

Scanning Logical Data with --LD

Example of scanning a RP66V1 file for Logical Record Segments, this gives just a summary:

```
$ tdrp66v1scan --LD example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
Cmd: tdrp66v1scan --LD example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
gnuplot version: "b'gnuplot 5.2 patchlevel 6'"
args: Namespace(EFLR=False, IFLR=False, LD=True, LR=False, LRSH=False, VR=False, dump_
↳ bytes=0, dump_raw_bytes=False, eflr_as_table=False, eflr_set_type=[], encrypted=False,
↳ frame_slice=',', gnuplot=None, iflr_set_type=[], keep_going=False, log_level=30, path_
↳ in='example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS', path_out='',
↳ recurse=False, verbose=0)
Use -v to see individual logical data.
Use -v and --dump-bytes to see actual first n bytes.
***** RP66V1 Logical Data Summary
↳ *****
===== RP66V1 Logical Data EFLR Summary
↳ =====
Total number of EFLR records: 30
Total length of EFLR records: 78,109
EFLR record type 0 lengths and count [1]:
    120:      1
EFLR record type 1 lengths and count [1]:
    1,279:    1
EFLR record type 3 lengths and count [1]:
    7,174:    1
EFLR record type 4 lengths and count [1]:
    572:      1
EFLR record type 5 lengths and count [10]:
    181:      1
    475:      1
    561:      1
    617:      1
    781:      1
$ tdrp66v1scan --LD example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
Cmd: tdrp66v1scan --LD example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
gnuplot version: "b'gnuplot 5.2 patchlevel 6'"
args: Namespace(EFLR=False, IFLR=False, LD=True, LR=False, LRSH=False, VR=False, dump_
↳ bytes=0, dump_raw_bytes=False, eflr_as_table=False, eflr_set_type=[], encrypted=False,
↳ frame_slice=',', gnuplot=None, iflr_set_type=[], keep_going=False, log_level=30, path_
↳ in='example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS', path_out='',
↳ recurse=False, verbose=0)
Use -v to see individual logical data.
Use -v and --dump-bytes to see actual first n bytes.
***** RP66V1 Logical Data Summary
↳ *****
===== RP66V1 Logical Data EFLR Summary
↳ =====
Total number of EFLR records: 30
```

(continues on next page)

(continued from previous page)

```

Total length of EFLR records: 78,109
EFLR record type 0 lengths and count [1]:
    120:          1
EFLR record type 1 lengths and count [1]:
    1,279:        1
EFLR record type 3 lengths and count [1]:
    7,174:        1
EFLR record type 4 lengths and count [1]:
    572:          1
EFLR record type 5 lengths and count [10]:
    181:          1
    475:          1
    561:          1
    617:          1
    781:          1
    1,409:        1
    1,497:        1
    1,620:        1
    3,637:        1
    14,149:       1
EFLR record type 128 lengths and count [2]:
    336:          1
    888:          1
EFLR record type 129 lengths and count [2]:
    111:          1
    1,226:        1
EFLR record type 132 lengths and count [9]:
    128:          1
    288:          1
    512:          1
    568:          2
    592:          1
    992:          1
    2,325:        1
    4,036:        1
    6,156:        1
EFLR record type 133 lengths and count [2]:
    999:          1
    24,312:       1

===== END RP66V1 Logical Data EFLR Summary
↪=====
===== RP66V1 Logical Data IFLR Summary
↪=====

Total number of IFLR records: 3,222
Total length of IFLR records: 440,173
IFLR record type 0 lengths and count [4]:
    25:          127
    26:          794
    180:         127
    181:         2,174

===== END RP66V1 Logical Data IFLR Summary
↪=====

```

(continues on next page)

(continued from previous page)

```
Total length EFLR/IFLR: 17.745%
***** END RP66V1 Logical Data Summary
↳ *****
      540,372      -1      0.059  -0.000%      115.4 False "example_data/RP66V1/206_
↳ 05a-_3_DWL_DWL_WIRE_258276498.DLIS"
Execution time =      0.060 (S)
Processed 1 files and 540,372 bytes, 115.8 ms/Mb
```

And with the -v option gives the Visible Records and Logical Record Segments. The letter ‘E’ is for EFLRs and ‘I’ for IFLRs, ‘Plain’ is for un-encrypted records and ‘Crypt’ for encrypted records:

```
$ tdrp66v1scan --LD -v example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
Use -v and --dump-bytes to see actual first n bytes.
***** RP66V1 Logical Data Summary
↳ *****
Visible R  LRSH      Typ      Length
-----
0x00000050 0x00000054  0 E Plain    120
           0x000000d0  1 E Plain   1,279
           0x000005d4  5 E Plain   1,497
           0x00000bb4 132 E Crypt    992
           0x00000f98 132 E Crypt    592
           0x000011ec  5 E Plain   1,409
           0x00001774 132 E Crypt    568
           0x000019b0 132 E Crypt    128
           0x00001a34 132 E Plain   2,325
0x00002050 0x00002358  5 E Plain   3,637
           0x00003194 132 E Crypt   4,036
0x00004050 0x00004164  5 E Plain  14,149
0x00006050 0x000078b8 132 E Crypt   6,156
0x0000804c 0x000090d0  5 E Plain    475
           0x000092b0 132 E Crypt    568
           0x000094ec  5 E Plain    781
           0x00009800 128 E Crypt    336
           0x00009954  5 E Plain    561
           0x00009b8c  5 E Plain    617
           0x00009dfc  5 E Plain   1,620
...
0x00081f70 0x00081f74  0 I Plain    181
           0x00082030  0 I Plain     26
           0x00082050  0 I Plain    181
           0x0008210c  0 I Plain    181
           0x000821c8  0 I Plain    181
           ...
           0x00083d3c  0 I Plain     26
           0x00083d5c  0 I Plain    181
           0x00083e18  0 I Plain    181
===== RP66V1 Logical Data EFLR Summary
↳ =====
Total number of EFLR records: 30
Total length of EFLR records: 78,109
EFLR record type 0 lengths and count [1]:
```

(continues on next page)

(continued from previous page)

```

    120:          1
EFLR record type 1 lengths and count [1]:
    1,279:        1
EFLR record type 3 lengths and count [1]:
    7,174:        1
EFLR record type 4 lengths and count [1]:
    572:          1
EFLR record type 5 lengths and count [10]:
    181:          1
    475:          1
    561:          1
    617:          1
    781:          1
    1,409:        1
    1,497:        1
    1,620:        1
    3,637:        1
    14,149:       1
EFLR record type 128 lengths and count [2]:
    336:          1
    888:          1
EFLR record type 129 lengths and count [2]:
    111:          1
    1,226:        1
EFLR record type 132 lengths and count [9]:
    128:          1
    288:          1
    512:          1
    568:          2
    592:          1
    992:          1
    2,325:        1
    4,036:        1
    6,156:        1
EFLR record type 133 lengths and count [2]:
    999:          1
    24,312:       1

===== END RP66V1 Logical Data EFLR Summary
↳=====
===== RP66V1 Logical Data IFLR Summary
↳=====
Total number of IFLR records: 3,222
Total length of IFLR records: 440,173
IFLR record type 0 lengths and count [4]:
    25:          127
    26:          794
    180:         127
    181:         2,174

===== END RP66V1 Logical Data IFLR Summary
↳=====
Total length EFLR/IFLR: 17.745%
***** END RP66V1 Logical Data Summary
↳*****

```

(continues on next page)

(continued from previous page)

The --dump-bytes combined with -v shows the initial bytes of each logical record, here the first 16 bytes are dumped:

```
$ tdrp66v1scan --LD -v --dump-bytes=16 example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_
↪ 258276498.DLIS
***** RP66V1 Logical Data Summary*****
↪ *****
Visible R  LRSH      Typ      Length
-----
0x00000050 0x00000054  0 E Plain      120 f00b 4649 4c45 2d48 4541 4445 5234 0f53 ..
↪ FILE-HEADER4.S
      0x000000d0  1 E Plain    1,279 f006 4f52 4947 494e 3c07 4649 4c45 2d49 ..
↪ ORIGIN<.FILE-I
      0x000005d4  5 E Plain    1,497 f809 4551 5549 504d 454e 5402 3531 3006 ..
↪ EQUIPMENT.510.
      0x00000bb4 132 E Crypt      992 0018 01b8 ced6 0000 be18 0000 8467 0000 .....
↪ .....g..
      0x00000f98 132 E Crypt      592 0018 01b8 dee9 0000 4916 0000 f16d 0000 .....
↪ ..I...m..
      0x000011ec  5 E Plain    1,409 f804 544f 4f4c 0235 3430 0a50 4152 414d ..
↪ TOOL.540.PARAM
      0x00001774 132 E Crypt      568 0018 01b8 9a99 0000 3c15 0000 877e 0000 .....
↪ ...<....~..
      0x000019b0 132 E Crypt      128 0018 01b8 acb3 0000 064d 0000 b74d 0000 .....
↪ ...M...M..
      0x00001a34 132 E Plain    2,325 f80b 3434 302d 4348 414e 4e45 4c02 3537 ..440-
↪ CHANNEL.57
0x00002050 0x00002358  5 E Plain    3,637 f809 5041 5241 4d45 5445 5202 3538 3006 ..
↪ PARAMETER.580.
      0x00003194 132 E Crypt    4,036 0018 01b8 9aa6 0000 c84d 0000 4364 0000 .....
↪ ...M...Cd..
0x00004050 0x00004164  5 E Plain   14,149 f809 5041 5241 4d45 5445 5202 3630 3006 ..
↪ PARAMETER.600.
0x00006050 0x000078b8 132 E Crypt    6,156 0018 01b8 565d 0000 0945 0000 3812 0000 ....
↪ V]...E...8...
0x0000804c 0x000090d0  5 E Plain      475 f809 5041 5241 4d45 5445 5202 3632 3006 ..
↪ PARAMETER.620.
      0x000092b0 132 E Crypt      568 0018 01b8 010d 0000 f57f 0000 890a 0000 .....
↪ .....
      0x000094ec  5 E Plain      781 f817 4341 4c49 4252 4154 494f 4e2d 4d45 ..
↪ CALIBRATION-ME
      0x00009800 128 E Crypt      336 0018 01b8 4550 0000 ae56 0000 3207 0000 ....
↪ EP...V...2...
      0x00009954  5 E Plain      561 f817 4341 4c49 4252 4154 494f 4e2d 434f ..
↪ CALIBRATION-CO
      0x00009b8c  5 E Plain      617 f817 4341 4c49 4252 4154 494f 4e2d 434f ..
↪ CALIBRATION-CO
      0x00009dfc  5 E Plain    1,620 f80b 4341 4c49 4252 4154 494f 4e02 3734 ..
↪ CALIBRATION.74
...

```

The raw bytes object is dumped of the --dump-raw-bytes flag is used along with --dump-bytes combined with -v.

This can be useful for creating test cases:

```
$ tdrp66v1scan --LD -v --dump-bytes=16 --dump-raw-bytes example_data/RP66V1/206_05a-_3_
↳DWL_DWL_WIRE_258276498.DLIS | head -n 40
***** RP66V1 Logical Data Summary
*****
Visible R  LRSH      Typ      Length
-----
0x00000050 0x00000054  0 E Plain    120 b'\xf0\x0bFILE-HEADER4\x0fS'
          0x000000d0  1 E Plain   1,279 b'\xf0\x06ORIGIN<\x07FILE-I'
          0x0000005d4  5 E Plain   1,497 b'\xf8\tEQUIPMENT\x02510\x06'
          0x000000bb4 132 E Crypt    992 b'\x00\x18\x01\xb8\xce\xd6\x00\x00\xbe\x18\
↳x00\x00\x84g\x00\x00'
          0x000000f98 132 E Crypt    592 b'\x00\x18\x01\xb8\xde\xe9\x00\x00I\x16\x00\
↳x00\x00\xflm\x00\x00'
          0x0000011ec  5 E Plain   1,409 b'\xf8\x04TOOL\x02540\nPARAM'
          0x000001774 132 E Crypt    568 b'\x00\x18\x01\xb8\x9a\x99\x00\x00<\x15\x00\
↳x00\x87~\x00\x00'
          0x0000019b0 132 E Crypt    128 b'\x00\x18\x01\xb8\xac\xb3\x00\x00\x06M\x00\
↳x00\x00\x00\x00'
          0x000001a34 132 E Plain   2,325 b'\xf8\x0b440-CHANNEL\x0257'
0x000002050 0x000002358  5 E Plain   3,637 b'\xf8\tPARAMETER\x02580\x06'
          0x000003194 132 E Crypt   4,036 b'\x00\x18\x01\xb8\x9a\xa6\x00\x00\xc8M\x00\
↳x00Cd\x00\x00'
0x000004050 0x000004164  5 E Plain  14,149 b'\xf8\tPARAMETER\x02600\x06'
0x000006050 0x0000078b8 132 E Crypt   6,156 b'\x00\x18\x01\xb8VJ\x00\x00\tE\x00\x008\x12\
↳x00\x00'
0x00000804c 0x0000090d0  5 E Plain    475 b'\xf8\tPARAMETER\x02620\x06'
          0x0000092b0 132 E Crypt    568 b'\x00\x18\x01\xb8\x01\r\x00\x00\xf5\x7f\x00\
↳x00\x89\n\x00\x00'
          0x0000094ec  5 E Plain    781 b'\xf8\x17CALIBRATION-ME'
          0x000009800 128 E Crypt    336 b'\x00\x18\x01\xb8EP\x00\x00\xaeV\x00\x002\
↳x07\x00\x00'
          0x000009954  5 E Plain    561 b'\xf8\x17CALIBRATION-CO'
          0x000009b8c  5 E Plain    617 b'\xf8\x17CALIBRATION-CO'
          0x000009dfc  5 E Plain   1,620 b'\xf8\x0bCALIBRATION\x0274'
...

```

Scanning Explicitly Formatted Logical Records with --EFLR

Example of scanning a RP66V1 file for Logical Record Segments, this gives just a summary:

```
$ tdrp66v1scan --EFLR example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
Use -v to see individual logical data.
***** RP66V1 EFLR and IFLR Data Summary
*****
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'FILE-HEADER' name: b'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'ORIGIN' name: b'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'EQUIPMENT' name: b'51'>
Encrypted EFLR: VR: 0x00000050 LRSH: 0x000000bb4
Encrypted EFLR: VR: 0x00000050 LRSH: 0x000000f98
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'TOOL' name: b'54'>

```

(continues on next page)

(continued from previous page)

```

Encrypted EFLR: VR: 0x00000050 LRSH: 0x00001774
Encrypted EFLR: VR: 0x00000050 LRSH: 0x000019b0
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-CHANNEL' name: b'57'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'PARAMETER' name: b'58'>
Encrypted EFLR: VR: 0x00002050 LRSH: 0x00003194
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'PARAMETER' name: b'60'>
Encrypted EFLR: VR: 0x00006050 LRSH: 0x000078b8
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'PARAMETER' name: b'62'>
Encrypted EFLR: VR: 0x0000804c LRSH: 0x000092b0
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CALIBRATION-MEASUREMENT' name: b'64'>
Encrypted EFLR: VR: 0x0000804c LRSH: 0x00009800
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CALIBRATION-COEFFICIENT' name: b'72'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CALIBRATION-COEFFICIENT' name: b'73'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CALIBRATION' name: b'74'>
Encrypted EFLR: VR: 0x0000a04c LRSH: 0x0000a45c
Encrypted EFLR: VR: 0x0000a04c LRSH: 0x0000a7d8
Encrypted EFLR: VR: 0x0000a04c LRSH: 0x0000a8fc
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'PROCESS' name: b'78'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-OP-CORE_TABLES' name: b'79'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-OP-CORE_REPORT_FORMAT' name: b'330'>
<'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CHANNEL' name: b''>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-PRESENTATION-DESCRIPTION' name: b'375'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-OP-CHANNEL' name: b'377'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'FRAME' name: b''>
***** END RP66V1 EFLR and IFLR Data Summary
*****

```

The -v flag can be added to see the initial data:

```

$ tdrp66v1scan --EFLR -v example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
***** RP66V1 EFLR and IFLR Data Summary
*****
Visible R  LRSH      Typ      Length
-----
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'FILE-HEADER' name: b''>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'ORIGIN' name: b''>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'EQUIPMENT' name: b'51'>
Encrypted EFLR: <FileLogicalData VR: 0x00000050 LRSH: 0x00000bb4 LR type 132 E y len
0x03e0 Idx 0x0000 0018 01b8 ced6 0000 be18 0000 8467 0000 .....g..>
Encrypted EFLR: <FileLogicalData VR: 0x00000050 LRSH: 0x00000f98 LR type 132 E y len
0x0250 Idx 0x0000 0018 01b8 dee9 0000 4916 0000 f16d 0000 .....I...m..>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'TOOL' name: b'54'>
Encrypted EFLR: <FileLogicalData VR: 0x00000050 LRSH: 0x00001774 LR type 132 E y len
0x0238 Idx 0x0000 0018 01b8 9a99 0000 3c15 0000 877e 0000 .....<....~..>
Encrypted EFLR: <FileLogicalData VR: 0x00000050 LRSH: 0x000019b0 LR type 132 E y len
0x0080 Idx 0x0000 0018 01b8 acb3 0000 064d 0000 b74d 0000 .....M...M..>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-CHANNEL' name: b'57'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'PARAMETER' name: b'58'>
Encrypted EFLR: <FileLogicalData VR: 0x00002050 LRSH: 0x00003194 LR type 132 E y len
0x0fc4 Idx 0x0000 0018 01b8 9aa6 0000 c84d 0000 4364 0000 .....M..Cd..>

```

(continues on next page)

(continued from previous page)

```

<ExplicitlyFormattedLogicalRecord EFLR Set type: b'PARAMETER' name: b'60'>
Encrypted EFLR: <FileLogicalData VR: 0x00006050 LRSH: 0x000078b8 LR type 132 E y len
↳0x180c Idx 0x0000 0018 01b8 565d 0000 0945 0000 3812 0000 ....V]...E..8...>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'PARAMETER' name: b'62'>
Encrypted EFLR: <FileLogicalData VR: 0x0000804c LRSH: 0x000092b0 LR type 132 E y len
↳0x0238 Idx 0x0000 0018 01b8 010d 0000 f57f 0000 890a 0000 .....>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CALIBRATION-MEASUREMENT' name: b'64'>
Encrypted EFLR: <FileLogicalData VR: 0x0000804c LRSH: 0x00009800 LR type 128 E y len
↳0x0150 Idx 0x0000 0018 01b8 4550 0000 ae56 0000 3207 0000 ....EP...V..2...>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CALIBRATION-COEFFICIENT' name: b'72'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CALIBRATION-COEFFICIENT' name: b'73'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CALIBRATION' name: b'74'>
Encrypted EFLR: <FileLogicalData VR: 0x0000a04c LRSH: 0x0000a45c LR type 128 E y len
↳0x0378 Idx 0x0000 0018 01b8 eff6 0000 fd5c 0000 123e 0000 .....>
Encrypted EFLR: <FileLogicalData VR: 0x0000a04c LRSH: 0x0000a7d8 LR type 132 E y len
↳0x0120 Idx 0x0000 0018 01b8 4644 0000 ad4c 0000 4f31 0000 ....FD...L..01...>
Encrypted EFLR: <FileLogicalData VR: 0x0000a04c LRSH: 0x0000a8fc LR type 132 E y len
↳0x0200 Idx 0x0000 0018 01b8 abb7 0000 d01c 0000 6b36 0000 .....k6...>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'PROCESS' name: b'78'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-OP-CORE_TABLES' name: b'79'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-OP-CORE_REPORT_FORMAT' name: b'330
↳'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CHANNEL' name: b''>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-PRESENTATION-DESCRIPTION' name: b
↳'375'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'440-OP-CHANNEL' name: b'377'>
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'FRAME' name: b''>
***** END RP66V1 EFLR and IFLR Data Summary
↳*****

```

The `--eflr-set-type` can be used to select only specific EFLRs:

```

$ tdrp66v1scan --EFLR -v --eflr-set-type=ORIGIN --eflr-as-table example_data/RP66V1/206_
↳05a-_3_DWL_DWL_WIRE_258276498.DLIS
***** RP66V1 EFLR and IFLR Data Summary
↳*****
Visible R  LRSH      Typ      Length
-----
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'ORIGIN' name: b''>
***** END RP66V1 EFLR and IFLR Data Summary
↳*****

```

Scanning Implicitly Formatted Logical Records with --IFLR

Example of scanning a RP66V1 file for Logical Record Segments, this gives just a summary:

```
$ tdrp66v1scan --IFLR example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
Use -v to see individual logical data.
Use -v and --dump-bytes to see actual first n bytes.
***** RP66V1 EFLR and IFLR Data Summary
<IndirectlyFormattedLogicalRecord b'2000T'   frame:      1 free data[ 16]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:      1 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:      2 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'2000T'    frame:      2 free data[ 16]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:      3 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'2000T'    frame:      3 free data[ 16]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:      4 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:      5 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:      6 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:      7 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'2000T'    frame:      4 free data[ 16]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:      8 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:      9 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'2000T'    frame:      5 free data[ 16]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:     10 free data[ 172]>
...
<IndirectlyFormattedLogicalRecord b'2000T'    frame:     920 free data[ 16]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:    2,298 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:    2,299 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'2000T'    frame:     921 free data[ 16]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:    2,300 free data[ 172]>
<IndirectlyFormattedLogicalRecord b'800T'     frame:    2,301 free data[ 172]>
***** END RP66V1 EFLR and IFLR Data Summary
540,372      -1    0.435  -0.000%    844.6 False "example_data/RP66V1/206_
05a-_3_DWL_DWL_WIRE_258276498.DLIS"
Execution time =    0.436 (S)
Processed 1 files and 540,372 bytes, 845.2 ms/Mb
```

Scanning Everything with --LR

This reads every byte in the file and writes a very verbose output of each EFLR and a summary of each Log Pass. For example:

```
$ tdrp66v1scan --LR example_data/RP66V1/206_05a-_3_DWL_DWL_WIRE_258276498.DLIS
***** RP66V1 File Data Summary
StorageUnitLabel:
  Storage Unit Sequence Number: 1
    DLIS Version: b'V1.00'
  Storage Unit Structure: b'RECORD'
    Maximum Record Length: 8192
  Storage Set Identifier: b'Default Storage Set
```

(continues on next page)

(continued from previous page)

```

===== Logical File [0/1]
<TotalDepth.RP66V1.core.LogicalFile.LogicalFile object at 0x104d3f6a0>
----- EFLR [0/19] at VR: 0x00000050 LRSH:
0x00000054 -----
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'FILE-HEADER' name: b''>
  Template [2]:
    CD: 001 10100 L: b'SEQUENCE-NUMBER' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 10100 L: b'ID' C: 1 R: 20 (ASCII) U: b'' V: None
  Objects [1]:
    OBNAME: O: 2 C: 0 I: b'5'
    CD: 001 00001 L: b'SEQUENCE-NUMBER' C: 1 R: 20 (ASCII) U: b'' V: [b'          197']
    CD: 001 00001 L: b'ID' C: 1 R: 20 (ASCII) U: b'' V: [b'MSCT_197LTP
    ']
----- END EFLR [0/19] at VR: 0x00000050 LRSH:
0x00000054 -----
----- EFLR [1/19] at VR: 0x00000050 LRSH:
0x000000d0 -----
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'ORIGIN' name: b''>
  Template [20]:
    CD: 001 11100 L: b'FILE-ID' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11100 L: b'FILE-SET-NAME' C: 1 R: 19 (IDENT) U: b'' V: None
    CD: 001 11100 L: b'FILE-SET-NUMBER' C: 1 R: 18 (UVARI) U: b'' V: None
    CD: 001 11100 L: b'FILE-NUMBER' C: 1 R: 18 (UVARI) U: b'' V: None
    CD: 001 11100 L: b'FILE-TYPE' C: 1 R: 19 (IDENT) U: b'' V: None
    CD: 001 11100 L: b'PRODUCT' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11100 L: b'VERSION' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11100 L: b'PROGRAMS' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11100 L: b'CREATION-TIME' C: 1 R: 21 (DTIME) U: b'' V: None
    CD: 001 11100 L: b'ORDER-NUMBER' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11000 L: b'DESCENT-NUMBER' C: 1 R: 19 (IDENT) U: b'' V: None
    CD: 001 11000 L: b'RUN-NUMBER' C: 1 R: 19 (IDENT) U: b'' V: None
    CD: 001 11100 L: b'WELL-ID' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11100 L: b'WELL-NAME' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11100 L: b'FIELD-NAME' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11100 L: b'PRODUCER-CODE' C: 1 R: 16 (UNORM) U: b'' V: None
    CD: 001 11100 L: b'PRODUCER-NAME' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11100 L: b'COMPANY' C: 1 R: 20 (ASCII) U: b'' V: None
    CD: 001 11100 L: b'NAME-SPACE-NAME' C: 1 R: 19 (IDENT) U: b'' V: None
    CD: 001 11100 L: b'NAME-SPACE-VERSION' C: 1 R: 18 (UVARI) U: b'' V: None
  Objects [1]:
    OBNAME: O: 2 C: 0 I: b'DLIS_DEFINING_ORIGIN'
    CD: 001 00001 L: b'FILE-ID' C: 1 R: 20 (ASCII) U: b'' V: [b'MSCT_197LTP
    ']
    CD: 001 00001 L: b'FILE-SET-NAME' C: 1 R: 19 (IDENT) U: b'' V: [b'FAROE_PETROLEUM/
    206_05A-3']
    CD: 001 00001 L: b'FILE-SET-NUMBER' C: 1 R: 18 (UVARI) U: b'' V: [41]
    CD: 001 00001 L: b'FILE-NUMBER' C: 1 R: 18 (UVARI) U: b'' V: [167]
    CD: 001 00001 L: b'FILE-TYPE' C: 1 R: 19 (IDENT) U: b'' V: [b'STATION LOG']
    CD: 001 00001 L: b'PRODUCT' C: 1 R: 20 (ASCII) U: b'' V: [b'OP']
    CD: 001 00001 L: b'VERSION' C: 1 R: 20 (ASCII) U: b'' V: [b'19C0-187']
    CD: 001 01001 L: b'PROGRAMS' C: 4 R: 20 (ASCII) U: b'' V: [b'MSCT: Mechanical
    Sidewall Coring Tool', b'SGTP: Scintillation Gamma-Ray - P', b'LEHQT: Logging
    Equipment Head - QT', b'WELLCAD: WellCAD file generator']

```


(continued from previous page)

```

1000.100 [ 0]:
1000.200 [ 0]:
1000.300 [ 0]:
1000.400 [ 0]:
1000.500 [ 0]:
1000.600 [ 0]:
1000.700 [ 0]:
1000.800 [ 0]:
1000.900 [ 1]:
Frames [921] from: 16677259.000 to 17597260.000 Interval: 1000.000 b'ms'
Frame spacing: <Slice on length=921 start=0 stop=921 step=1> number of frames: 921 numpy
↳size: 14,736 bytes
Channel      Size  Absent      Min      Mean      Std.Dev.      Max      Unit
↳Units      dtype
-----
↳-----
TIME         921      0  16677259.000  17137260.404  265869.810  17597260.000  b
↳'ms'        float32
TDEP         921      0    852606.000   872468.708   17513.899   893302.000   b'0.
↳1 in'       float32
TENS_SL      921      0     1825.000    2145.789    198.506     2594.000     b
↳'lbf'       float32
DEPT_SL      921      0    852606.000   872467.735   17513.909   893303.000   b'0.
↳1 in'       float32

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA END Frame Array [0/2] AAAAAAAAAAAAAAAAA
↳AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA Frame Array [1/2] AAAAAAAAAAAAAAAAA
↳AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

FrameArray: ID: OBNAME: 0: 2 C: 0 I: b'800T' b''
  FrameChannel: OBNAME: 0: 2 C: 5 I: b'TIME' Rc: 2 Co: 1 Un: b'ms'
↳ Di: [1] b'400 milli-second time channel'
  FrameChannel: OBNAME: 0: 2 C: 5 I: b'TDEP' Rc: 2 Co: 1 Un: b'0.1 in'
↳ Di: [1] b'MSCT depth channel'
  FrameChannel: OBNAME: 0: 2 C: 1 I: b'ETIM' Rc: 2 Co: 1 Un: b's'
↳ Di: [1] b'Elapsed Logging Time'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'LMVL' Rc: 2 Co: 1 Un: b'V'
↳ Di: [1] b'Lower Motor Voltage Limit'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'UMVL' Rc: 2 Co: 1 Un: b'V'
↳ Di: [1] b'Upper Motor Voltage Limit'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'CFLA' Rc: 2 Co: 1 Un: b' '
↳ Di: [1] b'Coring Flag'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'OCD' Rc: 2 Co: 1 Un: b'ft'
↳ Di: [1] b'Observed Core Depth'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'RCMD' Rc: 2 Co: 1 Un: b'V'
↳ Di: [1] b'Raw Coring Motor Downhole Voltage'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'RCPP' Rc: 2 Co: 1 Un: b'in'
↳ Di: [1] b'Raw Kinematics Piston Position'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'CMRT' Rc: 2 Co: 1 Un: b'h'
↳ Di: [1] b'Coring Motor Run Time'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'RCNU' Rc: 2 Co: 1 Un: b' '
↳ Di: [1] b'Raw Core Number'

```

(continues on next page)

(continued from previous page)

FrameChannel: OBNAME: 0: 2 C: 0 I: b'DCFL'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Down Command Flag'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'DFS'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Data Full Scale'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'DZER'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Data Zero'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'RHMD'	Rc: 2 Co: 1 Un: b'V'	└
↳ Di: [1] b'Raw Hydraulic Motor Downhole Voltage'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'HMRT'	Rc: 2 Co: 1 Un: b'h'	└
↳ Di: [1] b'Hydraulic Motor Run Time'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'RHV'	Rc: 2 Co: 1 Un: b'V'	└
↳ Di: [1] b'Raw Head Voltage'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'RLSW'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Raw Limit Switch'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'MNU'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Marker Number'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'S1CY'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Solenoid 1 Cycles'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'S2CY'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Solenoid 2 Cycles'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'RSCU'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Raw Solenoid Current'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'RSTS'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Raw Solenoid Status'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'UCFL'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Up Command Flag'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'CARC'	Rc: 2 Co: 1 Un: b'mA'	└
↳ Di: [1] b'Cartridge Current'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'CMDV'	Rc: 2 Co: 1 Un: b'V'	└
↳ Di: [1] b'Coring Motor Downhole Voltage'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'CMPP'	Rc: 2 Co: 1 Un: b'in'	└
↳ Di: [1] b'Kinematics Piston Position'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'CNV'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Core Number'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'HMDV'	Rc: 2 Co: 1 Un: b'V'	└
↳ Di: [1] b'Hydraulic Motor Downhole Voltage'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'HV'	Rc: 2 Co: 1 Un: b'V'	└
↳ Di: [1] b'Head Voltage'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'LSWI'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Limit Switch'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'SCUR'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Solenoid Current'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'SSTA'	Rc: 2 Co: 1 Un: b' '	└
↳ Di: [1] b'Solenoid Status'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'RCMP'	Rc: 2 Co: 1 Un: b'psi'	└
↳ Di: [1] b'Raw Coring Motor Pressure'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'RHPP'	Rc: 2 Co: 1 Un: b'psi'	└
↳ Di: [1] b'Raw Hydraulic Pump Pressure'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'RRPP'	Rc: 2 Co: 1 Un: b'psi'	└
↳ Di: [1] b'Raw Kinematics Pressure'		
FrameChannel: OBNAME: 0: 2 C: 0 I: b'CMPP'	Rc: 2 Co: 1 Un: b'psi'	└
↳ Di: [1] b'Coring Motor Pressure'		

(continues on next page)

(continued from previous page)

```

FrameChannel: OBNAME: 0: 2 C: 0 I: b'HPPR'          Rc:  2 Co:  1 Un: b'psi'
↳ Di: [1] b'Hydraulic Pump Pressure'
FrameChannel: OBNAME: 0: 2 C: 0 I: b'RPPV'          Rc:  2 Co:  1 Un: b'psi'
↳ Di: [1] b'Kinematics Pressure'
FrameChannel: OBNAME: 0: 2 C: 0 I: b'SMSC'          Rc: 14 Co:  1 Un: b' '
↳ Di: [1] b'MSCT Status Word'
FrameChannel: OBNAME: 0: 2 C: 0 I: b'CMCU'          Rc:  2 Co:  1 Un: b'mA'
↳ Di: [1] b'Coring Motor Current'
FrameChannel: OBNAME: 0: 2 C: 0 I: b'HMCU'          Rc:  2 Co:  1 Un: b'mA'
↳ Di: [1] b'Hydraulic Motor Current'
FrameChannel: OBNAME: 0: 2 C: 0 I: b'CMLP'          Rc:  2 Co:  1 Un: b'in'
↳ Di: [1] b'Coring Motor Linear Position'
X Axis summary (all IFLRs):
Min: 16677259.0 Max: 17597260.0 [b'ms'] Count: 2301
X Axis spacing summary:
Min: 400.0 Max: 401.0 Mean: 400.0004347826087 Median: 400.0
Normal: 2300
Duplicate: 0
Skipped: 0
Back: 0
Spacing histogram
Value [ N]: Relative Frequency
400.000 [ 2299]:
↳ *****
400.100 [  0]:
400.200 [  0]:
400.300 [  0]:
400.400 [  0]:
400.500 [  0]:
400.600 [  0]:
400.700 [  0]:
400.800 [  0]:
400.900 [  1]:
Frames [2301] from: 16677259.000 to 17597260.000 Interval: 400.000 b'ms'
Frame spacing: <Slice on length=2301 start=0 stop=2301 step=1> number of frames: 2301
↳ numpy size: 395,772 bytes
Channel  Size  Absent      Min      Mean      Std.Dev.      Max
↳ Units    dtype
-----  -
↳ -----
TIME    2301      0  16677259.000  17137261.698  265696.737  17597260.000  b
↳ 'ms'   float32
TDEP    2301      0   852606.000   872468.805   17512.407   893304.000  b'0.
↳ 1 in'  float32
ETIM    2301      0      0.000     460.001     265.697     920.001
↳ b's'   float32
LMVL    2301      0     585.000     585.000      0.000     585.000
↳ b'V'   float32
UMVL    2301      0     635.000     635.000      0.000     635.000
↳ b'V'   float32
CFLA    2301      0      0.000     13.361      5.757      18.000
↳ b' '   float32

```

(continues on next page)

(continued from previous page)

OCD	2301	0	6789.050	7153.751	165.517	7433.008	b
↪ 'ft'	float32						
RCMD	2301	0	0.000	191.060	305.260	704.275	└
↪ b'V'	float32						
RCPP	2301	0	0.443	0.853	0.649	2.598	b
↪ 'in'	float32						
CMRT	2301	0	0.637	0.676	0.018	0.708	└
↪ b'h'	float32						
RCNU	2301	0	20.000	20.756	0.532	22.000	└
↪ b' '	float32						
DCFL	2301	0	0.000	1.229	12.818	143.000	└
↪ b' '	float32						
DFS	2301	0	209.000	209.464	0.499	210.000	└
↪ b' '	float32						
DZER	2301	0	0.000	0.002	0.042	1.000	└
↪ b' '	float32						
RHMD	2301	0	0.000	345.934	320.677	674.725	└
↪ b'V'	float32						
HMRT	2301	0	1.490	1.563	0.032	1.628	└
↪ b'h'	float32						
RHV	2301	0	142.319	151.464	1.880	159.428	└
↪ b'V'	float32						
RLSW	2301	0	0.000	0.377	0.485	1.000	└
↪ b' '	float32						
MNU	2301	0	24.000	24.757	0.533	26.000	└
↪ b' '	float32						
S1CY	2301	0	24.000	25.240	0.479	26.000	└
↪ b' '	float32						
S2CY	2301	0	27.000	28.939	0.788	30.000	└
↪ b' '	float32						
RSCU	2301	0	21.000	74.272	62.645	174.000	└
↪ b' '	float32						
RSTS	2301	0	0.000	0.707	0.882	2.000	└
↪ b' '	float32						
UCFL	2301	0	128.000	132.961	6.559	143.000	└
↪ b' '	float32						
CARC	2301	0	178.238	201.121	12.822	211.238	b
↪ 'mA'	float32						
CMDV	2301	0	0.000	191.060	305.260	704.275	└
↪ b'V'	float32						
CMPP	2301	0	-0.004	0.407	0.651	2.158	b
↪ 'in'	float32						
CNU	2301	0	20.000	20.756	0.532	22.000	└
↪ b' '	float32						
HMDV	2301	0	0.000	345.934	320.677	674.725	└
↪ b'V'	float32						
HV	2301	0	142.319	151.464	1.880	159.428	└
↪ b'V'	float32						
LSWI	2301	0	0.000	0.377	0.485	1.000	└
↪ b' '	float32						
SCUR	2301	0	21.000	74.272	62.645	174.000	└
↪ b' '	float32						

(continues on next page)

SSTA	2301	0	0 000	0 707	0 882	2 000
------	------	---	-------	-------	-------	-------

IMPORTING TOTALDEPTH INTO YOUR PYTHON PROJECT

6.1 Processing Western Atlas BIT Files

6.1.1 Reading a BIT File Log Data

Suppose that we have a file in `~/tmp/BIT.bit` and you want to read the frame data from a particular log pass, first import what we need:

```
>>> from TotalDepth.BIT import ReadBIT
>>> import os
```

Then read the BIT file:

```
>>> fpath = os.path.expanduser('~/.tmp/BIT.bit')
>>> frame_arrays = ReadBIT.create_bit_frame_array_from_path(fpath)
>>> frame_arrays
[<TotalDepth.BIT.ReadBIT.BITFrameArray object at 0x10f7041c0>, <TotalDepth.BIT.ReadBIT.
↳BITFrameArray object at 0x10f6d6880>]
```

In this file there is are two frame arrays, we can get the description of the first one using `long_str()`:

```
>>> print(frame_arrays[0].long_str())
BITFrameArray: ident="0"
  Unknown head: b'\x00\x02\x00\x00'
  Description: b'SHELL EXPRO U.K.      24 OCT 84      MANSFIELD/DODDS
↳ '
  Unknown A: b'\x00\n\x00\x18\x00'
  Unknown B: b'T  2 9 / 1 0 - 3
↳ '
  Unknown C: b'\x00\x12\x00\x0b\x00\x06 '
  Channels [10]: ['COND', 'SN ', 'SP ', 'GR ', 'CAL ', 'TEN ', 'SPD ', 'ACQ ', 'AC ',
↳ 'RT ']
  BIT Log Pass: LogPassRange(depth_from=14950.000891089492, depth_to=14590.000869631818,
↳ spacing=0.2500000149011621, unknown_a=0.0, unknown_b=16.000000953674373)
  Unknown tail: b'MN239J 1'
  Frame count: 1472
  Frame array:      FrameArray: ID: 0 b'SHELL EXPRO U.K.      24 OCT 84
↳ MANSFIELD/DODDS
    <FrameChannel: 'X ' "Computed X-axis" units: 'b'' count: 1 dimensions: (1,)
↳ frames: 1472>
    <FrameChannel: 'COND' "COND" units: 'b'' count: 1 dimensions: (1,) frames: 1472>
```

(continues on next page)

(continued from previous page)

```

<FrameChannel: 'SN'  ' "SN  " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'SP'  ' "SP  " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'GR'  ' "GR  " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'CAL' ' "CAL " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'TEN' ' "TEN " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'SPD' ' "SPD " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'ACQ' ' "ACQ " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'AC'  ' "AC  " units: 'b''' count: 1 dimensions: (1,) frames: 1472>
<FrameChannel: 'RT'  ' "RT  " units: 'b''' count: 1 dimensions: (1,) frames: 1472>

```

To get the actual values in the frame we can access the numpy array directly, either by channel ordinal or by name (spaces are significant):

```

>>> sp = frame_arrays[0].frame_array['SP  '].array
array([[ -2.49709030e+02],
       [ -2.49709030e+02],
       [ -2.49709030e+02],
       ...,
       [  9.99999962e-05],
       [  9.99999962e-05],
       [  9.99999962e-05]])

```

Now, if you are familiar with numpy then all normal operations are possible, for example get the min:

```

>>> sp.min()
-2.49709030e+02

```

Note: The `LogPass.FrameArray` is universal, but `BIT` can only represent one value per frame per channel. To access that value you need to use `array[frame_index][0]`.

References:

`BITFrameArray`: `TotalDepth.BIT.ReadBIT.BITFrameArray`

`FrameArray`: `TotalDepth.common.LogPass.FrameArray`

6.2 Processing LAS Files

6.2.1 Reading a LAS File Log Data

There are some example files in `example_data/LAS/data`, let's read one:

```

from TotalDepth.LAS.core import LASRead

las_file_path = os.path.join('example_data', 'LAS', 'data', 'BASIC_FILE_0_50.las')
las_file = LASRead.LASRead(las_file_path, las_file_path, raise_on_error=False)
# If there is an array section it will be initialised as a LogPass.FrameArray
if las_file.frame_array is not None:
    # Can iterate through the channels...
    for channel in las_file.frame_array.channels:

```

(continues on next page)

(continued from previous page)

```
# channel.array is a numpy masked array.
array = channel.array
print(f'{channel.ident:4} [{channel.units:4}] Shape: {array.shape} Minimum:
↪{array.min():8g}')
```

The output will typically be:

```
DEPT [m   ] Shape: (649, 1) Minimum:  2889.4
TENS [lbs ] Shape: (649, 1) Minimum: 5292.04
ETIM [min ] Shape: (649, 1) Minimum:   0.019
DHTN [lbs ] Shape: (649, 1) Minimum: 2562.92
GR   [api ] Shape: (649, 1) Minimum:  43.201
```

Note: The `LogPass.FrameArray` is universal, but LAS can only represent one value per frame per channel. To access that value you need to use `array[frame_index][0]`.

References:

FrameArray: `TotalDepth.common.LogPass.FrameArray`

Todo: Complete this.

6.3 Processing LIS Files

To use TotalDepth:

```
import TotalDepth
```

6.3.1 Important Concepts

Due to the size of data TotalDepth makes extensive use of lazy evaluation and generators, this means that compute power is only used where necessary.

Log Pass

A *Log Pass* represents a typical, independent, logging recording consisting of a format specification plus binary data. The format specification defines how to interpret the binary data (number of channels, name and units for each channel). In the LIS format the formatting record is known as the *Data Format Specification Record (DFSR)*. A “Repeat Section” plus a “Main Log” are two, distinct, independent Log Passes.

Frame Set

A *Frame Set* is the logging data converted to the platforms internal types (usually C doubles) and stored in memory as a table where each row represents a particular depth (or time) and each column is the output of a specific channel. Each row is often referred to as a *Frame*

6.3.2 Important LIS Concepts

LIS File

A LIS file is a wrapper round the platforms native I/O system.

Reference: `TotalDepth.LIS.core.File.FileRead`

LIS Index

The LIS file format is designed for recording rather than processing. As such it is a sequential self announcing format where to understand any part of it you have to have processed all of the preceeding data. This becomes very expensive with large files.

To avoid this an index is first created for the file that then allows random access to any part of that file including an individual frame. All access to the file is using this index. This index is very fast to create and its size is typically 1% of the original file size.

Reference: `TotalDepth.LIS.core.FileIndexer.FileIndex`

6.3.3 Reading a LIS File Table Data

Suppose that we have a file in `~/tmp/LIS.lis` and you want to read the well site data. First import what we need:

```
>>> import TotalDepth
>>> from TotalDepth.LIS.core import File
>>> from TotalDepth.LIS.core import FileIndexer
>>> import os
```

Then open the LIS file and create an index from it:

```
>>> fpath = os.path.expanduser('~tmp/LIS.lis')
>>> lis_file = TotalDepth.LIS.core.File.FileRead(fpath)
>>> lis_index = TotalDepth.LIS.core.FileIndexer.FileIndex(lis_file)
```

There is a method on the index `genAll()` that iterates through all the records, filter this for only table data:

```
>>> from TotalDepth.LIS.core import LogiRec
>>> cons_records = [lr for lr in lis_index.genAll() if lr.lrType in LogiRec.LR_TYPE_
↳ TABLE_DATA]
>>> cons_records
[<TotalDepth.LIS.core.FileIndexer.IndexTable object at 0x103ac3dd8>]
```

In this file there is only one table record. Now read it:

```
>>> lis_file.seekLr(cons_records[0].tell)
>>> table = LogiRec.LrTableRead(lis_file)
```

Now we can explore the table:

```
>>> table.desc
'Well site data'
>>> table.value
b'CONS'
>>> table.colLabels()
odict_keys([b'MNEM', b'ALLO', b'PUNI', b'TUNI', b'VALU'])
>>> table.rowLabels()
dict_keys([b'HIDE', b'HID1', b'HID2', b'CN ', b'WN ', ..., b'C30 '])
```

Notice all the entries are represented as Python bytes objects (b'...'), this is because LIS does not support Unicode. LIS is also a bit shouty.

To get a specific value, say the well name:

```
>>> print(table[b'WN '][b'VALU'])
CB: type=69 rc=65 size=16 mnem=b'VALU' EngValRc: b'GUSHER'
>>> table[b'WN '][b'VALU'].value
b'GUSHER'
```

You can index by integer:

```
>>> table[4][0].value
b'WN '
>>> table[4][4].value
b'GUSHER'
>>> [v.value for v in table[4]]
[b'WN ', b'ALLO', b' ', b' ', b'GROSSENKNETEN Z2']
```

You can index by slice:

```
>>> [v.value for v in table[4][:2]]
[b'WN ', b'ALLO']
```

To print the whole table there are some generators for this:

```
>>> for row in table.genRows():
...     for col in row.genCells():
...         print(col.value, ' ', end='')
...     print()
...
b'HIDE' b'ALLO' b' ' b' ' b'MAIN LOG'
b'HID1' b'ALLO' b' ' b' ' b'RAW DATA'
b'HID2' b'ALLO' b' ' b' ' b''
b'CN ' b'ALLO' b' ' b' ' b'BIG COMPANY'
b'WN ' b'ALLO' b' ' b' ' b'GUSHER'
...
```

Reference: `TotalDepth.LIS.core.LogiRec.LrTable`

6.3.4 Reading a LIS File Log Data

Suppose that we have a file in `~/tmp/LIS.lis` and you want to read the frame data from a particular log pass, first import what we need:

```
>>> import TotalDepth
>>> from TotalDepth.LIS.core import File
>>> from TotalDepth.LIS.core import FileIndexer
>>> import os
```

Then open the LIS file and create an index from it:

```
>>> fpath = os.path.expanduser('~tmp/LIS.lis')
>>> lis_file = TotalDepth.LIS.core.File.FileRead(fpath)
>>> lis_index = TotalDepth.LIS.core.FileIndexer.FileIndex(lis_file)
```

There is a method on the index `genLogPasses()` that iterates through the log passes, lets get them all:

```
>>> log_passes = list(lis_index.genLogPasses())
>>> print(log_passes)
[<TotalDepth.LIS.core.FileIndexer.IndexLogPass object at 0x103ac3e80>]
```

In this file there is only one log pass, we can get the description of it using `longstr()`:

```
>>> print(log_passes[0].logPass.longStr())
<TotalDepth.LIS.core.LogPass.LogPass object at 0x103ae10b8>:
  DFSR: <TotalDepth.LIS.core.LogiRec.LrDFSRRRead object at 0x103ac3eb8>: "Data_
↳ format specification record"
  Frame plan: <TotalDepth.LIS.core.Type01Plan.FrameSetPlan object at 0x103ae10f0>: indr=0_
↳ frame length=24 channels=6
    Channels: [b'DEPT', b'SP ', b'SN ', b'ILD ', b'CILD', b'DT ']
    RLE: <TotalDepth.LIS.core.Rle.RLEType01 object at 0x103ae1128>: func=None:_
↳ [RLEItemType01: datum=8592 stride=1014 repeat=7 frames=42, RLEItemType01: datum=16704_
↳ stride=None repeat=0 frames=39]
    X axis: first=2052.983 last=1995.986 frames=375 overall spacing=-0.1524 in optical_
↳ units=b'M ' (actual units=b'M ')
  Frame set: None
```

Note the last line `Frame set: None`, this is because the log pass is a lightweight object which does not (yet) contain all the frame data. To read all the frame data from the file we call `setFrameData(LisFile)` on the log pass:

```
>>> log_passes[0].logPass.setFrameSet(lis_file)
```

Note: The call to `setFrameSet()` will raise an Exception if there is no frame data in the Log Pass. The call should be wrapped in a check for frame data:

```
if log_passes[0].logPass.totalFrames > 0:
    log_passes[0].logPass.setFrameSet(lis_file)
    # Your code here...
```

Now the frame set is fully populated:

```
>>> print(list(log_passes[0].logPass.genFrameSetScNameUnit()))
[('DEPT', 'M '), ('SP ', 'MV '), ('SN ', 'OHMM'), ('ILD ', 'OHMM'), ('CILD', 'MMHO
↪ '), ('DT ', 'US/M')]
```

To get the actual vales in the frame we can access the numpy array directly:

```
>>> data = log_passes[0].logPass.frameSet.frames
>>> data
array([[ 2.05298340e+03, -4.54907703e+00,  1.34538269e+00,
         1.26347518e+00,  3.86598633e+02, -9.99250000e+02],
       [ 2.05283105e+03, -5.13720322e+00,  1.36061692e+00,
         1.29521227e+00,  5.00510803e+02, -9.99250000e+02],
       [ 2.05267871e+03, -6.66747475e+00,  1.38543439e+00,
         1.45785594e+00,  5.95623291e+02, -9.99250000e+02],
       ...,
       [ 1.99629077e+03, -9.99250000e+02, -9.99250000e+02,
        -9.99250000e+02, -9.99250000e+02, -9.99250000e+02],
       [ 1.99613843e+03, -9.99250000e+02, -9.99250000e+02,
        -9.99250000e+02, -9.99250000e+02, -9.99250000e+02],
       [ 1.99598608e+03, -9.99250000e+02, -9.99250000e+02,
        -9.99250000e+02, -9.99250000e+02, -9.99250000e+02]])
```

Now, if you are familiar with numpy then all normal operations are possible, for example get the X axis:

```
>>> data[:,0]
array([ 2052.98339844,  2052.83105469,  2052.67871094,  2052.52636719,
        2052.37402344,  2052.22167969,  2052.06933594,  2051.91650391,
        2051.76416016,  2051.61181641,  2051.45947266,  2051.30712891,
        ...,
        1996.29077148,  1996.13842773,  1995.98608398])
```

Find the min, mean, max:

```
>>> data.min(axis=0)
array([ 1995.98608398, -999.25      , -999.25      , -999.25      ,
        -999.25      , -999.25      ])
>>> data.mean(axis=0)
array([ 2024.48480339, -305.07223682, -326.84234802, -324.20620109,
        206.05499658,  28.2555695  ])
>>> data.max(axis=0)
array([ 2.05298340e+03, -7.69491196e-01,  1.98412299e+00,
        2.34852839e+00,  1.75242944e+03,  4.59522583e+02])
```

References:

LogPass: TotalDepth.LIS.core.LogPass.LogPass

FrameSet: TotalDepth.LIS.core.FrameSet.FrameSet

6.4 Processing RP66V1 Files

This describes how you can programmatically access the contents of an RP66V1 file with TotalDepth with worked examples. First, here is how TotalDepth represents the structure of a RP66V1 file.

6.4.1 RP66V1 File Structure

A physical RP66V1 file (typically a file on disk) consists of a *RP66V1.Storage Unit Label* followed of any number of *RP66V1.Explicitly Formatted Logical Record* (EFLRs) and any number of *RP66V1.Indirectly Formatted Logical Record* (IFLRs) in any order.

At this level the file conceptually looks like this:

```
Physical File
|-> Storage Unit Label
|-> EFLR
|-> EFLR
|-> IFLR
|-> EFLR
|-> IFLR
|-> EFLR
...
```

The EFLRs and IFLRs are organised into one or more any number of *Logical Files*. Specifically a Logical File starts with an FILE-HEADER EFLR followed by an ORIGIN or WELL-REFERENCE EFLR.

A Logical File might represent all the data recorded in, say, a “Repeat Section” and Logical Files are independent from each other¹.

This conceptual model for the Logical File delimited RP66V1 file is something like this:

```
Physical File
|-> Storage Unit Label
|-> Logical File[0] "Repeat Section"
    |-> EFLR type FILE-HEADER
    |-> EFLR type ORIGIN
    |-> EFLR
    |-> EFLR
    |-> IFLR
    |-> EFLR
    |-> IFLR
    ...
|-> Logical File[1] "Main Log"
    |-> EFLR type FILE-HEADER
    |-> EFLR type ORIGIN
    |-> EFLR
    ...
```

The collection of IFLRs for any Logical File can be further organised into a single *Log Pass*.

¹ RP66V1 provides a method of collecting together physical files by using the *RP66V1.Storage Unit Label* where the fields *RP66V1.Storage Set Identifier* and *RP66V1.Storage Unit Sequence Number* provide a means of linking physical files. In practice this has not been seen but if this is your use case then the class `TotalDepth.RP66V1.core.pFile.FileRead` can minimally read the Storage Unit Label and its fields.

Log Pass

A *Log Pass* represents a typical, independent, logging recording. A “Repeat Section” and a “Main Log” are two, distinct, independent Log Passes. The *Log Pass* is defined by two EFLRs, a CHANNEL that describes all the available measurements for that Logical File and a FRAME that describes the sets of these channels that go into a recording, the index and the spacing. The *Log Pass* represents a single data collection run (example: “Repeat Section”) and contains one or more *Frame Arrays*.

Frame Array

A *Frame Array* contains a number of frames of channel data converted to the platforms internal types (usually C doubles) and stored in memory as a table. In this table each row represents a particular value on the X axis (typically depth or time) and each column is the output of a specific channel. Each row is often referred to as a *Frame*. There can be multiple values for any channel in a frame except for the X axis which has a single value per frame. A value can be an *Absent Value* indicating that no data was recorded for this channel and frame.

Conceptual Model Presented by TotalDepth

By example here is a RP66V1 file that contains two Logical Files representing, say, the “Repeat Section” and the “Main Log”. Each of these has a number of EFLRs and a Log Pass that contains two Frame Arrays, one is sampled every inch in depth and the other every six inches. The two Frame Arrays might have different channels.

This model of the physical RP66V1 file will be something like:

```
Physical File
|-> Storage Unit Label
|-> Logical File[0] "Repeat Section"
|   |-> EFLRs
|   |-> ...
|   |-> Log Pass
|       |-> Frame Array at 1 inch spacing with channels A, B, C
|       |-> Frame Array at 6 inch spacing with channels B, D, E
|-> Logical File[1] "Main Log"
|   |-> EFLRs
|   |-> ...
|   |-> Log Pass
|       |-> Frame Array at 1 inch spacing with channels A, B, C, X
|       |-> Frame Array at 6 inch spacing with channels B, D, E, F
```

This model is exposed with the class `TotalDepth.RP66V1.core.LogicalFile.LogicalIndex`. The `LogicalIndex` contains a sequence of `TotalDepth.RP66V1.core.LogicalFile.LogicalFile` objects that allows random access to all parts of the file.

The following examples show how to iterate through a RP66V1 file with the `LogicalIndex` and access the tables and frame data. The code snippets here are all in `example_data/RP66V1/demo_read.py`

Warning: At this version, 0.4.0, these APIs are provisional, not final.

6.4.2 Basic Pattern for Reading RP66V1 Files with TotalDepth

All these examples take the following pattern where a `LogicalIndex` is created as a context manager.

This can be done with a file path as a string:

```
from TotalDepth.RP66V1.core import LogicalFile

with LogicalFile.LogicalIndex(path) as logical_index:
    # Do something
```

Or an open, binary, file:

```
from TotalDepth.RP66V1.core import LogicalFile

with open(path, 'rb') as fobj:
    with LogicalFile.LogicalIndex(fobj) as logical_index:
        # Do something
```

Example Data

There are some example RP66V1 files distributed in `example_data/RP66V1/data`, for example:

```
from TotalDepth.RP66V1.core import LogicalFile

path = os.path.join('example_data', 'RP66V1', 'data', '206_05a-_3_DWL_DWL_WIRE_258276498.
↳DLIS')
with LogicalFile.LogicalIndex(path) as logical_index:
    # Do something
```

There is also some example RP66V1 binary data in the module `tests.unit.RP66V1.core.test_data`, for example:

```
from TotalDepth.RP66V1.core import LogicalFile
from tests.unit.RP66V1.core import test_data

file_object = io.BytesIO(test_data.BASIC_FILE)
with LogicalFile.LogicalIndex(file_object) as logical_index:
    # Do something
```

6.4.3 Inspecting the Logical File

Once a `TotalDepth.RP66V1.core.LogicalFile.LogicalIndex` has been created the `TotalDepth.RP66V1.core.LogicalFile.LogicalFile` object can be accessed, for example:

```
from TotalDepth.RP66V1.core import LogicalFile
from tests.unit.RP66V1.core import test_data

file_object = io.BytesIO(test_data.BASIC_FILE)
with LogicalFile.LogicalIndex(file_object) as logical_index:
    for l, logical_file in enumerate(logical_index.logical_files):
        print(f'LogicalFile [{l}]: {logical_file}')
```

Produces the single Logical File in `BASIC_FILE`:

```
LogicalFile [0]: <TotalDepth.RP66V1.core.LogicalFile.LogicalFile object at 0x11ca5ec50>
```

The Logical File object has at least these properties:

Property	Description
file_header_logical_record	The FILE_HEADER Logical Record that defines the Logical File. See: [RP66V1 Section 5.1 File Header Logical Record (FHLR)].
origin_logical_record	The ORIGIN Logical Record that defines the origin of the Logical Record. See [RP66V1 Section 5.2 Origin Logical Record (OLR)].
defining_origin	the Defining Origin of the Logical File. This is the first row of the ORIGIN Logical Record. From [RP66V1 Section 5.2.1 Origin Objects]: “ <i>The first Object in the first ORIGIN Set is the Defining Origin for the Logical File in which it is contained, and the corresponding Logical File is called the Origin’s Parent File. It is intended that no two Logical Files will ever have Defining Origins with all Attribute Values identical.</i> ”
channel	The CHANNEL EFLR or None.
frame	The FRAME EFLR or None.
has_log_pass	True if this Logical File has both a CHANNEL and a FRAME EFLR.

Notes:

- **TotalDepth implements an EFLR as the class:** `TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord`
- **That class has a method, used below, that provides verbose information about the table:** `TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord.str_long()`

Here is an example of accessing all of the above properties for the BASIC_FILE:

```
from TotalDepth.RP66V1.core import LogicalFile
from tests.unit.RP66V1.core import test_data

file_object = io.BytesIO(test_data.BASIC_FILE)
with LogicalFile.LogicalIndex(file_object) as logical_index:
    for l, logical_file in enumerate(logical_index.logical_files):
        print(f'***** logical_file.file_header_logical_record.str_long():')
        print(logical_file.file_header_logical_record.str_long())
        print()
        print(f'***** logical_file.origin_logical_record.str_long():')
        print(logical_file.origin_logical_record.str_long())
        print()
        print(f'***** logical_file.defining_origin:')
        print(logical_file.defining_origin)
        print()
        if logical_file.channel is not None:
            print(f'***** logical_file.channel.str_long():')
            print(logical_file.channel.str_long())
            print()
        if logical_file.frame is not None:
            print(f'***** logical_file.frame.str_long():')
            print(logical_file.frame.str_long())
            print()
        print(f'***** logical_file.has_log_pass:')
```

(continues on next page)

(continued from previous page)

```
print(logical_file.has_log_pass)
print()
```

Gives:

```
***** logical_file.file_header_logical_record.str_long():
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'FILE-HEADER' name: b''>
Template [2]:
  CD: 001 10100 L: b'SEQUENCE-NUMBER' C: 1 R: 20 (ASCII) U: b'' V: None
  CD: 001 10100 L: b'ID' C: 1 R: 20 (ASCII) U: b'' V: None
Objects [1]:
  OBNAME: O: 2 C: 0 I: b'1'
  CD: 001 00001 L: b'SEQUENCE-NUMBER' C: 1 R: 20 (ASCII) U: b'' V: [b'00000000001']
  CD: 001 00001 L: b'ID' C: 1 R: 20 (ASCII) U: b'' V: [b'HES INSITE.1
  '']

***** logical_file.origin_logical_record.str_long():
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'ORIGIN' name: b''>
Template [20]:
  CD: 001 11000 L: b'FILE-ID' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'FILE-SET-NAME' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'FILE-SET-NUMBER' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'FILE-NUMBER' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'FILE-TYPE' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'PRODUCT' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'VERSION' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'PROGRAMS' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'CREATION-TIME' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'ORDER-NUMBER' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'DESCENT-NUMBER' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'RUN-NUMBER' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'WELL-ID' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'WELL-NAME' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'FIELD-NAME' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'PRODUCER-CODE' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'PRODUCER-NAME' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'COMPANY' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'NAME-SPACE-NAME' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 11000 L: b'NAME-SPACE-VERSION' C: 0 R: 19 (IDENT) U: b'' V: None
Objects [1]:
  OBNAME: O: 2 C: 0 I: b'0'
  CD: 001 01101 L: b'FILE-ID' C: 1 R: 20 (ASCII) U: b'' V: [b'HES INSITE.1']
  CD: 001 01101 L: b'FILE-SET-NAME' C: 1 R: 19 (IDENT) U: b'' V: [b'BURU ENERGY
  LIMITED/VALHALLA NORTH 1']
  CD: 001 01101 L: b'FILE-SET-NUMBER' C: 1 R: 18 (UVARI) U: b'' V: [257346645]
  CD: 001 01101 L: b'FILE-NUMBER' C: 1 R: 18 (UVARI) U: b'' V: [1]
  CD: 001 01101 L: b'FILE-TYPE' C: 1 R: 19 (IDENT) U: b'' V: [b'PLAYBACK']
  CD: 001 01101 L: b'PRODUCT' C: 1 R: 20 (ASCII) U: b'' V: [b'HES INSITE']
  CD: 001 01101 L: b'VERSION' C: 1 R: 20 (ASCII) U: b'' V: [b'R5.1.4']
  CD: 000 00000 L: b'PROGRAMS' C: 0 R: 19 (IDENT) U: b'' V: None
  CD: 001 01101 L: b'CREATION-TIME' C: 1 R: 21 (DTIME) U: b'' V: [<<class
  'TotalDepth.RP66V1.core.RepCode.DateTime'> 2012-03-07 10:00:49.000 STD>]
```

(continues on next page)

(continued from previous page)

```

CD: 001 01101 L: b'ORDER-NUMBER' C: 1 R: 20 (ASCII) U: b'' V: [b'9262611']
CD: 000 00000 L: b'DESCENT-NUMBER' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 000 00000 L: b'RUN-NUMBER' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 01101 L: b'WELL-ID' C: 1 R: 20 (ASCII) U: b'' V: [b'N/A']
CD: 001 01101 L: b'WELL-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'VALHALLA NORTH 1']
CD: 001 01101 L: b'FIELD-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'VALHALLA']
CD: 001 01101 L: b'PRODUCER-CODE' C: 1 R: 16 (UNORM) U: b'' V: [280]
CD: 001 01101 L: b'PRODUCER-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'Halliburton']
CD: 001 01101 L: b'COMPANY' C: 1 R: 20 (ASCII) U: b'' V: [b'BURU ENERGY LIMITED']
CD: 000 00000 L: b'NAME-SPACE-NAME' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 000 00000 L: b'NAME-SPACE-VERSION' C: 0 R: 19 (IDENT) U: b'' V: None

**** logical_file.defining_origin:
OBNAME: 0: 2 C: 0 I: b'0'
CD: 001 01101 L: b'FILE-ID' C: 1 R: 20 (ASCII) U: b'' V: [b'HES INSITE.1']
CD: 001 01101 L: b'FILE-SET-NAME' C: 1 R: 19 (IDENT) U: b'' V: [b'BURU ENERGY LIMITED/
↳ VALHALLA NORTH 1']
CD: 001 01101 L: b'FILE-SET-NUMBER' C: 1 R: 18 (UVARI) U: b'' V: [257346645]
CD: 001 01101 L: b'FILE-NUMBER' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01101 L: b'FILE-TYPE' C: 1 R: 19 (IDENT) U: b'' V: [b'PLAYBACK']
CD: 001 01101 L: b'PRODUCT' C: 1 R: 20 (ASCII) U: b'' V: [b'HES INSITE']
CD: 001 01101 L: b'VERSION' C: 1 R: 20 (ASCII) U: b'' V: [b'R5.1.4']
CD: 000 00000 L: b'PROGRAMS' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 01101 L: b'CREATION-TIME' C: 1 R: 21 (DTIME) U: b'' V: [<<class 'TotalDepth.
↳ RP66V1.core.RepCode.DateTime'> 2012-03-07 10:00:49.000 STD>]
CD: 001 01101 L: b'ORDER-NUMBER' C: 1 R: 20 (ASCII) U: b'' V: [b'9262611']
CD: 000 00000 L: b'DESCENT-NUMBER' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 000 00000 L: b'RUN-NUMBER' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 01101 L: b'WELL-ID' C: 1 R: 20 (ASCII) U: b'' V: [b'N/A']
CD: 001 01101 L: b'WELL-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'VALHALLA NORTH 1']
CD: 001 01101 L: b'FIELD-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'VALHALLA']
CD: 001 01101 L: b'PRODUCER-CODE' C: 1 R: 16 (UNORM) U: b'' V: [280]
CD: 001 01101 L: b'PRODUCER-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'Halliburton']
CD: 001 01101 L: b'COMPANY' C: 1 R: 20 (ASCII) U: b'' V: [b'BURU ENERGY LIMITED']
CD: 000 00000 L: b'NAME-SPACE-NAME' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 000 00000 L: b'NAME-SPACE-VERSION' C: 0 R: 19 (IDENT) U: b'' V: None

**** logical_file.channel.str_long():
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'CHANNEL' name: b''>
Template [8]:
CD: 001 11100 L: b'LONG-NAME' C: 0 R: 20 (ASCII) U: b'' V: None
CD: 001 11100 L: b'PROPERTIES' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 11100 L: b'REPRESENTATION-CODE' C: 0 R: 15 (USHORT) U: b'' V: None
CD: 001 11100 L: b'DIMENSION' C: 0 R: 18 (UVARI) U: b'' V: None
CD: 001 11100 L: b'ELEMENT-LIMIT' C: 0 R: 18 (UVARI) U: b'' V: None
CD: 001 11100 L: b'UNITS' C: 0 R: 27 (UNITS) U: b'' V: None
CD: 001 11100 L: b'AXIS' C: 0 R: 23 (OBNAME) U: b'' V: None
CD: 001 11100 L: b'SOURCE' C: 0 R: 24 (OBJREF) U: b'' V: None
Objects [5]:
OBNAME: 0: 2 C: 0 I: b'DEPT'
CD: 001 01101 L: b'LONG-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'DEPT/Depth']
CD: 000 00000 L: b'PROPERTIES' C: 0 R: 19 (IDENT) U: b'' V: None

```

(continues on next page)

(continued from previous page)

```

CD: 001 01001 L: b'REPRESENTATION-CODE' C: 1 R: 15 (USHORT) U: b'' V: [7]
CD: 001 01001 L: b'DIMENSION' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'ELEMENT-LIMIT' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'UNITS' C: 1 R: 27 (UNITS) U: b'' V: [b'm']
CD: 000 00000 L: b'AXIS' C: 0 R: 23 (OBNAME) U: b'' V: None
CD: 000 00000 L: b'SOURCE' C: 0 R: 24 (OBJREF) U: b'' V: None
OBNAME: 0: 2 C: 0 I: b'TENS'
CD: 001 01101 L: b'LONG-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'TENS/Tension']
CD: 000 00000 L: b'PROPERTIES' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 01001 L: b'REPRESENTATION-CODE' C: 1 R: 15 (USHORT) U: b'' V: [2]
CD: 001 01001 L: b'DIMENSION' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'ELEMENT-LIMIT' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'UNITS' C: 1 R: 27 (UNITS) U: b'' V: [b'lbs']
CD: 000 00000 L: b'AXIS' C: 0 R: 23 (OBNAME) U: b'' V: None
CD: 001 01001 L: b'SOURCE' C: 1 R: 24 (OBJREF) U: b'' V: [ObjectReference(T=b'TOOL
↳', N=ObjectName(O=2, C=0, I=b'DEP'))]]
OBNAME: 0: 2 C: 0 I: b'ETIM'
CD: 001 01101 L: b'LONG-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'ETIM/Elapsed Time']
CD: 000 00000 L: b'PROPERTIES' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 01001 L: b'REPRESENTATION-CODE' C: 1 R: 15 (USHORT) U: b'' V: [7]
CD: 001 01001 L: b'DIMENSION' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'ELEMENT-LIMIT' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'UNITS' C: 1 R: 27 (UNITS) U: b'' V: [b'min']
CD: 000 00000 L: b'AXIS' C: 0 R: 23 (OBNAME) U: b'' V: None
CD: 001 01001 L: b'SOURCE' C: 1 R: 24 (OBJREF) U: b'' V: [ObjectReference(T=b'TOOL
↳', N=ObjectName(O=2, C=0, I=b'DEP'))]]
OBNAME: 0: 2 C: 0 I: b'DHTN'
CD: 001 01101 L: b'LONG-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'DHTN/CH Tension']
CD: 000 00000 L: b'PROPERTIES' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 01001 L: b'REPRESENTATION-CODE' C: 1 R: 15 (USHORT) U: b'' V: [2]
CD: 001 01001 L: b'DIMENSION' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'ELEMENT-LIMIT' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'UNITS' C: 1 R: 27 (UNITS) U: b'' V: [b'lbs']
CD: 000 00000 L: b'AXIS' C: 0 R: 23 (OBNAME) U: b'' V: None
CD: 001 01001 L: b'SOURCE' C: 1 R: 24 (OBJREF) U: b'' V: [ObjectReference(T=b'TOOL
↳', N=ObjectName(O=2, C=0, I=b'RWCH'))]]
OBNAME: 0: 2 C: 0 I: b'GR'
CD: 001 01101 L: b'LONG-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'GR/Gamma API']
CD: 000 00000 L: b'PROPERTIES' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 01001 L: b'REPRESENTATION-CODE' C: 1 R: 15 (USHORT) U: b'' V: [2]
CD: 001 01001 L: b'DIMENSION' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'ELEMENT-LIMIT' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 001 01001 L: b'UNITS' C: 1 R: 27 (UNITS) U: b'' V: [b'api']
CD: 000 00000 L: b'AXIS' C: 0 R: 23 (OBNAME) U: b'' V: None
CD: 001 01001 L: b'SOURCE' C: 1 R: 24 (OBJREF) U: b'' V: [ObjectReference(T=b'TOOL
↳', N=ObjectName(O=2, C=0, I=b'D4TG'))]]

***** logical_file.frame.str_long():
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'FRAME' name: b''>
Template [8]:
CD: 001 11100 L: b'DESCRIPTION' C: 0 R: 20 (ASCII) U: b'' V: None
CD: 001 11100 L: b'CHANNELS' C: 0 R: 23 (OBNAME) U: b'' V: None

```

(continues on next page)

(continued from previous page)

```
CD: 001 11100 L: b'INDEX-TYPE' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 11100 L: b'DIRECTION' C: 0 R: 19 (IDENT) U: b'' V: None
CD: 001 11100 L: b'SPACING' C: 0 R: 7 (FDOUBL) U: b'' V: None
CD: 001 11100 L: b'ENCRYPTED' C: 0 R: 15 (USHORT) U: b'' V: None
CD: 001 11100 L: b'INDEX-MIN' C: 0 R: 7 (FDOUBL) U: b'' V: None
CD: 001 11100 L: b'INDEX-MAX' C: 0 R: 7 (FDOUBL) U: b'' V: None
Objects [1]:
  OBNAME: 0: 2 C: 0 I: b'50'
    CD: 000 00000 L: b'DESCRIPTION' C: 0 R: 20 (ASCII) U: b'' V: None
    CD: 001 01001 L: b'CHANNELS' C: 5 R: 23 (OBNAME) U: b'' V: [ObjectName(O=2, C=0, I=b'DEPT'), ObjectName(O=2, C=0, I=b'TENS'), ObjectName(O=2, C=0, I=b'ETIM'), ObjectName(O=2, C=0, I=b'DHTN'), ObjectName(O=2, C=0, I=b'GR')]
    CD: 001 01001 L: b'INDEX-TYPE' C: 1 R: 19 (IDENT) U: b'' V: [b'BOREHOLE-DEPTH']
    CD: 001 01001 L: b'DIRECTION' C: 1 R: 19 (IDENT) U: b'' V: [b'INCREASING']
    CD: 001 01111 L: b'SPACING' C: 1 R: 7 (FDOUBL) U: b'm' V: [0.1]
    CD: 000 00000 L: b'ENCRYPTED' C: 0 R: 15 (USHORT) U: b'' V: None
    CD: 000 00000 L: b'INDEX-MIN' C: 0 R: 7 (FDOUBL) U: b'' V: None
    CD: 000 00000 L: b'INDEX-MAX' C: 0 R: 7 (FDOUBL) U: b'' V: None

**** logical_file.has_log_pass:
True
```

6.4.4 More about RP66V1.EFLR Tables

An *RP66V1.Explicitly Formatted Logical Record* is a table of data organised in rows and columns.

Table	
Row	
Value	
Value	
...	
Row	
Value	
Value	
...	
...	

This is implemented by TotalDepth as:

- Table: TotalDepth.RP66V1.core.LogicalRecord.EFLR.ExplicitlyFormattedLogicalRecord
- Row is an Object: TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object
- Value is an Attribute: TotalDepth.RP66V1.core.LogicalRecord.EFLR.Attribute

Reading EFLR Contents

Each value in a row/column is known as an *RP66V1.Attribute*

This is implemented by `TotalDepth.RP66V1.core.LogicalRecord.EFLR.Attribute` which has the following properties:

Property	Type	Description
label	bytes	The label identifying the Attribute.
count	int	The number of the values the Attribute has.
rep_code	int	The Representation Code of the values of the Attribute.
units	bytes	The units of the value.
value	list	The value itself as a list of instances of the Representation Code.

These Attributes are iterable, for example the following code accesses the contents of every PARAMETER EFLR:

```
from TotalDepth.RP66V1.core import LogicalFile
from tests.unit.RP66V1.core import test_data

file_object = io.BytesIO(test_data.BASIC_FILE)
with LogicalFile.LogicalIndex(file_object) as logical_index:
    for logical_file in logical_index.logical_files:
        for position, eflr in logical_file.eflrs:
            # eflr is a TotalDepth.RP66V1.core.LogicalRecord.EFLR.
            ↳ ExplicitlyFormattedLogicalRecord
            if eflr.set.type == b'PARAMETER':
                print(eflr)
                for row in eflr.objects:
                    # row is a TotalDepth.RP66V1.core.LogicalRecord.EFLR.Object
                    print(f'    Row: {row.name.I}')
                    for attr in row.attrs:
                        # attr is a TotalDepth.RP66V1.core.LogicalRecord.EFLR.Attribute
                        print(f'        Attr: {attr.label} = {attr.value} ({attr.units})
            ↳ ')
```

Will produce something like this (output truncated):

```
<ExplicitlyFormattedLogicalRecord EFLR Set type: b'PARAMETER' name: b''>
  Row: b'LOC'
    Attr: b'LONG-NAME' = [b'LOCATION'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b"LATITUDE: 18DEG 01' 32.8'' S"] (b'')
  Row: b'SVCO'
    Attr: b'LONG-NAME' = [b'SERVICECONAME'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'Halliburton'] (b'')
  Row: b'IQVR'
    Attr: b'LONG-NAME' = [b'WLIQ VERSION'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
```

(continues on next page)

(continued from previous page)

```

    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'R3.2.0'] (b'')
Row: b'STAT'
    Attr: b'LONG-NAME' = [b'STATE NAME'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'WA'] (b'')
Row: b'COUN'
    Attr: b'LONG-NAME' = [b'COUNTRY NAME'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'AUSTRALIA'] (b'')
Row: b'SON'
    Attr: b'LONG-NAME' = [b'JOB NUMBER'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'9262611'] (b'')
Row: b'SECT'
    Attr: b'LONG-NAME' = [b'SECTION'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'N/A'] (b'')
Row: b'TOWN'
    Attr: b'LONG-NAME' = [b'TOWNSHIP'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'N/A'] (b'')
Row: b'RANG'
    Attr: b'LONG-NAME' = [b'RANGE'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'N/A'] (b'')
Row: b'APIN'
    Attr: b'LONG-NAME' = [b'API S/N'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'N/A'] (b'')
Row: b'CN'
    Attr: b'LONG-NAME' = [b'CUSTOMER NAME'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'BURU ENERGY LIMITED'] (b'')
Row: b'WN'

```

(continues on next page)

(continued from previous page)

```

Attr: b'LONG-NAME' = [b'WELL NAME'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [b'VALHALLA NORTH 1'] (b'')
Row: b'FN'
Attr: b'LONG-NAME' = [b'FIELD NAME'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [b'VALHALLA'] (b'')
Row: b'RIG'
Attr: b'LONG-NAME' = [b'RIG NAME'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [b'ENSIGN RIG #32'] (b'')
Row: b'PDAT'
Attr: b'LONG-NAME' = [b'PERMANENT DATUM'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [b'MSL'] (b'')
Row: b'LMF'
Attr: b'LONG-NAME' = [b'LOG MEAS FROM'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [b'RT'] (b'')
Row: b'DMF'
Attr: b'LONG-NAME' = [b'DRILL MEAS FROM'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [b'RT'] (b'')
Row: b'FL1'
Attr: b'LONG-NAME' = [b'LOCATIONLINE1'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [b"LATITUDE: 18DEG 01' 32.8'' S"] (b'')
Row: b'FL2'
Attr: b'LONG-NAME' = [b'LOCATIONLINE2'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [b"LONGITUDE: 124DEG 43' 47.1'' E"] (b'')
Row: b'FL3'
Attr: b'LONG-NAME' = [b'LOCATIONLINE3'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')

```

(continues on next page)

(continued from previous page)

```

    Attr: b'VALUES' = [b'EASTING: 683112'] (b'')
Row: b'FL4'
    Attr: b'LONG-NAME' = [b'LOCATIONLINE4'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'NORTHING: 8006107'] (b'')
Row: b'FL5'
    Attr: b'LONG-NAME' = [b'LOCATIONLINE5'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'GDA ZONE 51'] (b'')
Row: b'DATE'
    Attr: b'LONG-NAME' = [b'DATE'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'06-Mar-2012'] (b'')
Row: b'LCC'
    Attr: b'LONG-NAME' = [b'PRODUCER-CODE'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [b'280'] (b'')
Row: b'EDF'
    Attr: b'LONG-NAME' = [b'DF ELEV'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [114.90000015258789] (b'm')
Row: b'EPD'
    Attr: b'LONG-NAME' = [b'ELEVATION'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [0.0] (b'm')
Row: b'EGL'
    Attr: b'LONG-NAME' = [b'GL ELEV'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [109.0] (b'm')
Row: b'GVFD'
    Attr: b'LONG-NAME' = [b'GRAVITY FIELD'] (b'')
    Attr: b'DIMENSION' = [1] (b'')
    Attr: b'AXIS' = None (b'')
    Attr: b'ZONES' = None (b'')
    Attr: b'VALUES' = [1.0] (b'g')
Row: b'EKB'
    Attr: b'LONG-NAME' = [b'KB ELEV'] (b'')
    Attr: b'DIMENSION' = [1] (b'')

```

(continues on next page)

(continued from previous page)

```

Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [114.90000015258789] (b'm')
Row: b'TVDS'
Attr: b'LONG-NAME' = [b'TVDSS CORRECTN'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [5.900000057220459] (b'm')
Row: b'APD'
Attr: b'LONG-NAME' = [b'DEPATH ABOVE PD'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [5.900000057220459] (b'm')
Row: b'DDEV'
Attr: b'LONG-NAME' = [b'MAX INC'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [1.82000000524520874] (b'deg')
Row: b'DDEG'
Attr: b'LONG-NAME' = [b'MAX INC DEPTH'] (b'')
Attr: b'DIMENSION' = [1] (b'')
Attr: b'AXIS' = None (b'')
Attr: b'ZONES' = None (b'')
Attr: b'VALUES' = [2225.169921875] (b'm')
...

```

Or the Attributes can be extracted by identity or integer index, for example:

```

file_object = io.BytesIO(test_data.BASIC_FILE)
with LogicalFile.LogicalIndex(file_object) as logical_index:
    for logical_file in logical_index.logical_files:
        for position, eflr in logical_file.eflrs:
            if eflr.set.type == b'PARAMETER':
                print(eflr[0])
                print()
                print(eflr[0][0])

```

Gives:

```

OBNAME: 0: 2 C: 0 I: b'LOC'
CD: 001 01101 L: b'LONG-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'LOCATION']
CD: 001 01001 L: b'DIMENSION' C: 1 R: 18 (UVARI) U: b'' V: [1]
CD: 000 00000 L: b'AXIS' C: 0 R: 23 (OBNAME) U: b'' V: None
CD: 000 00000 L: b'ZONES' C: 0 R: 23 (OBNAME) U: b'' V: None
CD: 001 01101 L: b'VALUES' C: 1 R: 20 (ASCII) U: b'' V: [b"LATITUDE: 18DEG 01' 32.8'' S
↪"]

CD: 001 01101 L: b'LONG-NAME' C: 1 R: 20 (ASCII) U: b'' V: [b'LOCATION']

```

6.4.5 Reading the Frame Data and Accessing the numpy Arrays

Here is an example of accessing the numpy arrays and using `np.describe()` to describe each array:

```
import numpy as np

from TotalDepth.RP66V1.core import LogicalFile

with LogicalFile.LogicalIndex(path_in) as logical_index:
    for logical_file in logical_index.logical_files:
        if logical_file.has_log_pass:
            for frame_array in logical_file.log_pass:
                print(frame_array)
                frame_count = logical_file.populate_frame_array(frame_array)
                print(
                    f'Loaded {frame_count} frames and {len(frame_array)} channels'
                    f' from {frame_array.ident} using {frame_array.sizeof_array} bytes.'
                )
                for channel in frame_array.channels:
                    print(channel)
                    # channel.array is a numpy array
                    np.info(channel.array)
                    print()
```

The output will be:

```
Loaded 921 frames and 4 channels from OBNAME: 0: 2 C: 0 I: b'2000T' using 14736 bytes.
```

```
FrameChannel: OBNAME: 0: 2 C: 4 I: b'TIME'          Rc:  2 Co:    1 Un: b'ms'
↳Di: [1] b'1 second River Time'
```

```
class: ndarray
shape: (921, 1)
strides: (4, 4)
itemsize: 4
aligned: True
contiguous: True
fortran: True
data pointer: 0x7faa710b6000
byteorder: little
byteswap: False
type: float32
```

```
FrameChannel: OBNAME: 0: 2 C: 4 I: b'TDEP'          Rc:  2 Co:    1 Un: b'0.1 in'
↳Di: [1] b'1 second River Depth'
```

```
class: ndarray
shape: (921, 1)
strides: (4, 4)
itemsize: 4
aligned: True
contiguous: True
fortran: True
data pointer: 0x7faa710b7000
byteorder: little
byteswap: False
```

(continues on next page)

(continued from previous page)

```

type: float32

FrameChannel: OBNAME: 0: 2 C: 0 I: b'TENS_SL'          Rc: 2 Co: 1 Un: b'lbf'
↳Di: [1] b'Cable Tension'
class: ndarray
shape: (921, 1)
strides: (4, 4)
itemsize: 4
aligned: True
contiguous: True
fortran: True
data pointer: 0x7fae6c8c2600
byteorder: little
byteswap: False
type: float32

FrameChannel: OBNAME: 0: 2 C: 0 I: b'DEPT_SL'          Rc: 2 Co: 1 Un: b'0.1 in'
↳Di: [1] b'Station logging depth'
class: ndarray
shape: (921, 1)
strides: (4, 4)
itemsize: 4
aligned: True
contiguous: True
fortran: True
data pointer: 0x7fae6c8c3600
byteorder: little
byteswap: False
type: float32
...

```

Making Calculations on the numpy Data

Very similar to the above we can make some calculations using standard numpy calls:

```

import numpy as np

from TotalDepth.RP66V1.core import LogicalFile

with LogicalFile.LogicalIndex(path_in) as logical_index:
    for logical_file in logical_index.logical_files:
        if logical_file.has_log_pass:
            for frame_array in logical_file.log_pass:
                print(frame_array)
                frame_count = logical_file.populate_frame_array(frame_array)
                print(
                    f'Loaded {frame_count} frames and {len(frame_array)} channels'
                    f' from {frame_array.ident} using {frame_array.sizeof_array} bytes.'
                )
                for channel in frame_array.channels:
                    print(channel.ident, channel.long_name, channel.units)

```

(continues on next page)

(continued from previous page)

```
# channel.array is a numpy array
print(f'Min: {channel.array.min():12.3f} Max: {channel.array.
↪max():12.3f}')
```

Would give this output:

```
FrameArray: ID: OBNAME: 0: 2 C: 0 I: b'2000T' b''
  FrameChannel: OBNAME: 0: 2 C: 4 I: b'TIME'          Rc:  2 Co:    1 Un: b'ms'      ↪
↪ Di: [1] b'1 second River Time'
  FrameChannel: OBNAME: 0: 2 C: 4 I: b'TDEP'          Rc:  2 Co:    1 Un: b'0.1 in'   ↪
↪ Di: [1] b'1 second River Depth'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'TENS_SL'        Rc:  2 Co:    1 Un: b'lbf'      ↪
↪ Di: [1] b'Cable Tension'
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'DEPT_SL'        Rc:  2 Co:    1 Un: b'0.1 in'   ↪
↪ Di: [1] b'Station logging depth'
Loaded 921 frames and 4 channels from OBNAME: 0: 2 C: 0 I: b'2000T' using 14736 bytes.
OBNAME: 0: 2 C: 4 I: b'TIME' b'1 second River Time' b'ms'
Min: 16677259.000 Max: 17597260.000
OBNAME: 0: 2 C: 4 I: b'TDEP' b'1 second River Depth' b'0.1 in'
Min:  852606.000 Max:  893302.000
OBNAME: 0: 2 C: 0 I: b'TENS_SL' b'Cable Tension' b'lbf'
Min:   1825.000 Max:   2594.000
OBNAME: 0: 2 C: 0 I: b'DEPT_SL' b'Station logging depth' b'0.1 in'
Min:  852606.000 Max:  893303.000

FrameArray: ID: OBNAME: 0: 2 C: 0 I: b'800T' b''
  FrameChannel: OBNAME: 0: 2 C: 5 I: b'TIME'          Rc:  2 Co:    1 Un: b'ms'      ↪
↪ Di: [1] b'400 milli-second time channel'
  FrameChannel: OBNAME: 0: 2 C: 5 I: b'TDEP'          Rc:  2 Co:    1 Un: b'0.1 in'   ↪
↪ Di: [1] b'MSCT depth channel'
  FrameChannel: OBNAME: 0: 2 C: 1 I: b'ETIM'          Rc:  2 Co:    1 Un: b's'        ↪
↪ Di: [1] b'Elapsed Logging Time'
... Lots more omitted
  FrameChannel: OBNAME: 0: 2 C: 0 I: b'CMLP'          Rc:  2 Co:    1 Un: b'in'        ↪
↪ Di: [1] b'Coring Motor Linear Position'
Loaded 2301 frames and 43 channels from OBNAME: 0: 2 C: 0 I: b'800T' using 395772 bytes.
OBNAME: 0: 2 C: 5 I: b'TIME' b'400 milli-second time channel' b'ms'
Min: 16677259.000 Max: 17597260.000
OBNAME: 0: 2 C: 5 I: b'TDEP' b'MSCT depth channel' b'0.1 in'
Min:  852606.000 Max:  893304.000
OBNAME: 0: 2 C: 1 I: b'ETIM' b'Elapsed Logging Time' b's'
Min:    0.000 Max:   920.001
... Lots more omitted
OBNAME: 0: 2 C: 0 I: b'CMLP' b'Coring Motor Linear Position' b'in'
Min:   -0.927 Max:    2.891
```

Limiting the Amount of Data Read

The RP66V1 Frame Array can be very large so to make it more manageable the `TotalDepth.RP66V1.core.LogicalFile.LogicalIndex.populate_frame_array()` can take the following, optional, arguments:

- *channels*: A sequence of channel identifiers. Only these channels will be populated into the numpy arrays in the Frame Array. The other channels will have a zero length numpy array. Channel 0, the X axis, will always be populated.
- *frame_slice* to reduce the number of frames that are populated. You can use either of these classes:
 - `TotalDepth.common.Slice.Slice` which takes optional start, stop, step values that default to `(0, len(data), 1)`. For example if there are 128 frames available then `Slice(64, None, 2)` would populate every other frame from frame 64 to the end.
 - `TotalDepth.common.Slice.Split` which takes single integer, this is maximum number of frames to be populated and they will be evenly spaced throughout the Frame Array. For example if there are 128 available frames that `Split(8)` would populate each numpy array with every 16th frame producing 8 frames.

For example, adding the two highlighted lines which populates every 64th frame and channels 1 and 2:

```
from TotalDepth.RP66V1.core import LogicalFile
from TotalDepth.common import Slice

with LogicalFile.LogicalIndex(path_in) as logical_index:
    for logical_file in logical_index.logical_files:
        if logical_file.has_log_pass:
            for frame_array in logical_file.log_pass:
                frame_count = logical_file.populate_frame_array(
                    frame_array,
                    frame_slice=Slice.Slice(0, None, 64),
                    channels={frame_array.channels[1].ident, frame_array.channels[2].
↪ident}
                )
                print(
                    f'Loaded {frame_count} frames'
                    f' from {frame_array.ident} using {frame_array.sizeof_array} bytes.'
                )
                for channel in frame_array.channels:
                    if len(channel.array):
                        print(channel.ident, channel.long_name, channel.units)
                        print(f'Min: {channel.array.min():12.3f} Max: {channel.array.
↪max():12.3f}')
                print()
```

Gives:

```
Loaded 15 frames from OBNAME: 0: 2 C: 0 I: b'2000T' using 180 bytes.
OBNAME: 0: 2 C: 4 I: b'TIME' b'1 second River Time' b'ms'
Min: 16677259.000 Max: 17573260.000
OBNAME: 0: 2 C: 4 I: b'TDEP' b'1 second River Depth' b'0.1 in'
Min: 852606.000 Max: 892658.062
OBNAME: 0: 2 C: 0 I: b'TENS_SL' b'Cable Tension' b'lbf'
Min: 1877.000 Max: 2561.000

Loaded 36 frames from OBNAME: 0: 2 C: 0 I: b'800T' using 432 bytes.
```

(continues on next page)

(continued from previous page)

```
OBNAME: 0: 2 C: 5 I: b'TIME' b'400 milli-second time channel' b'ms'  
Min: 16677259.000 Max: 17573260.000  
OBNAME: 0: 2 C: 5 I: b'TDEP' b'MSCT depth channel' b'0.1 in'  
Min: 852606.000 Max: 893135.188  
OBNAME: 0: 2 C: 1 I: b'ETIM' b'Elapsed Logging Time' b's'  
Min: 0.000 Max: 896.001
```


TECHNICAL NOTES

This collect together various technical descriptions of how TotalDepth solves certain problems.

Contents:

7.1 Performance of TotalDepth

This technical note presents the results of some actual performance tests on TotalDepth.

7.2 Measuring Performance

This describes some principles used in establishing TotalDepth's performance and setting performance targets.

7.2.1 User's Perception of Performance

Users only want to pay for what they get and experienced users have a rough idea of the cost of what they ask the software to do. For example most users would regard these a cheap operations, and would not expect them to take much time. If they did the user is likely to regard the application as 'slow':

- Load file
- Show log header
- Plot small section

Most users would regard these more expensive operations and is more likely to accept that they would take more time.:

- Cross correlate multiple curves
- Dipmeter processing
- Deconvolution

Most users are aware of the size of the data set with which the are operating and appreciate that operations on larger data sets take longer time. Users do not appreciate $O(N^2)$ or worse behaviour. We don't like it either¹.

TotalDepth measures the cost of operations, in:

¹ It is very easy for software developers to fail to see this kind of behaviour. For example if the time for an operation is: $a + b * N + c * N^2$ and $c \ll b$. If the software test suit tests an insufficiently small size of N then it appears that the operation is $O(N)$. Along comes a user with a large data set and they see $O(N^2)$ behaviour. This (or worse) is quite commonly observed in many software products.

```

Execution time (ms)
-----
Size of input (Mb)

```

Example of LIS cost

Our measure is ms/Mb of input. 1Mb of LIS data is typically 250,000 values, or, to put this in context 200 feet of 10 curves (6" sampling) is 0.015 Mb. So, just as an example, the cost of plotting such data from a 20Mb file *might* work out as:

Operation	Cost (ms/Mb)	Data Size (Mb)	User's Time (ms)	Notes
Index a 20Mb file	12	20	240	One-off exercise
Reading 200 feet, 10 curves	1500	0.015	22	
Plotting what has been read	4000	0.015	60	
Total	.	.	322	

The rest of this tech note describes the performance of reading LIS files in these ms/Mb terms.

7.2.2 Performance Improvements

The low level performance of TotalDepth is pretty good. FrameSet performance is satisfactory. Further improvement is certain for *Indexing Performance Improvements* once the existing C code (in another project) is integrated into this one.

Populating the frame is a costly exercise and the current solution takes this path:

```

File bytes -> Cython convert to C double -> convert to Python float -> insert into a
numpy array.

```

All this boxing and unboxing is expensive and a faster (but with more code complexity) is to populate the numpy array directly so this all happens in C code:

```

File bytes -> Convert to C double -> copy directly into numpy memory space

```

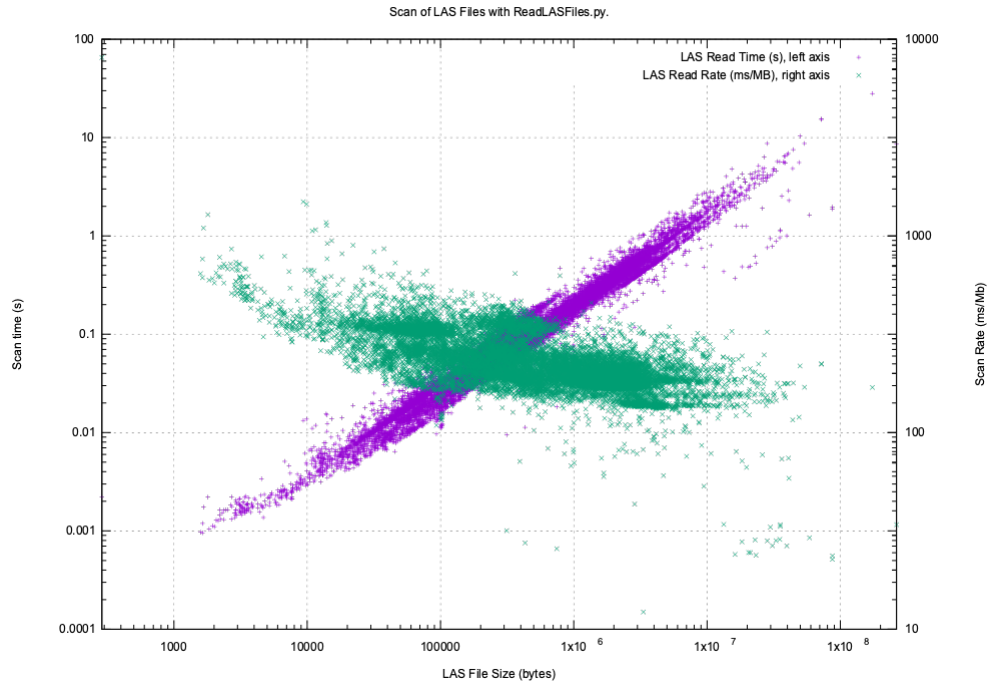
This should provide a great speedup.

The SVG creation is also worth looking at.

7.3 LAS Performance

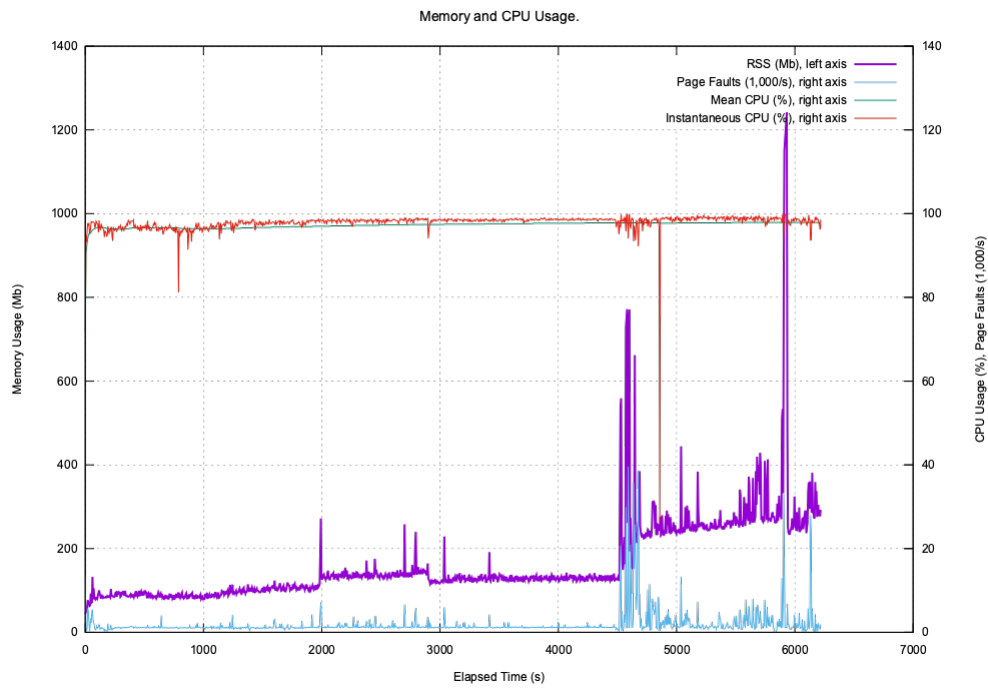
7.3.1 Reading LAS Files

Here is the execution time of reading LAS files with TotalDepth.LAS.ReadLASFiles as a single process on 23,000+ real world LAS files totalling 35+Gb, 170+m frames and 2.8+ billion data points. Success rate was >98%. Largest file was 260 Mb:



The overall performance is asymptotic to 180 ms/MB.

The memory usage of doing this follows this pattern:



7.4 Indexing LIS Files

Most petrophysical files are recorded in real time and the recording format is sequential, thus everything depends on what has gone before. This is not very satisfactory for the user who might well wish to access the data in a arbitrary manner - “give me these curves over this interval”. The solution is to create an index to the original file so that it can be accessed *as if* it is a random access file.

Here we describe how that indexing works and the performance it achieves for the user.

7.4.1 Introduction

The LIS file format is a binary, self describing, sequential format with multiple layers of encoding and, in practice, no forward references. LIS files can be large and, generally speaking, the greater part of the file consists of frame data whose format is invariant within any particular *Log Pass*.

In a dynamic situation, such as a user reading the file, the *instantaneous* amount of data needed from a LIS file is small compared with the file size. For example plotting 200 feet of a 50Mb file might need only 1/3000 of the data in the file. As the LIS file format is geared to sequential recording, not random access, accessing such a small amount efficiently needs additional software cunning.

TotalDepth’s approach to this is to use *indexing*. The essential requirements for an indexer are:

- Fast to create an index.
- The index is small.
- The index has sufficiently useful granularity.
- The index can be serialised in a number of ways (as a minimum; XML, binary (e.g. ‘pickle’), JSON?).
- The indexer design is flexible and extensible.

Apart from the cost of design and coding a solution the cost/benefit of indexing can be measured thus:

- The time to creating an index.
- The space required by the index, in-memory or serialised in some form.
- The time, in $O(N)$, terms of accessing N bytes of data.

A design that has low time/space requirements is regarded as a ‘good’ design.

7.4.2 Indexing Design

TotalDepth’s LIS indexer works on several levels:

Level	Description
File Level	Indexing of the position of all Logical Records
Logical Records with frame data	Runs of these can be efficiently indexed with Run Length Encoding
Within a Logical Record containing frame data	Accessing a particular frame and channel can be by computation with the help of a frame index. In particular this can identify seek/read sequences of any value from any channel in $O(1)$ time.

Indexing a LIS File

The intention is to find the start position of each Logical Record and a minimal amount of information of that Logical Record. The start position is the `size_t` value of the file index of, either, the start of the TIF marker for the first Physical Record in the Logical Record (if TIF encoded), or, the start of the first Physical Record in the Logical Record (if *not* TIF encoded).

As well as recording the file position of the Logical Record the indexer retrieves part or all of the Logical Record contents. This is specialised by Logical Record type thus:

Logical Record	Index Contents
Reel/Tape/File Header/Trailer	The complete contents of the Logical Record. These are small (58 or 128 bytes) but possibly important.
Table records	The Logical Record header and the first Component Block. These are variable, large, but possibly important.
DFSR	The complete contents of the Logical Record. These are variable size, not particularly large and very important.
Logical Records with frame data	Indexed as part of a Log Pass with a RLE object. This records the Logical Record header and the first data word.
All other Logical Records	The Logical Record header only.

Module

The Python module that performs file indexing is `TotalDepth.LIS.core.FileIndexer`

For reference documentation see: `TotalDepth.LIS.core.FileIndexer`.

Example

The LIS package has an `Index.py` module that will index any LIS file. Here is some (selected) output of a single file:

```
===== All records =====
tell: 0x00000000 type=128 <TotalDepth.LIS.core.FileIndexer.IndexFileHead object at 0x101b09050>
tell: 0x0000004a type= 34 name=b'CONS' <TotalDepth.LIS.core.FileIndexer.IndexTable object at 0x101b090d0>
<TotalDepth.LIS.core.LogPass.LogPass object at 0x101b09450>
tell: 0x0006141c type=129 <TotalDepth.LIS.core.FileIndexer.IndexFileTail object at 0x101b0c110>
tell: 0x00061466 type=128 <TotalDepth.LIS.core.FileIndexer.IndexFileHead object at 0x101b0c190>
tell: 0x000614b0 type= 34 name=b'CONS' <TotalDepth.LIS.core.FileIndexer.IndexTable object at 0x101b0c210>
<TotalDepth.LIS.core.LogPass.LogPass object at 0x101b0c510>
tell: 0x00065a44 type=129 <TotalDepth.LIS.core.FileIndexer.IndexFileTail object at 0x101b0cad0>
tell: 0x00065a8e type=128 <TotalDepth.LIS.core.FileIndexer.IndexFileHead object at 0x101b0cb50>
tell: 0x00065ad8 type= 34 name=b'CONS' <TotalDepth.LIS.core.FileIndexer.IndexTable object at 0x101b0cbd0>
<TotalDepth.LIS.core.LogPass.LogPass object at 0x101b0cfd0>
```

(continues on next page)

(continued from previous page)

```
tell: 0x000d2c44 type=129 <TotalDepth.LIS.core.FileIndexer.IndexFileTail object at_
↪0x101b10450>
===== All records DONE =====
```

Note that the indexing of a Log Pass is separate since it covers an EFLR and zero or more IFLRs. This is the subject of the next section.

Indexing a Log Pass

A Log Pass is described by an EFLR and zero or more IFLRs. In LIS terms this means a DFSR and the associated binary IFLRs. This is a candidate for fairly aggressive optimisation since:

- In practice the Logical Records containing the IFLRs are normally adjacent and regular (same size).
- The frame structure is invariant.

Supposing a Log Pass has 18 data channels using representation code 68 (frame length 72 bytes) and each IFLR record contains 24 frames (24*72=1728 bytes). If there are 72 Logical Records then it is fairly easy to calculate where, in logical space, logical record x is in the file; $\text{start} + x * 1728$ ¹.

NOTE: The following describes some of the internals of Log Pass indexing, it is for information only as the LogPass and IndexLogPass objects do this automatically.

Run Length Encoding of Logical Records

Run Length Encoding (RLE) is an encoding system that is highly efficient at describing regularly spaced intervals. Since LIS is primarily a recording format the recording software will create and suitably sized buffer for an integer number of frames and other environmental parameters. During recording the buffer will be filled frame by frame. When the buffer is full the buffer will be flushed to file. So a continuous series of IFLRs tends to consist of a series of adjacent Logical Records of the same size followed, possibly, by a single terminating Logical Record that is shorter (by an integer number of frames) than the others.

Module

The Python module that performs RLE of Logical Records is `TotalDepth.LIS.core.Rle`

For reference documentation see: `TotalDepth.LIS.core.Rle`.

Indexing Frame Data

This describes how an index is created to find arbitrary values in a Logical Record containing Frame Data.

Supposing a Log Pass has 18 data channels using representation code 68 (frame length 72 bytes) and each IFLR record contains 24 frames. If there are 72 Logical Records then it is a simple² task to calculate where, in logical space, the bytes for channel x and frame y are.

¹ Illustrative only, it is slightly more complicated than this.

² Well not that simple, there is indirect X-axis to take into account and several other things.

Module

The Python module that performs indexing within a Logical Record containing frame data is `TotalDepth.LIS.core.Type01Plan`.

For reference documentation see: `TotalDepth.LIS.core.Type01Plan`.

7.4.3 Indexing Performance

As mentioned above the cost of indexing can be measured with these independent measures:

- The time to creating an index.
- The space required by the index, in-memory or serialised in some form.
- The time per byte to access N bytes of data.
- Any deviation from $O(N)$ performance, N being total file size or size of data read.

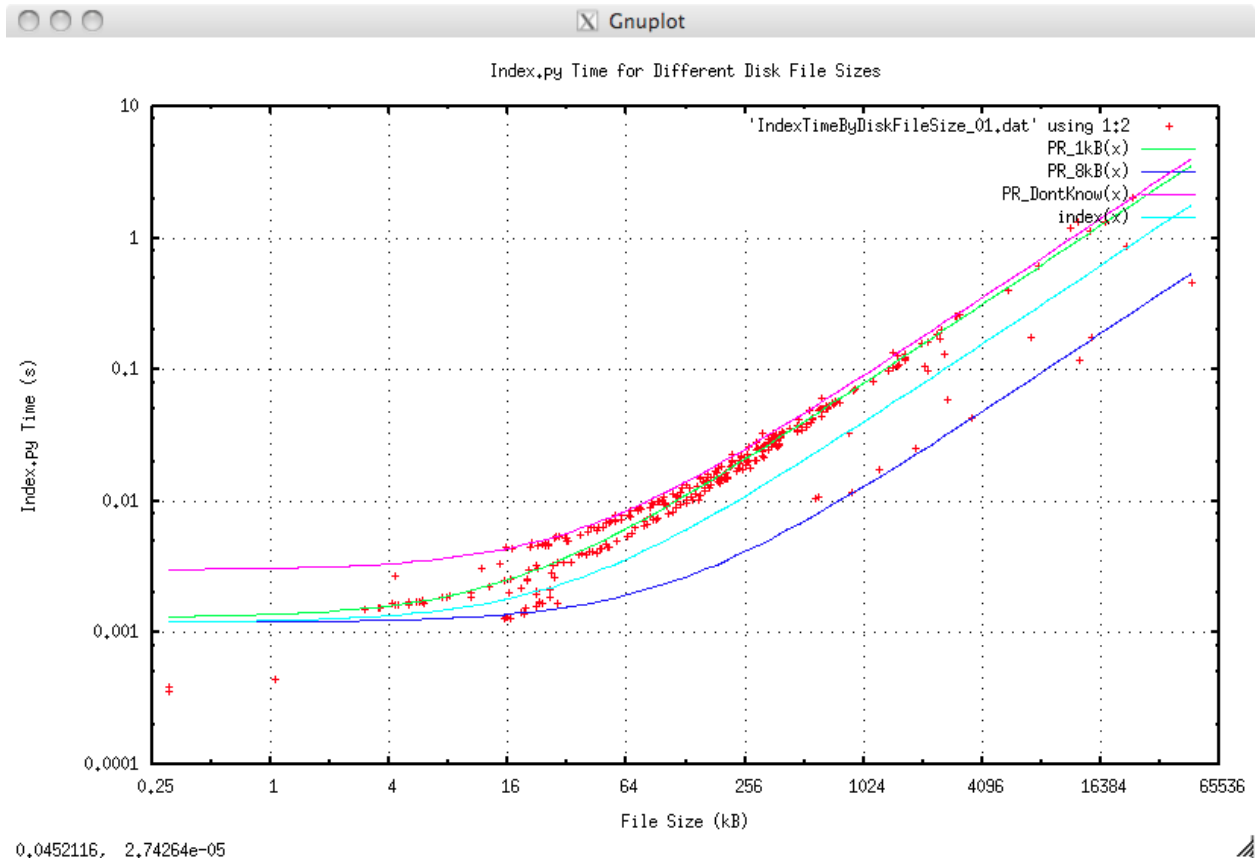
The following data was measured using 300+ LIS files totalling of around 300+Mb, the largest being around 50Mb. The LIS module `TotalDepth.LIS.RandomFrameSetRead.py` was used to conduct the tests.

The original targets for the cost of TotalDepth LIS indexing were:

Cost ms/Mb	Result
<15	Excellent
15-50	Good
50-100	Satisfactory
>100	Unsatisfactory

Index Time

This is simply the actual time taken to create a file level index:



The best fit of all points is the cyan line (index(x) on the plot legend). The best fit for the lines is:

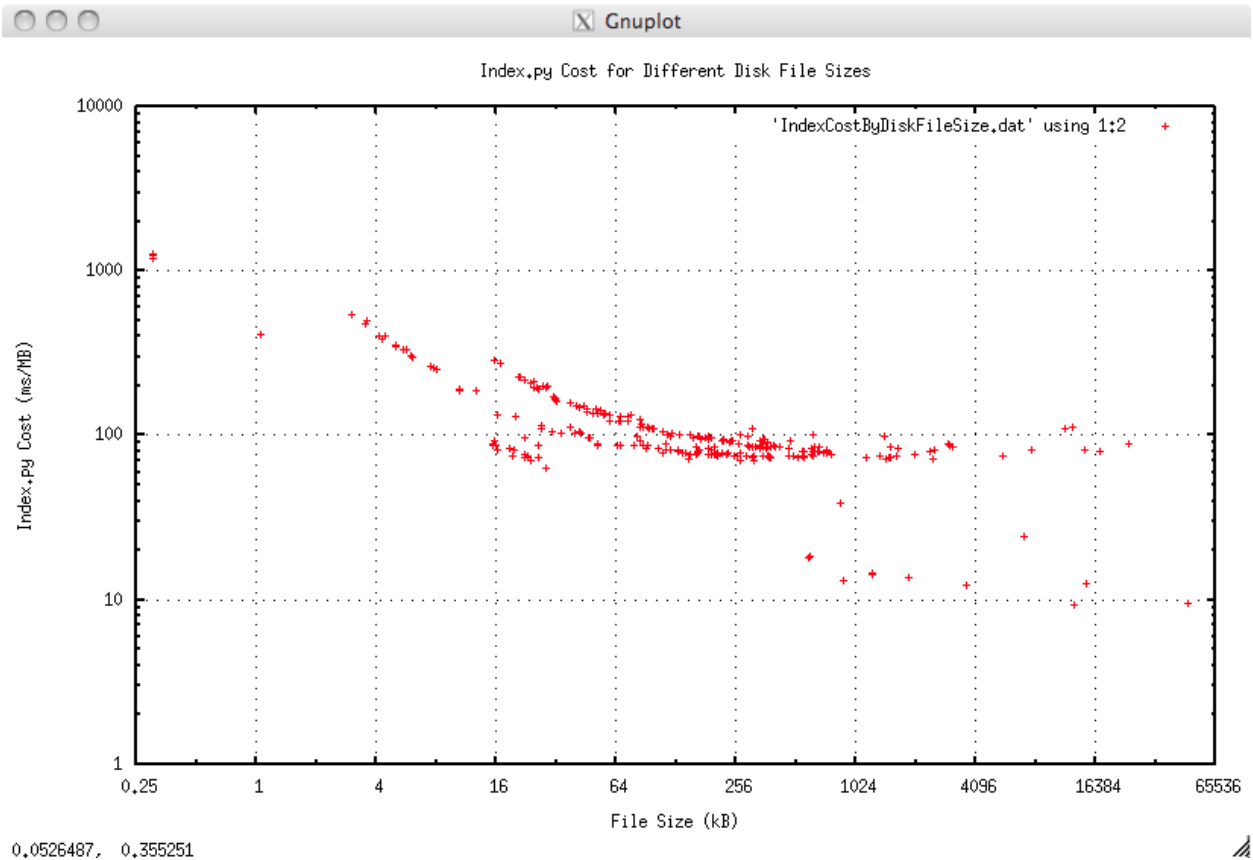
Line	Description	Colour	Latency (ms)	Cost (ms/Mb)	Result
PR_1kB(x)	Physical Record length = 1kb	Green	1.3	80	Satisfactory
PR_8kB(x)	Physical Record length = 8kb	Blue	1.2	12	Excellent
PR_DontKnow	Upper bound, worst case	Magenta	3.0	90	Satisfactory
index(x)	Regression fit on all points	Cyan	1.2	39.9	Good

However there are some other trends that can be teased out when separating files that have Physical Records of 1kb and those in the data set that have 8kb Physical Records (there were no larger Physical Record sizes in the data set). The green fit (PR_1kB(x) on the plot legend) is the best fit for those files that have Physical Records of 1kb. The blue line (PR_8kB(x) on the plot legend) is the best fit for files that have Physical Records of 8kb, it is almost exactly 8 times faster.

In fact in other tests, not presented here, it was shown that there was a linear speedup with Physical Record size up to their maximum size of 64kb. In fact the dominant factor in indexing time was the *number of physical records*. This suggests a couple of things:

- Before archiving or processing LIS files rewrite them to their maximum Physical Record size (64kb). This will pay off later when indexing prior to any read operation.
- Rewrite the Physical Record handler (and below) in C or Cython as it seems to be the bottleneck for indexing.

This is actual time taken to create a file level index divided by the file size. This gives some measure of ‘cost’ which is defined here as ms per Mb of data processed, here the size in Mb is the total file size:

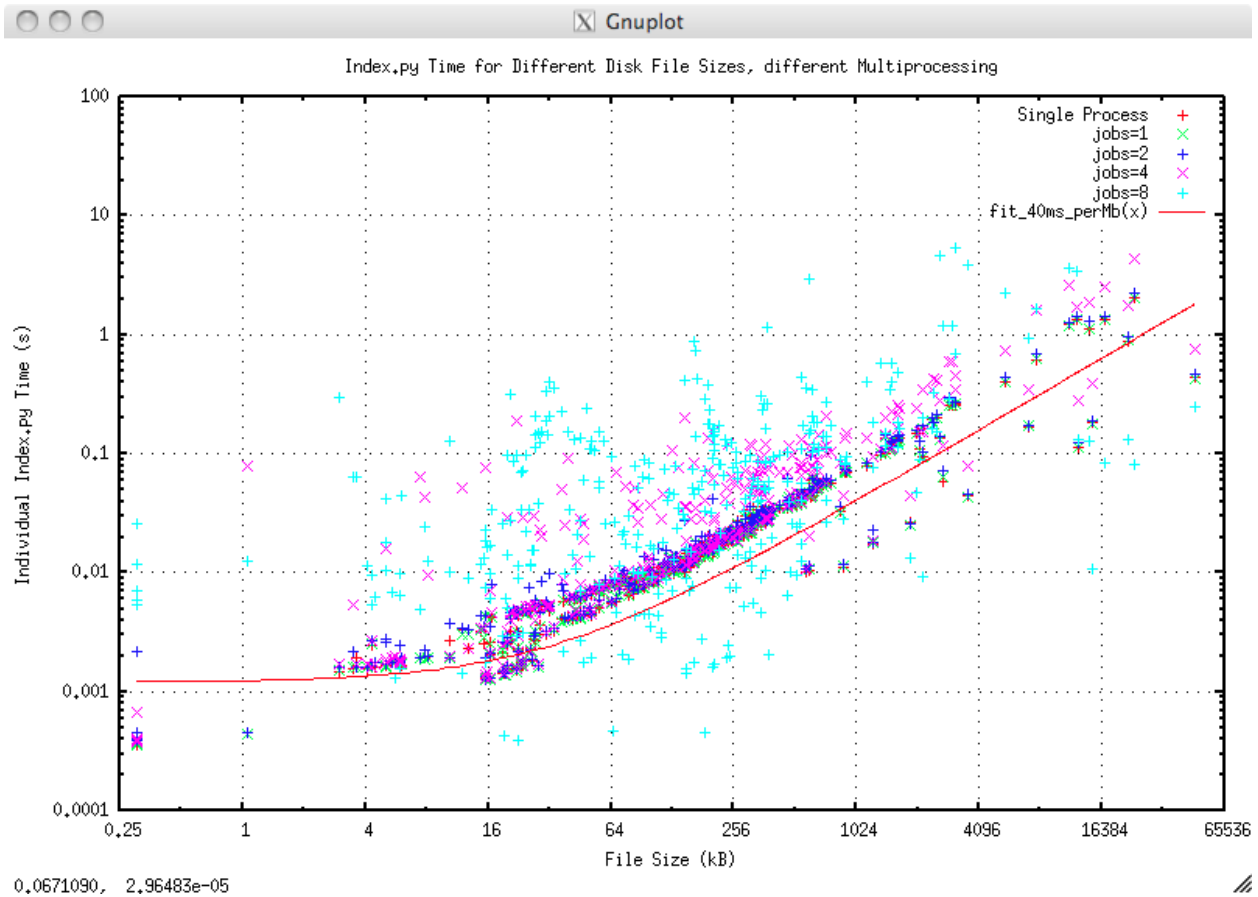


The rising costs for smaller files is no real cause for concern, this represents the startup cost of indexing and reading of around 1 to 2 ms. This graph clearly demonstrates $O(N)$, or better, behaviour.

Again the advantage in having 8kb Physical Records is evident in the lower right hand corner. The cost would be further reduced to around 1 to 1.5 ms/Mb by having 64kb Physical Records.

Multi-Processing

Most of TotalDepth software supports parallel processing. The LIS command line tool Index.py can create indexes in parallel. The following graph shows the file size plotted against total time to index when indexing around 300 LIS files with different numbers of simultaneous processes. The red line is the best fit for single process indexing that costs about 40 ms/Mb:



The index time rises slightly with increasing number of processes and the scatter rises dramatically with 8 processes presumably because of I/O contention.

Actual wall clock times for indexing all those LIS files for different numbers of simultaneous processes on a four core machine with hyper-threading are:

Processes	Total time to index (s)	Improvement over No multiprocessing
No multiprocessing	18.4	Datum
1	18.3	0.995
2	9.9	0.538
4	8.9	0.484
8	7.6	0.413

Moving to two processes gives an almost linear speedup, moving from two to four or eight processes gives only slight improvement presumably because the execution time becomes I/O bound.

Index Size

It is envisaged that the index will be persisted in some form. Once persisted then the LIS file would only be accessed via the index, any file write operation requires a suitable adjustment to the index.

Persistence techniques could be, for example:

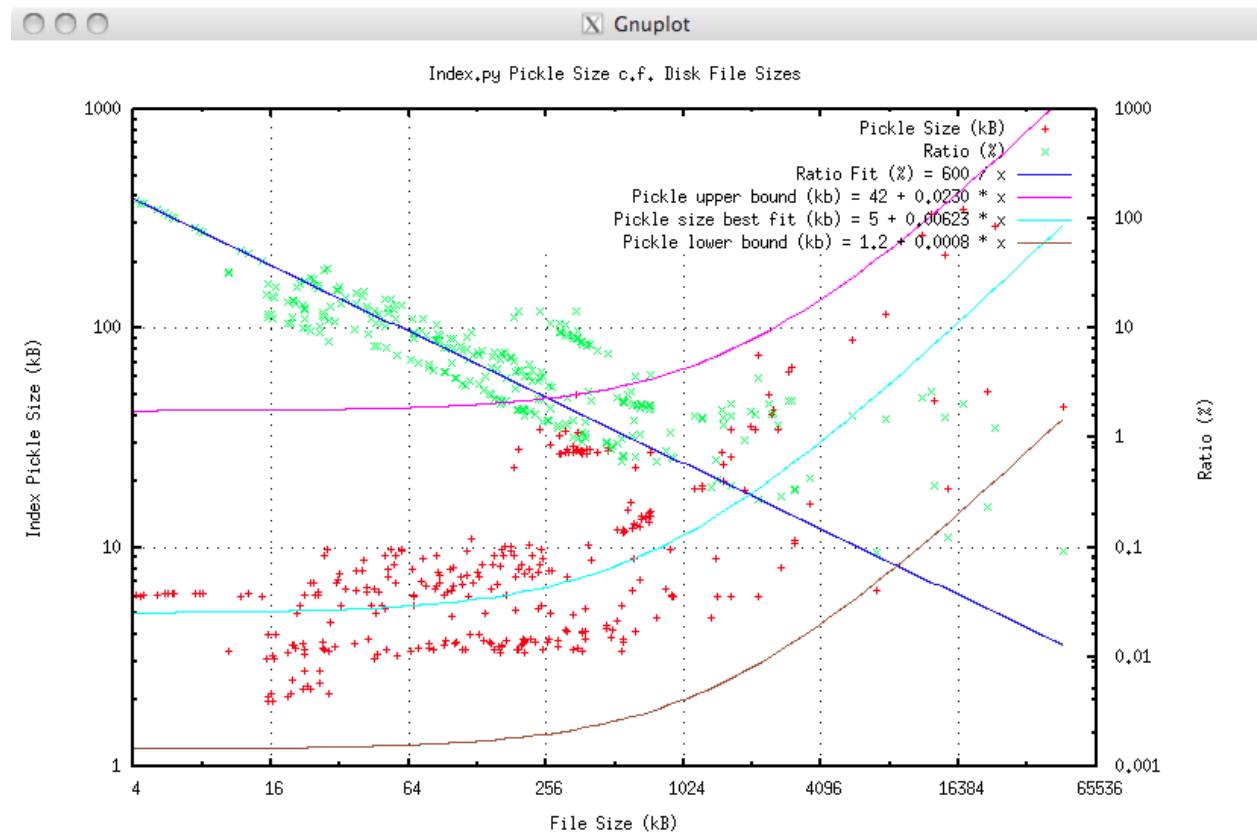
- Stored in a database.
- Serialised in binary form onto the file system.
- Serialised in human readable form, such as XML, onto the file system or database³.
- Serialised in binary form and attached as a Logical Record to the LIS file it refers to.

TotalDepth and SaaS

There is a further imperative to understanding index size; if TotalDepth were to be used, as it was always intended, as Software as a Service (SaaS) where the bulk of the processing is with the data file on the client machine and the processing done on the server then part of the bootstrap process of any transaction is for the client to index the file and send the index to the server. In that case it is important to keep the index size small.

Thus the size of the index content is a significant consideration.

The following graph measures the size of the index when serialised with Python's cPickle module, Pickled size is in kb (red markers, left scale) and compared as a percentage of the original file size (green markers, right scale):



³ JSON can not handle the index in its current form.

The pickled size shows a wide range that is representative of the wide range of LIS inputs. The best fit for size (albeit with a large scatter) is 5kb + 0.6% of the file size which is entirely satisfactory [the upper bound being 42kb + 2.3% and the lower bound being just 1.2kb + 0.08% with larger files tending towards the lower bound].

The relative size of the index shows a strong downward trend (blue line) for files below 4Mb, before levelling off at the 0.1 to 3% mark. This is quite satisfactory for the use cases described above including SaaS. This graph clearly demonstrates $O(N)$, or better, behaviour.

Indexing Performance Improvements

As noted above there is a substantial improvement in indexing when large Physical Record sizes are used.

It is also likely that significant improvement could be made if the RawStream, TifMarker and PhysRec were to be rewritten in C/C++ or Cython. The PhysRec module has a dependency on the RepCode module (easily removed). All three modules have a dependency on the struct module so the limited functionality that they use from there would have to be reproduced, thats pretty easy since it only involves integer manipulation.

So given an ‘average’ cost of indexing of 40 ms/Mb (i.e. a ‘good’ rating) the performance improvements could be:

- Moving to larger Physical Record sizes: x8?
- Integrate the existing (in another project) code in C that handles Raw Stream/TIF/Physical Record handling into this project. This is known to be about x100 faster (and the index has a lower memory footprint).

The performance improvements would not necessarily combine as they are mutually dependent but the combination might reduce the cost to around 2 to 8 ms/MB, an exceptionally good performance indeed.

See [Performance Improvements](#) for other performance improvements

LIS Read Performance via an Index

This is described here [LIS Performance](#)

Summary

Indexing is not free, it incurs an overhead, but this overhead is acceptable. The overhead is worst for small data sizes where the performance is high in any case. The overhead is low, and the benefit is very great for large or complex data sizes where the performance, without indexing, could be very poor indeed.

7.5 LIS Performance

Actual performance results of some LIS file operations.

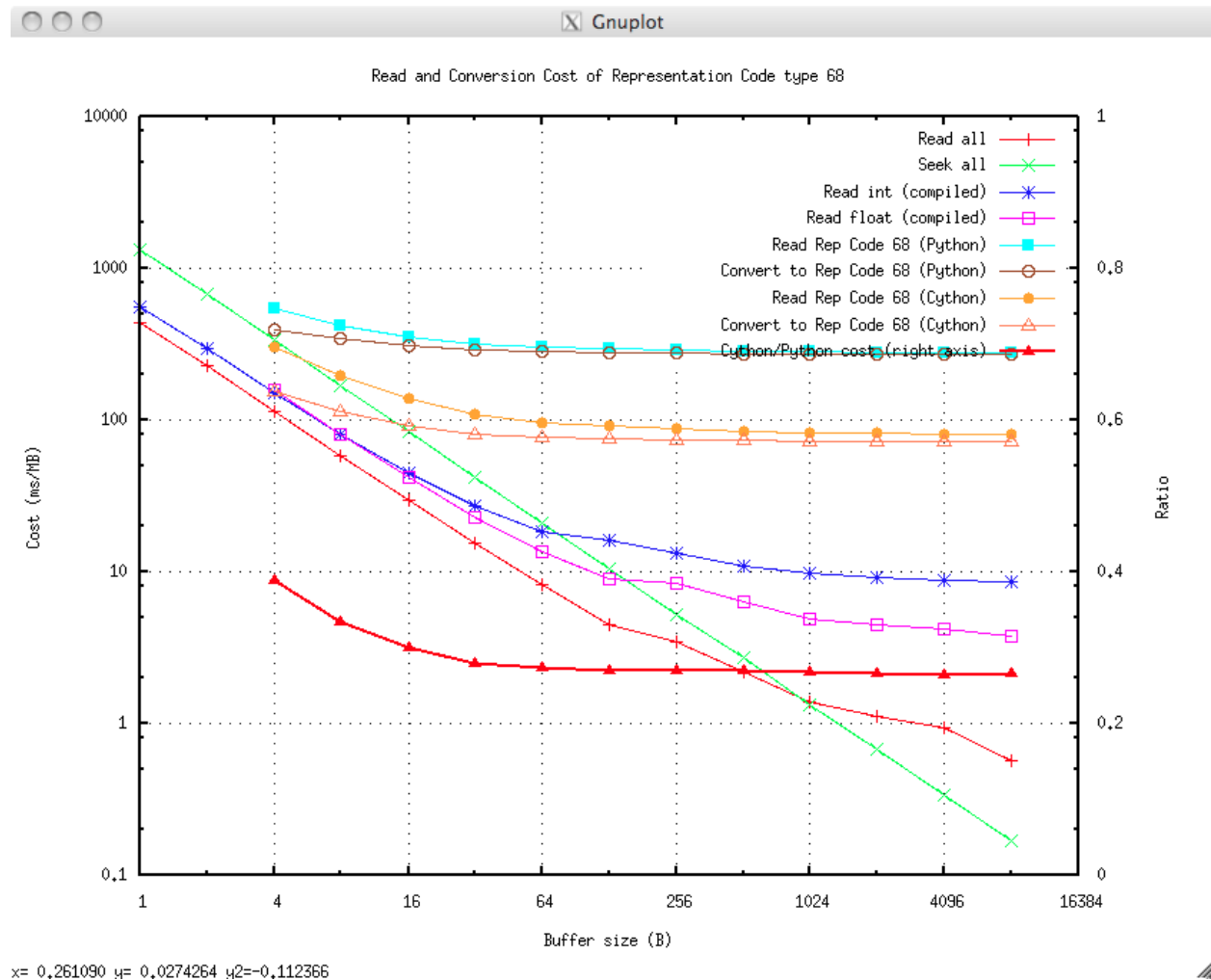
7.5.1 File Read Cost

The measure here is low level reading and conversion from binary words to internal number formats. Three operations are being measured:

- Raw read speed of bytes.
- Above, and converting to internal integers and floats.
- Above and converting from Rep Code 68 to an internal float.

The input data was about 100 binary files with random data of around 100 Mb in total.

The x-axis of the graph below is the buffer length, in other words how much data is swallowed in a single atomic read.



Raw File Read

The 'Read all' graph shows the cost of reading bytes at different buffer sizes from a file and discarding it. This is the baseline and represents the crude storage access time. Large buffer sizes achieve <1 ms/Mb (1 Gb/s).

Raw File seek()

The 'Seek all' graph shows the cost of traversing a file using seek() at different seek lengths. No data is read. Combined with the read information this can be used to estimate the minimum time to make a series of read/seek operations.

Using the struct module

The “Read int (compiled)” and “Read float (compiled)” data shows the cost of reading bytes into a buffer and then converting them to a list of integers and floats using the `struct` module (the type declaration is compiled). They show an asymptotic cost, in addition to the read cost, of 8 and 3 ms/Mb respectively. This shows very good performance for reading builtin types with the struct module. As most RP66 types are builtins (or types supported by the struct module) then the struct module should be used for RP66.

It is slightly surprising that reading an integer is slower than reading a floating point number.

Reading Rep Code 68

This establishes the cost of converting a bytes object in Representation Code 68 to a internal floating point value.

The cost of using Python code and Cython code is examined and in each case the the total cost or read+convert is presented and then the integer `struct` read cost (above) is subtracted to get the conversion cost.

Python

The data sets “Read Rep Code 68 (Python)” and “Convert to Rep Code 68 (Python)” plots to the cost of reading bytes and converting them to Representation code 68, the latter curve has the read cost subtracted (from above) so it represents merely the conversion cost of in-memory from bytes to Rep Code 68. This is asymptotic to around 270 ms/Mb.

Cython

The data sets “Read Rep Code 68 (Cython)” and “Convert to Rep Code 68 (Cython)” is as immediately above but using Cython code rather than Python code. This is asymptotic to around 72 ms/Mb. This is significantly longer than using the struct module to interpret a IEEE float so it could be that some improvement could be made with a pure C implementation.

Cython vs Python

The “Cython/Python cost (right axis)” plots the ratio of converting Rep Code 68 with Python and Cython. The Cython code takes 0.27 of the cost of the Python code. Well worth it.

7.5.2 Frame Read Cost

The operation being timed here is, given a LIS file index, read a certain set of frame data then convert each value to an internal floating point number and then populate the internal FrameSet object. This is a basic operation before doing any plotting, channel editing, recalibration etc. The amount of data read is the cumulative size in LIS bytes of all the values read from the file. 250,000 values would constitute typically 1Mb of LIS data.

A number of different methods were use to read the LIS data:

- Method A: All frames all channels. Here indexing provides no optimisation as it is a simple sequential read operation of everything.
- Method C: A sequential subset of the frames and a sequential subset of the channels. This requires a logical seek operation to the start of the frames then, within each frame a seek then read then seek operation.
- Method D: A sequential subset of the frames and a non-sequential subset of the channels. This requires a logical seek operation to the start of the frames then, within each frame multiple seek/read operations.

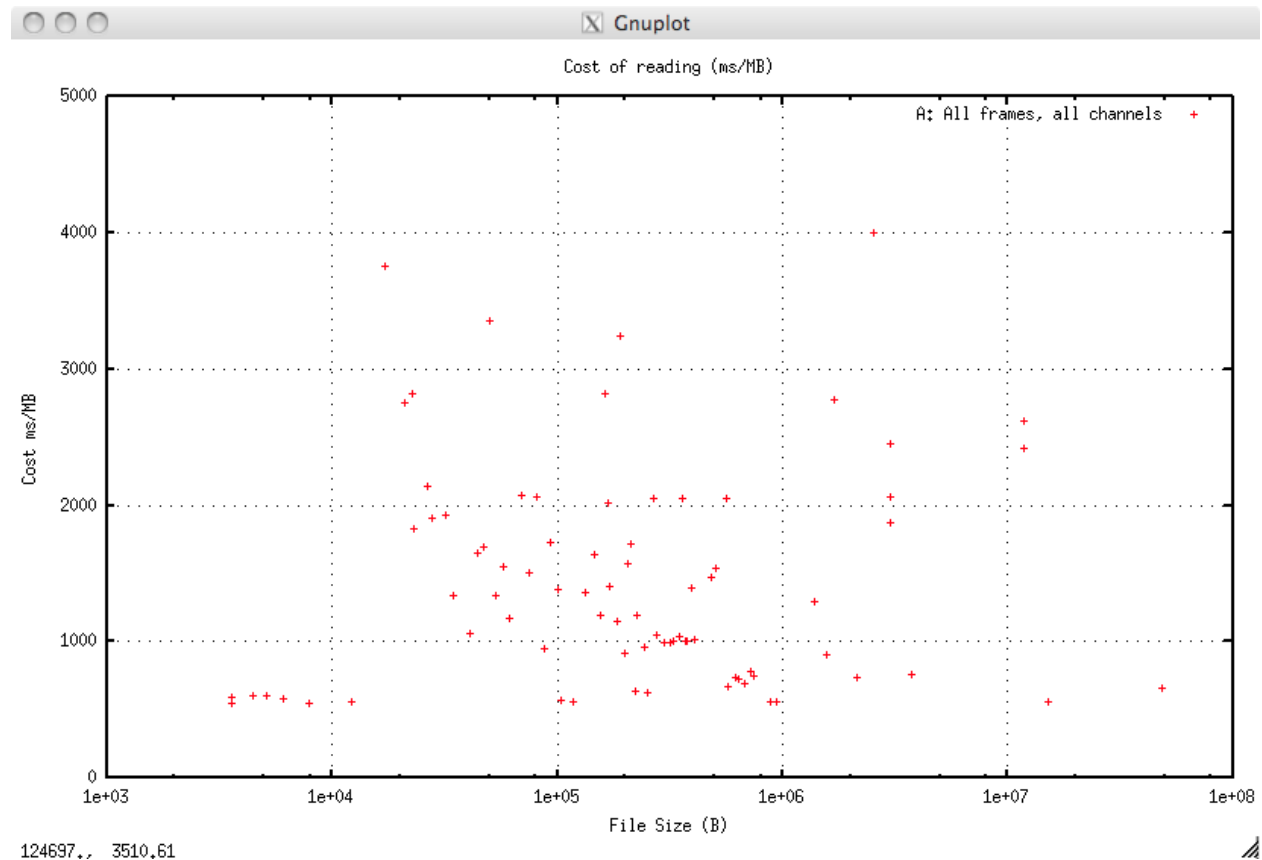
- Method E: A non-sequential subset of the frames and a non-sequential subset of the channels. This requires a set of logical seek/read operations to each frame then, within each frame multiple seek/read operations.

Indexing can provide an optimisation for methods C-E as it hugely reduces the amount of read operations to that just sufficient for the required frames and channels. The important measure is that the cost of indexing, computing and executing seek/read operations does not outweigh the reduction in data read.

For example if a frame has 40 channels and only 5 are required then, if the cost per byte stays the same then the user sees a 8 fold speed improvement. If the cost per byte doubles then the user still sees a 4 fold speed improvement. If the cost per byte rises 8 fold there is no advantage over doing a sequential read of all channels. Beyond an eight fold rise in indexing cost the user sees a performance *reduction*.

A: All Frames, all Channels

This is the baseline, indexing plays no part as this is a straightforward sequential read operation of every channel in every frame.



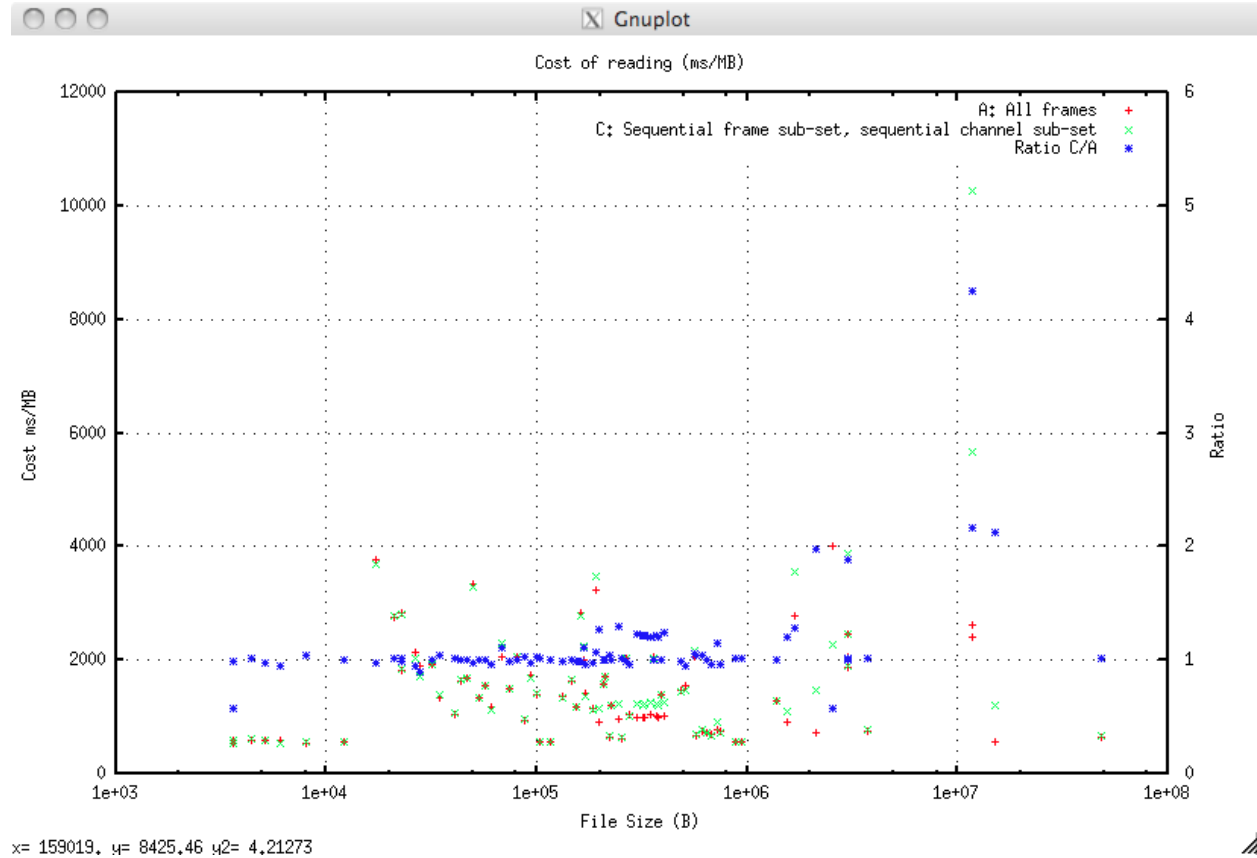
Although there is quite a spread there is, gratifyingly, no obvious trend for the cost to increase as the file size does which would indicate worse than $O(N)$ behaviour. The behaviour is clearly $O(N)$. The cost in ms/Mb is:

A:	Cost ms/Mb
Minimum	541.8
Mean	1446.9
Maximum	4003.3

C: Adjacent Frames and Channels

This is taking a random, but sequential, subset of the frames and a random, but sequential, subset of the channels. This requires the indexer to do a logical seek operation to the start of the frames then, within each frame a seek then read then seek operation.

The results are plotted compared to the baseline and the ratio of the cost of C relative to the cost of A for each file is plotted on the right hand axis.



The cost in ms/Mb is:

Value	A: Cost ms/Mb	C: Cost ms/Mb	Ratio C/A
Minimum	541.8	545.7	0.57
Mean	1446.9	1561.3	1.13
Maximum	4003.3	4485.8	4.25

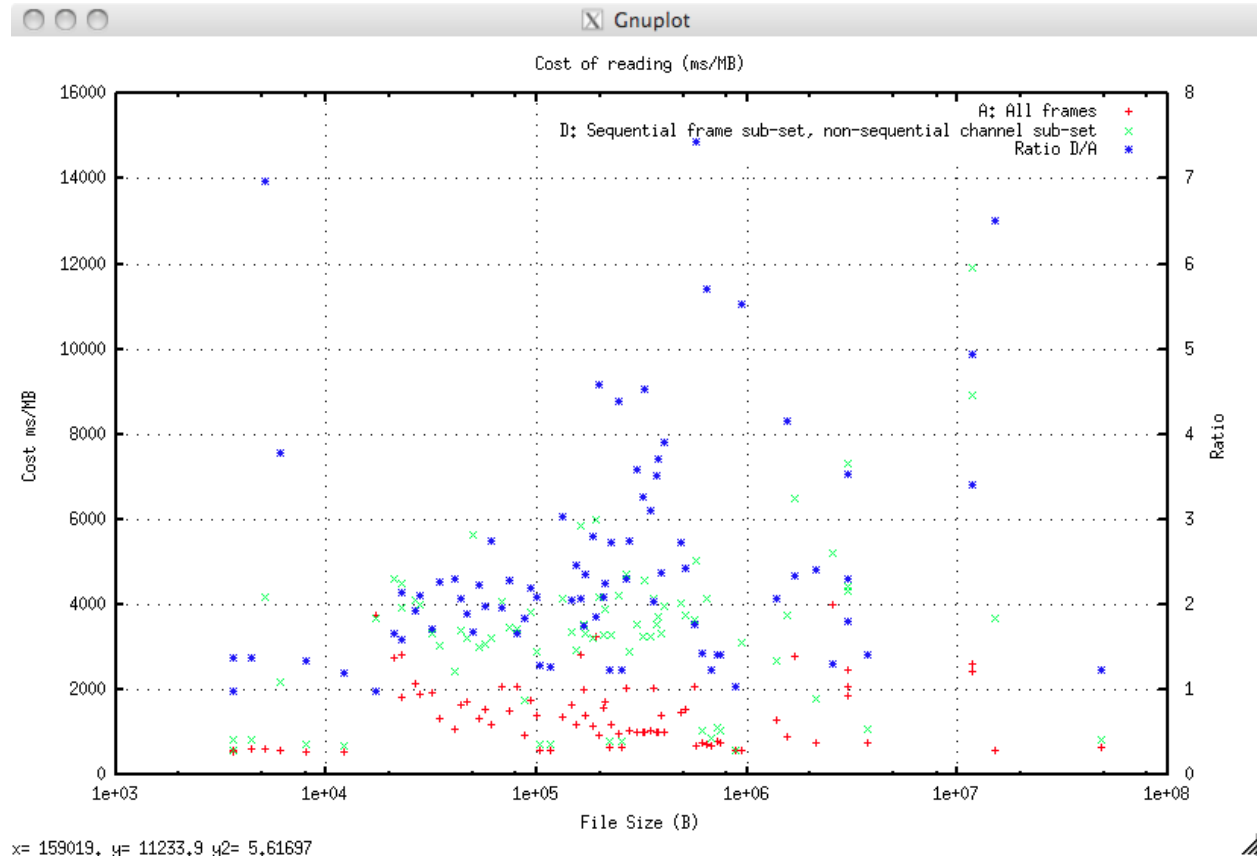
This means that a small increase in cost (13%) is the price of a hugely reduced data set.

D: Adjacent Frames, any Channels

This is taking a random, but sequential, subset of the frames and a random, but non-sequential, subset of the channels. This requires the indexer to do a logical seek operation to the start of the frames then, within each frame multiple seek/read operations.

This is highly representative of the indexing operation performed when plotting, say, any 200 foot section of a number of selected channels.

The results are plotted compared to the baseline and the ratio of the cost of D relative to the cost of A for each file is plotted on the right hand axis.



The cost in ms/Mb is:

Value	A: Cost ms/Mb	D: Cost ms/Mb	Ratio D/A
Minimum	541.8	581.8	0.98
Mean	1446.9	3410.5	2.55
Maximum	4003.3	11928.6	7.44

It should be noted that this test is deliberately harsh in that non-sequential channels are always chosen. If some of the channels are adjacent then the cost reduces, for those channels, more towards C (typically half the cost of D).

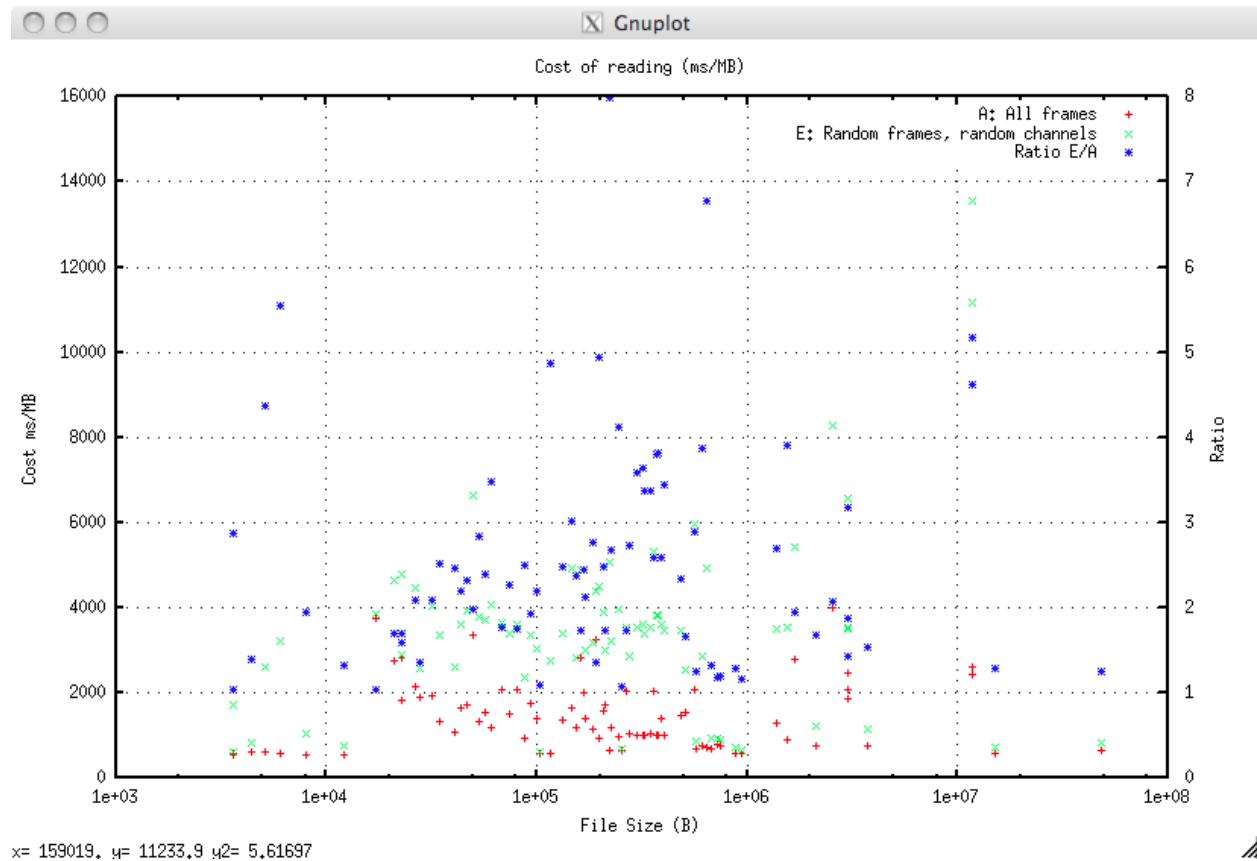
Even so provided the number of channels of interest is less than 40% of the total indexing this way provides a real speed improvement. The important point being this may not produce an improvement (and may actually be slower) for small frame sizes; they are not the problem precisely because they are small. For the seriously problematic large frame sizes then indexing will always be faster, often dramatically so.

E: Any Frames, any Channels

This is taking a random, non-sequential, subset of the frames and a random, but non-sequential, subset of the channels. This requires the indexer to perform a set of logical seek/read operations to each frame then, within each frame multiple seek/read operations.

Essentially this is the same as test D but skipping intermediate frames.

The results are plotted compared to the baseline and the ratio of the cost of E relative to the cost of A for each file is plotted on the right hand axis.



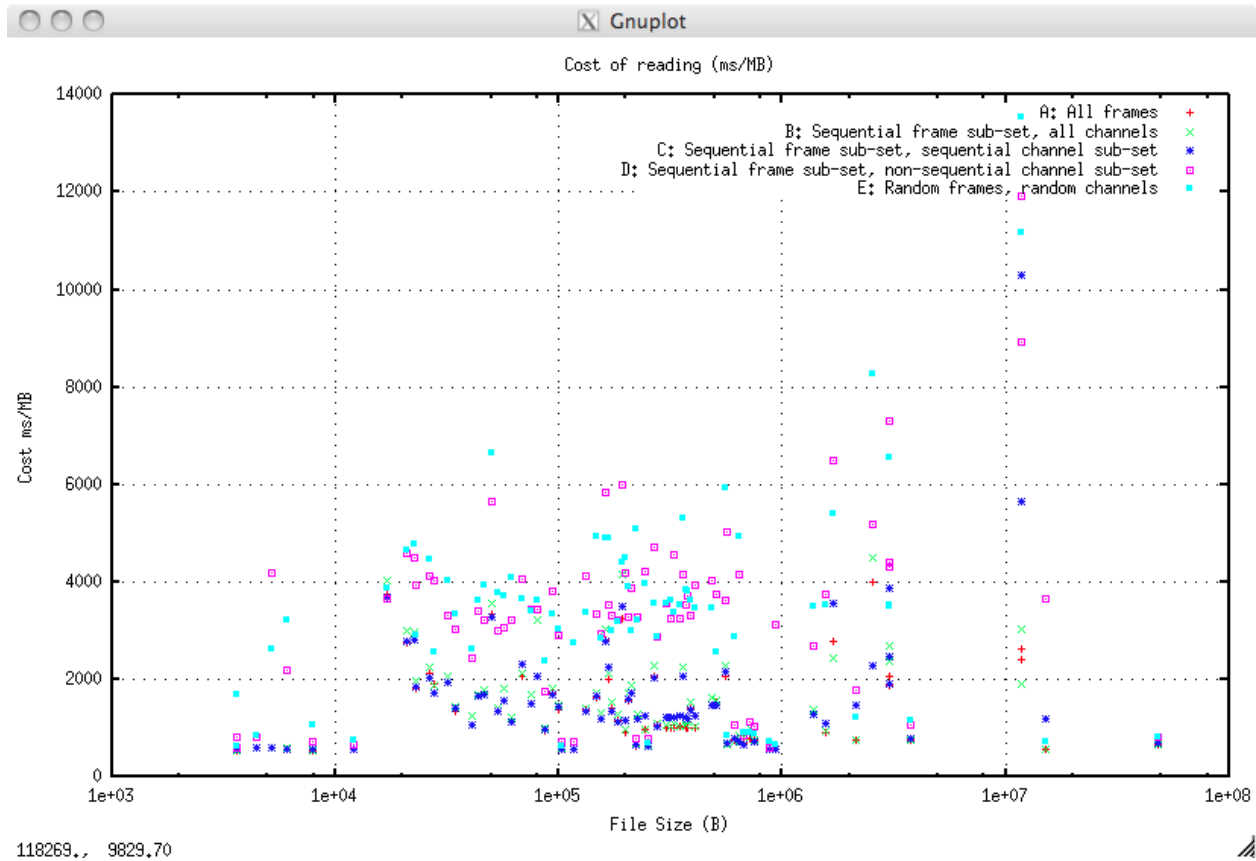
The cost in ms/Mb is:

Value	A: Cost ms/Mb	E: Cost ms/Mb	Ratio E/A
Minimum	541.8	619.9	1.03
Mean	1446.9	3491.7	2.57
Maximum	4003.3	13552.5	7.98

There is a very slight (around 1-5%) additional cost over D but the huge benefit is being able to skip intermediate frames. This means that a low resolution plot (say 1:500) of a high resolution log (say sampling every 1.2 inches) could read every fifth frame (six inch sampling) and that would be plotted every 0.012" (say 1 pixel) as a speedup of almost 500%.

All Measurements

For the sake of completeness, here are all the results:



7.5.3 Performance Improvements

todo:: Complete this

7.6 Indexing RP66V1 Files

This describes how RP66V1 binary files are indexed to provide random access to any part of their data structure. This also describes the performance of TotalDepth indexing a test set of around 100+ files that ranged in size from 80kb to 4GB totalling 17GB. The average file size was about 150Mb. These tests were run on a 2.7 GHz Intel Core i7 machine with 4 cores and hyper-threading. This data refers to version 0.3.0 and may not be relevant to the current version, 0.4.0.

7.6.1 Low-Level Index

There are multiple levels of a RP66V1 index in TotalDepth, the lowest being a simple index of the start of every Logical Record. The index is implemented by `TotalDepth.RP66V1.core.pIndex.LogicalRecordIndex`

There is one entry on the index for each Logical Record and the entry consists of:

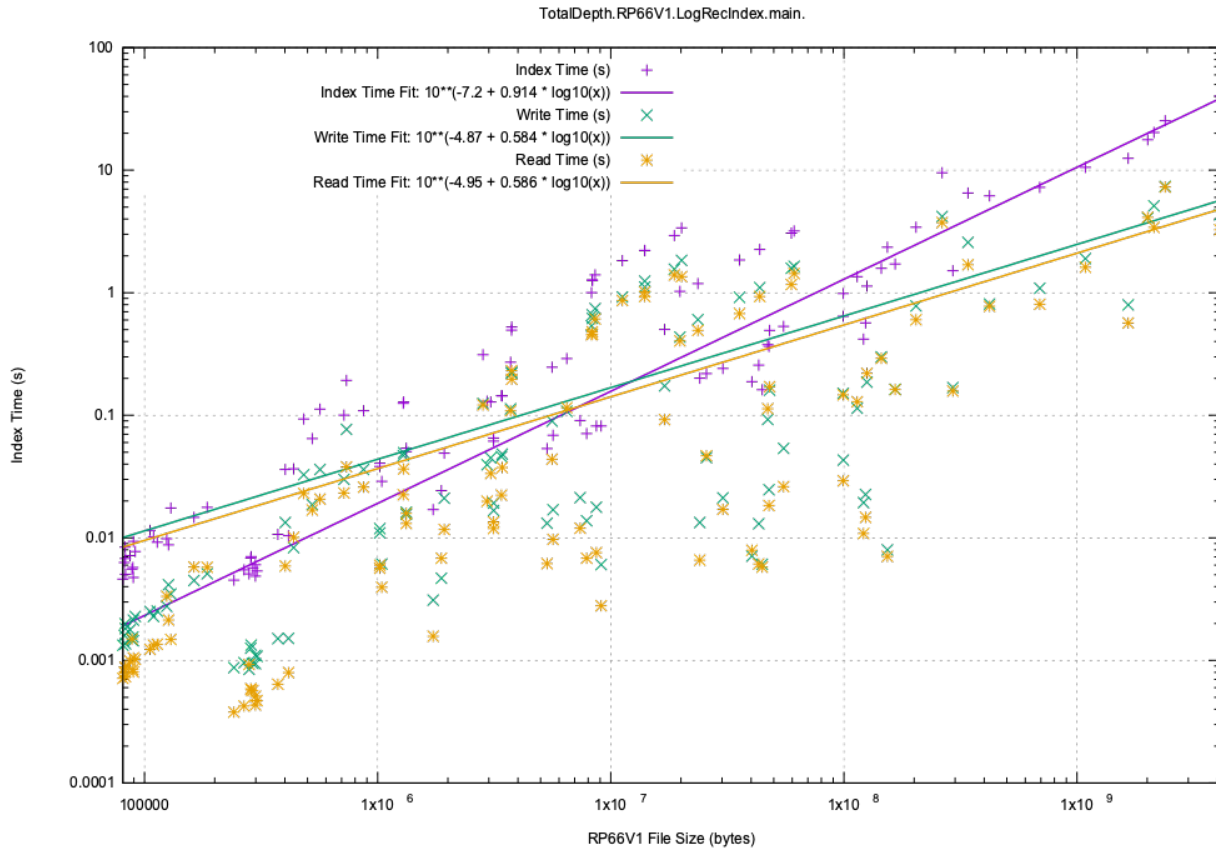
- Integer absolute file position of the immediately prior Visible Record.
- Integer absolute file position of the first Logical Record Segment Header.
- One byte value of the first Logical Record Segment Header attributes.
- One byte value of the first Logical Record Segment Header type.
- Integer length of the Logical Data.

The following performance data was gathered by `TotalDepth.RP66V1.LogRecIndex`. The performance is assessed by:

- The time it takes to index the file, persist it and read it back.
- The size of the persistent index.
- Memory usage.
- The value of multi-processing.

Execution Time

Here is the execution time(s) for creating, persisting and reading back the persisted index compared to RP66V1 files size:

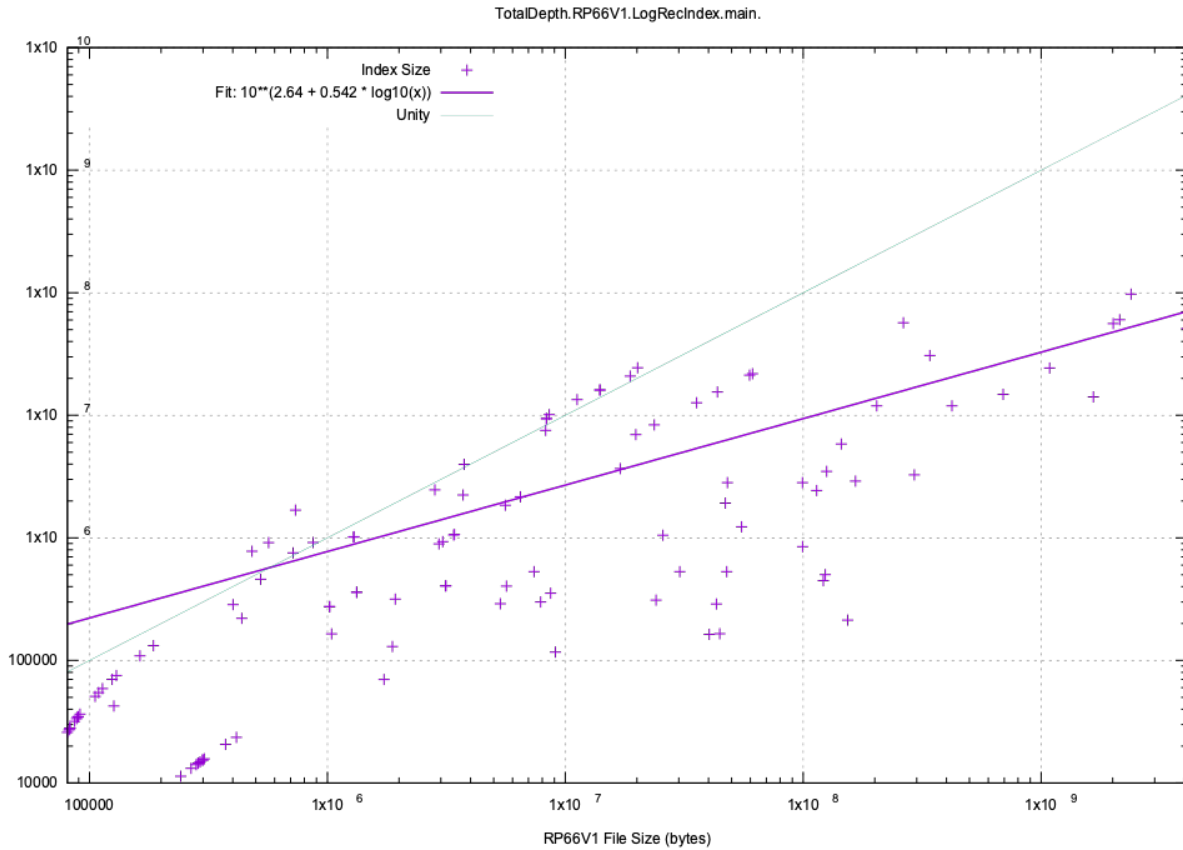


While there is quite a variation by a factor of 10 the *average asymptotic* execution time trends to:

- Create the index: 10.6 ms/Mb
- Persist the index: 2.4 ms/Mb
- Read the persistent index: 2.1 ms/Mb

Index Size

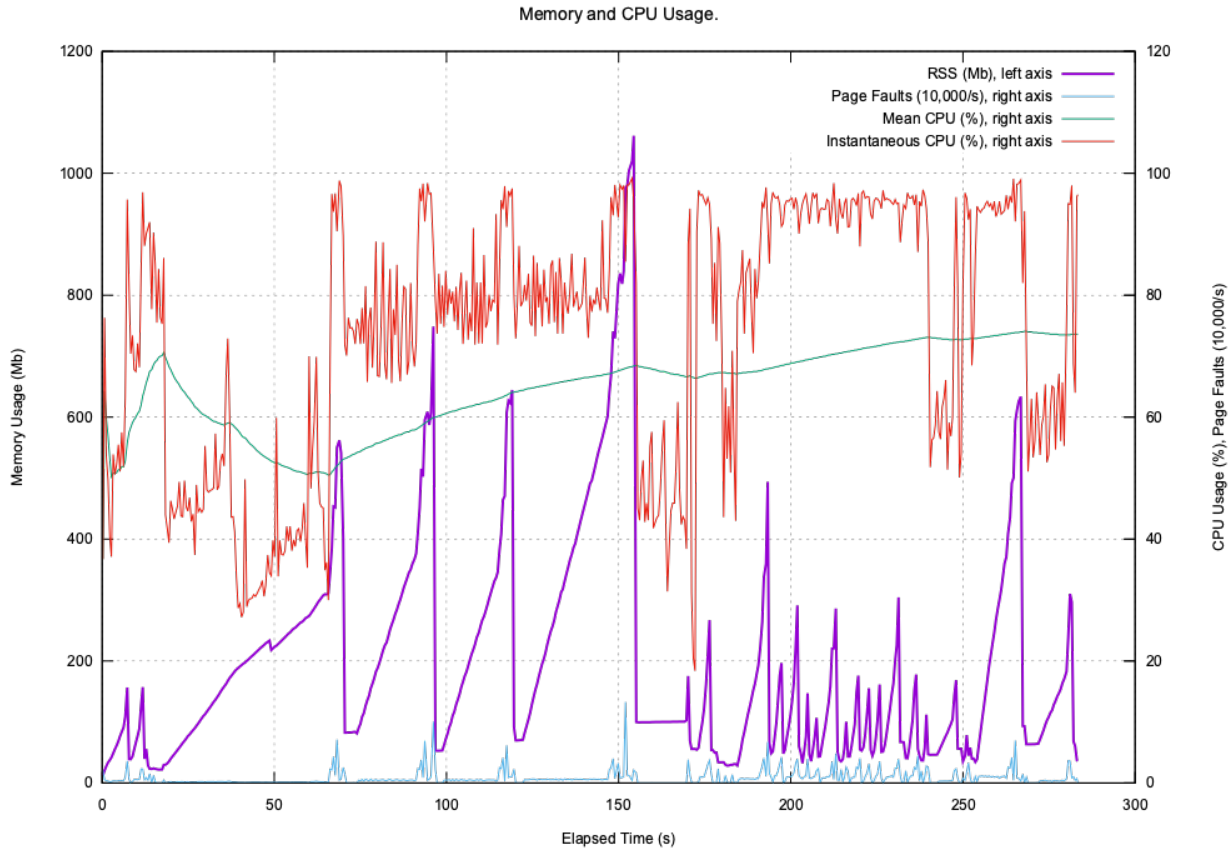
The pickled index size compared to the original file size is shown here. The green line shows where the index size would be equal to the input file size:



This index size trends to around 10% of the file size. The large size of the index reflects to cost of Python's general purpose pickle protocol. Each entry takes about 160 bytes whereas the C/C++ implementation takes only around 24 bytes.

Memory and CPU Usage

Indexing the test set makes this memory and CPU demands:

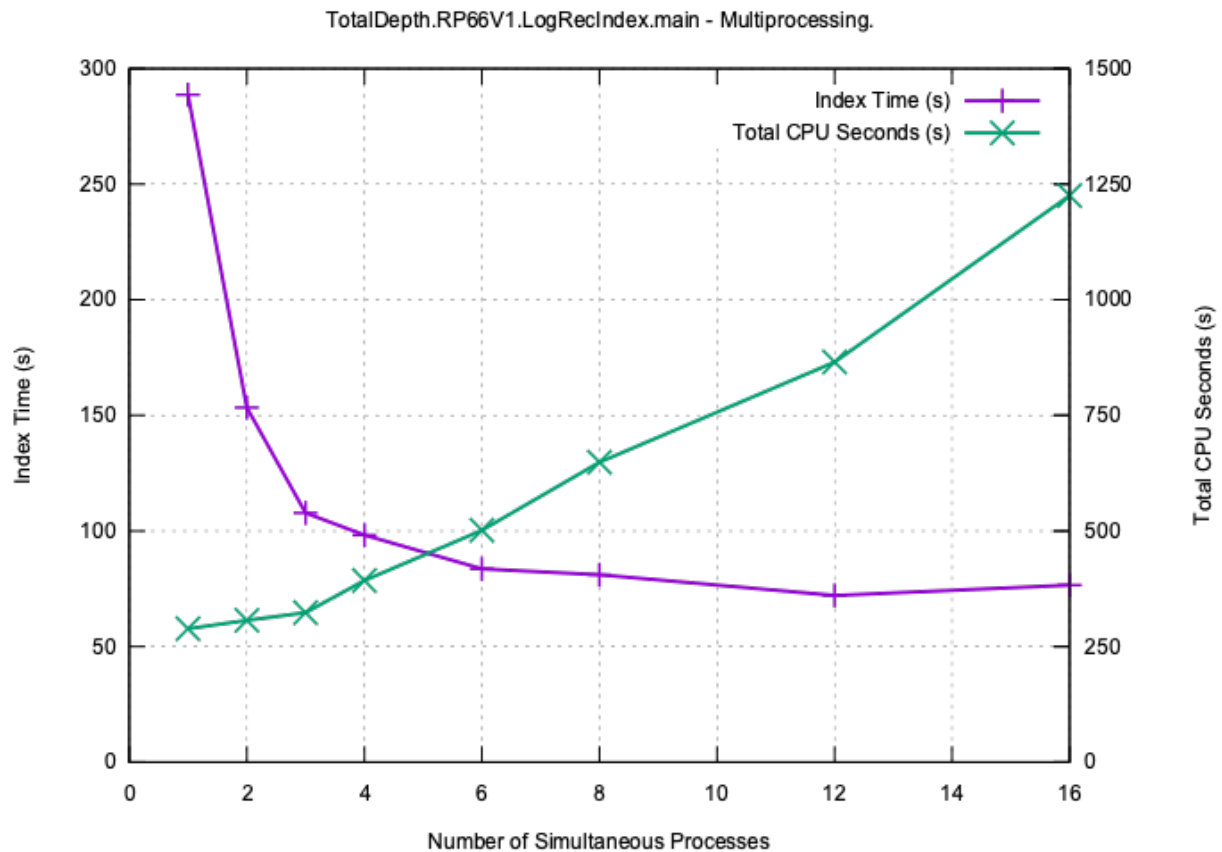


The extra spikes at the end of indexing are due to Python's pickle module that pickles in-memory (even if pickling to a file).

The current version, 0.4.0, this index is implemented in pure Python but in a future release the C/C++ indexer will be used. This is far faster and has a much smaller persistent index.

Multiprocessing

Each index is independent so multi-processing can speed up index creation enormously. The number of simultaneous processes can be specified with the `--jobs=` option. Here is the same test data set being indexed including pickling the index to disk with a different number of simultaneous processes.



As expected beyond four processes the improvement is marginal.

7.6.2 Mid-Level Index

The mid-level index wraps up the low-level index with richer information and is one that the user normally interacts with.

The additional information it contains is:

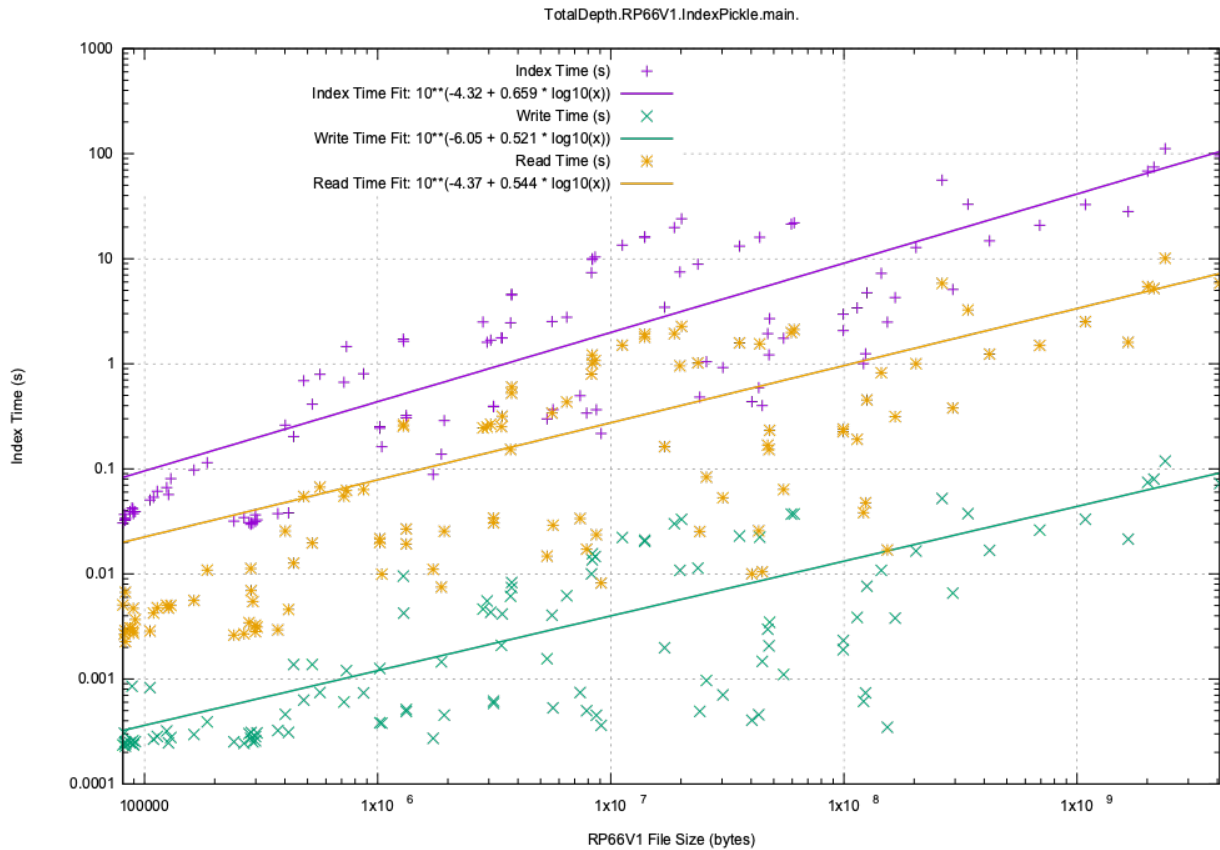
- Every *RP66V1.Explicitly Formatted Logical Record* (EFLR) as a full internal representation.
- A reference to every *RP66V1.Indirectly Formatted Logical Record* (IFLR) as a partial internal representation. This does not contain the entire IFLR but does have each X axis value.

Naturally enough this index takes longer to build and uses more memory (and more disc space when persisted).

The index is implemented by `TotalDepth.RP66V1.core.LogicalFile.LogicalIndex`. The following performance data was gathered by `TotalDepth.RP66V1.IndexPickle`.

Execution Time

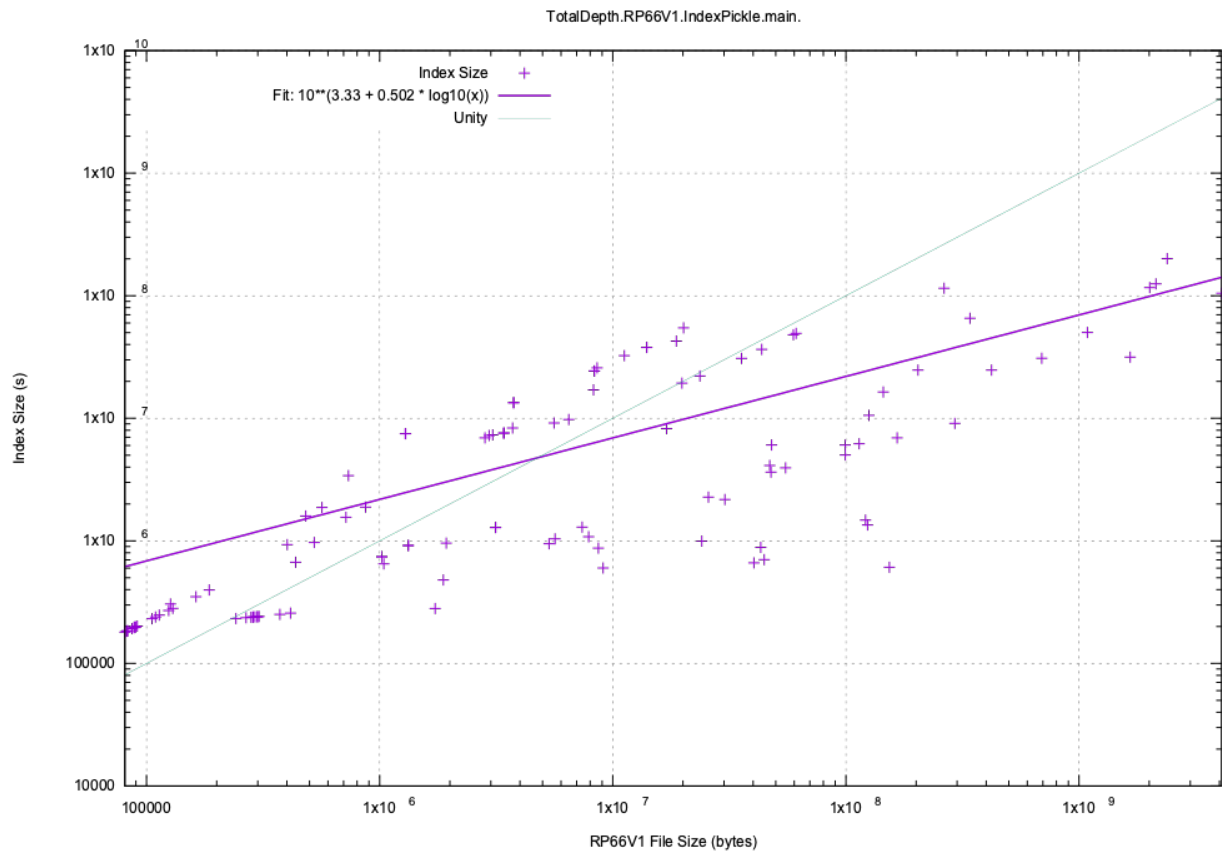
Firstly time to create, persist and read back the persisted index:



While there is quite a variation by a factor of 10 the *average asymptotic* execution time trends to:

- Create the index: 42 ms/Mb (this includes the low level index of 10.6 ms/Mb).
- Persist the index: 0.07 ms/Mb (this seems remarkably small).
- Read the persistent index: 4 ms/Mb

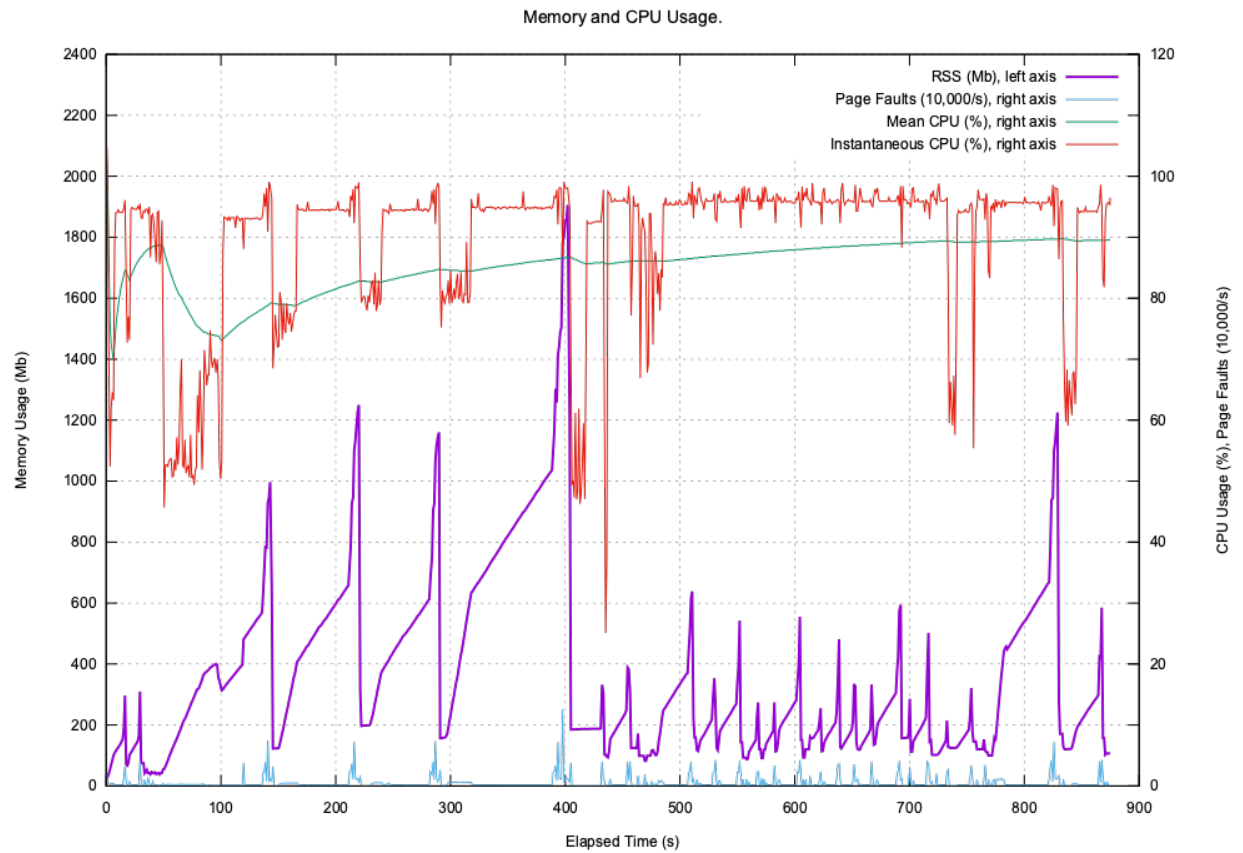
Index Size



The index is roughly twice the size of the low-level index (although that ratio will increase dramatically when the C/C++ low-level index code is merged).

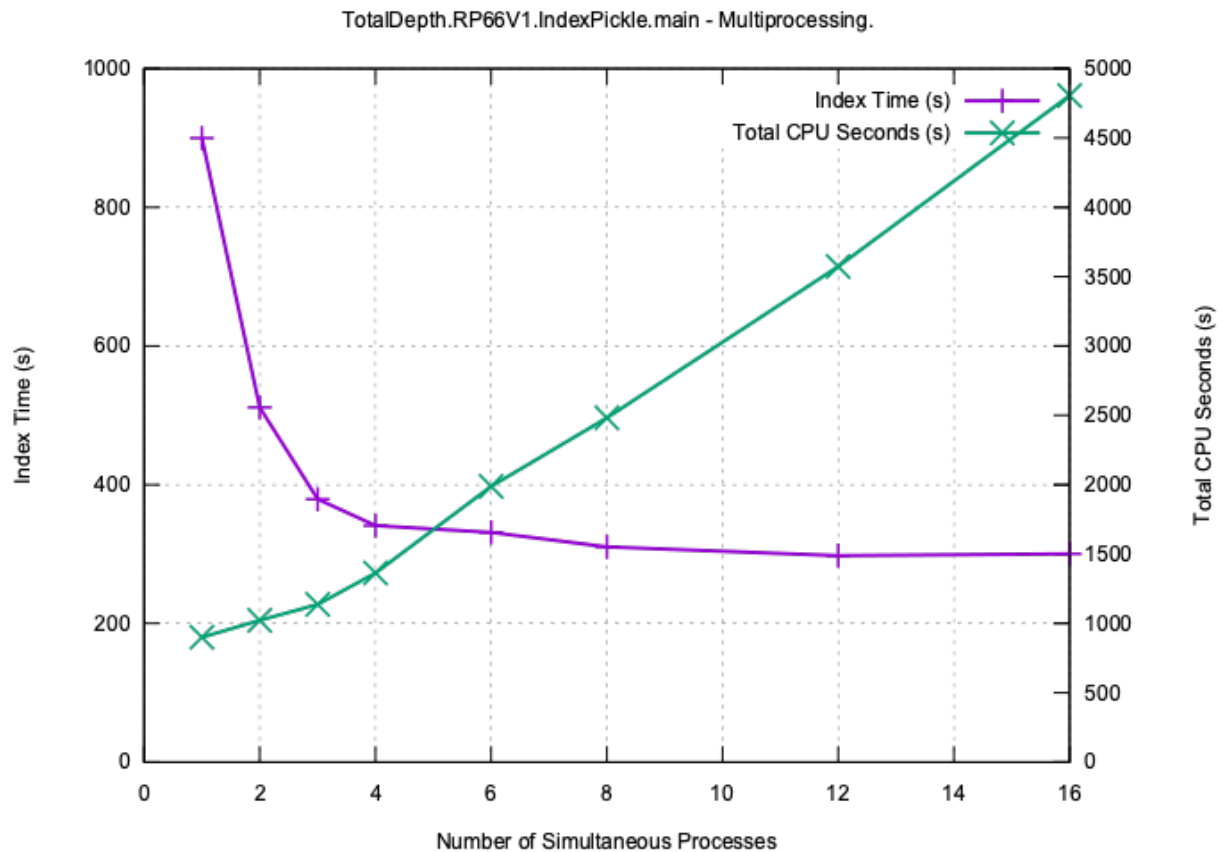
Memory and CPU Usage

Indexing the test set makes this memory and CPU demands:



Multiprocessing

Here is the same test data set being indexed including pickling the index to disk with a different number of simultaneous processes.



Because the I/O is so much higher the improvement is more limited, at best x3 faster with 4 cores but still useful.

7.7 RP66V1 Performance

This describes the performance of processing RP66V1 binary files by TotalDepth.

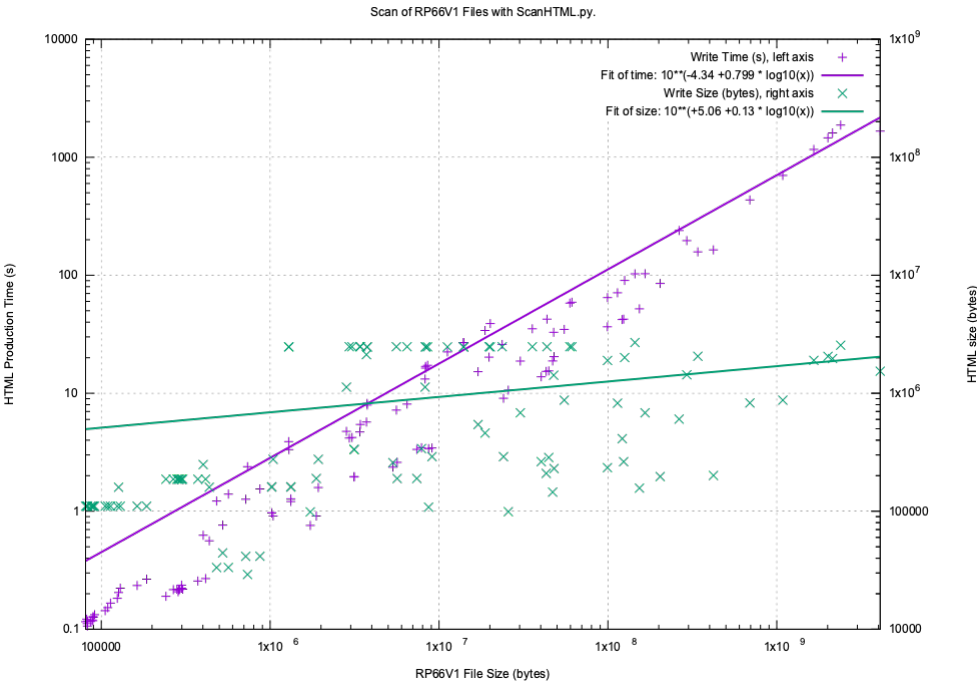
Note: This data refers to version 0.3.0 and may or may not be relevant to the current version, 0.4.0.

7.7.1 Scanning RP66V1 to produce an HTML Summary

The test set was around 100+ files that ranged in size from 80kb to 4GB totalling 17GB. The average file size was about 150Mb. These tests were run on a 2.7 GHz Intel Core i7 machine with 4 cores and hyper-threading.

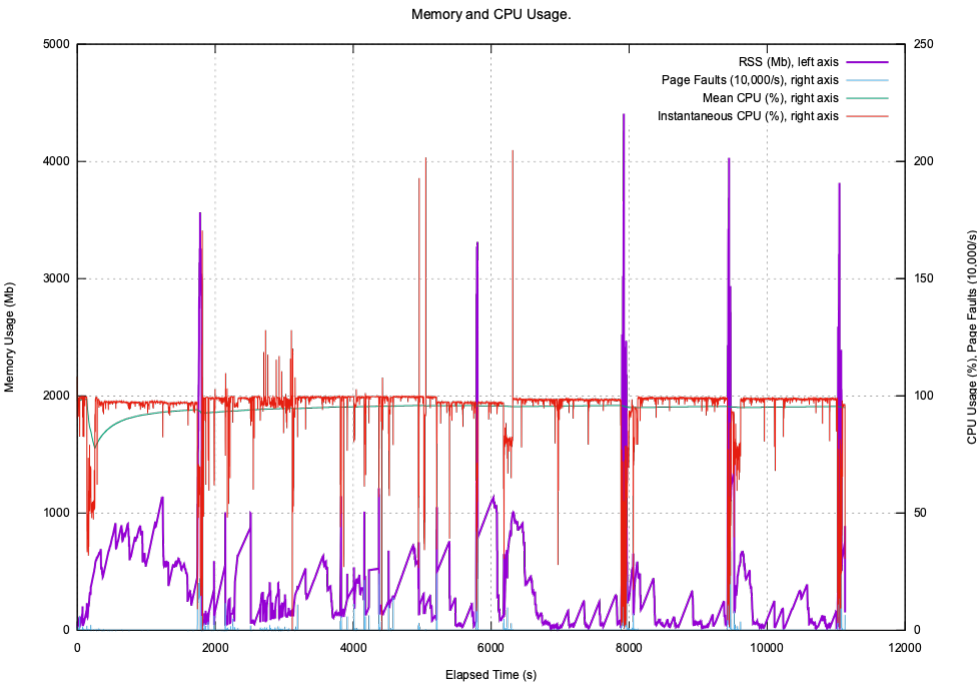
An archive of RP66V1 can be scanned to provide a summary in HTML by `TotalDepth.RP66V1.ScanHTML`. This produces an HTML page with every EFLR and a summary of all the log frame data. This is exposed as a command line tool `tdrp66v1scanhtml`.

By default this processes every byte of the file so can take a long time with large files. Here is the execution time for processing all frames by RP66V1 file size and the size of each HTML file produced.



The asymptotic processing rate is around 800 ms/Mb.

And here is the memory and CPU usage:



Performance Improvements

With very large sets of frame data not every frame needs to be processed. The option `--frame-slice` can be used to sample a subset of frames. For example:

- `--frame-slice=1024,2048,64` will process every 64th frame from frame 1024 to 2048
- `--frame-slice=64` will process only 64 frames of those available (roughly evenly spaced from those available).

Multiprocessing will help proportionally. In one test case the processing time for the archive fell from 11,000 seconds to 800 seconds using `--frame-slice=, ,64` and `--jobs=4` on a four core machine. The performance improvement of around x15 was attributed to x5 (frame slicing) and x3 (multiprocessing).

7.7.2 Converting RP66V1 to LAS

The test set was around 52 files that ranged in size from 400kb to 2.2GB totalling 12GB in all. The average file size was about 235Mb. These tests were run on a 2.7 GHz Intel Core i7 machine with 4 cores and hyper-threading.

An Example File

An example file in the tests archive is 2GB in size and contains this Log Pass, the size of the numpy frame to hold all the data is also shown:

Frame Array	Channels	Frames	Spacing	Size of Numpy array (bytes)
1B (O: 35 C: 0)	12	653880	0.1 inch	31,386,240
2B (O: 35 C: 0)	6	326940	0.2 inch	7,846,560
10B (O: 35 C: 0)	19	65388	1 inch	10,200,528
15B (O: 35 C: 0)	14	43592	1.5 inch	2,441,152
20B (O: 35 C: 0)	83	32694	2 inch	10,462,080
60B (O: 35 C: 0)	204	10899	6 inch	2,294,980,632
120B (O: 35 C: 0)	6	5449	12 inch	11,028,776

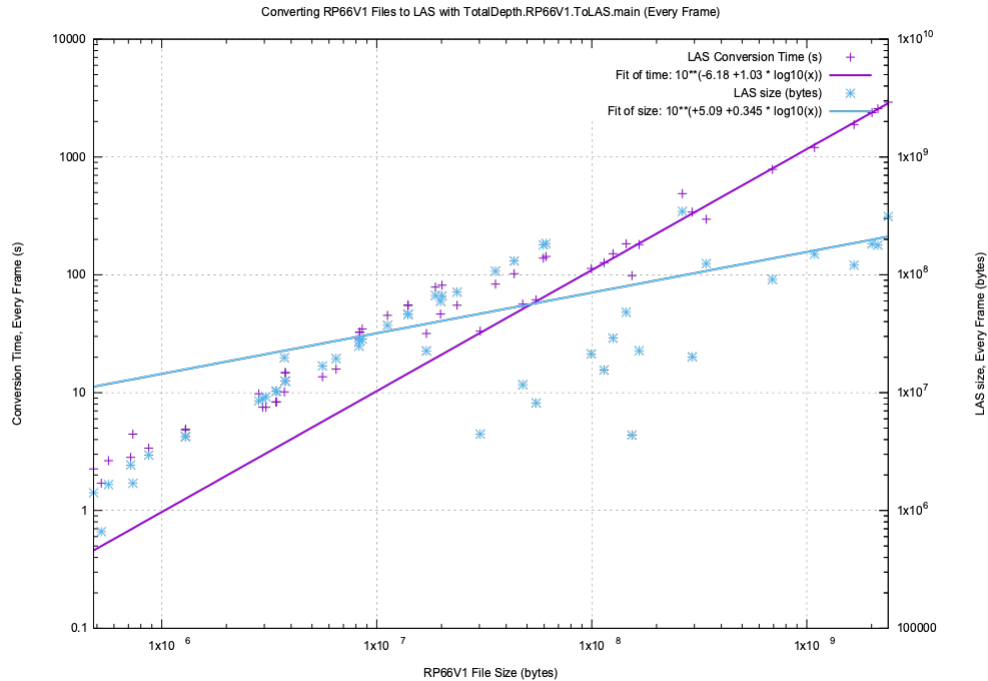
The 60B Frame Array contains some complex waveform data. Processing this produces seven LAS files, each LAS file contains all the parameter data and data from a single Frame Array. Depending on the frame slice the following processing times and LAS sizes were observed.

Frame Slice	Time (s)	LAS Size (bytes)	ms/Mb
1	2922	312,604,951	1282
4	855	100,570,379	375
16	303	47,562,819	133.4
64	188	34,309,331	82.8
256	156	30,995,584	68.6
512	152	30,444,007	66.9
1024	148	30,166,341	65.0

And of course limiting the number of channels has a proportionately similar effect.

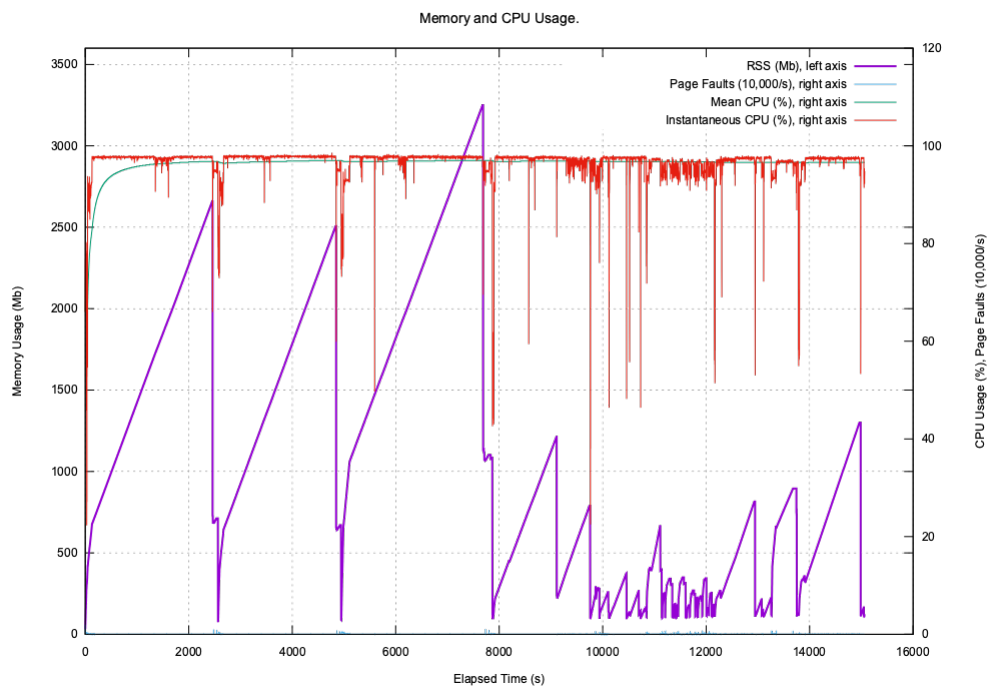
Processing the Test Archive to LAS

Here is the time taken to process each file plotted against the RP66V1 file size. Also plotted on the right scale is the total size of the LAS file(s). This is converting every frame to LAS (click to see the original):



The asymptotic processing rate is around 1230 ms/Mb. LAS files (in this test set) below 10Mb tend to be larger than the original, above 100Mb they tend to be around 20% smaller.

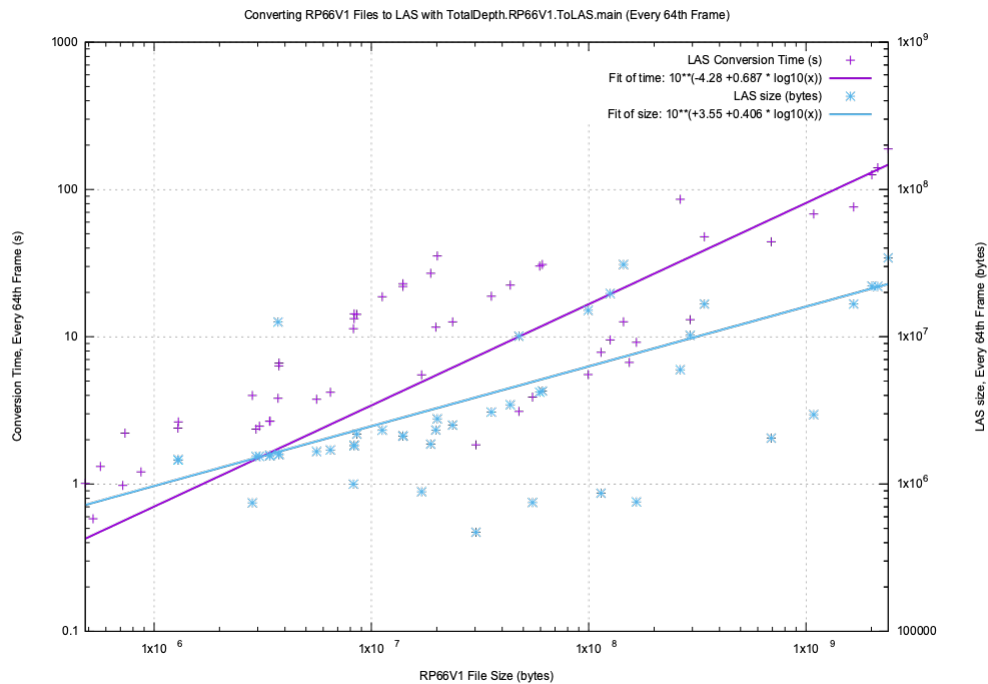
And this is the memory usage (click to see the original):



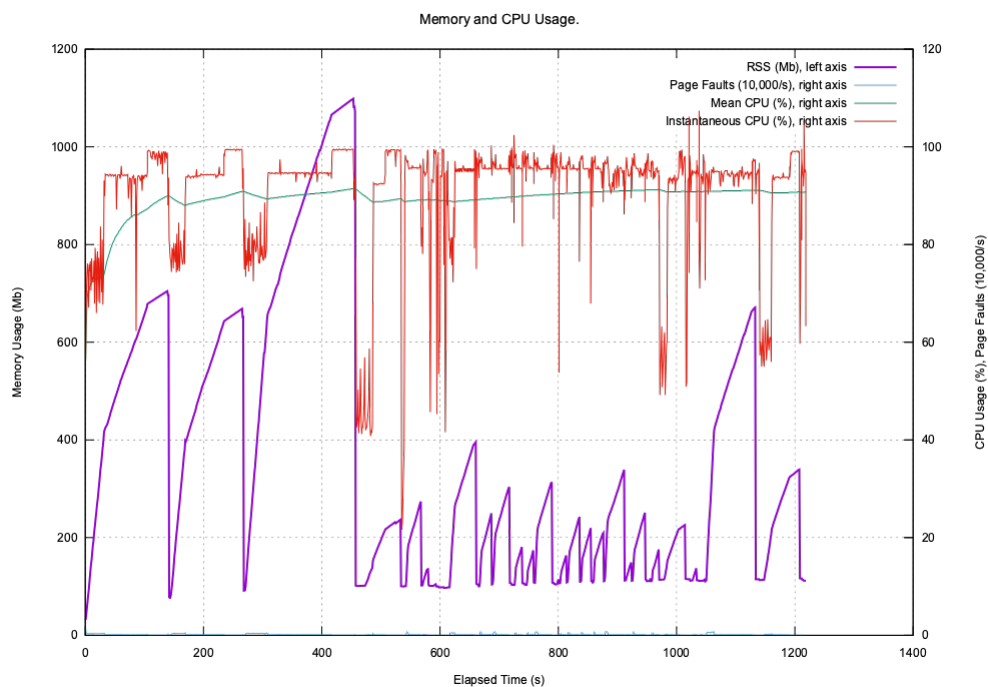
The peaks are caused by the multi-gigabyte Numpy arrays needed for some files.

Sub-sampling

Here is the performance of the same data with `--frame-slice=64` that just writes every 64th frame to LAS (click to see the original):

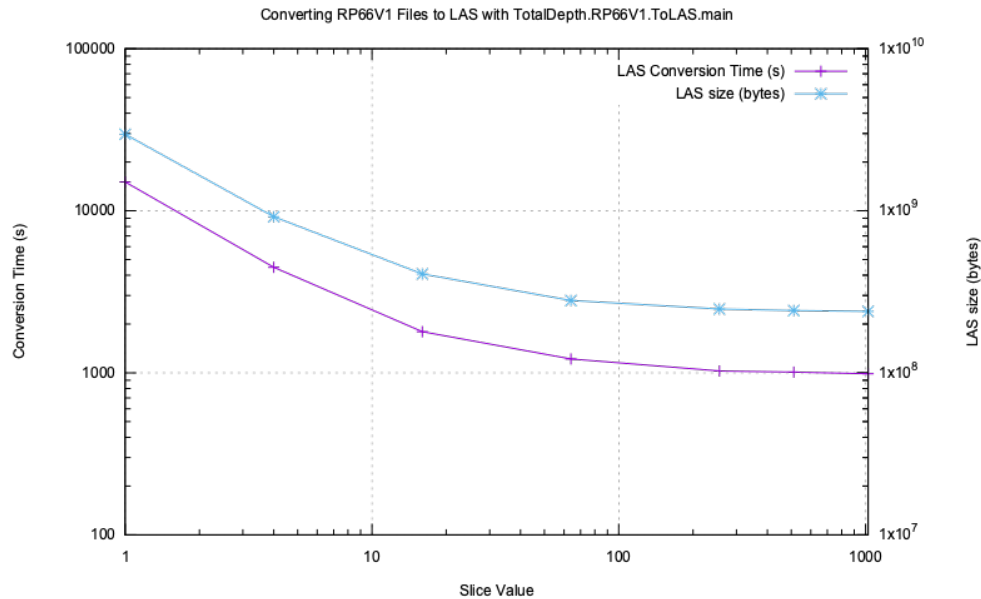


And this is the memory usage (click to see the original):



This is vastly reduced from the every frame case.

Frame slicing can improve the performance dramatically. Here is the time to process the test archive and the size of the finished LAS archive by slice, for example 64 on the X-axis is write only every 64th frame:



Multi-Processing

Using `--jobs` can also improve performance proportionally if you have lots of cores and good I/O.

7.8 Plotting Wireline Data

Aside from the LogPass (the data structure that holds the frame data) the plot layout needs to be specified. There are two ways that TotalDepth does this:

- A plot specification that is *integral* to the file. For example; LIS files may well have FILM and PRES tables within them that configure the plot as the recording engineer (or software) intended. In this case TotalDepth can make those plots directly.
- A plot specification that is specified *externally* to the file. TotalDepth supports one such way; using LgFormat XML files and these are described below.

7.8.1 Wireline Files With Integral Plot Data

This applies to LIS (and possibly RP66 files). The LAS file specification does not describe any plot specification at all.

LIS Plot Specification

There are a number of table-like Logical Records (type 34) within a Log Pass that can specify the plot layout. As a minimum TotalDepth.LIS needs a FILM and a PRES record.

FILM Record

A dump of a FILM Logical Record might look like this:

Table record (type 34) type: FILM				
MNEM	GCOD	GDEC	DEST	DSCA

1	EEE	----	PF2	D200
2	E4E	-4--	PF1	DM

The columns are:

Col- umn	Description
MNEM	The (logical) name of the output, TotalDepth uses this as part of the plot file name.
GCOD	The coding of the tracks. E4E means three tracks which have linear, log (4 decades), linear. With three tracks the X axis track (depth for example) appears between the first and second tracks.
GDEC	The number of logarithmic decades for each track, - for linear.
DEST	The physical destination (in the logging unit) of the plot. Ignored.
DSCA	The depth scale, encoded. For example D200 means 1:200.

PRES Record

A dump of a PRES Logical Record might look like this:

Table record (type 34) type: PRES									
MNEM	OUTP	STAT	TRAC	CODI	DEST	MODE	FILT	LEDG	REDG

SP	SP	ALLO	T1	LLIN	1	SHIF	0.500000	-80.0000	20.0000
CALI	CALI	ALLO	T1	LDAS	1	SHIF	0.500000	5.00000	15.0000
MINV	MINV	DISA	T1	LLIN	1	SHIF	0.500000	30.0000	0.00000
MNOR	MNOR	DISA	T1	LDAS	1	SHIF	0.500000	30.0000	0.00000
LLD	LLD	ALLO	T23	LDAS	1	GRAD	0.500000	0.200000	2000.00
LLDB	LLD	ALLO	T2	HDAS	1	GRAD	0.500000	2000.00	200000.
LLG	LLG	DISA	T23	LDAS	1	GRAD	0.500000	0.200000	2000.00
LLGB	LLG	DISA	T2	HDAS	1	GRAD	0.500000	2000.00	200000.
LLS	LLS	ALLO	T23	LSPO	1	GRAD	0.500000	0.200000	2000.00
LLSB	LLS	ALLO	T2	HSPO	1	GRAD	0.500000	2000.00	200000.
MSFL	MSFL	ALLO	T23	LLIN	1	GRAD	0.500000	0.200000	2000.00
11	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
12	DUMM	DISA	T1	LLIN	NEIT	NB	0.500000	0.00000	1.00000
...									

Column	Description
MNEM	The (logical) name of the output curve. Note multiple curves might come from one OUTP.
OUTP	The source of the curve.
STAT	Status, is this curve to be plotted.
TRAC	Which track the curve should be plotted on.
CODE	The line coding, dot, dash etc.
DEST	The logical film that this curve is sent to. Can be BOTH and so on.
MODE	What to do when the curve goes off scale (wrap round the track for example).
FILT	Filtering. Unused.
LEDG	Value of the left edge of the plot.
REDG	Value of the right edge of the plot.

Records Needed for a LIS Plot

As a minimum TotalDepth needs a FILM and PRES plot, using these TotalDepth can produce a plot identical to that when the data was recorded.

In future releases TotalDepth might also support these Logical Records for plotting:

- AREA: Describes shading between curves on a plot.
- PIP: Describes integration marks on the plot such as integrated transit time, integrated hole volume etc.
- VDL: Describes presentation of waveform amplitude data.

7.8.2 Wireline Files With External Plot Data

This can apply to any wireline file and it means using an external configuration file to decide the plot layout. For example; LAS files do not have any plot specification (the LAS standard precludes this). The presentation of the plot is made with an external plot specification.

These external plot specifications are numerous and varied. TotalDepth currently supports one such specification: the XML *LgFormat* or *LgSchema* files.

LgFormat XML Files

These appear to originate from some of Schlumberger's "free" software.

Here is an example LgFormat XML file for plotting [HDT logs](#). TotalDepth supplies an number of examples of these.

There are a number of problems¹ with the LgFormat which is why TotalDepth only supports a subset of it while we look for something better.

¹ For example the LgCurve element content of <LeftLimit> and <RightLimit> is numeric, without units. This robs a compliant implementation of the opportunity of converting frame data, say in "V/V " to plot data say "PU " which might be far more appropriate. Truth in advertising prompts me to say that LIS PRES tables have the same flaw but they have a reasonable excuse that they are *per file* specification rather than a global specification.

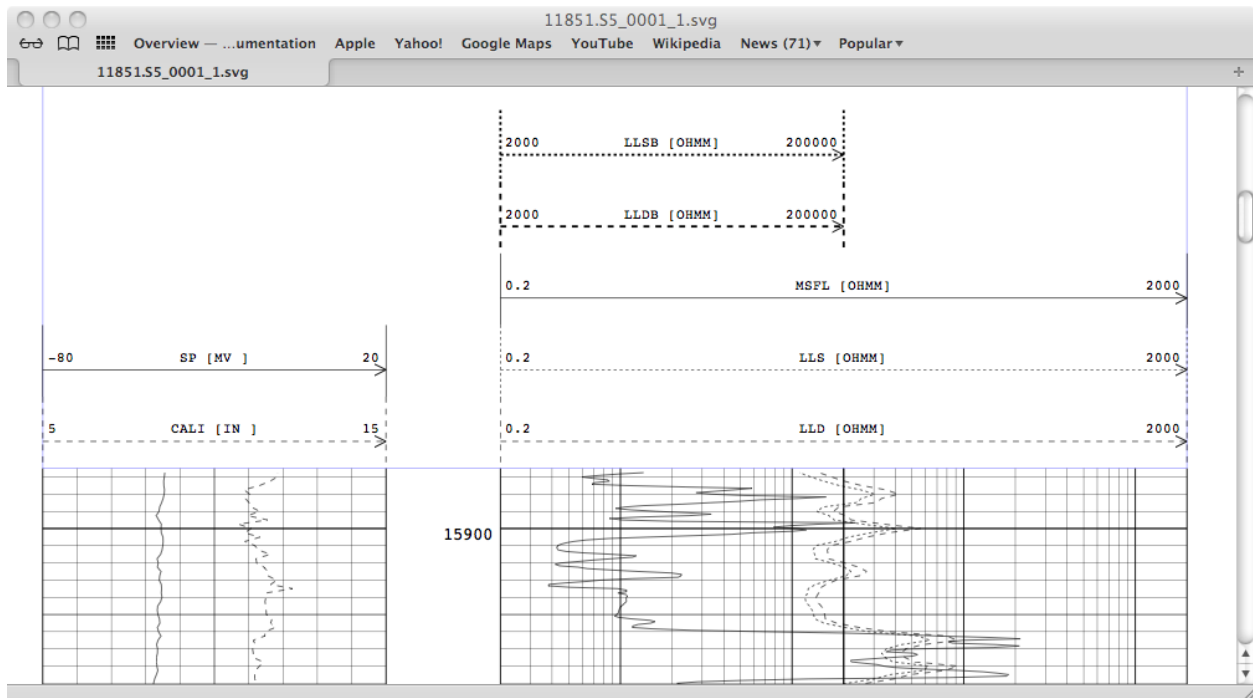
7.8.3 Internal Plot Algorithms

This describes some algorithms used by the Plot module. This is for information only as these algorithms are internal and the user has little or no control over them.

Plotting the Legends (Scales)

These are at each end of the plot per curve and may span a number of tracks or be limited to half a track. Here is an example which has seven curves:

TODO: Finish this



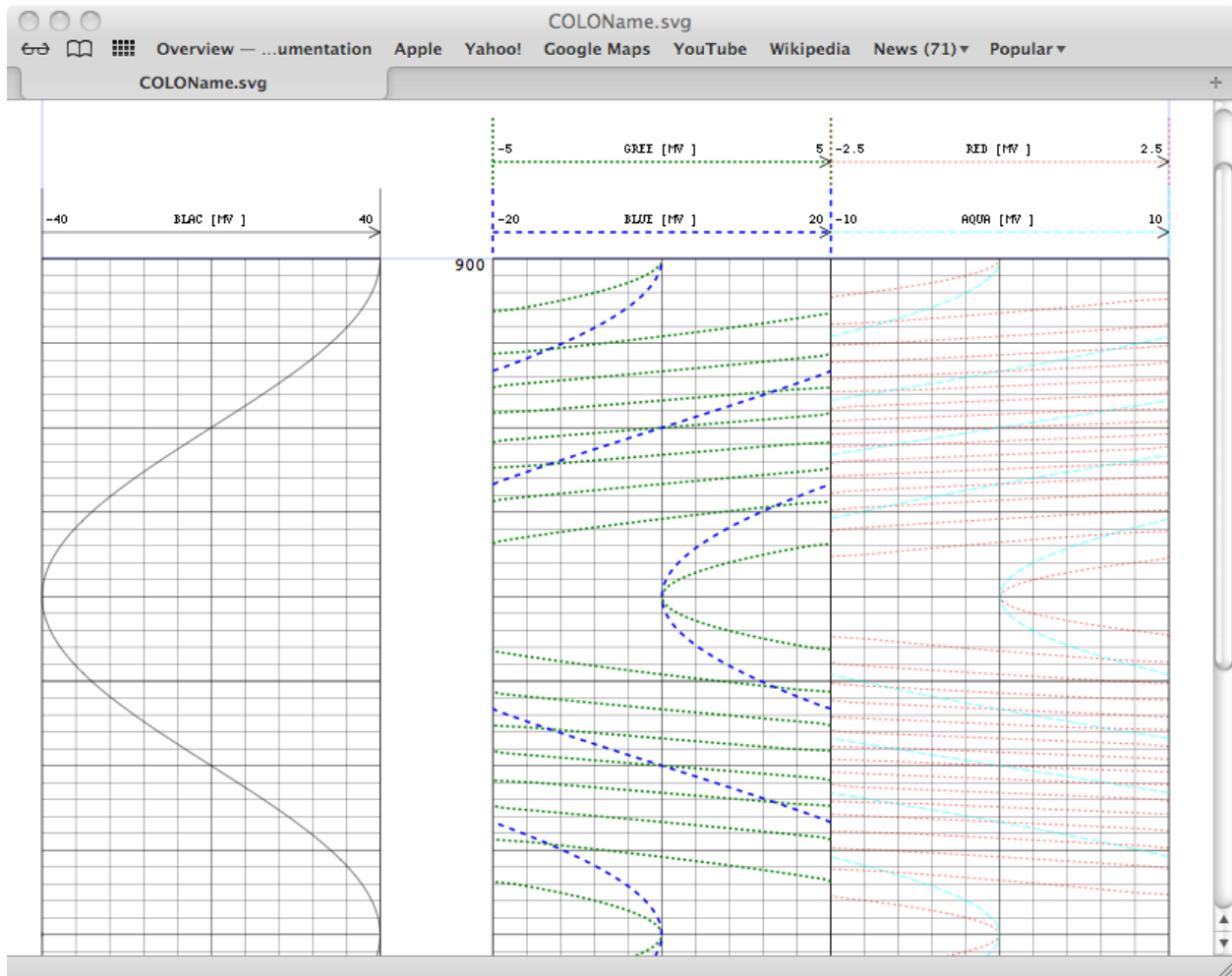
There are two important methods that implement this algorithm: `Plot.Plot._retCurvePlotScaleOrder()` and `Plot.Plot._plotScale()`

Interpolating Backups

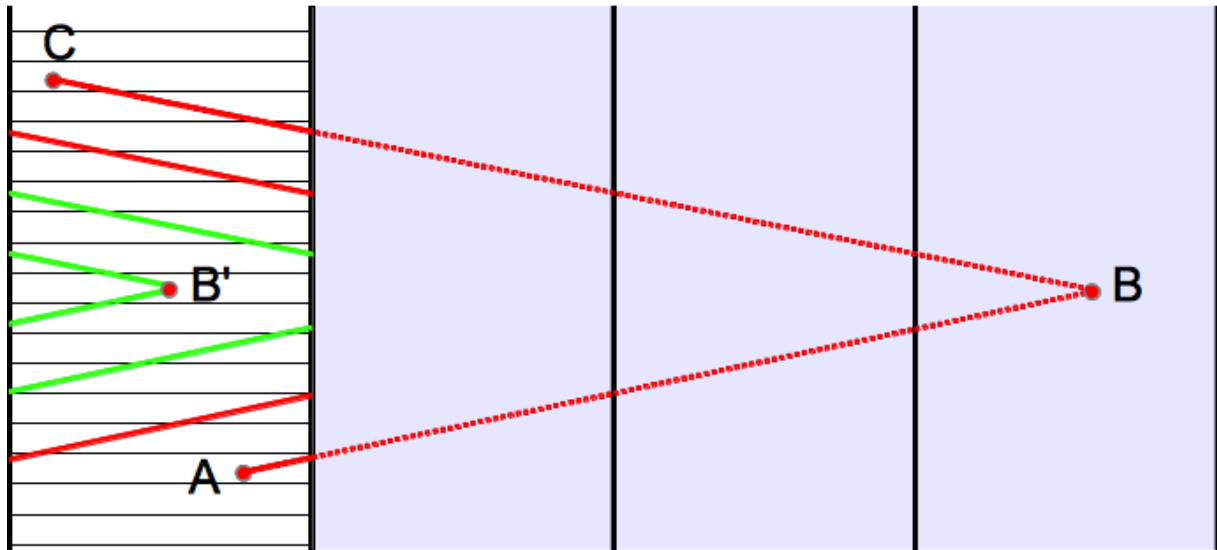
Industry standard wireline plots have the notion of curve backups where if the trace for a particular curve goes off scale on one side of the track it might well reappear from the alternate side.

Here is a simulated example generated by `TestPlot.py` where a single curve (a sin curve with an amplitude of ± 40) is plotted in full on track one. The same curve is plotted fully backed up:

- In green on track two with a scale of ± 5
- In blue on track two with a scale of ± 20
- In red on track three with a scale of ± 2.5
- In aqua on track three with a scale of ± 10



The algorithm that does this is probably best illustrated by example. The diagram below shows a curve (in the white track on the left) that goes from point A goes off-scale to the right to point B in 'real' space. The line A-B is subdivided by track widths and these *crossing* lines are plotted by interpolation in the track space making the line space A-B' (so-called 'FlyBack' lines are omitted in the diagram). If the *Backup Mode* is *wrap* then both red and green lines in the track are plotted. In this case if the *Backup Mode* is *1* then only the red lines are plotted in the track i.e. only one backup allowed.



On the return B-C a similar process happens in reverse.

Backup Specification

A backup specification is expected for any plot *per-curve*. Internally a backup specification is a pair of integers. If the virtual track position is less than the value (if left) or greater than the value (if right) then it is off scale. Zero is a special case in that all virtual track positions are on scale.

Various values have been predefined and there are existing mappings from LIS data and LgFormat XML files. The following table describes all of this.

Spec.	Symbol	Description	LIS Map- ping	LgFormat Mapping
(1, -1)	BACKUP_NONE	No backup	'NB', 'GRAD'	
(0, 0)	BACKUP_ALL	Every backup i.e. 'wrap' Note: Plot.py has a way of limiting ludicrous backup lines to a sensible number; say 4	'WRAP'	'LG_WRAPPED', 'LG_X10'
(-1, 1)	BACKUP_ONCE	Single backup to left or right	'SHIF'	'1'
(-2, 2)	BACKUP_TWICE	Two backups to left or right		'2'
(0, -1)	BACKUP_LEFT	Single backup to left only		'LG_LEFT_WRAPPED'
(1, 0)	BACKUP_RIGHT	Single backup to right only		'LG_RIGHT_WRAPPED'

Note that it is also common to have one curve with no backup and a second curve driven from the same output with a different scale acting as a backup. This has the advantage that a different coding and colour can be assigned to it. See [Plotting the Legends \(Scales\)](#) above for an example with a Laterlog plot.

Algorithm

The backup algorithm works as follows:

- For every point plotted an integer wrap value is calculated.
- If the wrap value is different from the previous wrap value then some interpolation is required and the `Plot._interpolateBackup()` method is called (below).
- Otherwise the point is plotted.

`_interpolateBackup`

The `Plot._interpolateBackup()` method manages all the different cases where a backup is needed given the backup configuration. It relies on `Plot._retInterpolateWrapPoints()` to generate a list of interpolated lines, including *crossing lines*, between one point and another.

`_filterCrossLineList`

A further refinement to the algorithm is limiting the number of *crossing lines*. Consider the SP curve represented by this data taken from a real example:

Depth (ft)	SP (mV)
10245.0	332.724
10245.5	13716948.000
10246.0	41150204.000
10246.5	68583456.000
10247.0	96016712.000
10247.5	334.176

Clearly some episode happened between 10245.0 and 10245.5 feet that caused a jump of 13716615.276 mV. This could have been an recording disturbance (unlikely as where would you find 96kV at 10247 feet?) or an editing error. In any case on a scale of -20 to 80 mV means 137,166 (mostly spurious) wrap lines crossing the track. This can turn a 1.6Mb file into a 91Mb file! To stop this a arbitrary limit is made to the number of *crossing lines* (e.g. 4) between each Xaxis interval. This limit filters the *crossing line* list to an evenly distributed list of 4 (or thereabouts).

This is also applicable to RFT plots where it is common to plot the pressure modulo 10 psi in RHT3.

The method that does this is The `Plot._filterCrossLineList()` method

7.9 Data Quality

TODO:

7.10 TotalDepth and SaaS

This design shows the potential for running TotalDepth as *Software as a Service* (SaaS) or, if you prefer, *Cloud Computing*.

The design envisages a server running the full TotalDepth software and any number of clients running sub-sets of that software. Client software could be in-browser or on-device.

Network speed (i.e. latency+bandwidth) is crucial to the performance of this system and the network connection between the client and the server could be any of these (in order of slowness):

- A client and a server both running on the client's machine.
- A client device and a server running on a high speed LAN.
- A client device communicating with a server via a VPN.
- A client device communicating with a remote server via the internet.

TotalDepth accommodates all of these.

7.10.1 Processing Server Side Files

There is no particular challenge here as the server has all it needs to process any client instruction. The big disadvantage is that the client has to upload the file to the server before they can do anything. For small jobs on large files that can impose an disproportionate cost.

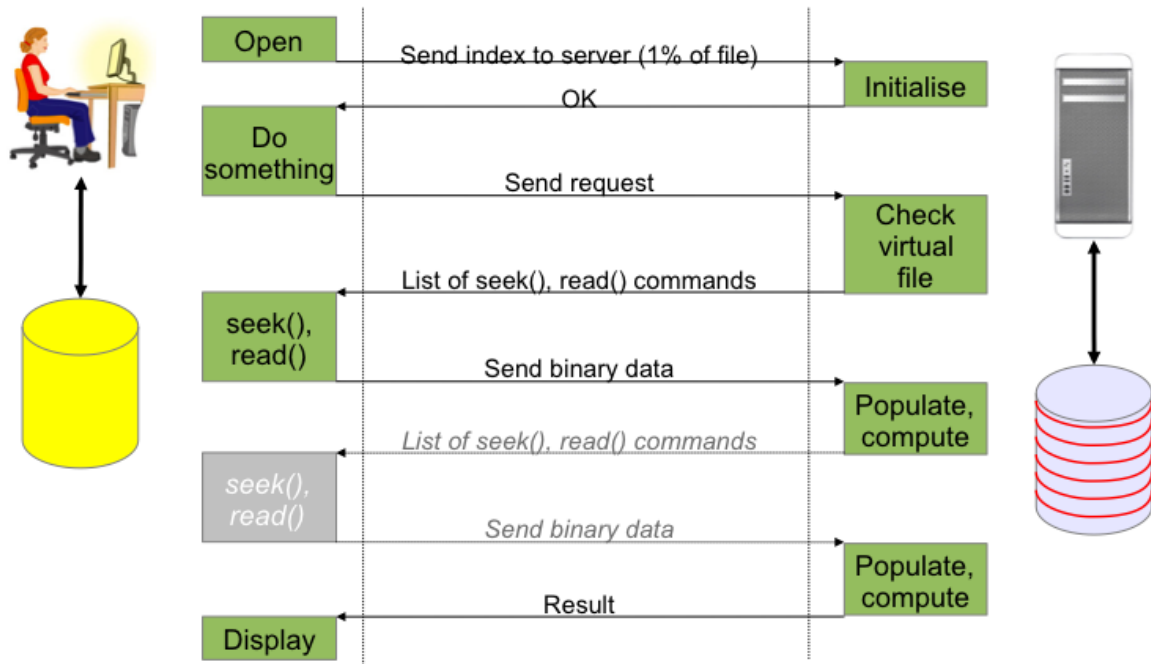
7.10.2 Processing Files that Remain on the Client's Device

The real challenge is: the client has a (very large?) file and wants to do something small (or at least incremental) with it, for example "Plot curves XYZ from 5840 to 5600 feet using format ABC" and the server only needs a tiny fraction of that file. Regardless of the network connection there should be some cunning in extracting the data that the client needs in minimal time. TotalDepth's indexing technique makes this possible.

This design shows how the client can work with the server without the tedious business of the client having to upload the complete file to the server. Instead the client and server cooperate to make sure that the minimum data is transferred for the immediate task in hand.

The design is illustrated below, the client is on the left:

TotalDepth SaaS



Starting the session

The session is initialised by the user requesting the client to operate on a particular file on the clients device. The client passes basic file information to the server and receives a unique session ID.

The Session ID

The session ID could be a portable file ID, such as a checksum. Thus two users using the same file could benefit from each others read transactions. This would require copy-on-write and session ID modification.

Another optimisation when using a checksum is that if the server already has an exact copy of the file (using the checksum as a proxy for that) then the server could provide data from its own copy of the file without the user having to transfer any data to the server at all.

Session Bootstrapping

Given a valid session ID the client code reads the minimum structural information for the server to comprehend (and mirror) the file structure. This will vary according to the file format (and will vary according to file size).

LIS

The client code must be capable of processing TIF markers, Physical Record headers and trailers and Logical Record headers. The client passes this data as an ordered list of pairs [(tell, bytes), ...] where tell is the `size_t` position in the of the Physical Record (or TIF marker) that starts the Logical Record and bytes is the raw two bytes of the Logical Record Header.

LAS

As LAS files are small and do not lend themselves easily to efficient indexing then, most likely, it is worth passing the entire client file to the server.

RP66

This will be similar to LIS bootstrapping with the client code processing Visible Record Headers and Logical Record segments.

Index Completion

The bootstrap information passed to the server allows the server to construct a virtual, partial, logical image of the file. The server may request further information in the form of an ordered list of pairs [(file position, number of bytes), ...]. The client does a series of seek/read operations and passes the data back to the server as a list of pairs [(file position, bytes), ...]. This allows the server to complete the file index.

The total size of the data transferred to the server at this stage is typically 0.6% of the file size. See [Indexing LIS Files](#) for a description of how LIS files are indexed and [Multi-Processing](#) for a study of index sizes.

7.10.3 Rest of Session

Once bootstrapped any user request is passed by the client to the server, for example: “Plot curves ABC from 5840 to 5600 feet”.

The server then consults its virtual file image to see if it has the data to satisfy the request. If so then the result of the request is sent to the client.

If the server does not have sufficient data in its virtual file image then the server will send to the client an (ordered) list of pairs [(file position, number of bytes), ...]. The client does a series of seek/read operations on each pair and passes the data back to the server as a list of pairs [(file position, bytes), ...]. The server adds these to its virtual file image. The server then can complete the request.

Over time the virtual file image would grow but only as fast as the user’s (new) demands.

7.10.4 Data Persistence

It is possible for the server to persist the virtual file image over any number of sessions, this would improve the performance even further.

7.10.5 SaaS Pros and Cons

Advantages

- Low barrier to use: browser based, no installation.
- Cross Platform: desktop, tablet, mobile etc.
- Minimum client code.
- Continuous software version update from the server.
- Integrated with other Internet services for example mapping data.
- Tailored appearance per client.
- Cloud availability behind the server.
- All usage data is available on server logs.

Disadvantages

- Requires network connection (could have a server version running locally).
- User agent variability.
- Speed/performance limited by network.
- Server infrastructure and investment.
- Continuous maintenance and support.
- Security.

7.11 Process Monitoring with TotalDepth.common.process

TotalDepth.common.process can monitor the memory and CPU usage of a running process. It does this by creating a thread which, at regular intervals, reports process data to the log file in JSON. The basic use is like this:

```
from TotalDepth.common import process

with process.log_process(1.0):
    # Your code here
```

Then TotalDepth.common.process will write process data as a single line in the log file every 1.0 seconds. The JSON data is preceded by the following, recognisable, entry in the log file:

```
2019-10-31 14:10:06,051 - process.py - 86611 - (ProcMon  ) - INFO      -
↪ProcessLoggingThread-JSON
```

The JSON data looks like this example (but on one line):

```
{
  "timestamp": "2019-10-31 14:10:06.051630",
  "memory_info":
    {
      "rss": 11939840,
      "vms": 4404228096,
      "pfaults": 531770,
      "pageins": 0
    },
  "cpu_times": {
    "user": 0.583945792,
    "system": 0.66087648,
    "children_user": 0.0,
    "children_system": 0.0
  },
  "elapsed_time": 12.42771601676941
}
```

7.11.1 Command Line Tools

Command line tools can add process capability with an argument parser created by `TotalDepth.common.cmn_cmd_opts arg_parser()`:

```
process.add_process_logger_to_argument_parser(parser)
```

This makes the `--log-process` option available which takes a numeric value as a float in seconds (default zero which means no process logging) for the logging interval. Your code pattern is then:

```
args = parser.parse_args()
if args.log_process > 0.0:
    with process.log_process(args.log_process):
        # Do something
```

7.11.2 Plotting the Data

`TotalDepth.common.process` can be used from the command line to extract the data from the log file and plot it with Gnuplot.

Example

Here we will create eight large, randomly sized strings and simulate doing some work:

```
import random
import time

from TotalDepth.common import process

with process.log_process(0.1):
    for i in range(8):
        size = random.randint(128, 128 + 256) * 1024 ** 2
```

(continues on next page)

(continued from previous page)

```
s = ' ' * (size)
# Simulate 0.5 to 1.5 seconds of work.
time.sleep(0.5 + random.random())
del s
# Simulate 0.25 to 0.75 seconds of work.
time.sleep(0.25 + random.random() / 2)
```

This will produce a log such as:

```
2019-10-31 14:09:53,726 - process.py - 86611 - (ProcMon  ) - INFO      -_
↳ProcessLoggingThread-JSON {"timestamp": "2019-10-31 14:09:53.726676", "memory_info": {
↳"rss": 11898880, "vms": 4404228096, "pfaults": 3624, "pageins": 0}, "cpu_times": {"user
↳": 0.07540488, "system": 0.020255324, "children_user": 0.0, "children_system": 0.0},
↳"elapsed_time": 0.10263395309448242}
2019-10-31 14:09:53,896 - process.py - 86611 - (ProcMon  ) - INFO      -_
↳ProcessLoggingThread-JSON {"timestamp": "2019-10-31 14:09:53.896083", "memory_info": {
↳"rss": 162922496, "vms": 4555227136, "pfaults": 40495, "pageins": 0}, "cpu_times": {
↳"user": 0.108017992, "system": 0.056484236, "children_user": 0.0, "children_system": 0.
↳0}, "elapsed_time": 0.27210497856140137}
2019-10-31 14:09:53,997 - process.py - 86611 - (ProcMon  ) - INFO      -_
↳ProcessLoggingThread-JSON {"timestamp": "2019-10-31 14:09:53.996930", "memory_info": {
↳"rss": 162930688, "vms": 4555227136, "pfaults": 40497, "pageins": 0}, "cpu_times": {
↳"user": 0.10846144, "system": 0.05655662, "children_user": 0.0, "children_system": 0.0}
↳, "elapsed_time": 0.373028039932251}
...
2019-10-31 14:10:06,051 - process.py - 86611 - (ProcMon  ) - INFO      -_
↳ProcessLoggingThread-JSON {"timestamp": "2019-10-31 14:10:06.051630", "memory_info": {
↳"rss": 11939840, "vms": 4404228096, "pfaults": 531770, "pageins": 0}, "cpu_times": {
↳"user": 0.583945792, "system": 0.66087648, "children_user": 0.0, "children_system": 0.
↳0}, "elapsed_time": 12.42771601676941}
```

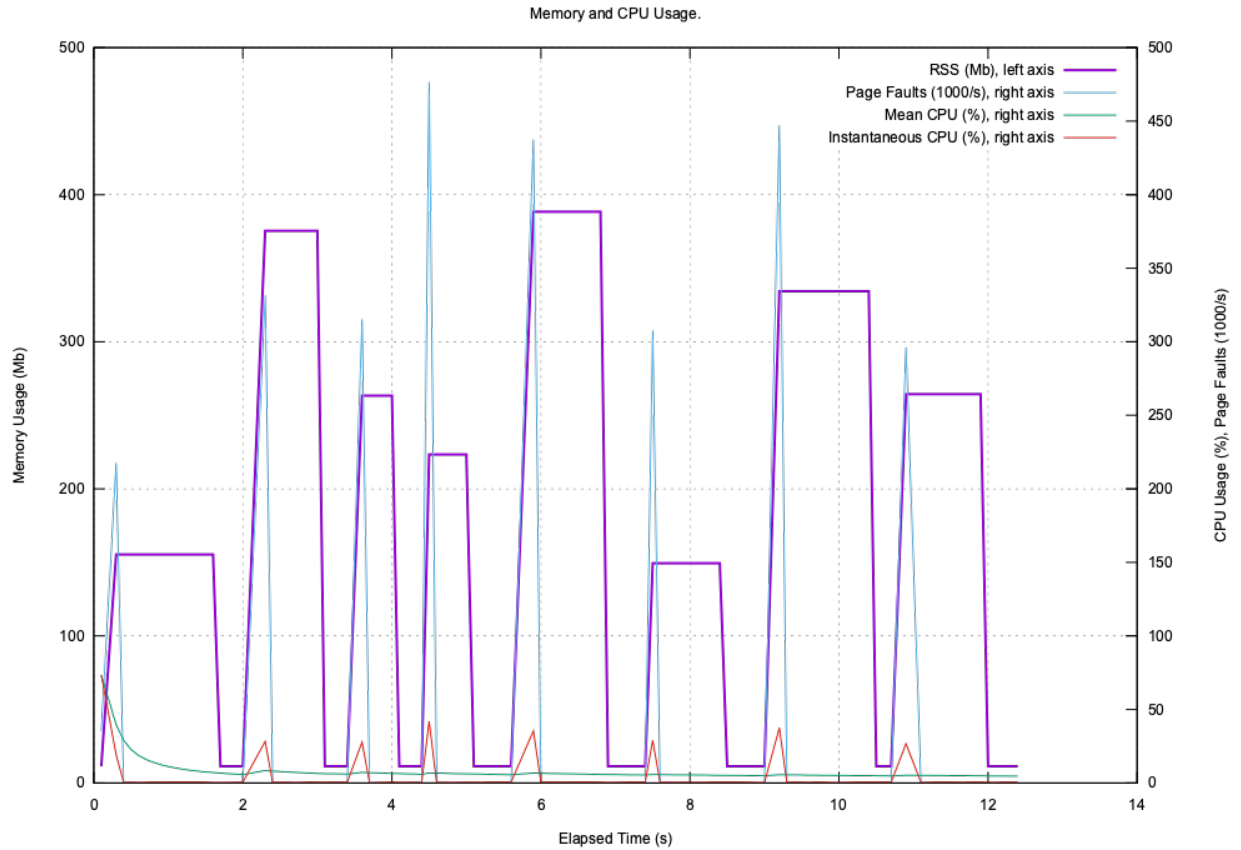
Then run `TotalDepth.common.process` CLI entry point with two arguments, the log file and a directory to write the Gnuplot data to.

```
$ python src/TotalDepth/common/process.py tmp/process_C.log tmp/gnuplot_process
2019-10-31 14:11:29,737 - gnuplot.py - 86631 - (MainThread) - INFO      - gnuplot stdout:_
↳None
2019-10-31 14:11:29,741 - gnuplot.py - 86631 - (MainThread) - INFO      - Writing gnuplot_
↳data "process_C.log" in path tmp/gnuplot_process
2019-10-31 14:11:29,782 - gnuplot.py - 86631 - (MainThread) - INFO      - gnuplot stdout:_
↳None
```

In the output directory there is the log data extracted as `.dat` file, the Gnuplot specification as `.plt` file, and the plot itself in SVG as `process_C.log.svg`:

```
$ ls -l tmp/gnuplot_process/
total 360
-rw-r--r-- 1 xxxxxxxx staff 13679 31 Oct 14:11 process_C.log.dat
-rw-r--r-- 1 xxxxxxxx staff 1067 31 Oct 14:11 process_C.log.plt
-rw-r--r--@ 1 xxxxxxxx staff 30878 31 Oct 14:11 process_C.log.svg
-rw-r--r-- 1 xxxxxxxx staff 32100 31 Oct 14:11 test.svg
```

Here is `process_C.log.svg`:



7.11.3 Adding Events as Graph Labels

You can also inject events into `TotalDepth.common.process` as string messages and these will be reproduced on the plot as labels. So adding one line of code:

```
import random
import time

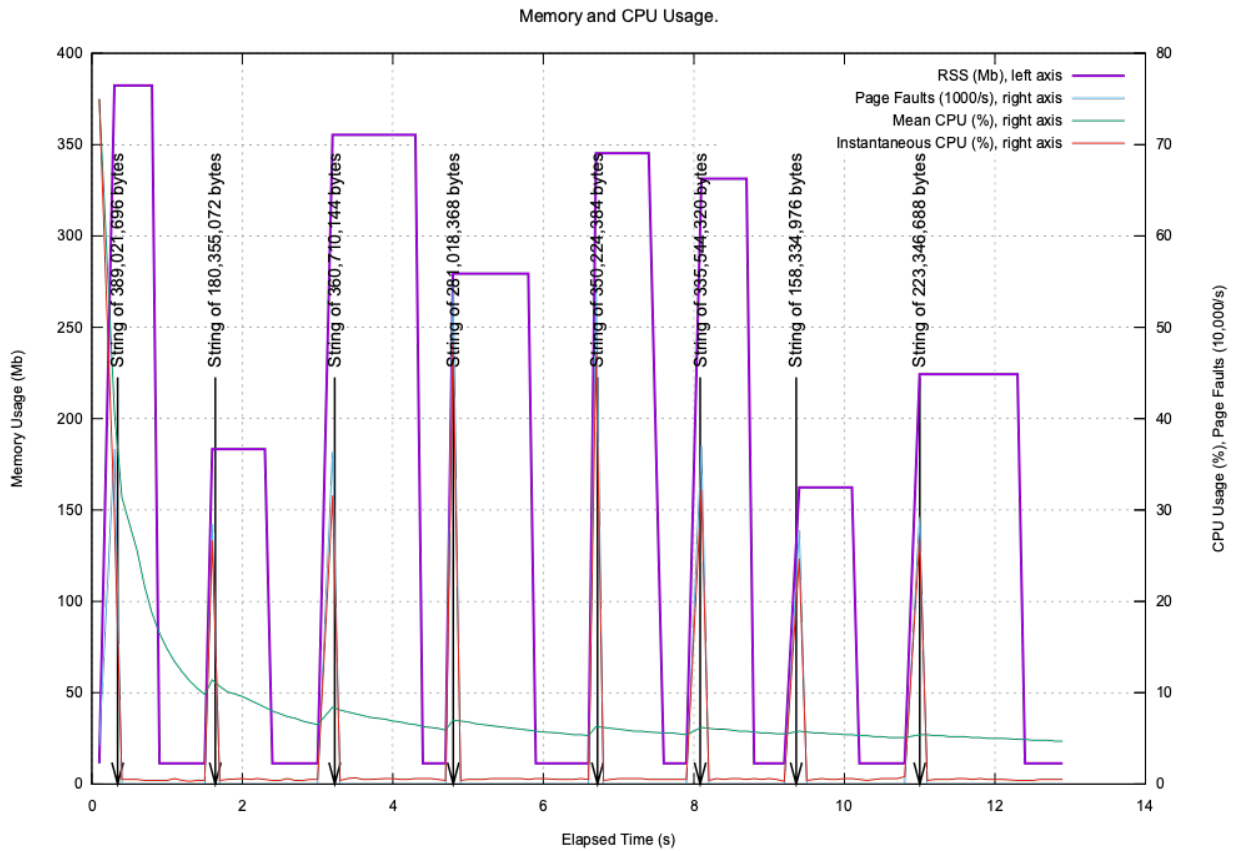
from TotalDepth.common import process

with process.log_process(0.1):
    for i in range(8):
        size = random.randint(128, 128 + 256) * 1024 ** 2
        process.add_message_to_queue(f'String of {size:,d} bytes')
        s = ' ' * (size)
        # Simulate 0.5 to 1.5 seconds of work.
        time.sleep(0.5 + random.random())
        del s
        # Simulate 0.25 to 0.75 seconds of work.
        time.sleep(0.25 + random.random() / 2)
```

Adds that label into the JSON on the next write:

```
{
  "label": 'String of xxx bytes'
  # ...
}
```

When plotted these labels appear on the plot:

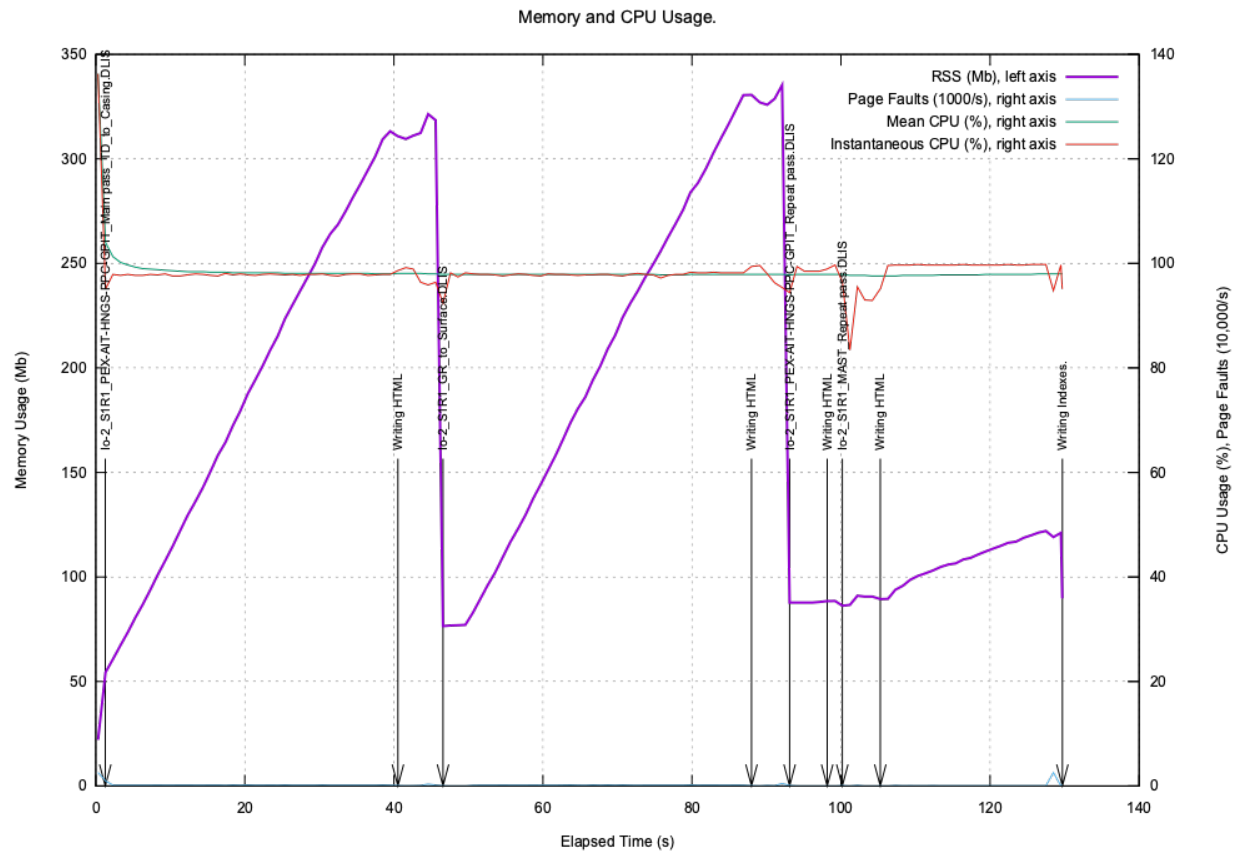


A Real World Example

Here is an example of running `TotalDepth.RP66V1.ScanHTML` on four files of sizes 75, 80, 8 and 500 MB. `TotalDepth.RP66V1.ScanHTML` essentially does two things:

- Creates an index of the RP66V1 file.
- Then iterates across that index writing HTML, this includes reading a (potentially) large number of frames depending on the file.

The start points of these operations are labeled on the graph.



The graphs clearly shows that for the last file reading the index is very quick but writing the HTML is comparatively slow. This is because that is an unusual file that deserves further investigation.

7.12 Performance Graphs via Gnuplot with TotalDepth.util.gnuplot

Todo: Finish this.

7.13 Total Depth and Units

This describes how TotalDepth handles unit conversions.

7.13.1 LIS Units

LIS units are identified by a four character, uppercase ASCII string. TotalDepth supports all known LIS units with the exchange values hard coded in `TotalDepth.LIS.core.Units`. The API is described there.

References:

LIS unit conversion: `TotalDepth.LIS.core.Units`

7.13.2 General Units

Clearly hard coded data is somewhat limiting. RP66V2 specifies a unit convention whereby fundamental units can be combined, for example 'm/s' to 'ft/min', then the conversion factors could be calculated parsing those strings.

TotalDepth does not do this, instead opting for a short cut of pre-built lookup tables. This uses an online source of unit conversion, the primary source is [Schlumberger's Oilfield Services Data Dictionary \(OSDD\)](#)

This essentially provides tables such as:

Code	Name	Standard Form	Dimension	Scale	Offset
DEGC	'degree celsius'	degC	Temperature	1	-273.15
DEGF	'degree fahrenheit'	degF	Temperature	0.555555555555556	-459.67
DEGK	'kelvin'	K	Temperature	1	0
DEGR	'degree rankine'	degR	Temperature	0.555555555555556	0

So conversion from, say DEGC to DEGF of 0.0 is:

```
((value - DEGC.offset) * DEGC.scale) / DEGF.scale + DEGF.offset
((0.0 - -273.15) * 1.0) / 0.555555555555556 + -459.67 == 32.0
```

Unit Code

The unit is uniquely identified by the code, for example 'DEGC'. This is almost always the identifier in log data.

Standard Form

The 'standard form' is the RP66V2 definition of the name of the units. This can have multiple codes depending on the origin of the data. For 'degC' this is:

Code	Name	Standard Form	Dimension	Scale	Offset
DEGC	'degree celsius'	degC	Temperature	1	-273.15
deg C	'degree celsius'	degC	Temperature	1	-273.15
oC	'GeoFrame legacy unit'	degC	Temperature	1	-273.15

Internal Representation of the Data

Each unit conversion is represented by a Unit class: `TotalDepth.common.units.Unit`. The entire conversion table is represented internally by a `Dict[str, Unit]` where the key is the Unit Code. At runtime the OSDD web page is read, parsed and the internal data structure cached. This happens the first time any of the APIs is accessed. On demand a data structure `typing.Dict[str, typing.List[str]]` mapping Standard Form to a list of Unit Codes is created and cached.

If the OSDD web page can not be read, perhaps because the user is offline, the code falls back to reading a static JSON file at `src/TotalDepth/common/data/osdd_units.json` that contains the last version of the OSDD. This file is refreshed every time the integration test `tests.integration.common.test_units.test_slb_units_write_to_json()` is run.

Units Conversion

The simplest form of conversion is:

```
from TotalDepth.common import units

unit_from = units.slb_units('DEGC')
unit_to = units.slb_units('DEGF')
result = units.convert(100.0, unit_from, unit_to) # Gives 212.0
```

To convert all the values in a numpy array inplace there is `units.convert_array(array: np.ndarray, unit_from: Unit, unit_to: Unit)`.

References:

Unit conversion: `TotalDepth.common.units`

7.14 Schlumberger Mnemonics

TotalDepth provides a Python API to the [Schlumberger's Oilfield Services Data Dictionary \(OSDD\)](#)

To quote the site:

This evergreen database delivers descriptions of more than 50,000 Schlumberger logging tools, analytical software, and log curves and parameters. It also provides definitions of physical property measurements and relevant units of measurement. Special tables enumerate mineral properties and depositional environments.

TotalDepth provides Python APIs to access this data online. The values are cached so that repeated calls do not refer to the website. Here are the `TotalDepth.common.lookup_mnemonic` APIs:

Table 1: **Lookup Mnemonic API**

Function	Argument	Return Value	Typical Size	Cache
<code>slb_data_channel()</code>	A data channel name from a logging tool.	Channel	128	
<code>slb_parameter()</code>	A well site parameter.	Parameter	256	
<code>slb_logging_tool()</code>	A logging tool name.	LoggingTool	64	

Examples:

```

>>> from TotalDepth.common import lookup_mnemonic

>>> lookup_mnemonic.slb_data_channel('RHOB')
Channel(
  channel='RHOB',
  description='Bulk Density',
  unit_quantity='Density',
  property='Bulk_Density',
  related_tools=(
    ToolDescription(tool='ADN', description='Bulk Density'),
    ToolDescription(tool='ADN4', description='Bulk Density'),
    ToolDescription(tool='ADN6C', description='Bulk Density'),
    ToolDescription(tool='ADN8', description='Bulk Density'),
    ToolDescription(tool='APS-C', description='Bulk Density'),
    ToolDescription(tool='DV6MT', description='Bulk Density'),
    ToolDescription(tool='ECO6', description='Bulk Density'),
    ToolDescription(tool='FGTC', description='Bulk Density'),
    ToolDescription(tool='HLDS-D', description='Bulk Density'),
    ToolDescription(tool='HLDT', description='Bulk Density'),
    ToolDescription(tool='HLDTA', description='Bulk Density'),
    ToolDescription(tool='ILDT-A', description='Bulk Density'),
    ToolDescription(tool='ILDT-B', description='Bulk Density'),
    ToolDescription(tool='LDS-C', description='Bulk Density'),
    ToolDescription(tool='LDT', description='Bulk Density'),
    ToolDescription(tool='LDTA', description='Bulk Density'),
    ToolDescription(tool='LDTTC', description='Bulk Density'),
    ToolDescription(tool='LDTD', description='Bulk Density'),
    ToolDescription(tool='PGT', description='Bulk Density'),
    ToolDescription(tool='PGTE', description='Bulk Density'),
    ToolDescription(tool='PGTK', description='Bulk Density'),
    ToolDescription(tool='SADN8', description='Bulk Density'),
    ToolDescription(tool='TBT-A', description='Bulk Density'),
    ToolDescription(tool='V475', description='Bulk Density')
  ),
  related_products=(
    ProductDescription(product='CMR_DMRP', description='Bulk Density'),
    ProductDescription(product='CMR_DMRRXO', description='Bulk Density'),
    ProductDescription(product='ELAN-PLUS', description='Bulk Density'),
    ProductDescription(product='GEOSHARE', description='Bulk Density'),
    ProductDescription(product='HISTEC', description='Bulk Density'),
    ProductDescription(product='IESX', description='Bulk Density'),
    ProductDescription(product='IMPACT', description='Bulk Density'),
    ProductDescription(product='INVASION_FACTOR', description='Bulk Density'),
    ProductDescription(product='PETROSONIC', description='Bulk Density'),
    ProductDescription(product='POR_TX', description='Bulk Density'),
    ProductDescription(product='PREPLUS2_EC', description='Bulk Density'),
    ProductDescription(product='ROCKSOLID', description='Bulk Density'),
    ProductDescription(product='RUNIT_SYNTHETICS', description='Bulk Density'),
    ProductDescription(product='RWA_CLAY_CORR', description='Bulk Density'),
    ProductDescription(product='RWAC', description='Bulk Density'),
    ProductDescription(product='SCQ', description='Bulk Density'),
    ProductDescription(product='SONFRAC', description='Bulk Density'),
    ProductDescription(product='STPERM', description='Bulk Density')
  )
)

```

(continues on next page)

(continued from previous page)

```

    )
)

>>> lookup_mnemonic.slb_parameter('LATI')
Parameter(
  code='LATI',
  description='Latitude',
  unit_quantity='Dimensionless',
  property='Latitude',
  related_products=(
    ProductDescription(product='CSUD_WSD', description='Latitude'),
    ProductDescription(product='MAXIS_WSD', description='Latitude')
  )
)

>>> lookup_mnemonic.slb_logging_tool('HDT')
LoggingTool(
  code='HDT',
  technology='Dipmeter',
  discipline='Geology',
  method='WIRELIN',
  description='High Resolution Dipmeter Tool',
  related_channels=(
    ChannelDescription(channel='AZIM', description='Measured Azimuth'),
    ChannelDescription(channel='AZIX', description='Azimuth of Reference Sensor (Pad
↪1, if available)'),
    ChannelDescription(channel='C1', description='Caliper 1'),
    ChannelDescription(channel='C2', description='Caliper 2'),
    ChannelDescription(channel='DEVI', description='Hole Deviation'),
    ChannelDescription(channel='EWDR', description='East West Drift Component'),
    ChannelDescription(channel='FC0', description='HDT Fast Channel 0 (Speed Button)
↪),
    ChannelDescription(channel='FC1', description='HDT Fast Channel 1'),
    ChannelDescription(channel='FC2', description='HDT Fast Channel 2'),
    ChannelDescription(channel='FC3', description='HDT Fast Channel 3'),
    ChannelDescription(channel='FC4', description='HDT Fast Channel 4'),
    ChannelDescription(channel='FEP', description='Far Electrode Potential'),
    ChannelDescription(channel='FEP1', description='Far Electrode Potential 1'),
    ChannelDescription(channel='FEP2', description='Far Electrode Potential 2'),
    ChannelDescription(channel='HAZI', description='Hole Azimuth Relative to True
↪North'),
    ChannelDescription(channel='NSDR', description='North South Drift'),
    ChannelDescription(channel='PlAZ', description='Pad 1 Azimuth in Horizontal
↪Plane (0 = True North)'),
    ChannelDescription(channel='PP', description='Pad Pressure'),
    ChannelDescription(channel='RAZI', description='Raw Azimuth'),
    ChannelDescription(channel='RB', description='Relative Bearing'),
    ChannelDescription(channel='RC', description='Reference Check'),
    ChannelDescription(channel='RC1', description='Raw C1'),
    ChannelDescription(channel='RC2', description='Raw C2'),
    ChannelDescription(channel='RDEV', description='Raw Deviation'),
    ChannelDescription(channel='REFE', description='Reference'),

```

(continues on next page)

(continued from previous page)

```

ChannelDescription(channel='RHDT', description='Raw HDT Data Block'),
ChannelDescription(channel='RRB', description='Raw Relative Bearing'),
ChannelDescription(channel='SDEV', description='Sonde Deviation'),
ChannelDescription(channel='TEMP', description='Computed Borehole Temperature'),
ChannelDescription(channel='ZB', description='Zero Button')
),
related_parameters=(
    ParameterDescription(parameter='AMOD', description='Averaging Mode Selection'),
    ParameterDescription(parameter='AZIM', description='Well Section Azimuth'),
    ParameterDescription(parameter='DANG', description='Dip Angle of the Bedding'),
    ParameterDescription(parameter='DAZI', description='Dip Azimuth of the Bedding'),
    ParameterDescription(parameter='DEVI', description='Well Section Deviation'),
    ParameterDescription(parameter='DISO', description='DIP Information Source'),
    ParameterDescription(parameter='HDTT', description='HDT Sonde Type'),
    ParameterDescription(parameter='LATD', description='Latitude (N=+ S=-)'),
    ParameterDescription(parameter='LOND', description='Longitude'),
    ParameterDescription(parameter='MCT', description='Mechanical Cartridge Type'),
    ParameterDescription(parameter='MDEC', description='Magnetic Field Declination'),
    ParameterDescription(parameter='MFIN', description='Magnetic Field Intensity'),
    ParameterDescription(parameter='MINC', description='Magnetic Field Inclination'),
    ParameterDescription(parameter='MTD', description='Measured Tie Depth'),
    ParameterDescription(parameter='RANO', description='Resistivity Anomaly Selection
→ '),
    ParameterDescription(parameter='SFAN', description='Scale Factor of Anomalies'),
    ParameterDescription(parameter='STYP', description='Sonde Type')
)
)

```

References:

Unit conversion: `TotalDepth.common.lookup_mnemonic`

TODO'S

TotalDepth is work-in-progress. This is a gathering place for features that would be nice to have in future releases. Priority numbers are 0 (not going to be done) and >0 which is an ever more important priority. Work is the estimated amount of work from 1 upwards.

If you find a bug or need a feature then raise an [issue with TotalDepth](#).

TotalDepth is currently at **Beta**, development version 0.4.0, release version 0.4.0.

8.1 TotalDepth Improvements (General)

Table 1: **TotalDepth Improvements (General)**

ID	Description	Priority	Work	Status
TD.Cython	Remove Cython as a dependency. Merge C++ replacement code.	1	1	
TD.SQL	Extract data to database.	1	3	
TD.GEO	Extract Latitude and Longitude as trustworthy metadata.	1	3	
TD.svfs	Merge Sparse Virtual File System C/C++ code.	1	4	
TD.test.slow	Move slow LIS test to tests/integration/ or mark as @slow.	1	1	DONE: v0.3.0
TD.test.benchmark	Move benchmarking tests to benchmark/.	1	2	

8.2 LAS Improvements

Table 2: LAS Improvements

ID	Description	Priority	Work	Status
LAS.fast	Merge the fast array parser with ~50x performance.	2	3	
LAS.zip	Read directly from .zip files.	1	1	
LAS.v3	Support version 3.0. However this is barely used by the industry.	0	N/A	
LAS.merge_O_P	Merge ~0 section into ~P if correct format.	1	1	
LAS.consist	Consistency checking of mutual data such as STRT/STOP/STEP.	1	1	

8.3 LIS Improvements

Table 3: LIS Improvements

ID	Description	Priority	Work	Status
LIS.HDT	Expand/contract 'sub-channels' to actual channels and use the universal Frame Array.	2	3	
LIS.XNAM	Full XNAM direct support for LIS-A.	0	3	No. These only occur in a small number from a minority producer using specialised software. They do not occur in mainstream LIS files.
LIS.index_c	Merge fast indexer in C for 100x performance.	2	3	
LIS.index_inline	Insert or append binary representation of the index to the LIS file.	1	3	
LIS.rc_over	Check Rep Code overflow/underflow on write.	1	1	
LIS.test.slow	Move slow LIS test to tests/integration/ or mark as @slow.	1	1	DONE: v0.3.0

8.4 RP66V1 Improvements

Table 4: **RP66V1 Improvements**

ID	Description	Priority	Work	Status
RP66V1.test.benchmark	Write benchmarking tests in benchmark/.	2	2	
RP66V1.index_0	Add the multi-level index code implemented in C/C++ that is much faster and smaller. See 2019-11-12.	2	3	
RP66V1.index_1	Mid level index implemented in C/C++.	1	3	
RP66V1.RepCode	Add in the code that does Representation Code conversion in C/C++.	1	3	
RP66V1.numpy.read	Directly populate the Frame Array in Numpy from C/C++. See also Frame.buffer.	1	3	
RP66V1.units	Conformance of unit conversion with the RP66V1 and, possibly, RP66V2 standard. NOTE: The RP66V2 standard is expanded on RP66V1 but barely used. Many producers deviate from these standards in any case.	1	2	
RP66V1.plot	Plot RP66V1 files. See Plot.spec.	1	3	
RP66V1.fail	When a file deviates from the standard then the user can specify what deviations are acceptable. Examples: UNITS Rep Code, multiple ORIGIN and CHANNEL records.	1	2	

8.5 Plotting Improvements

Table 5: Plotting Improvements

ID	Description	Priority	Work	Status
Plot.spec	There is quite a lot of technical debt built up since we added LgFormat support, this area needs a review. Implement the XML design.	2	4	
Plot.head	Header: Some mud parameters being dropped.	1	2	
Plot.perf	Benchmarks to characterise execution time and profiling.	1	3	
Plot.cXML	Integrate the existing XML writer written in C for x4 speedup.	1	3	
Plot.hover	Display values when hovering over curves in SVG.	1	3	
Plot.PDF	PDF output of plots. Probably use reportlab.	1	4	

8.6 File Formats

Table 6: File Format Support

ID	Description	Priority	Work	Status
Format.RP66V2	Unused by the industry.	0	N/A	
Format.WellLogML	Unused by the industry.	0	N/A	
Format.ATLAS_BIT	Legacy format.	1	1	DONE v0.4.0
Format.SEGY	Other FOSS projects specialise in this.	0	N/A	
Format.SEGD	Used at all?	0	N/A	
Format.DAT	An informal format used for mud logs.	1	1	DONE v0.4.0

8.7 Frame Array Improvements

Table 7: Frame Array Improvements

ID	Description	Priority	Work	Status
Frame.common	Common Frame Array code for all file formats.	1	3	
Frame.buffer	Implement frame processing in C++ using the buffer protocol. Also shared memory with multi-processing.	1	3	

CONTRIBUTING

Contributions are welcome, and they are greatly appreciated! Every little bit helps, and credit will always be given. You can contribute in many ways:

9.1 Types of Contributions

9.1.1 Report Bugs

Report bugs at <https://github.com/paulross/TotalDepth/issues>.

If you are reporting a bug, please include:

- Your operating system name and version.
- Any details about your local setup that might be helpful in troubleshooting.
- Detailed steps to reproduce the bug.

9.1.2 Fix Bugs

Look through the GitHub issues for bugs. Anything tagged with “bug” and “help wanted” is open to whoever wants to implement it.

9.1.3 Implement Features

Look through the GitHub issues for features. Anything tagged with “enhancement” and “help wanted” is open to whoever wants to implement it.

9.1.4 Write Documentation

TotalDepth could always use more documentation, whether as part of the official TotalDepth docs, in docstrings, or even on the web in blog posts, articles, and such.

9.1.5 Submit Feedback

The best way to send feedback is to file an issue at <https://github.com/paulross/TotalDepth/issues>.

If you are proposing a feature:

- Explain in detail how it would work.
- Keep the scope as narrow as possible, to make it easier to implement.
- Remember that this is a volunteer-driven project, and that contributions are welcome :)

9.2 Get Started!

Ready to contribute? Here's how to set up *TotalDepth* for local development.

1. Fork the *TotalDepth* repo on GitHub.
2. Clone your fork locally:

```
$ git clone git@github.com:paulross/TotalDepth.git
```

3. Install your local copy into a virtualenv. Assuming you have virtualenvwrapper installed, this is how you set up your fork for local development:

```
$ mkvirtualenv TotalDepth
$ cd TotalDepth/
$ python setup.py develop
```

4. Create a branch for local development:

```
$ git checkout -b name-of-your-bugfix-or-feature
```

Now you can make your changes locally.

5. When you're done making changes, check that your changes pass flake8 and the tests, including testing other Python versions with tox:

```
$ flake8 TotalDepth tests
$ python setup.py test or py.test
$ tox
```

To get flake8 and tox, just pip install them into your virtualenv.

6. Commit your changes and push your branch to GitHub:

```
$ git add .
$ git commit -m "Your detailed description of your changes."
$ git push origin name-of-your-bugfix-or-feature
```

7. Submit a pull request through the GitHub website.

9.2.1 Pull Request Guidelines

Before you submit a pull request, check that it meets these guidelines:

1. The pull request should include tests.
2. If the pull request adds functionality, the docs should be updated. Put your new functionality into a function with a docstring, and add the feature to the list in README.rst.
3. The pull request should work for Python 3.3 and above (we don't support Python 2.x), and for PyPy. Check https://travis-ci.org/paulross/TotalDepth/pull_requests and make sure that the tests pass for all supported Python versions.

9.2.2 Tips

To run a subset of tests:

```
$ pytest tests.test_TotalDepth
```

9.3 Release Checklist

In the following example the version we are moving to, in `Major.Minor.Patch` format, is `0.2.1`.

Current version should be something like `M.m.(p)rcX`, for example `0.2.1rc0`.

9.3.1 Increment version

Change the version to `M.m.p` in these places:

- `setup.cfg`
- `setup.py`
- `src/TotalDepth/__init__.py`
- `/docs/source/conf.py` (two places).

In `src/TotalDepth/__init__.py` change `VERSION = (0, 2, 1)`

Update the history:

- `HISTORY.rst`
- `src/TotalDepth/__init__.py`

Update any Trove classifiers in `setup.py`, https://pypi.python.org/pypi?%3Aaction=list_classifiers

9.3.2 Build and Test

Build

Build, in this case for Python 3.6:

```
$ python3.6 -m venv ~/venvs/TotalDepth_00/  
$ . ~/venvs/TotalDepth_00/bin/activate  
(TotalDepth_00) $ pip install --upgrade pip  
(TotalDepth_00) $ pip install --user --upgrade setuptools wheel  
(TotalDepth_00) $ pip install -r requirements.txt  
(TotalDepth_00) $ python setup.py develop
```

Test

As a minimal test:

```
(TotalDepth_00) $ pytest tests/
```

This should take under a minute.

A full test:

```
(TotalDepth_00) $ pytest --runslow tests/
```

This can take 10 to 30 minutes.

9.3.3 Build a Development Environment

This includes the full testing (including test coverage) and documentation environment.

As above plus:

```
(TotalDepth_00) $ pip install -r requirements-dev.txt
```

Test Coverage

With test coverage:

```
(TotalDepth_00) $ pytest --cov=TotalDepth --cov-report html tests --runslow
```

This can take 10 to 30 minutes.

Documentation

Build the docs HTML to test them, from an environment that has Sphinx:

```
(Sphinx) $ cd docs
(Sphinx) $ make clean
(Sphinx) $ make html
```

9.3.4 Commit and Tag

Commit, tag and push:

```
$ git add .
$ git commit -m 'Release version 0.2.1'
$ git tag -a v0.2.1 -m 'Version 0.2.1'
$ git push
$ git push origin v0.2.1
```

9.3.5 PyPi

Prepare release to PyPi for Python 3.6:

Build the egg and the source distribution:

```
(TotalDepth_00) $ python setup.py install sdist
```

Check the contents of `dist/*`, unpack into `tmp/` if you want:

```
$ cp dist/* tmp/
$ cd tmp/
$ unzip TotalDepth-0.2.1-py3.6-macosx-10.6-intel.egg -d py27egg
$ tar -xzf TotalDepth-0.2.1.tar.gz
```

Release to PyPi, <https://pypi.python.org/pypi/TotalDepth>:

```
(TotalDepth_00) $ twine upload dist/*
```

9.3.6 ReadTheDocs

Build the documentation: <https://readthedocs.org/projects/TotalDepth/builds/>

9.3.7 Prepare Next Release Candidate

Finally change the version to `M.m. (p+1)rc0`, in this example `0.2.2rc0` in these places:

- `setup.cfg`
- `setup.py`
- `src/TotalDepth/__init__.py`
- `/docs/source/conf.py` (two places).

In *src/TotalDepth/__init__.py* change `VERSION = (0, 2, 2, 'rc0')`

Commit and push:

```
$ git add .  
$ git commit -m 'Release candidate v0.2.2rc0'  
$ git push
```

TESTING TOTALDEPTH

Contents:

10.1 Unit Tests

This describes how unit testing is done in TotalDepth.

TotalDepth uses the following test frameworks:

- [pytest](#) - This is a basic minimum.
- [pytest-cov](#) - Pytest's wrapper around Ned Batchelor's *excellent* coverage tool.
- [pytest-benchmark](#) - For micro benchmarks.
- [Airspeed Velocity](#) - For timeline benchmarks.

10.1.1 Basic Testing

Simply:

```
$ pytest tests/
```

This should take only a few tens of seconds. If you include the slow tests with `--runslow` this will take many minutes.

10.1.2 Testing With Test Coverage

For complete coverage you need to run the slow tests and with the following command:

```
$ pytest --cov=TotalDepth --cov-report html --runslow tests
```

10.2 Testing the Plot Package

This is a comprehensive testing of your installation to see if LIS/LAS files can be written, read and plotted. The code writes specific LIS/LAS files in memory, reads them back with the appropriate parser then plots them as SVG files.

10.2.1 Running the test

TestPlot is a unit test which also has performance tests within it. It is invoked thus:

```
$ python tests/unit/test_util/test_plot/TestPlot.py
```

The response should be something like:

```
TestClass.py script version "0.1.0", dated 2010-08-02
Author: Paul Ross
Copyright (c) Paul Ross

test_00 (__main__.TestPlotRollStatic)
TestPlotRollStatic.test_00(): Tests setUp() and tearDown(). ... ok
test_01 (__main__.TestPlotRollStatic)
TestPlotRollStatic.test_01(): viewBox. ... ok

... 8<----- snip, snip, snip ----->8

test_01 (__main__.TestPlotReadLIS_SingleSinCurve) ... ExecTimerList [6]:
Loading FrameSet          Size:    0.002 (MB) Time:    0.010 (s) Cost:   3838.393 (ms/
↳MB)
Initialising LIS plot     Size:    0.000 (MB) Time:    0.001 (s) Cost:         N/A (ms/
↳MB)
Plotting Tracks           Size:    0.000 (MB) Time:    0.005 (s) Cost:         N/A (ms/
↳MB)
Plotting XGrid            Size:    0.000 (MB) Time:    0.023 (s) Cost:         N/A (ms/
↳MB)
Plotting scales (legends) Size:    0.000 (MB) Time:    0.013 (s) Cost:         N/A (ms/
↳MB)
Plotting curves           Size:    0.002 (MB) Time:    0.141 (s) Cost:   9338.957 (ms/
↳MB) ok
test_00 (__main__.TestPlotReadLIS_SingleSquareCurveLowFreq)
TestPlotReadLIS_SingleSquareCurve.test_00(): Tests setUp() and tearDown(). ... ok

... 8<----- snip, snip ----->8

test_12 (__main__.TestPlotReadLAS_XML_LgFormat)
TestPlotReadLAS_XML_LgFormat.test_12(): Plot from XML LgFormat files - density, porosity,
↳and multiple gamma ray curves. ... ExecTimerList [6]:
Initialising LAS plot: "Porosity_GR_3Track" Size:    0.000 (MB) Time:    0.001 (s)
↳Cost:         N/A (ms/MB)
Plotting API Header              Size:    0.000 (MB) Time:    0.016 (s)
↳Cost:         N/A (ms/MB)
Plotting Tracks                  Size:    0.000 (MB) Time:    0.005 (s)
↳Cost:         N/A (ms/MB)
Plotting XGrid                  Size:    0.000 (MB) Time:    0.055 (s)
↳Cost:         N/A (ms/MB)
```

(continues on next page)

(continued from previous page)

Plotting scales (legends)	Size:	0.000 (MB)	Time:	0.013 (s)
↪Cost: N/A (ms/MB)				
Plotting curves	Size:	0.020 (MB)	Time:	0.096 (s)
↪Cost: 5009.433 (ms/MB) ok				

Ran 107 tests in 34.318s				
OK				
CPU time = 34.179 (S)				
Bye, bye!				

The important thing is that there should be no reported failures.

10.2.2 Results

You should find in the directory `<TOTALDEPTH>/tests/unit/test_util/test_plot/test_svg` a set of test plots.

Index

This directory has an `<TOTALDEPTH>/tests/unit/test_util/test_plot/test_svg/index.html` that looks like this:

file://localhost/Users/paulross/Documents/workspace/TotalDepth/src/TotalDepth/util/plot/test_s...

API Headers

1. [link](#) TestLogHeaderLIS.test_05(): Empty API header from LIS data (upright).
2. [link](#) TestLogHeaderLIS.test_06(): Empty API header from LIS data (rotated).
3. [link](#) TestLogHeaderLIS.test_10(): API header with CONS information from LIS data (upright).
4. [link](#) TestLogHeaderLIS.test_11(): API header with CONS information from LIS data (rotated).
5. [link](#) TestLogHeaderLAS.test_05(): Empty API header from LAS data (upright).
6. [link](#) TestLogHeaderLAS.test_06(): Empty API header from LAS data (rotated).
7. [link](#) TestLogHeaderLAS.test_10(): API header with CONS information from LAS data (upright).
8. [link](#) TestLogHeaderLAS.test_11(): API header with CONS information from LAS data (rotated).

Plots from LIS files

1. [link](#) TestPlotReadLIS_SingleSinCurve.test_01(): Sinusoidal SP plotted on a number of different scales.
2. [link](#) TestPlotReadLIS_SingleSquareCurveLowFreq.test_01(): Square wave with 4' spacing to check wrap interpolation.
3. [link](#) TestPlotReadLIS_SingleSquareCurveHighFreq.test_01(): Square wave with 0.5' spacing to check wrap interpolation.
4. [link](#) TestPlotReadLIS_HDT.test_01(): 200 feet of HDT.
5. [link](#) TestPlotReadLIS_SuperSampled.test_01(): Channels at 4' frame spacing, single, x8, x32 super-sampling.
6. [link](#) TestPlotReadLIS_COLO_Named.test_01(): Sinusoidal SP plotted on a number of different named colours.
7. [link](#) TestPlotReadLIS_COLO_Numbered.test_01(): Numbered colours 400 (red), 040 (green), 004 (blue).
8. [link](#) TestPlotReadLIS_COLO_Numbered_Comp.test_01(): Numbered colours 440 (yellow), 404 (magenta), 044 (cyan).
9. [link](#) TestPlotReadLIS_Perf_00.test_01(): Film 1 2000' of 10 curves, linear scale.
10. [link](#) TestPlotReadLIS_Perf_00.test_02(): Film 2 2000' of 10 curves, linear and log scale.
11. [link](#) LgFormat: "Triple_Combo_00" 2000 of 10 curves, linear and log scale.
12. [link](#) LgFormat: "Resistivity_3Track_Logrithmic.xml_00" 2000' of 10 curves, linear and log scale.
13. [link](#) TestPlotReadLIS_SingleSinCurve.test_01(): Sinusoidal SP plotted on a number of different scales with API header.

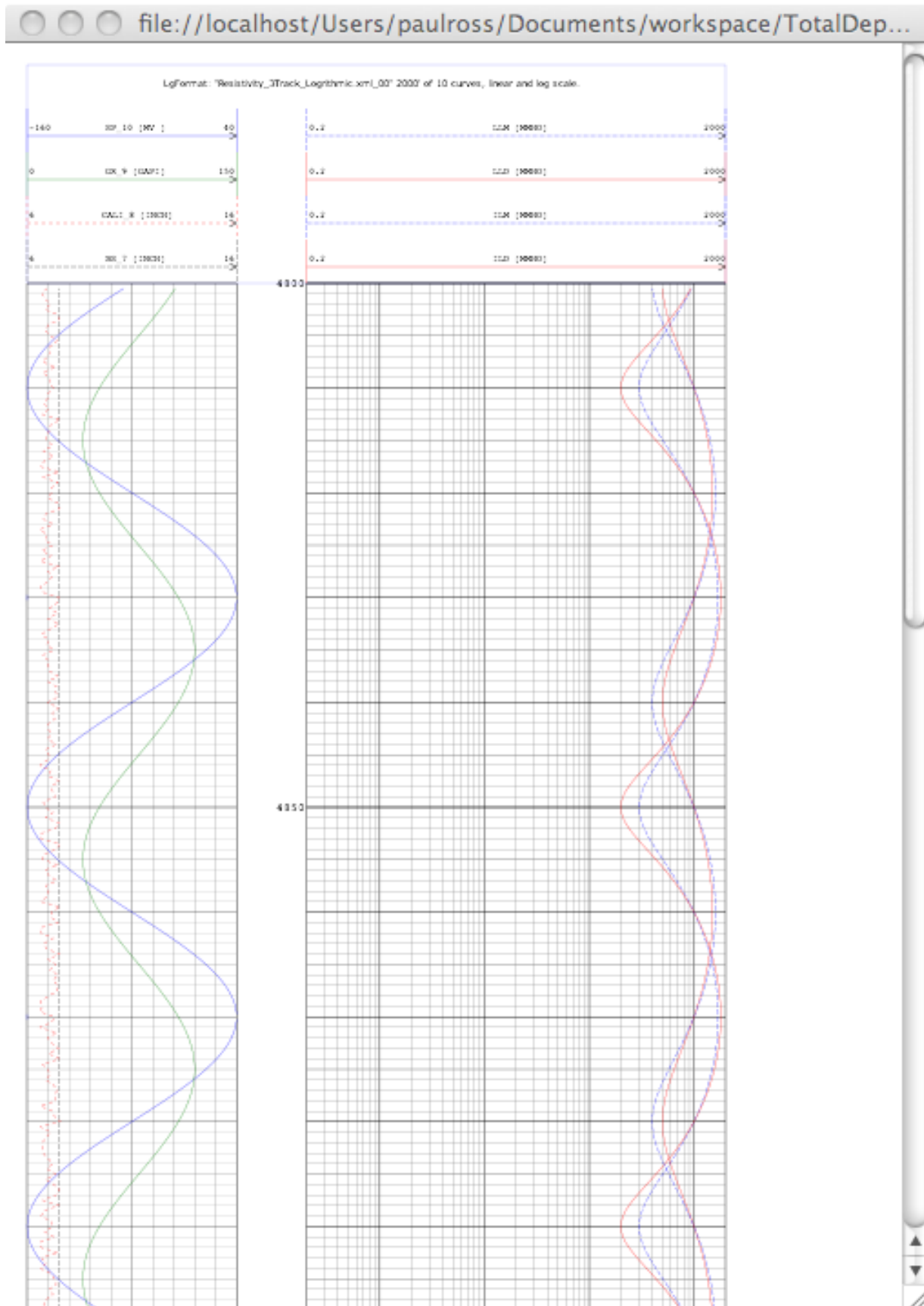
Plots from LAS files

1. [link](#) LgFormat: "Triple_Combo" 200 of 15 curves, linear and log scale from LAS file, DOWN log, no header.
2. [link](#) LgFormat: "Resistivity_3Track_Logrithmic.xml" 200' of 15 curves, linear and log scale from LAS file, DOWN log, no header.
3. [link](#) LgFormat: "Triple_Combo" 200 of 10 curves, linear and log scale from LAS file, DOWN log, with header.
4. [link](#) LgFormat: "Resistivity_3Track_Logrithmic.xml" 200' of 15 curves, linear and log scale from LAS file, DOWN log, with header.
5. [link](#) LgFormat: "Triple_Combo" 200 of 15 curves, linear and log scale from LAS file, UP log, no header.
6. [link](#) LgFormat: "Resistivity_3Track_Logrithmic.xml" 200' of 15 curves, linear and log scale from LAS file, UP log, no header.
7. [link](#) LgFormat: "Triple_Combo" 200 of 15 curves, linear and log scale from LAS file, UP log, with header.
8. [link](#) LgFormat: "Resistivity_3Track_Logrithmic.xml" 200' of 15 curves, linear and log scale from LAS file, UP log, with header.
9. [link](#) LgFormat: "Triple_Combo" 1000 of 15 curves, linear and log scale from LAS file, large down log, with header.
10. [link](#) LgFormat: "Resistivity_3Track_Logrithmic.xml" 1000' of 15 curves, linear and log scale from LAS file, large down log, with header.
11. [link](#) LgFormat: "Triple_Combo" 100 feet of 5 gamma ray curves.
12. [link](#) LgFormat: "Resistivity_3Track_Logrithmic.xml" 100 feet of 5 gamma ray curves.
13. [link](#) LgFormat: "Porosity_GR_3Track" 100 feet of Density, porosity and 5 gamma ray curves.

///

Example Plot

Navigate to a typical test LIS plot from that index such as:



Cool. If you see that then your TotalDepth installation is pretty good!

10.3 Testing TotalDepth With Recorded Data

TotalDepth is tested against a diverse data set of real world files. The test set is split into small/medium/large datasets.

10.3.1 Small Test Set

todo Complete this

10.3.2 Medium Test Set

The Medium Size Test Set is 20,000+ files (130Gb+) of typical oilfield data. Here is the approximate breakdown of the test set:

File Type	Files	Total Size	Notes
BIT	~500	~1.5Gb	Largest file is around 6Mb.
LAS v1.2	~500	~1Gb	Largest file is around 16Mb.
LAS v2.0	~20,000	~30Gb	Largest file is around 250Mb (RP66V1 converted files are much larger).
LAS v3.0	A few	~0	Rarely present, their absence is not considered significant.
LIS	~2000	~2GB	Largest file is around 60Mb. Around half have TIF markers.
DLIS (RP66V1)	~800	~100GB	Largest file is around 4GB. About one quarter are corrupted by TIF markers.
DLIS (RP66V2)	0	0	Not present, their absence is not considered significant.
Other	Various	Various	Various files such as PDF, TIFF, miscellaneous binary files and unstructured ASCII. If present then their contents is not considered significant but archives containing these should be processed without drama.

The layout of the test set is typical of an oilfield repository, typically by well, with a well having an unspecified directory structure and a mix of file types in each directory.

The aim is that TotalDepth can process 97+^o% of the files in this archive in the formats that TotalDepth supports.

10.3.3 Large Test Set

todo Complete this

10.3.4 Synthetic Test Set

TotalDepth can generate arbitrary sized files in a number of formats and this test set is used for performance analysis. This test set is not distributed with TotalDepth but the means to create it is.

HISTORY

11.1 0.4.0 (2021-09-18)

- **General**
 - Add SLB parameter and unit online lookup.
 - Add detection of CFBF, EBCDIC, RCD, SEG-Y, STK, PDS binary file types.
 - Python 3.6 no longer supported, although it will most likely work.
- **Specific File formats**
 - **BIT**
 - * Support Western Atlas BIT files.
 - * Add BIT file conversion to LAS.
 - * BIT float to bytes (ISINGL) encoding.
 - **DAT**
 - * Add DAT file support using the common FrameArray.
 - **LAS**
 - * Add tdlstohtml as an entry point.
 - * Parser improvements.
 - * LAS reader now ignores duplicate channels if requested.
 - * Add LAS variants to binary_file_type.
 - * LAS FrameArray writing now in TotalDepth.common.LogPass
 - **LIS**
 - * Add LIS to LAS conversion.
 - * Kill off XNAM LIS support.
 - * Better handling of LIS Physical Record padding.
 - * Fix for LIS indexer when the DFSR is missing.
 - * Adds generation of AREA patterns in SVG.
 - * Creates PNG pattern files from XML data. Provides an API to pattern files and Data URI Scheme inline implementations.
 - **RP66V1**

- * Prepare for RP66V1 C/C++ code. Update to Python 3.8, 3.9, 3.10.
- * Add units conversion.

11.2 0.3.1 (2020-06-15)

- Fixes for builds on Linux and Windows.

11.3 0.3.0 (2020-01-01)

- Adds full RP66V1 support.
- Tested against multi GB data set.

11.4 0.2.1 (2018-04-21)

- Minor fixes.

11.5 0.2.0 (2017-09-25)

- Moved to Github: <https://github.com/paulross/TotalDepth>
- First release on PyPI.

11.6 0.1.0 (2012-03-03)

- First release on Sourceforge: <https://sourceforge.net/projects/TotalDepth/> registered: 2011-10-02

Earlier versions (unreleased):

- OpenLis - 2010-11-11 to 2011-08-01
- PyLis - 2009 to 2010

For many years this project was hosted by [Sourceforge](https://sourceforge.net/). Thank you Sourceforge!

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

GLOSSARY

Note: Terms here starting with “RP66V1.” are taken directly from the [RP66V1 glossary](#). They are lightly edited, corrected and enhanced before being reproduced here for the convenience of the reader.

Absent Value A recorded value from a sensor that is not a true recorded value and should be ignored. A point of difficulty arises for a conformant application; within the [LIS-79](#) format this is specified in the [DFSR](#) so can be per frame array. For [RP66V1](#) the absent value is supposed to be represented by having an attribute count of zero. In practice this never happens, instead a bunch of ad-hoc values need to be presumed such as -999, -999.25 based on their [RepCode](#). See also [RP66V1.Absent Value](#)

Backup Mode A means of specifying what happens to plotted lines when they go off scale. Typical examples are None (all intermediate data is lost) and ‘wrap’ (all data is plotted with lines at modulo scale).

Data Format Specification Record [[LIS](#)] An EFLR that describes type 0 or type 1 binary IFLRs containing log data. A DFSR consists of a set of Entry Blocks followed by a list of Datum Specification Blocks.

NOTE: The LIS specification associates a particular DFSR type 0 | 1 with binary IFLRs of type 0 | 1. These collections will be independent of each other and thus permits the simultaneous recording of entirely different data sets. In practice there is no evidence that any LIS implementations supports this.

Datum Specification Block [[LIS](#)] A fixed format data block that defines the characteristics of a single, independent, data channel in a DFSR.

DFSR See [Data Format Specification Record](#)

DLIS This adds schema specific semantics to [RP66V1](#) or [RP66V2](#).

DSB See [Datum Specification Block](#).

EFLR See [Explicitly Formatted Logical Record](#).

Engineering Value A numeric value together with its units of measure.

Entry Block [[LIS](#)] A variable length block of data that describes a static value in DFSR. This value is local to a Log Pass. For example and Entry Block might describe the NULL or absent value for any channel in a Log Pass.

Explicitly Formatted Logical Record This is an internally complete, self-describing record. See also [RP66V1.Explicitly Formatted Logical Record](#)

FFLR See [Fixed Format Logical Record](#).

Fixed Format Logical Record This is an internally complete record whose format is described by a standard. Does not occur in [RP66V1](#).

Frame An array of values for each channel at a particular depth (or time).

Frame Set

Frame Array A set of frames representing multi-channel data that is typically depth or time series based.

IFLR

Indirectly Formatted Logical Record This is a Logical Record whose format is described by another EFLR. The EFLR that describes an IFLR might be identified formally; for example by a specific reference to an EFLR (as in RP66) or informally; by some heuristic (as in LIS) such as “the immediately prior Logical Record that is type 64 i.e. a *Data Format Specification Record*. See also *RP66V1.Indirectly Formatted Logical Record*

LAS Text based flog file format managed by the [Canadian Well Logging Society](#)

LIS

LIS-79 Log Information Standard. The original file format used by Schlumberger from 1979 onwards. It is a *proprietary, de-facto* standard.

LIS-A *Enhanced* Log Information Standard. This adds somewhat to the [\[LIS-79\]](#) standard.

Log Pass A TotalDepth term that describes a continuous body of logging data recorded simultaneously and independent of any other recording. Examples might be “Repeat Section” or “Main Log”. A Log Pass contains one or more *Frame Array* (s). The number of allowable Frame Arrays within a Log Pass depends on the log format:

LAS can only support a single Frame Array within a Log Pass.

In the *LIS* format the Log Pass is defined by a single *DFSR* Logical Record. This can describe up to two Log Passes (type 0 or type 1) Logical Records. In practice only type 0 exists so LIS has just one Frame Array per Log Pass.

RP66V1 supports any number of Frame Arrays within a Log Pass and usually does.

Logical Record [\[LIS\]](#) A formal record from a LIS file. Logical Records consist of a header and optional payload. The Logical Records *type* is identified in the header. The interpretation of the payload of (some) Logical Records types is defined in the LIS standard. Logical Records consist of the payloads of one or more Physical Records. Logical Records are either EFLR or IFLR records.

LRH Logical Record header. The bytes that describe the type and attributes of a Logical Record.

Mnem See *Mnemonic*.

Mnemonic [\[LIS\]](#) A four byte identifier that is human readable e.g. RHOB for Bulk Density.

Physical Record [\[LIS\]](#) A formal record in a LIS file. Physical Records consist of a header, optional payload and optional trailer. Logical Records consist of the payloads of one or more Physical Records.

RepCode

Rep Code See *Representation Code*.

Representation Code A code, usually an integer, that describes how a particular run of bytes should be interpreted as a value (number, string etc.). For example in the LIS standard representation code 68 describes a 32 bit floating point format.

RP66 *Recommended Practice 66* is an API standard that describes a more advanced file format for, among other things, wireline logs. Comes in two flavours version 1 and version 2. Often (and incorrectly) referred to as *DLIS*.

RP66V1 *Recommended Practice 66* version 1 is an API standard first published on MAY 1, 1991 that describes a (mostly) more advanced file format compared with *LIS*. The specification can be found online from the Petrotechnical Open Software Corporation here [RP66V1 standard](#) See also the [RP66V1 glossary](#)

RP66V1.Absent Value The Value of an Attribute is an Absent Value and is undefined when the Attribute Count is zero. A Channel Sample Value is an Absent Value and is undefined when its Dimension Count is zero. Under DLIS, Absent Values are explicitly absent and are not represented by specially-designated numeric quantities.

Warning: Unfortunately RP66V1 does not allow this to be set *per frame* (i.e. per [RP66V1.IFLR](#)) but only *per channel*. So data providers use a fixed and undeclared value such as -999 for integers and -999.25 for floats. This allows per-frame *and* per-channel absent values which means that RP66V1 files from all providers should be treated with some care in this area.

RP66V1.Attribute One of possibly many specific named items of information or data associated with an Object. An Attribute is similar in function to a column value of a row in a table or to a field of a record in a database relation. Its information content can be more general, however.

RP66V1.Channel A measured or computed quantity that occurs as a sequence of samples indexed against time, depth, or some other physical dimension of a well. Also a Set Type.

RP66V1.Characteristic A descriptive feature of a Set, an Object, or an Attribute. The Characteristics of a Set are its Name and Type, of an Object its Name, and of an Attribute its Label, Count, Representation Code, Units, and Value.

RP66V1.Company Code A numeric code assigned to a company that writes information under the DLIS format. Each company (or major division) is assigned a unique Company Code by the API.

RP66V1.Component The basic syntactic unit of an Explicitly Formatted Logical Record (EFLR). A Component consists of a Component Descriptor (one byte), followed by zero or more fields that contain Characteristics associated with a Set, Object, or Attribute.

RP66V1.Component Descriptor The first byte of a Component. It has a Role field (bits 1-3), which specifies whether the Component describes a Set, Object, or Attribute, and a Format field (bits 4-8), which indicate which Characteristics of the thing described are present in the remainder of the Component.

RP66V1.Component Format See [RP66V1.Component Descriptor](#).

RP66V1.Component Role See [RP66V1.Component Descriptor](#).

RP66V1.Compound Representation Code A Representation Code that is defined in terms of other simpler Representation Codes.

RP66V1.Consumer The system or application program or company that reads information recorded under the DLIS Logical format. The Consumer reads what the Producer writes.

RP66V1.Copy Number A number, having meaning only within the context of a Logical File, that is used to distinguish two Objects of the same Type that have the same Identifier and Origin. The Name of an Object consists

RP66V1.Count One of the five Characteristics of an Attribute. The Count specifies how many Elements are in the Value of the Attribute. When the Count is zero, the Attribute has an Absent Value.

RP66V1.Data Descriptor Reference The first field of an Indirectly Formatted Logical Record (IFLR). The Data Descriptor Reference is the Name of an Object that identifies and describes a sequence of IFLRs. Each IFLR Type is associated with a specific Set Type to which such Objects belong.

RP66V1.Defining Origin The first Origin Object in a Logical File. The Defining Origin describes the environment under which the Logical File was created.

RP66V1.Descriptor See [RP66V1.Component Descriptor](#).

RP66V1.Dictionary A Dictionary is a database in which Identifiers used under DLIS are administered. The standard does not specify the mechanisms for designing, creating, or managing Dictionaries. However, it does specify for which Set Types Identifiers should be managed. The statement, "Names of Set Type X are dictionary-controlled" means that Identifiers for such Objects have a persistent meaning in all Logical Files in which they occur (by a given Producer). Identifiers of Objects for Set Types that are not dictionary-controlled are considered void of meaning and are expected to be computer-generated.

RP66V1.Dimension This is a vector of integers which describe the form and size of a rectangular array that is represented elsewhere, for example as a Channel Sample. The first integer specifies the number of remaining integers

and the dimensionality of the array (i.e., 1-d, 2-d, etc.). The remaining integers specify the number of elements along each dimension (or coordinate) of the array. The Dimension of an array is typically contained in the Dimension Attribute of some Object that is associated with the array.

RP66V1.EFLR See *RP66V1.Explicitly Formatted Logical Record*

RP66V1.Element One of a vector of homogeneous quantities that make up the Value of an Attribute or of a Channel Sample. A Value or Sample may consist of one or more Elements. All Elements have the same Representation Code and Units. The number of Elements of an Attribute Value is specified by the Attribute Count. The number of Elements of a Channel Sample is specified by its associated Dimension.

RP66V1.Encryption Packet An optional sequence of bytes that follows the Logical Record Segment Header and precedes the Logical Record Segment Body and that contains information used to decrypt the Logical Record. The first two bytes of the Encryption Packet specify its length, and the next two bytes specify the Producer's Company Code. The remaining bytes are meaningful only to the Producer.

RP66V1.Explicitly Formatted Logical Record One of two kinds of Logical Record defined under DLIS. The Body of an EFLR is a sequence of Components that combine to describe a single Set of Objects. An EFLR is self-describing and can be interpreted without the use or knowledge of any other Logical Records. More simply put, an Explicitly Formatted Logical Record is a table of rows and columns. Each row/column contains a *RP66V1.Attribute*.

RP66V1.Format Version A two-byte field immediately following the Visible Record Length in Visible Records on Record Storage Units (e.g., standard 9-track tapes). This field is used to distinguish DLIS from other formats and to distinguish DLIS Version 1 from later major versions. The first byte of the Format Version contains the value FF (hex), and the second byte is the major version number of the standard (in the current case, 1).

RP66V1.Frame The Indirectly Formatted Data of an IFLR of Type FDATA (see Appendix A) is called a Frame. A Frame is made of a Frame Number, followed by a set of Channel sample values, one sample per Channel, all sampled at the same index value. One of the Channels may serve as an index. When this is the case, it is always the first Channel in the Frame. When there is no Channel index, then the Frame Number serves as an index.

RP66V1.Frame Data Information recorded in Frames is called Frame Data. This consists of Channel samples, one sample per Channel per Frame.

RP66V1.Frame Number A positive integer recorded at the beginning of each Frame. The Frame Number is a sequential index of the Frames of the same Frame Type. Frame *n* precedes Frame *n*+1, although other Logical Records may fall between.

RP66V1.Frame Type The Name (Origin, Copy Number, Identifier) of a Frame Object used to group Frames that have the same organization. This Name is also used as the Data Descriptor Reference in the Frames, and the Frames are known to be of the given Frame Type. Frames of a given Frame Type all contain samples of the same set of Channels and all in the same order. The Representation Code and Units used to record a Channel are the same in all Frames of a given Frame Type but may be different in another Frame Type. A Channel sample may change size (number of Elements) from Frame to Frame and may even become Absent when its number of Elements reduces to zero.

RP66V1.Header (Refer to Logical Record Segment Header) See *RP66V1.Logical Record Segment Header*.

RP66V1.Identifier That part of an Object Name that is textual. The Identifier is what commonly distinguishes one Object from another. Two Objects of the same Type may have the same Identifier, in which case the other Subfields of the Name are used to distinguish the Objects. Identifiers of certain Types of Objects uniquely identify the type of data represented in the Object, and such Identifiers (typically mnemonic in nature) are dictionary-controlled.

RP66V1.IFLR See *RP66V1.Indirectly Formatted Logical Record*

RP66V1.Index Channel The first Channel in a Frame, when the Frame has an Index Channel. A Frame may be indexed by Frame Number only and need not have an Index Channel. Whether or not a Frame has an Index

Channel is specified in the associated Frame Object. When a Frame has an Index Channel, then all Channel values in the Frame are considered to be sampled at the index indicated by the value of the Index Channel.

RP66V1.Indirectly Formatted Data That part of the Body of an Indirectly Formatted Logical Record (IFLR) that follows the Data Descriptor Reference.

RP66V1.Indirectly Formatted Logical Record One of two kinds of Logical Record defined under DLIS. The Body of an IFLR consists of a Data Descriptor Reference, followed by an arbitrary number of bytes of Indirectly Formatted Data. This data is not self-descriptive. Instead, its format is determined from information contained in the Object named by the Data Descriptor Reference and possibly related Objects. For example, the format of a Frame Data IFLR is specified by a Frame Object and by one or more Channel Objects referenced by the Frame Object.

RP66V1.Invisible Envelope Data recorded on the physical medium and used as a control interface by the processor I/O subsystem, but not visible through normal application read/write requests, for example tape marks on magnetic tape.

RP66V1.Lexicon A list of dictionary-controlled words or phrases applicable as Name Part Values for a particular Name Part Type. For example, each Producer manages a Lexicon of Entity names and another Lexicon of Quantity names.

RP66V1.Locus A sequence of distinct points in space and time, each of which has a three-dimensional Position coordinate and a Time coordinate.

RP66V1.Logical File A sequence of two or more contiguous Logical Records in a Storage Set that begins with a File Header Logical Record and contains no other File Header Logical Records. A Logical File must have at least one OLR (Origin) Logical Record immediately following the File Header Logical Record. A Logical File supports user-level organization of data.

RP66V1.Logical Format The view of DLIS data that is completely independent of any physical mapping. The DLIS Logical Format consists of a sequence of Logical Records organized into one or more Logical Files. This format is the same for any physical representation of the data.

RP66V1.Logical Record A sequence of one or more contiguous Logical Record Segments. A Logical Record supports application-level organization of data.

RP66V1.Logical Record Body The sequential concatenation of the Logical Record Segment Bodies from the Logical Record Segments that make up the Logical Record.

RP66V1.Logical Record Segment A sequence of contiguous 8-bit bytes organized to have a Logical Record Segment Header, followed (optionally) by an Encryption Packet, followed by a Logical Record Segment Body, followed (optionally) by a Logical Record Segment Trailer. Logical Record Segments are used to bind the Logical Format to a physical format.

RP66V1.Logical Record Segment Attributes Eight bits of binary data that describe the attributes of a Logical Record Segment.

RP66V1.Logical Record Segment Body The part of a Logical Record Segment that contains some or all of the data belonging to a Logical Record. The intersection of a Logical Record and one of its Logical Record Segments is the Logical Record Segment Body.

RP66V1.Logical Record Segment Encryption Packet An optional packet of information, following the Logical Record Segment Header, that contains encryption/decryption information for the Logical Record Segment. The Encryption Packet begins with its size in bytes and the Company Code of the Producer. Any additional data in the Encryption Packet is meaningful only to the Producer's organization.

RP66V1.Logical Record Segment Header The first part of a Logical Record Segment. It contains the Logical Record Segment Length, the Logical Record Segment Attributes, and the Logical Record Type.

RP66V1.Logical Record Segment Length A two-byte unsigned integer that specifies the combined length of all parts of the Logical Record Segment.

RP66V1.Logical Record Segment Trailer The last part of a Logical Record Segment. It contains three fields, all of which are optional: the Padding, the Checksum, and the Logical Record Segment Trailing Length.

RP66V1.Logical Record Structure One of the attributes specified in the Logical Record Segment Attributes. It specifies whether the Logical Record is an EFLR or an IFLR.

RP66V1.Logical Record Type A one-byte unsigned integer that indicates the general semantic content of the Logical Record.

RP66V1.Long Name A structured textual description that provides an understanding, to humans, of the named item, with enough detail to distinguish it from similar items that have different meanings. It is not a unique identifier. A Long Name is represented in a Long-Name Object.

RP66V1.Maximum Visible Record Length The maximum permitted length of a Visible on a Record Storage Unit. Its current value is 16,384 bytes.

RP66V1.Minimum Visible Record Length The minimum permitted length of a Visible Record on a Record Storage Unit. Its current value is 20 bytes, which is based on the minimum Logical Record Segment Length (16 bytes) plus the Visible Record Length (2 bytes) and the Format Version (2 bytes).

RP66V1.Name Used to refer to the Name Characteristic of a Set, Object, or Attribute. The Name of a Set or Attribute is a character string (Representation Code IDENT). The Name of an Object is an aggregate consisting of an integer Origin, an integer Copy Number, and a character Identifier.

RP66V1.Name Part Type A classification of the words or phrases that apply to a particular part of the Long Name structure, for example Entity or Quantity.

RP66V1.Name Part Value A word or phrase that applies to a particular part (Name Part Type) of a Long Name structure. For example, “Density” and “Porosity” are Name Part Values that apply to the Name Part Type “Quantity”.

RP66V1.Object A data entity that has a Name and a number of Attributes. An Object is like a row in a table of information. Its Attributes are like the column values in the row. Objects are recorded in EFLRs.

RP66V1.Object Component An Object Component indicates the beginning of a new Object in a Set and is followed by zero or more Attribute Components. The Attributes of an Object that has no Attribute Components are completely specified in the Template. The Object Component contains a single Characteristic, the Object Name, which is mandatory.

RP66V1.Origin As an Object in a Logical File, an Origin contains information describing the original circumstances under which that or another Logical File was created. Only one Origin Object in a Logical File, namely the first one, describes that Logical File. Additional Origin Objects describe other Logical Files from which data has been copied. Other Objects in a Logical File are keyed to their appropriate Origin Object by means of an integer Subfield in their Names, namely the Origin Subfield. This integer value matches the Origin Subfield of the appropriate Origin Object. This integer value is also commonly referred to as the Object’s Origin.

RP66V1.Pad Bytes Pad Bytes are part of the Logical Record Segment Trailer and are used to alter the size of a Logical Record Segment to satisfy minimum size requirements or more commonly to make the Logical Record Segment Length divisible by some integer. In all cases, the Logical Record Segment Length must be divisible by two. Additionally, certain encryption methods may require the length of the Logical Record Segment Body plus the Pad Bytes to be divisible by some other factor.

RP66V1.Pad Count This is the first byte of the Pad Bytes and indicates how many Pad Bytes there are. The maximum number of Pad Bytes may therefore not exceed 255.

RP66V1.Padding An informal reference to Pad Bytes.

RP66V1.Parent File The Logical File in which data are originally created. Some data in a Logical File may have been copied from other Logical Files.

RP66V1.Path A sequence of space-time coordinates, where space is typically represented by depth, radial distance from a vertical line, and angular displacement about the same vertical line. The vertical line used is the one

that goes through a well's Well Reference Point, a point used to identify the location of a well. A Path may be represented by a combination of Channels, each of which represents one of the above-mentioned coordinates.

RP66V1.Physical Format The way in which recorded data is located and organized on a particular physical medium such as a magnetic tape or disk file. With some I/O systems more than one organization and view of data is supported on the same medium. Each such view corresponds to a different physical format. For example, disk files may be viewed as having variable-length record structures, block structures, byte stream structures, etc., depending on the I/O facility that is used. The physical format determines the way in which Logical Record Segments are used but generally has no effect on Logical Records.

RP66V1.Predecessor Used to indicate the relation between successive Logical Record Segments. If two Logical Record Segments belong to the same Logical Record, then one of them — the one that comes first — is a Predecessor of the other. The first Logical Record Segment of a Logical Record has no Predecessor.

RP66V1.Private A Logical Record with Type ≥ 128 is said to be Private. In particular, the semantic content of such a Logical Record is decided upon solely by the Producer and not via any public standardization process. Private Logical Records are available to consumers in general, unless encrypted. The fact that a Logical Record is Private does not imply that it is also encrypted.

RP66V1.Producer The system or application program or company that records information under the DLIS Logical Format. The Producer writes what the Consumer reads.

RP66V1.Public A Logical Record with Type < 128 is said to be Public. In particular, the semantic content of such a Logical Record is agreed to by the all users via a standardization process administered by the API. Such Logical Records may not be used except in accordance with the standard definition. Public Logical Records may be encrypted or not, according to the needs of the Producer.

RP66V1.Radial Drift Radial Drift is the perpendicular distance of a point from a vertical line that passes through the Well Reference Point of a well.

RP66V1.Record Structure One of possibly many different physical formats. A Storage Unit is said to have Record Structure if data is written and read in sequential, variable-length records. For all Record Structure Storage Units each record must begin with a two-byte unsigned integer Visible Record Length, followed by a two-byte Format Version, followed by an integer number of Logical Record Segments. Other requirements, for example use of Tape Marks, depend on the particular physical medium.

RP66V1.Representation Code Each distinct piece of information in the Logical Format has a well-defined representation that extends across one or more bytes. Each different representation is identified by a one-byte Representation Code. Representation Codes are defined in Appendix B and identify the various floating point, integer, and text representations permitted under the DLIS.

RP66V1.Sample (of a Channel) A Channel Sample is one of a sequence of evaluations of a Channel. A Channel Sample may be a scalar sensor reading (i.e., a single number), or it may be an array representing a waveform or some other multi-dimensional data.

RP66V1.Semantics Semantics is the definition of what data means and how it is used. Whereas syntax provides rules for recording Objects in Sets, semantics defines the Objects that may be recorded, e.g., the File-Header, Origin, Channel, Frame, etc. Objects.

RP66V1.Set A data entity that has a Type and optionally a Name, and contains a number of Objects. A Set is like a table of information in which the Objects are the rows of the table. Each Set is contained in an EFLR (exactly one Set per EFLR), and there may be more than one Set with the same Type in a Logical File.

RP66V1.Set Component A Set Component indicates the beginning of a Set and is followed by one or more Template Attribute Components. A Set Component always contains a Type Characteristic and may contain a Name Characteristic.

RP66V1.Set Type A textual identifier of the type of Objects contained in a Set. The Objects in a Set are characterized by the Attributes in the Template of the Set. The Attributes associated with each given Set Type are specified in the standard.

RP66V1.Slot A data entity that has a Type and optionally a Name, and contains a number of Objects. A Set is like a table of information in which the Objects are the rows of the table. Each Set is contained in an EFLR (exactly one Set per EFLR), and there may be more than one Set with the same Type in a Logical File. One of a fixed number of positions in a Frame for recording a single Channel Sample value. Channels are assigned to Slots in a Frame in a specific order, and all Slots follow the Frame Number. The Index Channel, if there is one, is in the first Slot of a Frame.

RP66V1.Splice A Splice is the result of concatenating two or more instances of a Channel (e.g., from different runs) to obtain a resultant Channel defined over a larger domain or interval. The information associated with a Splice is represented in a Splice Object.

RP66V1.Static Information Static Information consists of Objects typically used to describe Channels and Frames, and information about Channels and Frames. Static Information is typically required by an application prior to the processing of Frames.

RP66V1.Storage Set A group of Storage Units that contain a common DLIS Logical Format (e.g., a sequence of Logical Files) and for which at least two Storage Units are spanned by a single Logical File.

RP66V1.Storage Set Identifier A 60-character ASCII field in the Storage Unit Label used to identify a Storage Set.

RP66V1.Storage Unit Something that contains DLIS data and is manageable as a unit at the human level, (e.g., a tape or disk file).

RP66V1.Storage Unit Label The first 80 bytes of the Visible Envelope of a Storage Unit. The Storage Unit Label consists of five fixed-length ASCII fields used to identify the Storage Unit and the Storage Set of which it is a part.

RP66V1.Storage Unit Sequence Number A positive integer (its ASCII representation) in the Storage Unit Label that indicates the order in which a Storage Unit occurs in a Storage Set.

RP66V1.Storage Unit Structure An ASCII keyword in the Storage Unit Label that reflects the Physical Format of the Storage Unit and indicates the binding mechanism between the Physical Format and the DLIS Logical Format.

RP66V1.Subfield A part of a datum for which the representation is described by a simple (not compound) Representation Code. For example, the Subfields of a datum having Representation Code OBNAM are, in order, an integer (UVARI), another integer (USHORT), and a string (IDENT).

RP66V1.Successor Used to indicate the relation between successive Logical Record Segments. If two Logical Record Segments belong to the same Logical Record, then one of them — the one that comes second — is a Successor of the other. The last Logical Record Segment of a Logical Record has no Successor.

RP66V1.Syntax Syntax is the definition of the rules for how to record data but not for what the data means (at the application level) or how it is to be used. Syntax does convey meaning of data, but at a level below applications. For example, the rules of syntax tell when a Component has a Type Characteristic and how to get it, but syntax provides no information on the meaning or use of the values the Type Characteristic may have.

RP66V1.Template A sequence of Attributes at the beginning of a Set that specify defaults for the Objects in the Set. Attributes in the Template must have Labels. Objects in the Set have no Attributes other than those identified in the Template.

RP66V1.Tool Zero Point A fixed point on the tool string (usually the bottom of the bottom tool) that stands opposite the Well Reference Point when Borehole Depth is zero.

RP66V1.Trailing Length The optional last field in the Logical Record Segment Trailer that contains a copy of the Logical Record Segment Length.

RP66V1.Transient Information Transient Information consists of Objects that correspond to events that occur during the processing of Frames. These events can affect Objects in the Static Information or can correspond to messages between the operator and the system.

RP66V1.Unzoned A Parameter or Computation Object is said to be Unzoned when it has the same value everywhere. This is the case when the Zones Attribute of the Object is absent.

RP66V1.Update An Update is a change made to data represented by an Object (e.g., a Parameter) previously recorded in a Logical File. The change and information related to the change are represented in an Update Object recorded in the same Logical File.

RP66V1.Value (of an Attribute) The Value of an Attribute is the data contained in its Value Characteristic. A Value may consist of one or more Elements, each of which has the same Units and Representation Code.

RP66V1.Vertical Depth Depth measured along the Vertical Generatrix from the Well Reference Point. Vertical depth increases in a downward direction and is negative above the Well Reference Point.

RP66V1.Vertical Generatrix A vertical line that passes through the Well Reference Point.

RP66V1.Visible Envelope Information on a Storage Unit that is provided to applications as normal data by the processor's I/O subsystem, but which is not part of the DLIS Logical Format. Information in the Visible Envelope includes the Storage Unit Label. Other information in the Visible Envelope may be used to define or enhance the binding of Logical Record Segments to the Physical Format.

RP66V1.Visible Record A Visible Record is a term that applies to Record Structure Storage Units. It consists of all the data bytes accessed by means of a record read operation from the system-specific file access subsystem.

RP66V1.Visible Record Length When DLIS information is recorded in variable length physical records, each Visible Record begins with a two-byte unsigned integer length of the Visible Record called the Visible Record Length. This length is considered to be external the DLIS Logical Format.

RP66V1.Well Reference Point A unique point that is the origin of a well's spatial coordinate system for information in a Logical File. This point is defined relative to some permanent vertical structure, such as ground level or mean sea level, and to three independent geographical coordinates, which typically include Latitude and Longitude. The same well may have different Well Reference Points in different Logical Files.

RP66V1.Zone A Zone is a single interval in depth or time. The depth coordinate may be either Vertical Depth or Borehole Depth.

RP66V1.Zoned A Parameter or Computation Object is said to be Zoned when it has different values in different intervals along a depth or time domain or is undefined in some interval of a depth or time domain. This is the case when the Zones Attribute of the Object is not absent.

RP66V2 *Recommended Practice 66* version 2 is an API standard first published in June 1996 that describes a (mostly) more advanced file format compared with [RP66V1](#). It is unused by the industry and will not be referenced here. The specification can be found online from the Petrotechnical Open Software Corporation here [RP66V2 standard](#) See also the [RP66V2 glossary](#)

UOM *Unit of Measure* for engineering values. In [LIS](#) these are a set of fixed terms organised into several categories, such as *Linear Length*. Values can only be converted between units of in the same category. In [RP66V1](#) these are composed by a BNF parseable string.

Xaxis

X Axis The index channel in an array, for example an array of frames. Typically depth or time.

LICENCE

B.1 OpenSource Licence

TotalDepth is released under GPL 2.:

GNU GENERAL PUBLIC LICENSE
Version 2, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users. This General Public License applies to most of the Free Software Foundation's software and to any other program whose authors commit to using it. (Some other Free Software Foundation software is covered by the GNU Lesser General Public License instead.) You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too, receive or can get the

(continues on next page)

(continued from previous page)

source code. And you must show them these terms so they know their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that redistributors of a free program will individually obtain patent licenses, in effect making the program proprietary. To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all.

The precise terms and conditions for copying, distribution and modification follow.

GNU GENERAL PUBLIC LICENSE
TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License. The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term "modification".) Each licensee is addressed as "you".

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

1. You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and give any other recipients of the Program a copy of this License along with the Program.

You may charge a fee for the physical act of transferring a copy, and

(continues on next page)

(continued from previous page)

you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

- a) You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.
- b) You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License.
- c) If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this License. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following:

- a) Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections

(continues on next page)

(continued from previous page)

1 and 2 above on a medium customarily used for software interchange; or,

b) Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

c) Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.

4. You may not copy, modify, sublicense, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense or distribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

5. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to

(continues on next page)

(continued from previous page)

these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

7. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Program at all. For example, if a patent license would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Program under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

9. The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and "any

(continues on next page)

(continued from previous page)

later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you may choose any version ever published by the Free Software Foundation.

10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

11. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>

(continues on next page)

(continued from previous page)

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.

Also add information on how to contact you by electronic and paper mail.

If the program is interactive, make it output a short notice like this when it starts in an interactive mode:

```
Gnomovision version 69, Copyright (C) year name of author
Gnomovision comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
This is free software, and you are welcome to redistribute it
under certain conditions; type `show c' for details.
```

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, the commands you use may be called something other than `show w' and `show c'; they could even be mouse-clicks or menu items--whatever suits your program.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the program, if necessary. Here is a sample; alter the names:

```
Yoyodyne, Inc., hereby disclaims all copyright interest in the program
`Gnomovision' (which makes passes at compilers) written by James Hacker.
```

```
<signature of Ty Coon>, 1 April 1989
Ty Coon, President of Vice
```

This General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License.

B.2 Other Licences

For other licences contact: apaulross@gmail.com

A

Absent Value, 205

B

Backup Mode, 205

D

Data Format Specification Record, 205

Datum Specification Block, 205

DFSR, 205

DLIS, 205

DSB, 205

E

EFLR, 205

Engineering Value, 205

Entry Block, 205

Explicitly Formatted Logical Record, 205

F

FFLR, 205

Fixed Format Logical Record, 205

Frame, 205

Frame Array, 206

Frame Set, 205

I

IFLR, 206

Indirectly Formatted Logical Record, 206

L

LAS, 206

LIS, 206

LIS-79, 206

LIS-A, 206

Log Pass, 206

Logical Record, 206

LRH, 206

M

Mnem, 206

Mnemonic, 206

P

Physical Record, 206

R

Rep Code, 206

RepCode, 206

Representation Code, 206

RP66, 206

RP66V1, 206

RP66V1.Absent Value, 206

RP66V1.Attribute, 207

RP66V1.Channel, 207

RP66V1.Characteristic, 207

RP66V1.Company Code, 207

RP66V1.Component, 207

RP66V1.Component Descriptor, 207

RP66V1.Component Format, 207

RP66V1.Component Role, 207

RP66V1.Compound Representation Code, 207

RP66V1.Consumer, 207

RP66V1.Copy Number, 207

RP66V1.Count, 207

RP66V1.Data Descriptor Reference, 207

RP66V1.Defining Origin, 207

RP66V1.Descriptor, 207

RP66V1.Dictionary, 207

RP66V1.Dimension, 207

RP66V1.EFLR, 208

RP66V1.Element, 208

RP66V1.Encryption Packet, 208

RP66V1.Explicitly Formatted Logical Record, 208

RP66V1.Format Version, 208

RP66V1.Frame, 208

RP66V1.Frame Data, 208

RP66V1.Frame Number, 208

RP66V1.Frame Type, 208

RP66V1.Header (*Refer to Logical Record Segment Header*), 208

RP66V1.Identifier, 208

RP66V1.IFLR, [208](#)
RP66V1.Index Channel, [208](#)
RP66V1.Indirectly Formatted Data, [209](#)
RP66V1.Indirectly Formatted Logical Record, [209](#)
RP66V1.Invisible Envelope, [209](#)
RP66V1.Lexicon, [209](#)
RP66V1.Locus, [209](#)
RP66V1.Logical File, [209](#)
RP66V1.Logical Format, [209](#)
RP66V1.Logical Record, [209](#)
RP66V1.Logical Record Body, [209](#)
RP66V1.Logical Record Segment, [209](#)
RP66V1.Logical Record Segment Attributes, [209](#)
RP66V1.Logical Record Segment Body, [209](#)
RP66V1.Logical Record Segment Encryption Packet, [209](#)
RP66V1.Logical Record Segment Header, [209](#)
RP66V1.Logical Record Segment Length, [209](#)
RP66V1.Logical Record Segment Trailer, [210](#)
RP66V1.Logical Record Structure, [210](#)
RP66V1.Logical Record Type, [210](#)
RP66V1.Long Name, [210](#)
RP66V1.Maximum Visible Record Length, [210](#)
RP66V1.Minimum Visible Record Length, [210](#)
RP66V1.Name, [210](#)
RP66V1.Name Part Type, [210](#)
RP66V1.Name Part Value, [210](#)
RP66V1.Object, [210](#)
RP66V1.Object Component, [210](#)
RP66V1.Origin, [210](#)
RP66V1.Pad Bytes, [210](#)
RP66V1.Pad Count, [210](#)
RP66V1.Padding, [210](#)
RP66V1.Parent File, [210](#)
RP66V1.Path, [210](#)
RP66V1.Physical Format, [211](#)
RP66V1.Predecessor, [211](#)
RP66V1.Private, [211](#)
RP66V1.Producer, [211](#)
RP66V1.Public, [211](#)
RP66V1.Radial Drift, [211](#)
RP66V1.Record Structure, [211](#)
RP66V1.Representation Code, [211](#)
RP66V1.Sample (*of a Channel*), [211](#)
RP66V1.Semantics, [211](#)
RP66V1.Set, [211](#)
RP66V1.Set Component, [211](#)
RP66V1.Set Type, [211](#)
RP66V1.Slot, [212](#)
RP66V1.Splice, [212](#)
RP66V1.Static Information, [212](#)
RP66V1.Storage Set, [212](#)
RP66V1.Storage Set Identifier, [212](#)
RP66V1.Storage Unit, [212](#)
RP66V1.Storage Unit Label, [212](#)
RP66V1.Storage Unit Sequence Number, [212](#)
RP66V1.Storage Unit Structure, [212](#)
RP66V1.Subfield, [212](#)
RP66V1.Successor, [212](#)
RP66V1.Syntax, [212](#)
RP66V1.Template, [212](#)
RP66V1.Tool Zero Point, [212](#)
RP66V1.Trailing Length, [212](#)
RP66V1.Transient Information, [212](#)
RP66V1.Unzoned, [212](#)
RP66V1.Update, [213](#)
RP66V1.Value (*of an Attribute*), [213](#)
RP66V1.Vertical Depth, [213](#)
RP66V1.Vertical Generatrix, [213](#)
RP66V1.Visible Envelope, [213](#)
RP66V1.Visible Record, [213](#)
RP66V1.Visible Record Length, [213](#)
RP66V1.Well Reference Point, [213](#)
RP66V1.Zone, [213](#)
RP66V1.Zoned, [213](#)
RP66V2, [213](#)

U
UOM, [213](#)

X
X Axis, [213](#)
Xaxis, [213](#)